Data Structure and Algorithm Design

ME/MSc (Computer) – Pokhara University

Prepared by:

Assoc. Prof. Madan Kadariya (NCIT)

Chapter 2:
Advanced Data Structures and Algorithms (11 hrs)

Outline

1. Advanced Trees
   - B and B+ Trees
2. Advanced Graph
   - Planar Graphs and its applications in Geographic and Circuit Layout Design
3. Network Flow Algorithms
   - Ford-Fulkerson Algorithm
   - Edmonds-Karp Algorithm
4. Advanced Shortest Path Algorithms
   - Bellman-Ford Algorithm
   - Johnson's algorithm
5. Graphical Data Structures
   - Quadtrees, KD Trees, R-Trees
6. Computational Geometry
   - Convex Hull and Voronoi Diagrams
7. Case Studies in Red-Black Tree and Its Applications, Applications of Directed Acyclic Graph, Applications of Minimum Spanning Tree in Network Design

Review of Tree Concepts

TREES

- Tree is a connected undirected graph with no simple circuits.
- Because a tree cannot have a simple circuit, a tree cannot contain multiple edges or loops. Therefore any tree must be a simple graph.

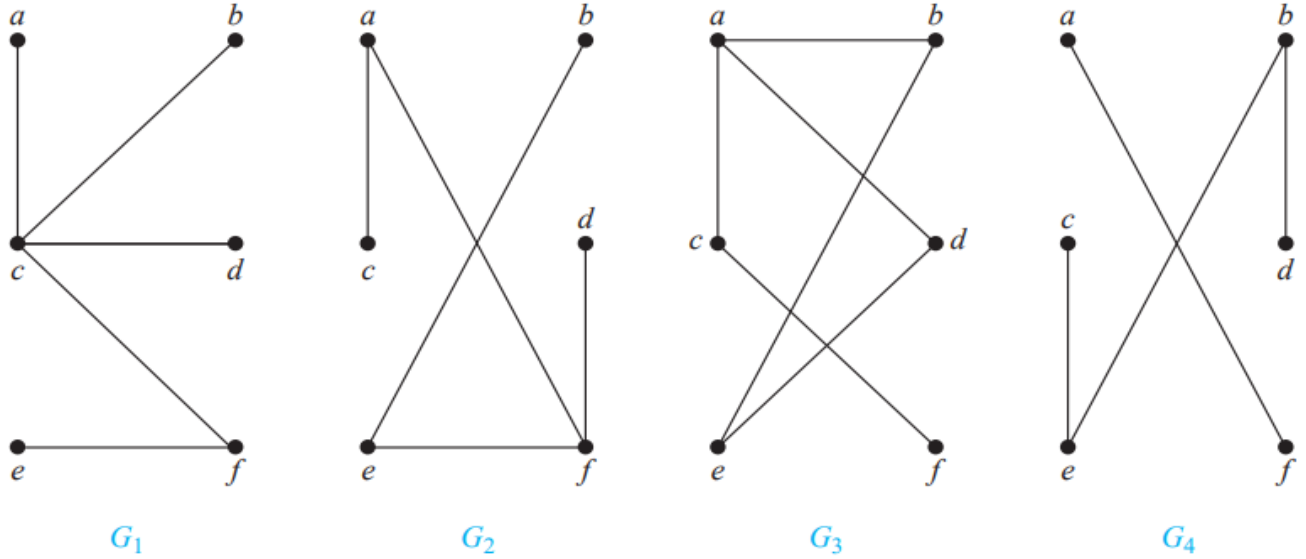


FIGURE 2 **Examples of Trees and Graphs That Are Not Trees.**

- G1 and G2 are trees
- G3 is not a tree because e, b, a, d, e is a simple circuit in this graph. Finally, G4 is not a tree because it is not connected.

Review of Tree Concepts

Rooted Trees:

- A rooted tree is a tree in which one vertex has been designated as the root and every edge is directed away from the root.
- A rooted tree is called an **m-ary** tree if every internal vertex has no more than m children. The tree is called a **full m-ary** tree if every internal vertex has exactly m children. An m-ary tree with m = 2 is called a binary tree.

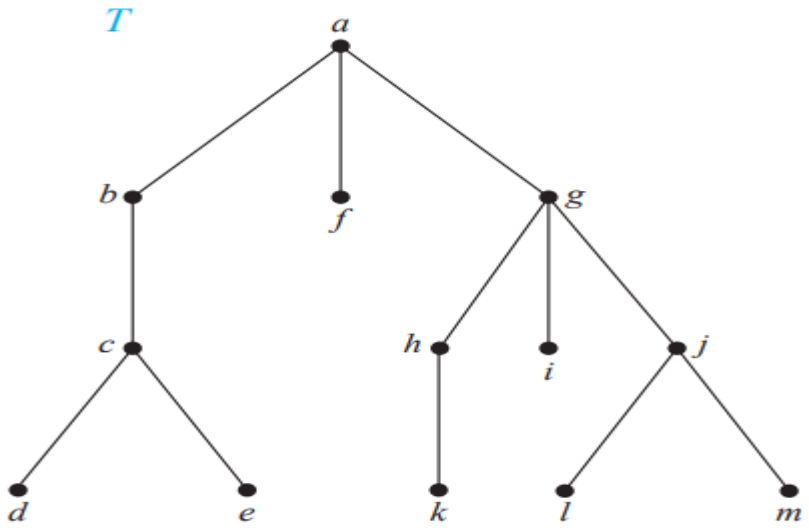
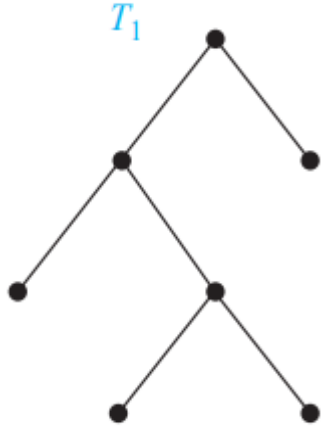


FIGURE 5 **A Rooted Tree *T*.**

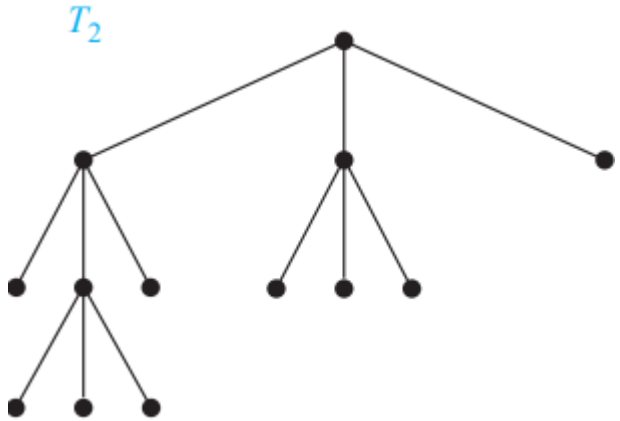
- A vertex of a rooted tree is called a **leaf** if it has no children.
- Vertices that have children are called **internal vertices**.
- Vertices with the same parent are called **siblings**
- The **ancestors** of a vertex other than the root are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root.
- The **descendants** of a vertex v are those vertices that have v as an ancestor

Review of Tree Concepts

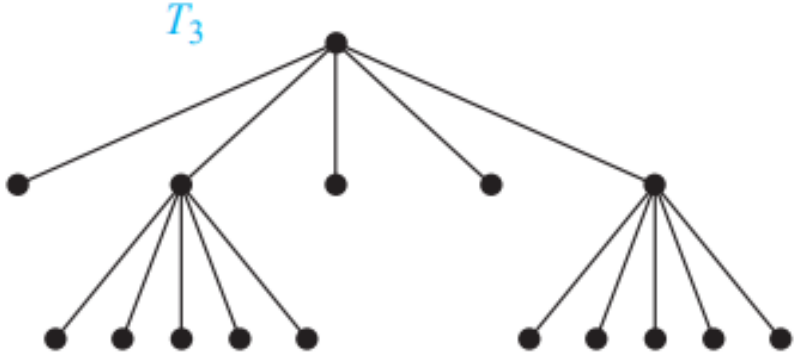
Rooted Trees:



T1 is a full binary tree because each of its internal vertices has two children



T2 is a full 3-ary tree because each of its internal vertices has three children



In T3 each internal vertex has five children, so T3 is a full 5-ary tree

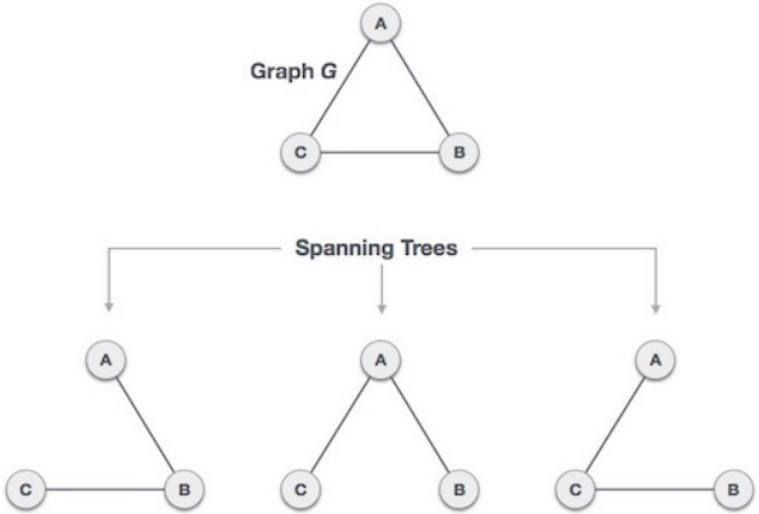
Spanning Trees:

- A **spanning tree** is a subset of Graph G, which has all the vertices covered with minimum possible number of edges. Formally, Let G be a simple graph. A spanning tree of G is a subgraph of G that is a tree containing every vertex of G.
- Every connected and undirected Graph G has at least one spanning tree. A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.

Review of Tree Concepts

Spanning Trees:

- A complete undirected graph can have maximum $\mathbf{n^{n-2}}$ number of spanning trees, where **n** is the number of nodes. In example, **n is 3,** hence $3^{3-2} = 3$ spanning trees are possible.



General Properties of Spanning Tree

- A connected graph G can have more than one spanning tree.
- All possible spanning trees of graph G, have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops).
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.
- Spanning tree has **n-1** edges, where **n** is the number of nodes (vertices).
- A complete graph can have maximum $\mathbf{n^{n-2}}$ number of spanning trees.

Indexing

- Indexing mechanisms are used to speedup access to desired data.
- Search Key-attribute to set of attributes used to lookup records in a file.
- An index file consists of records (called index entries) of the form

Search Key	Pointer

- Index files are typically much smaller than the original file
- Two basic kinds of indices:
  1. Ordered indices: search keys are stored in sorted order
  2. Hash indices: search keys are distributed uniformly across "buckets" using a "hash function".
- Index Evaluation Metrics:

  1. Access time 2. Insertion time 3. Deletion time 4. Space overhead

  5. Access types supported efficiently. E.g.,
  - records with a specified value in the attribute
  - or records with an attribute value falling in a specified range of values.
  - This strongly influences the choice of index, and depends on usage.

Indexing

Ordered Indices

- In an ordered index, index entries are stored sorted on the search key value. E.g., author catalog in library.

1. Primary index: in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
   - Also called clustering index
   - The search key of a primary index is usually but not necessarily the primary key.
2. Secondary index: an index whose search key specifies an order different from the sequential order of the file.
   - Also called non-clustering index.

- Index-sequential file: ordered sequential file with a primary index.

Indexing

- A **database index** is a data structure that improves the speed of data retrieval operations on a database table at the cost of additional writes and storage space to maintain the index data structure.

Types of Indexes

1. *Sparse Index*

- One entry per data block
- Identifies the first record of the block
- Requires data to be sorted

Advantages :

- Occupy much less space
- Can keep more of it in main memory
- *Faster access*

2. Dense Index

- One entry per record
- Data do not have to be sorted

Advantages:

- Can tell if a given record exists without accessing the file
- Do not require data to be sorted

Sparse vs Dense index

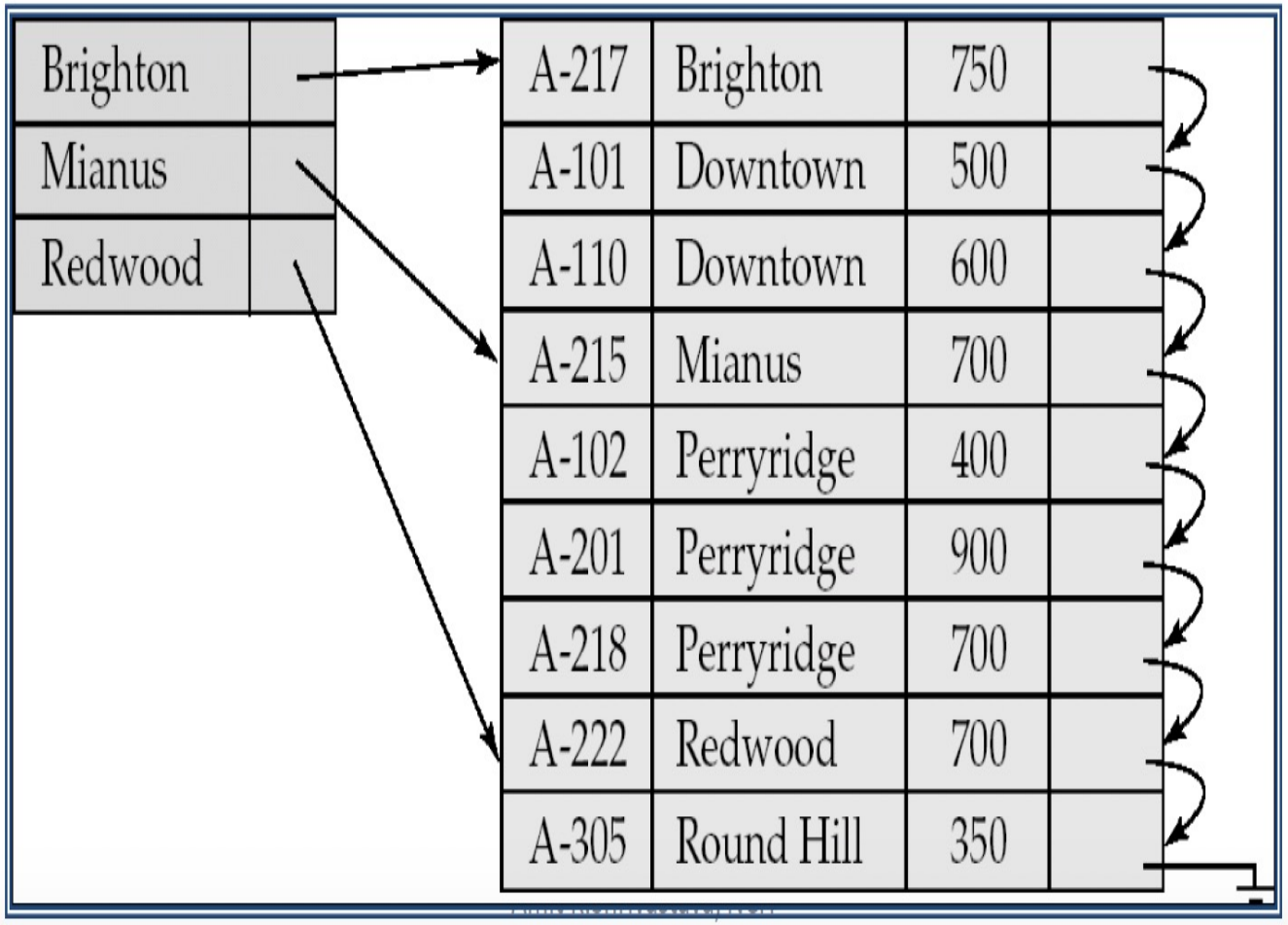


Fig: Sparse index

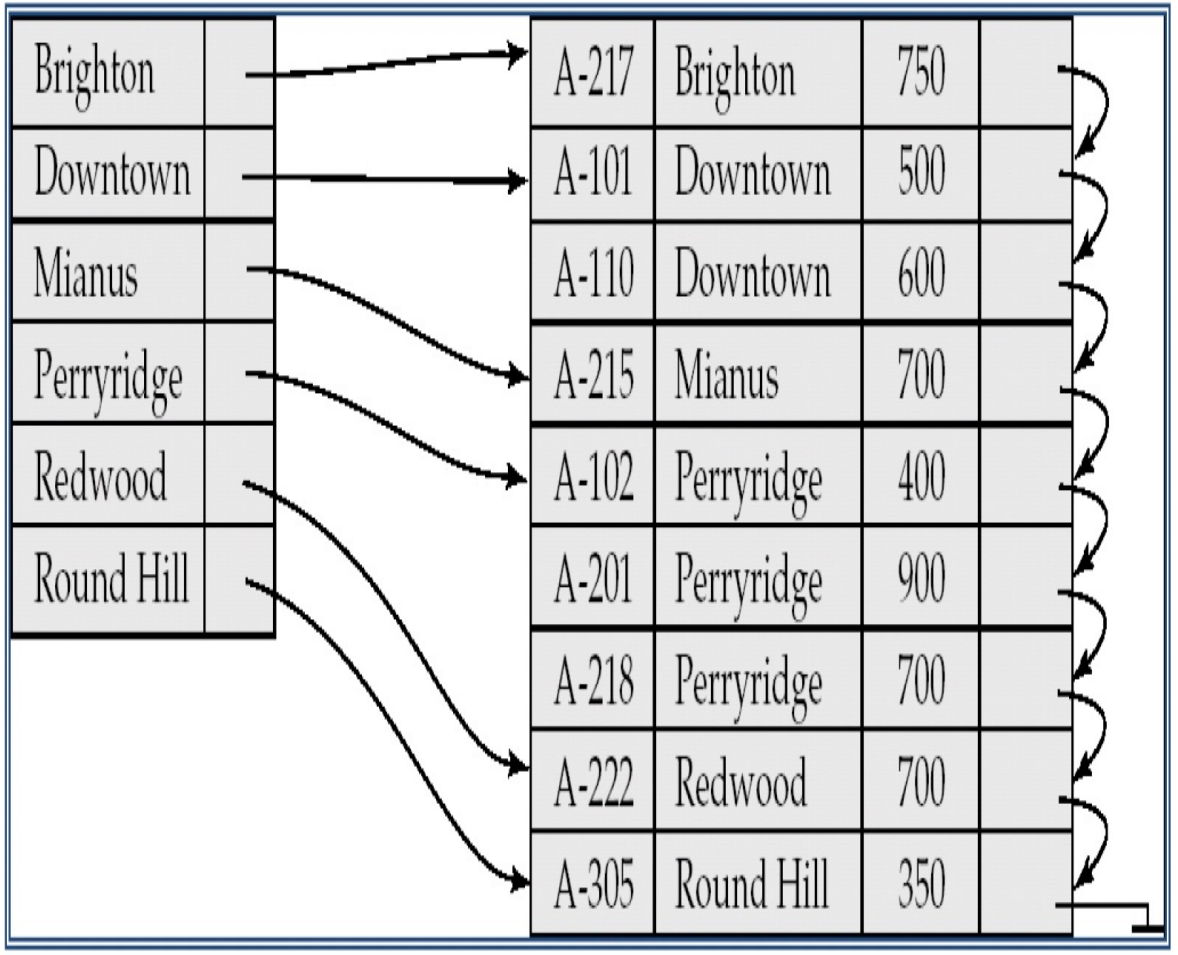


Fig: Dense index

Indexing

Database Indexes:

- Index is a pointer to data in a table. The pointer helps the storage engines to locate data with a reduced latency. Any kind of index usually slows down writes, because the index also needs to be updated every time data is inserted. Therefore, it is an important trade-off in storage systems to identify the required indexes for a dataset.

Disk Indexes:

- Data is ultimately stored in physical disks. Any block on the disk can be accessed via the sector and the track number. Data byte on a particular block can be located with the sector, track and the offset value.
- Example: a **block size of 512 bytes and a row size of 128 bytes**, a total of (512/128), 4 rows can be stored in a block. Now, to store 100 rows, a total of 25 blocks on the disk will be utilized.

Indexing

- Eg: To look for a particular key in the database, the engine needs to access at most 25 blocks. To to reduce access time, we need to index table where each row will have the key and pointer to every value in the database. Considering the size of each row of the index table to be 16 bytes, it would take at most 4 disk blocks.

- To locate a row, the database uses the index table (4 blocks) to find the key, then reads the record(1 block) from the data table, reducing access time from 25 blocks to 5. However, updating the index table is required for every new key insertion.

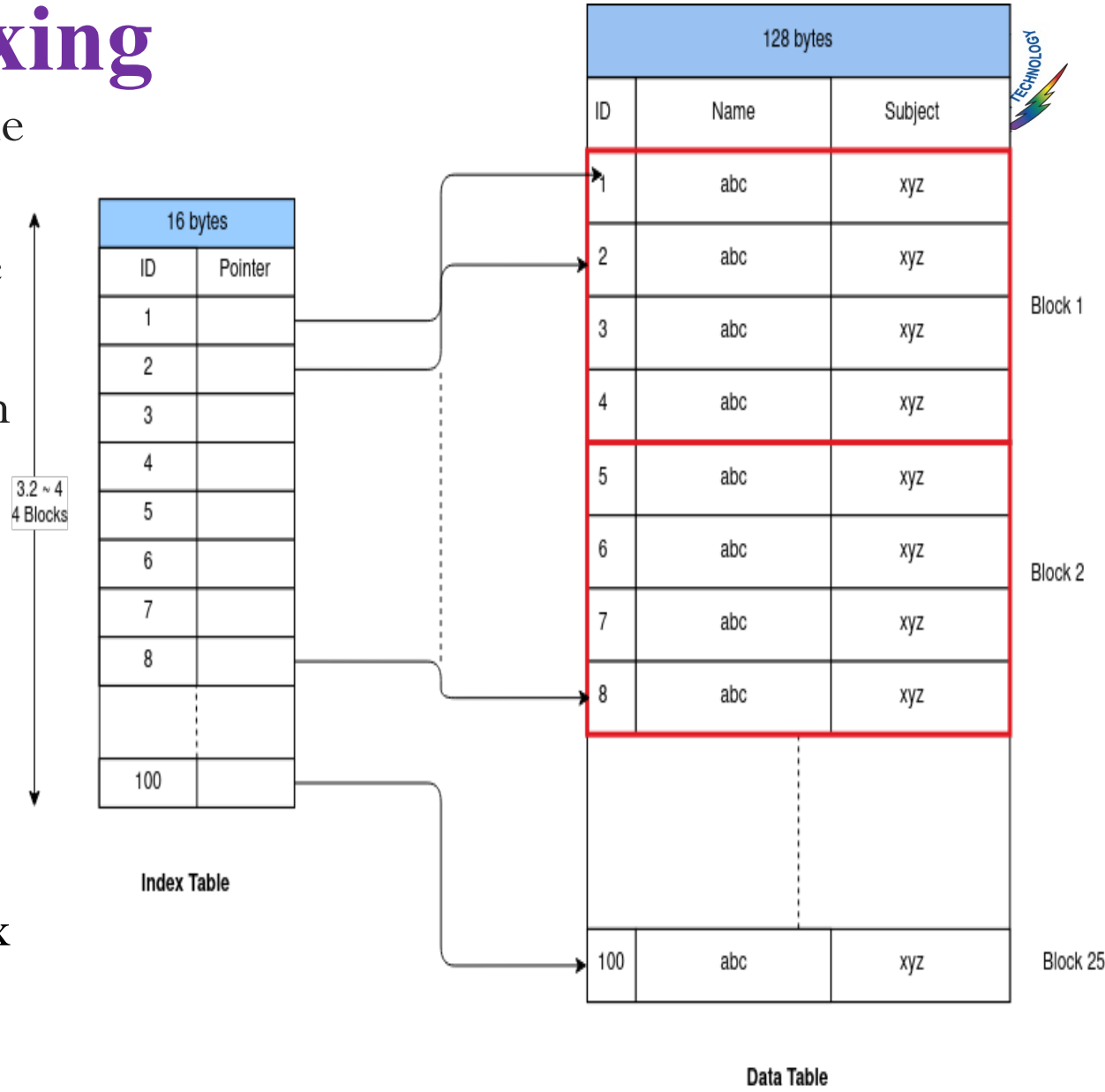


Fig: Example of a primary index

Illustration of Calculation

Table Storage:

Block Size: 512 bytes.

Row Size: 128 bytes.

- **Rows per Block:**
  Rows per block=Block size/Row size=512/128
  =4 rows per block.
- **Example Data Table:**
- Suppose the table has **100 rows**, each storing a key and associated data.
- To store all these rows:

Number of blocks required=Total rows/Rows per block=100/4=25 blocks.

Block 1: Row 1, Row 2, Row 3, Row 4
Block 2: Row 5, Row 6, Row 7, Row 8
…
Block 25: Row 97, Row 98, Row 99, Row 100

Key Lookup Without Indexing:

To find a particular key (e.g., Key 97), the database engine may need to search through all 25 blocks sequentially.

•This means **25 disk accesses**, which is time-consuming.

- **Index Table Design:**

Row Size in Index Table: 16 bytes (each row contains the key and a pointer to the data block).

Rows per Block in Index Table:

Rows per block=Block size/Row size=512/16=32 row s per block.

Storage for Index Table:

- For the original table of **100 rows**:
- Number of blocks required for the index table=

Total rows/Rows per block=100/32≈4 blocks.
Number of blocks required for the index table=

The index table is distributed over **4 blocks**:

Index Block 1: Index for Rows 1–32
Index Block 2: Index for Rows 33–64
Index Block 3: Index for Rows 65–96
Index Block 4: Index for Rows 97–100

Key Lookup With Indexing:

1.To find Key 97:

- First, access the **index table** (at most 4 blocks) to locate the block containing Key 97.
- The pointer in the index table directly points to **Block 25**, where Key 97 is stored.
- Access Block 25 to retrieve the actual data.

1.Total Disk Accesses:

- Access the **index table**: **4 blocks** (at most).
- Access the **data block**: **1 block** (Block 25).
- **Total disk accesses = 4 + 1 = 5** (significantly fewer than 25).

Indexing

Multi Level Indexing

- With an increase in the number of records (let's say 1000 rows), accessing the index table itself will require more disk blocks to be read. To further reduce this time, another level of indexing can be done. Another level of index can be created that stores the pointer to each block of the first-level index table.
- In example, another index table which stores the block address (*ID 1 to 32 and so on*) of the keys from the first index table.
- The multi-level index table requires only 2 blocks to get accessed, now the record can be accessed in (2 + 1 + 1) 4 block reads for a 1000 rows data table.
- As the scale of the dataset increases, more levels of indexing can be added

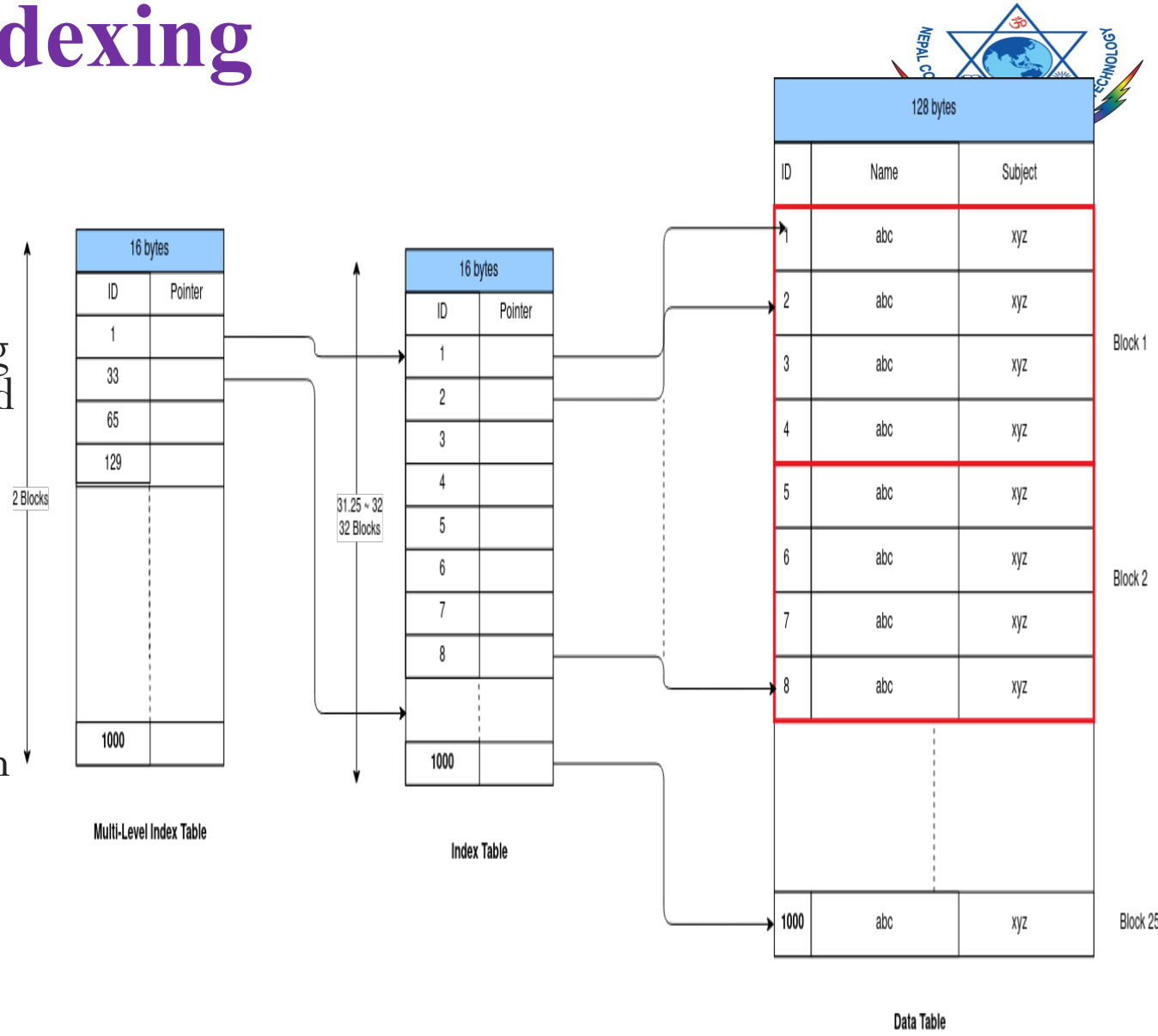
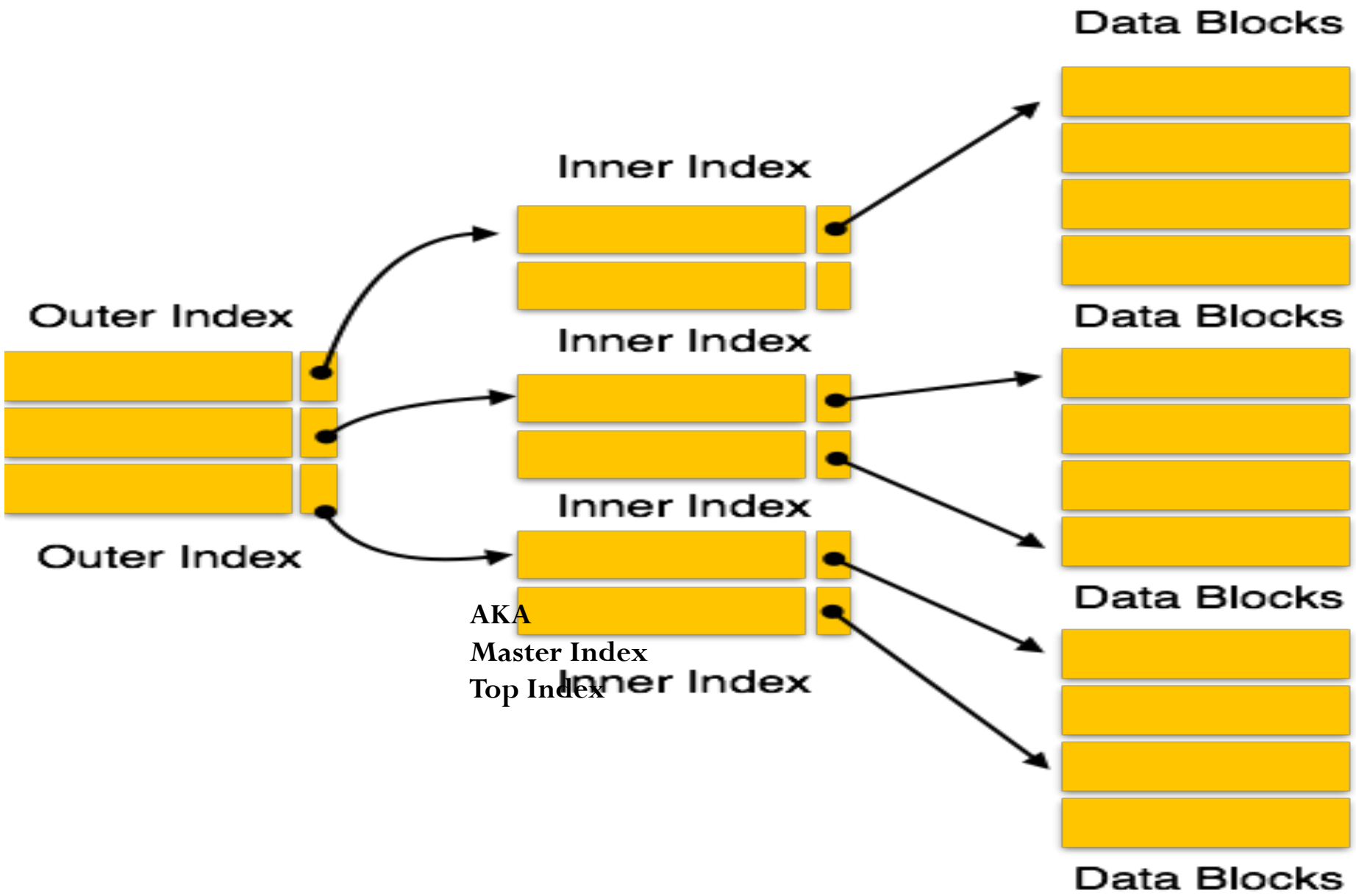


Fig: Multi-Level Indexing

Two levels Index Structure



Advance Tree

M-Way Trees

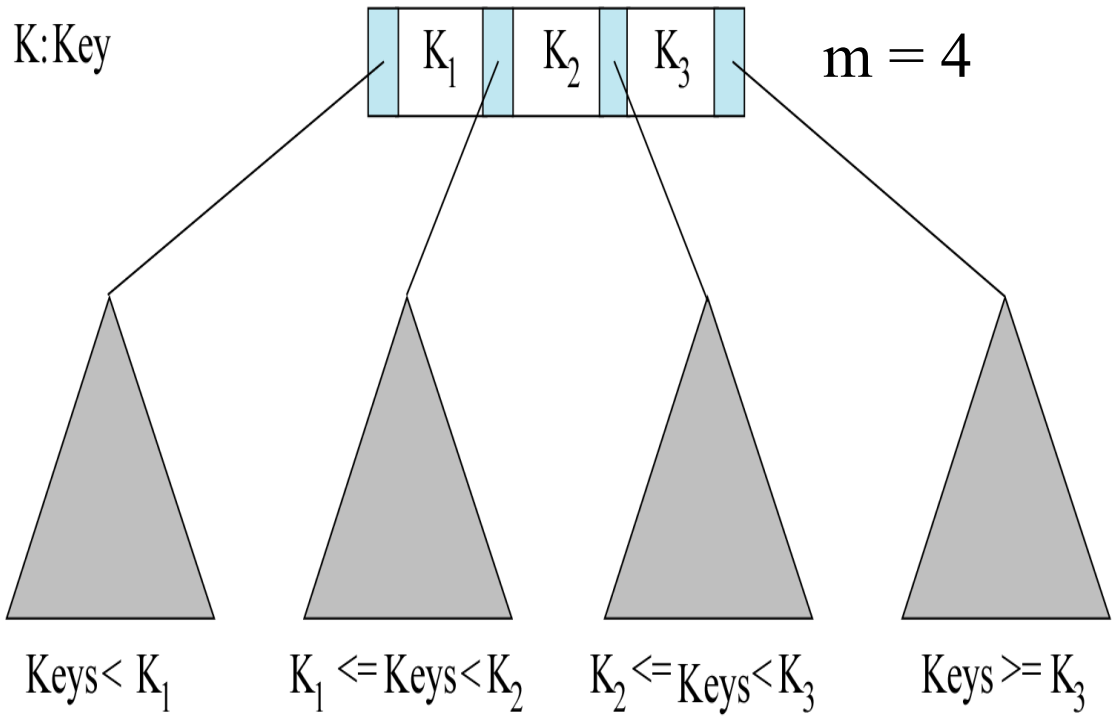
- M-way tree is a search tree.
- Each node of tree can have from zero to **m** subtrees.
- **m** is defined as the order of the tree.
- In an **m-Way** tree of order **m**, each node contains a maximum of **m – 1** elements and **m** children.
- The goal of **m-Way** search tree of height h calls for **O(h)** no. of accesses for an insert/delete/retrieval operation.
- The number of elements in an **m-Way** search tree of height **h** ranges from a minimum of **h** to a maximum of **m^h-1**

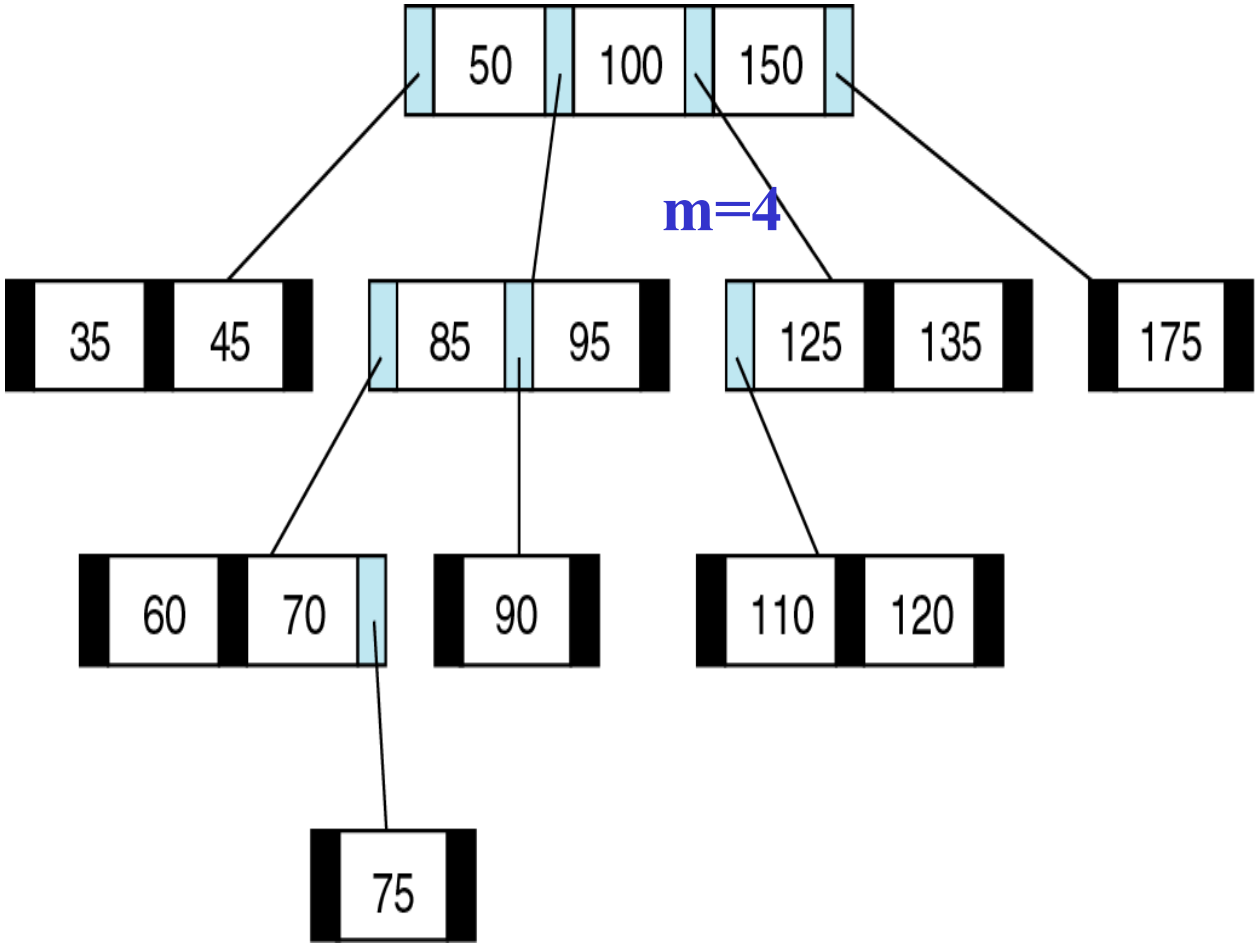
Given the nonempty multiway tree, the following properties are defined:

1. Each node has **0** to **m** subtrees.
2. Given a node contains **k** subtree pointers, some of which may be null, and **k-1** data entries.
3. The key values in the first subtree are all less than the key in the first entry, the key values in the other subtrees are all greater than or equal to the key in their parent entry.
4. The keys of the data entries are ordered.

Advance Tree

M-Way Trees





M-way tree of order 4. Key = (m-1)=3

Advance Tree (B-Tree)

Motivation for B-Trees

- Index structures for large datasets cannot be stored in main memory
- Storing it on disk requires different approach to efficiency
- Assuming that a disk spins at 3600 RPM, one revolution occurs in 1/60 of a second, or 16.7ms
- Crudely speaking, one disk access takes about the same time as 200,000 instructions.
- Assume that we use an AVL tree to store about 20 million records
- We end up with a **very** deep binary tree with lots of different disk accesses; $\log_2$ 20,000,000 is about 24, so this takes about 0.2 seconds
- We know we can't improve on the **log*n*** lower bound on search for a binary tree
- But, the solution is to use more branches and thus reduce the height of the tree!
  - As branching increases, depth decreases

Advance Tree (B-Tree)

Introduction of B-Trees

- A B-tree index is a self-balancing tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in **logarithmic time.**
- The B-tree index structure is widely used in databases and file systems.
- B-Tree indexing is widely used in relational databases such as MySQL and PostgreSQL.
- A B-tree of order *m* is an ***m*-way** tree (i.e., a tree where each node may have up to *m* children) in which:
  1. Every node can have at most m−1 keys and m children.
  2. All leaves are on the same level
  3. All non-leaf nodes except the root have at least $\lceil m/2 \rceil$ children
  4. All non-leaf nodes except the root have at least $\lceil m/2 \rceil - 1$ keys
  5. The root is either a leaf node, or it has from **two** to ***m*** children [min 1 key]
  6. A leaf node contains no more than $m - 1$ keys.
  7. Keys within a node are sorted.
- Time Complexity of B-Tree: **O(logn)** for search, insert and delete

Advance Tree (B-Tree)

Inserting into a B-Tree

- Attempt to insert the new key into a leaf
- If this would result in that leaf becoming too big, split the leaf into two, promoting the middle key to the leaf's parent
- If this would result in the parent becoming too big, split the parent into two, promoting the middle key
- This strategy might have to be repeated all the way to the top
- If necessary, the root is split in two and the middle key is promoted to a new root, making the tree one level higher

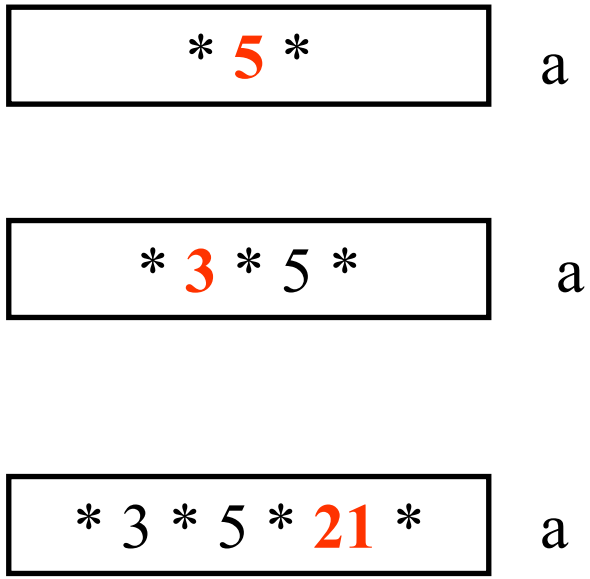
Advance Tree (B-Tree)

Insertion Operations

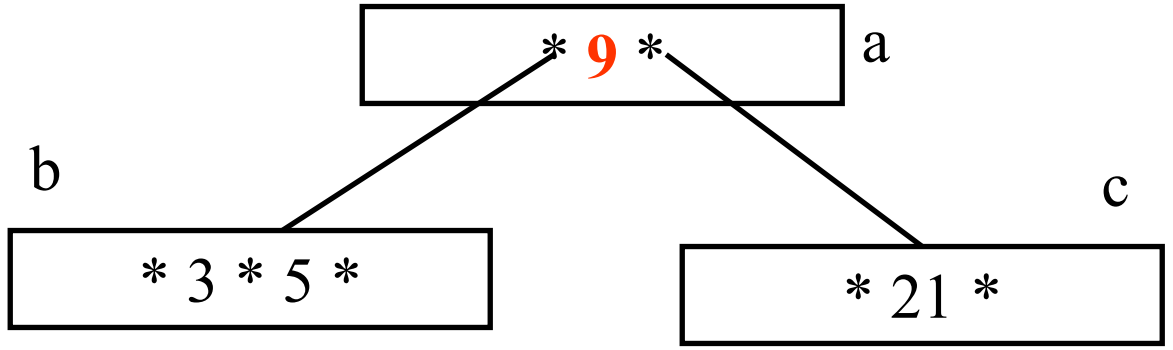
- B-Tree of order 4
  - Each node has at most 4 pointers and 3 keys, and at least 2 pointers and 1 key.
- Insert: 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8
- Delete: 2, 21, 10, 3, 4

Advance Tree (B-Tree)

Insert 5, 3, 21



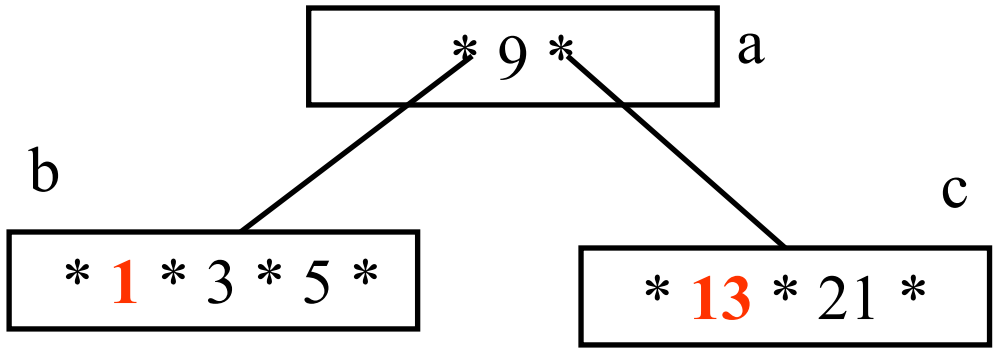
Insert 9



Node a splits creating 2 children: b and c

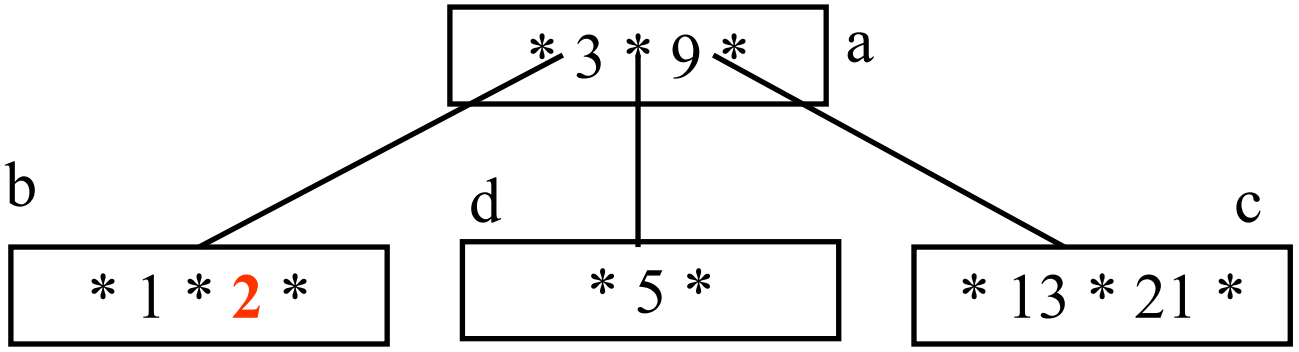
Advance Tree (B-Tree)

Insert 1, 13



Nodes b and c have room to insert more elements

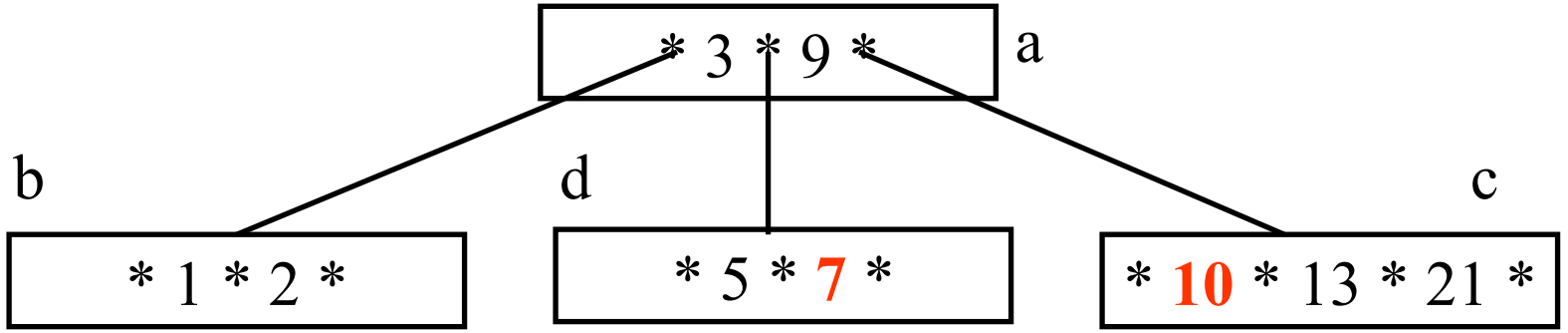
Insert 2



Node b has no more room, so it splits creating node d.

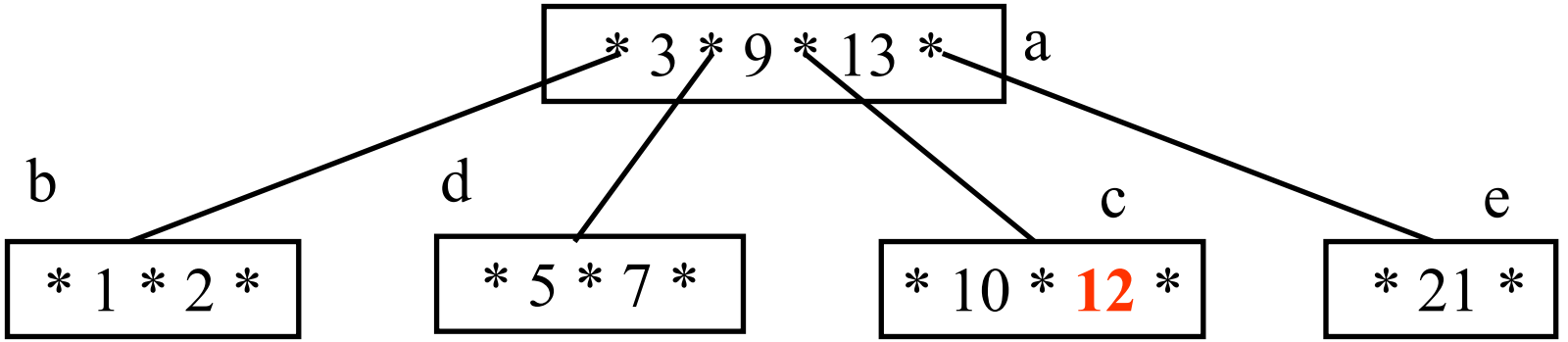
Advance Tree (B-Tree)

Insert 7, 10



Nodes d and c have room to add more elements

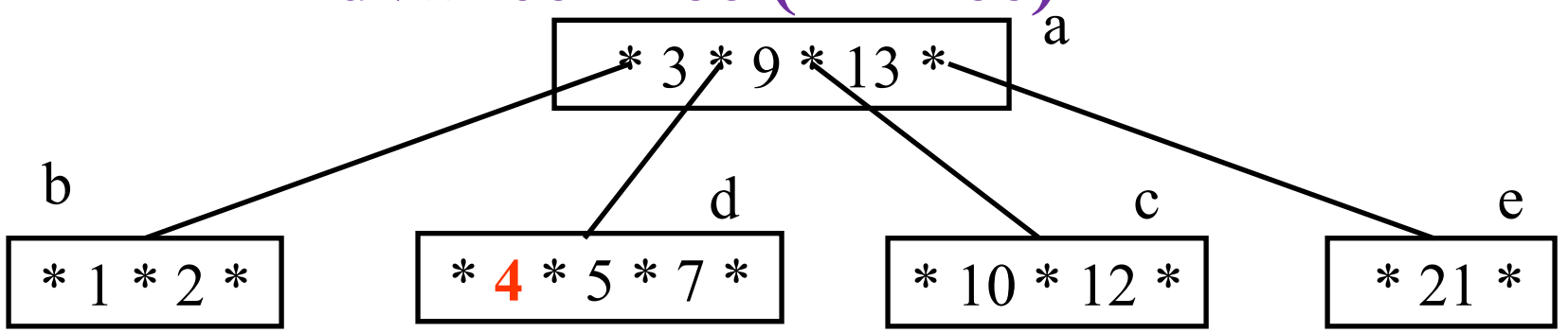
Insert 12



Nodes c must split into nodes c and e

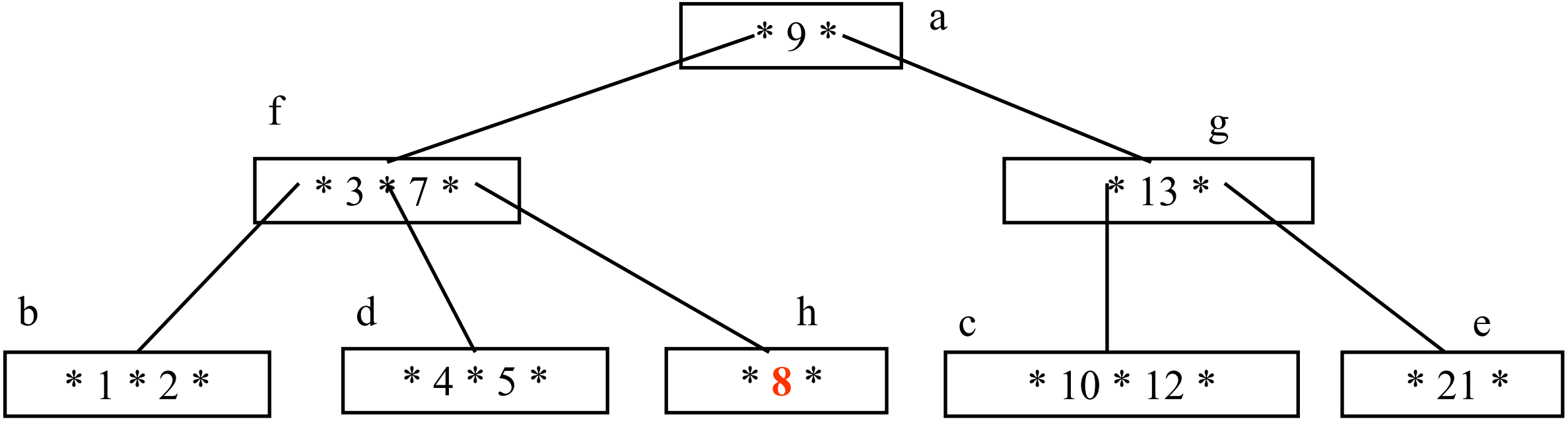
Advance Tree (B-Tree)

Insert 4



Node d has room for another element

Insert 8



Node d must split into 2 nodes. This causes node a to split into 2 nodes and the tree grows a level.

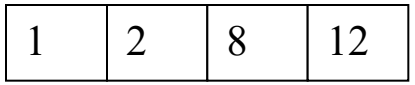
Advance Tree (B-Tree)

Try yourself

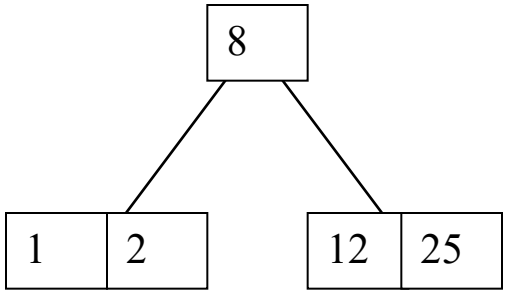
Construct a B-tree of order 5 from the following keys arrive in the :1 12 8 2 25 6 14 28 17 7 52 16 48 68 3 26 29 53 55 45

Advance Tree (B-Tree)

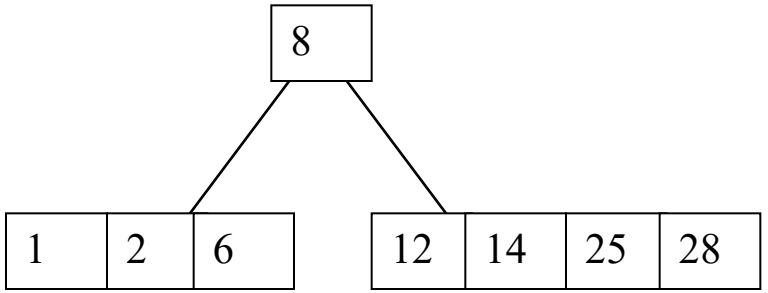
The first four items go into the root:



To put the fifth item in the root would violate condition 5
Therefore, when 25 arrives, pick the middle key to make a new root

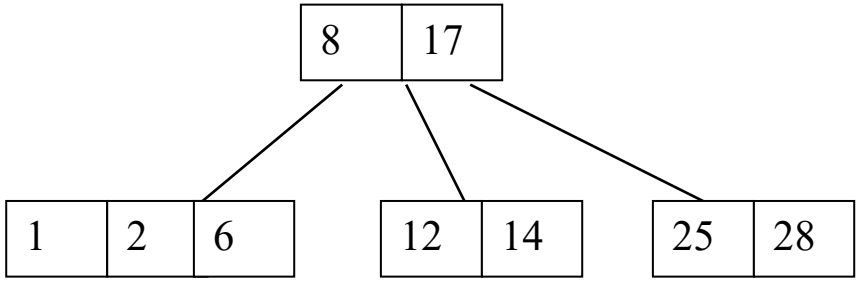


6, 14, 28 get added to the leaf nodes:

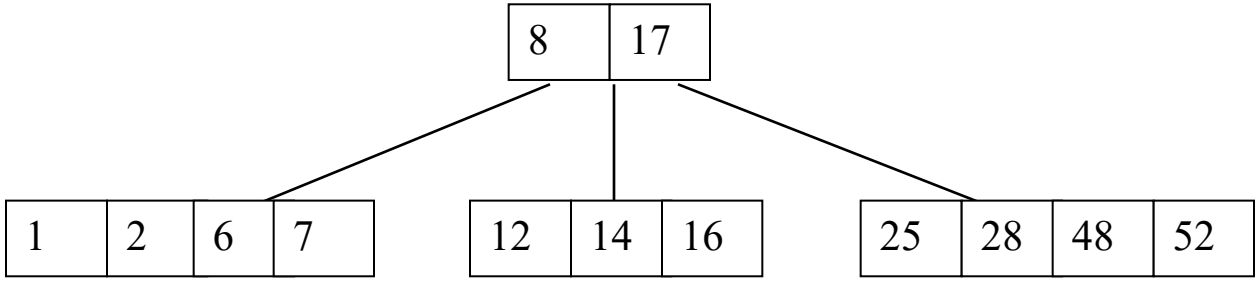


Advance Tree (B-Tree)

Adding 17 to the right leaf node would over-fill it, so we take the middle key, promote it (to the root) and split the leaf

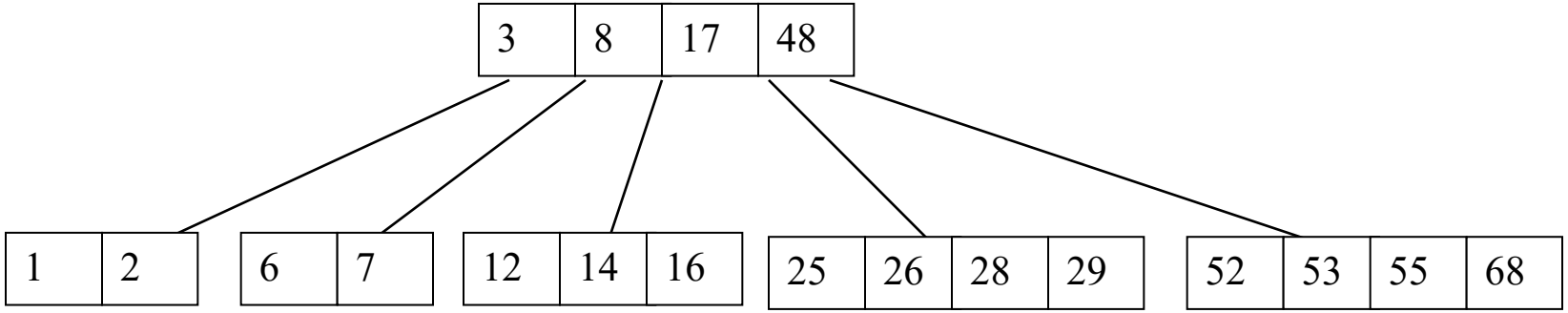


7, 52, 16, 48 get added to the leaf nodes

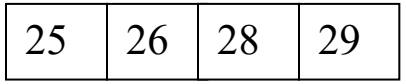


Advance Tree (B-Tree)

Adding 68 causes us to split the right most leaf, promoting 48 to the root, and adding 3 causes us to split the left most leaf, promoting 3 to the root; 26, 29, 53, 55 then go into the leaves



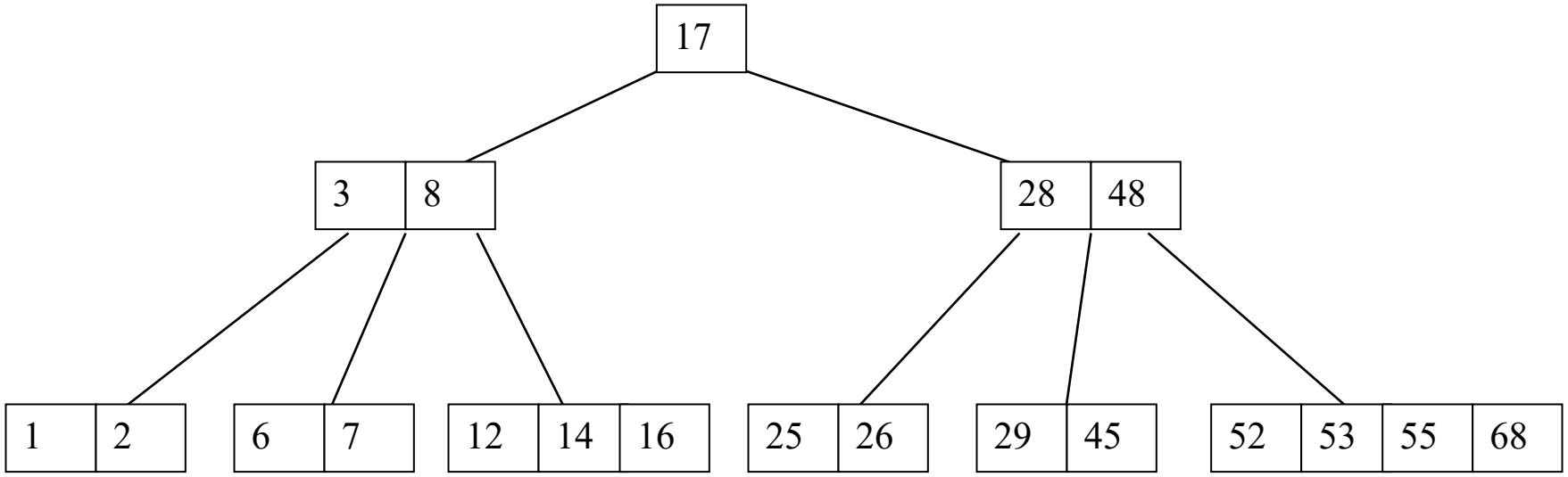
Adding 45 causes a split of



and promoting 28 to the root then causes the root to split

Advance Tree (B-Tree)

Final B-Tree of given try yourself



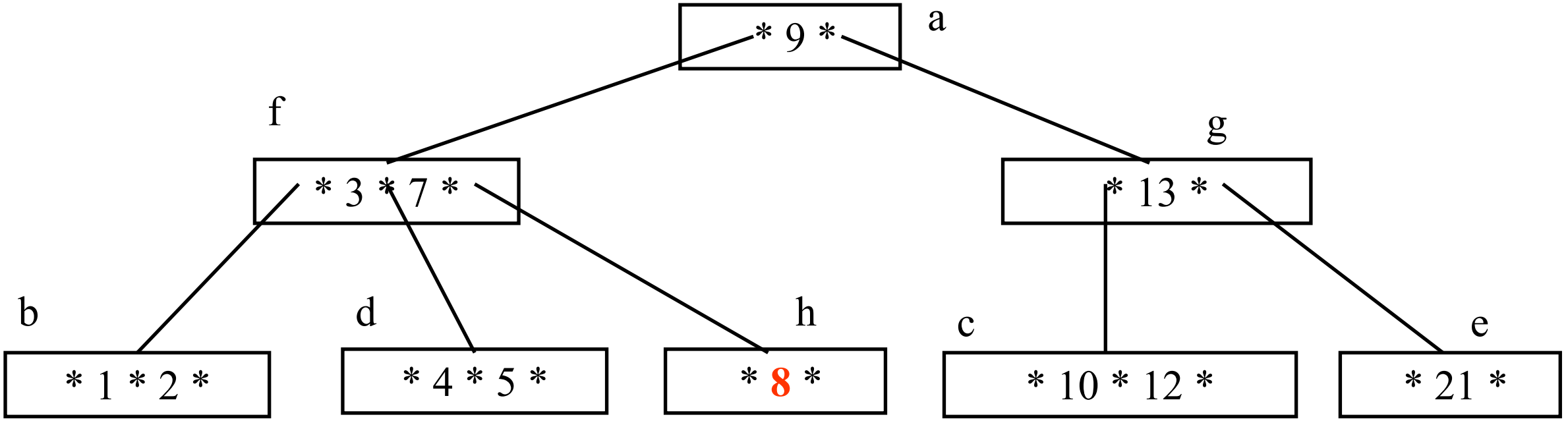
Advance Tree (B-Tree)

Removal from a B-tree (Delete Operation)

- During insertion, the key always goes *into* a *leaf*. For deletion we wish to remove *from* a leaf. There are three possible ways we can do this:
  1. If the key is already in a leaf node, and removing it doesn't cause that leaf node to have too few keys, then simply remove the key to be deleted.
  2. If the key is *not* in a leaf then it is guaranteed (by the nature of a B-tree) that its predecessor or successor will be in a leaf - in this case we can delete the key and promote the predecessor or successor key to the non-leaf deleted key's position.
- If (1) or (2) lead to a leaf node containing less than the minimum number of keys then we have to look at the siblings immediately adjacent to the leaf in question:
  3. If one of them has more than the min. number of keys then we can promote one of its keys to the parent and take the parent key into our lacking leaf
  4. If neither of them has more than the min. number of keys then the lacking leaf and one of its neighbours can be combined with their shared parent (the opposite of promoting a key) and the new leaf will have the correct number of keys; if this step leave the parent with too few keys then we repeat the process up to the root itself, if required

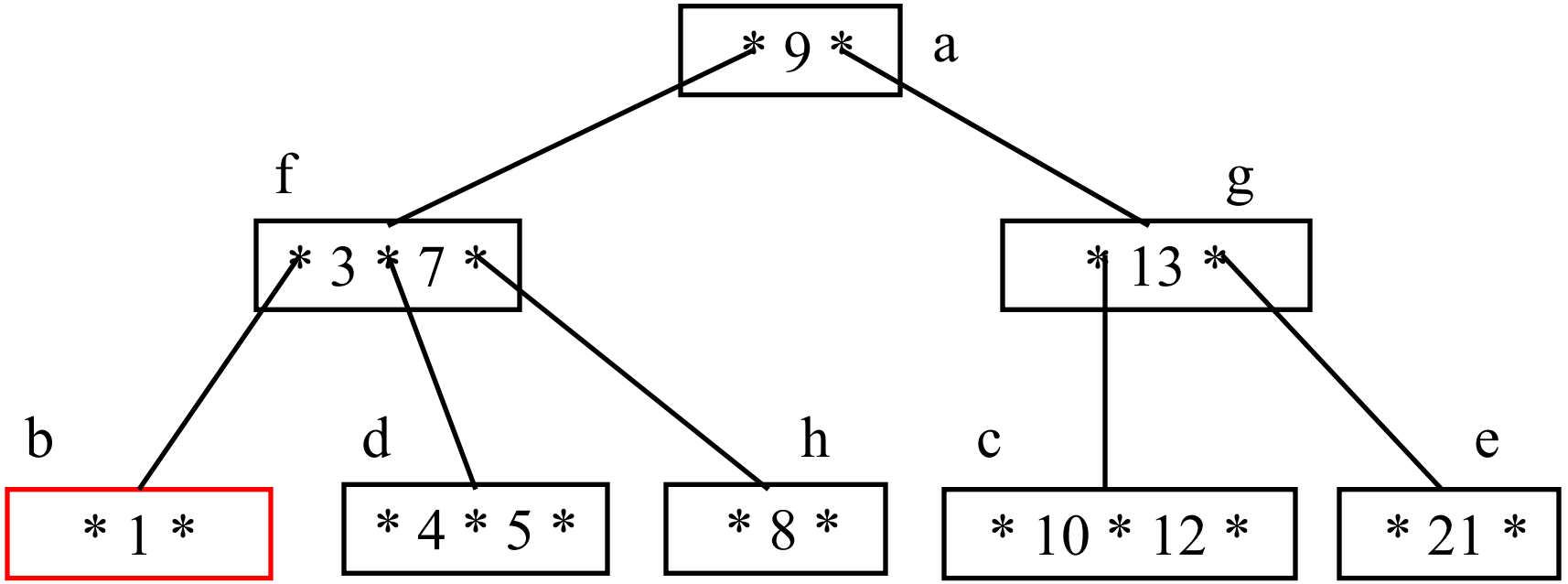
Advance Tree (B-Tree)

Delete: 2, 21, 10, 3, 4



B-Tree Deletion

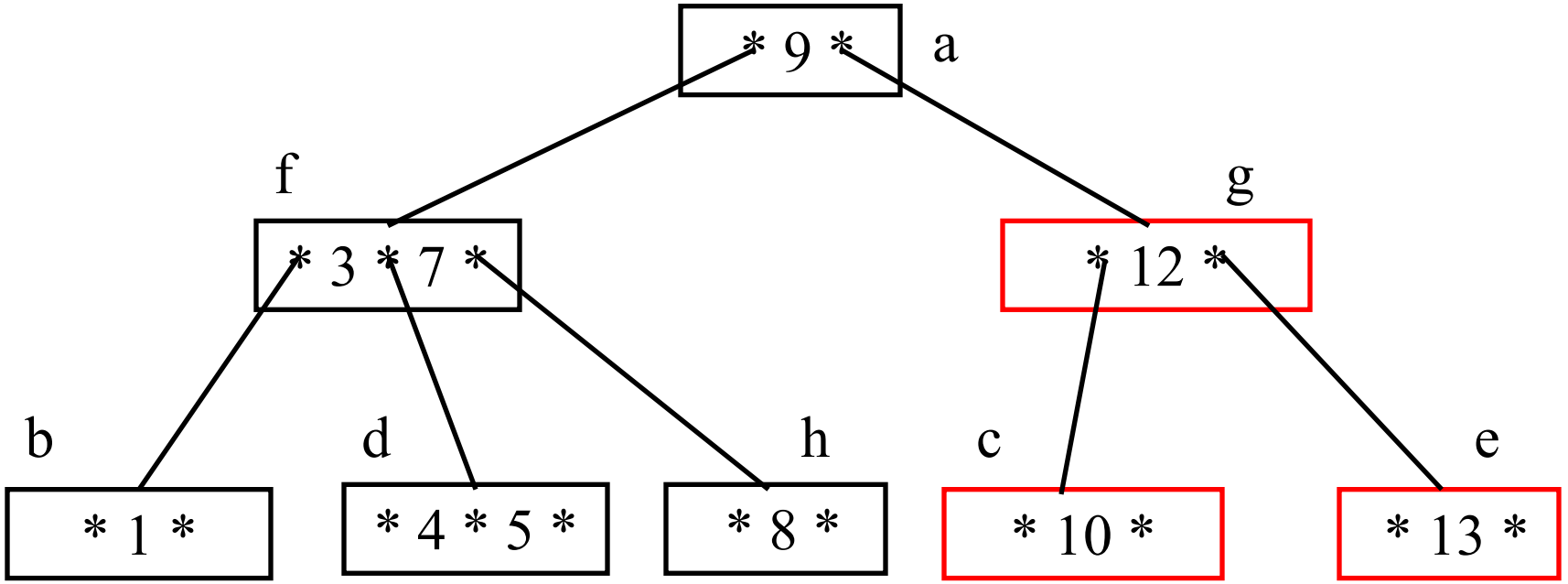
Delete 2



Node b can loose an element without underflow.

B-Tree Deletion

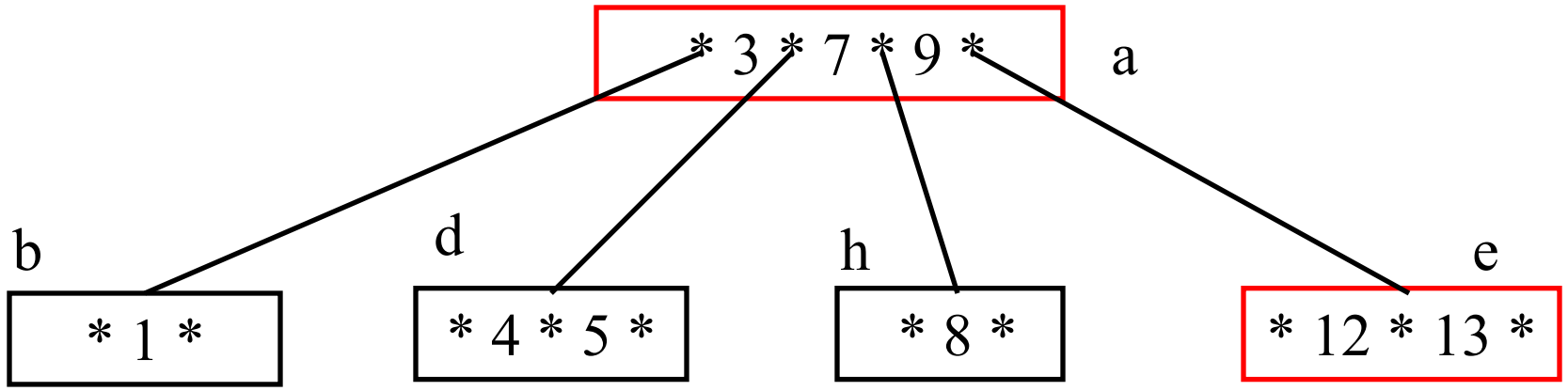
Delete 21



Deleting 21 causes node e to underflow, so elements are redistributed between nodes c, g, and e

B-Tree Deletion

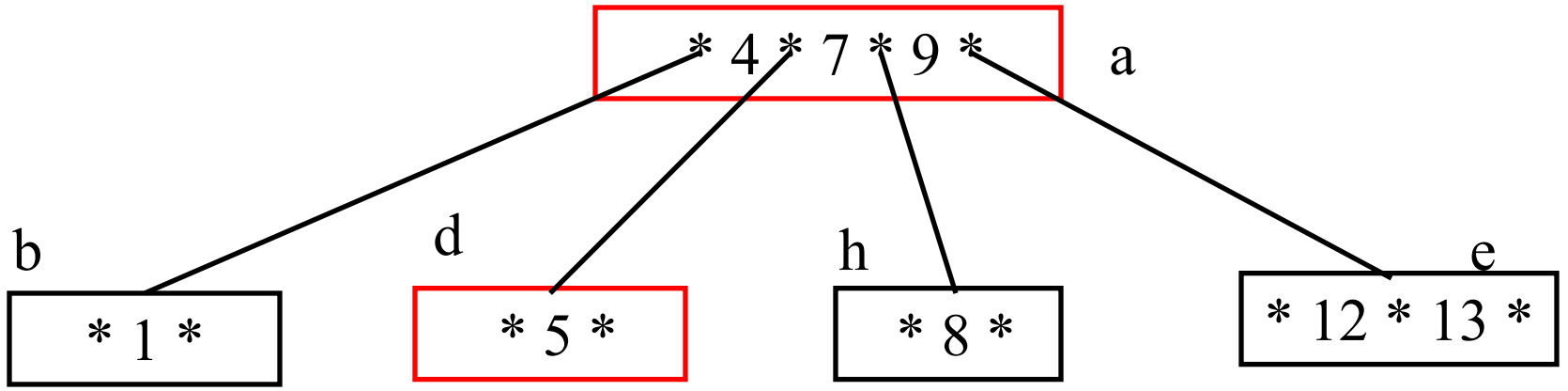
Delete 10



Deleting 10 causes node c to underflow. This causes the parent, node g to recombine with nodes f and a. This causes the tree to shrink one level.

B-Tree Deletion

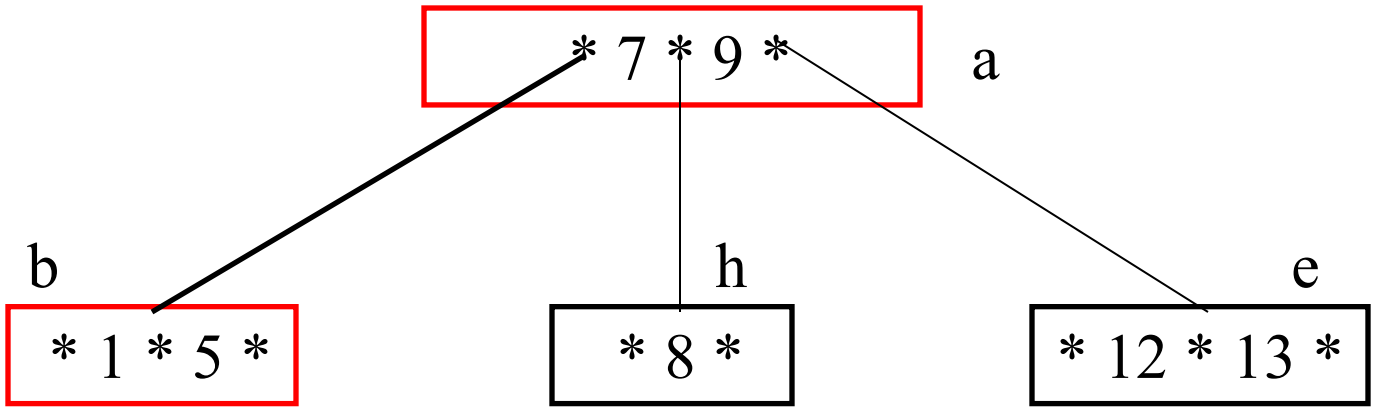
Delete 3



Because 3 is a pointer to nodes below it, deleting 3 requires keys to be redistributed between nodes a and d.

B-Tree Deletion

Delete 4



Deleting 4 requires a redistribution of the keys in the subtrees of 4; however, nodes b and d do not have enough keys to redistribute without causing an underflow. Thus, nodes b and d must be combined.

Applications of B-Tree

1. **Database Systems**
   - B-trees are commonly used to implement database indexing, allowing efficient access to large datasets. Used in systems like MySQL, PostgreSQL, and Oracle for indexing primary and secondary keys.
2. **File Systems**
   - Used to manage file systems where quick access to metadata (e.g., file locations) is critical.
   - Examples include NTFS (Windows) and HFS+ (macOS).
3. **Operating Systems**
   - For managing paging and virtual memory, where fast lookup of page tables is required.
   - Helps in organizing data structures used by kernels.
4. **Multilevel Indexing**
   - B-trees support multi-level indexing, making them ideal for hierarchical data storage, such as directory structures.
5. **Data Warehousing and Analytics**
   - Useful for indexing large datasets, especially for OLAP (Online Analytical Processing) systems.
6. **Search Engines**
   - To index and search massive amounts of data efficiently.
7. **Geographic Information Systems (GIS)**
   - For managing spatial data where multidimensional range queries are performed.
8. **Networking**
   - Applied in routing tables for fast lookup and update of IP routing entries.

B+ Tree

- B+ tree is an extension of the B tree.
- In B tree the keys and records can be stored as internal as well as **leaf nodes** whereas in B+ trees, the records are stored **as leaf nodes** and the **keys are stored only in internal nodes**.
- The leaf nodes are also linearly connected in B+ trees to improve range-query performance.
- B+-tree indices are an alternative to indexed-sequential files.
- Disadvantage of indexed-sequential files
  - performance degrades as file grows, since many overflow blocks get created.
  - Periodic reorganization of entire file is required.

Advantage of B+-tree index files:

- automatically reorganizes itself with small, local, changes, in the face of insertions and deletions.
- Reorganization of entire file is not required to maintain performance.
- (Minor) disadvantage of B+-trees: extra insertion and deletion overhead, space overhead.

B+ Tree Index Files

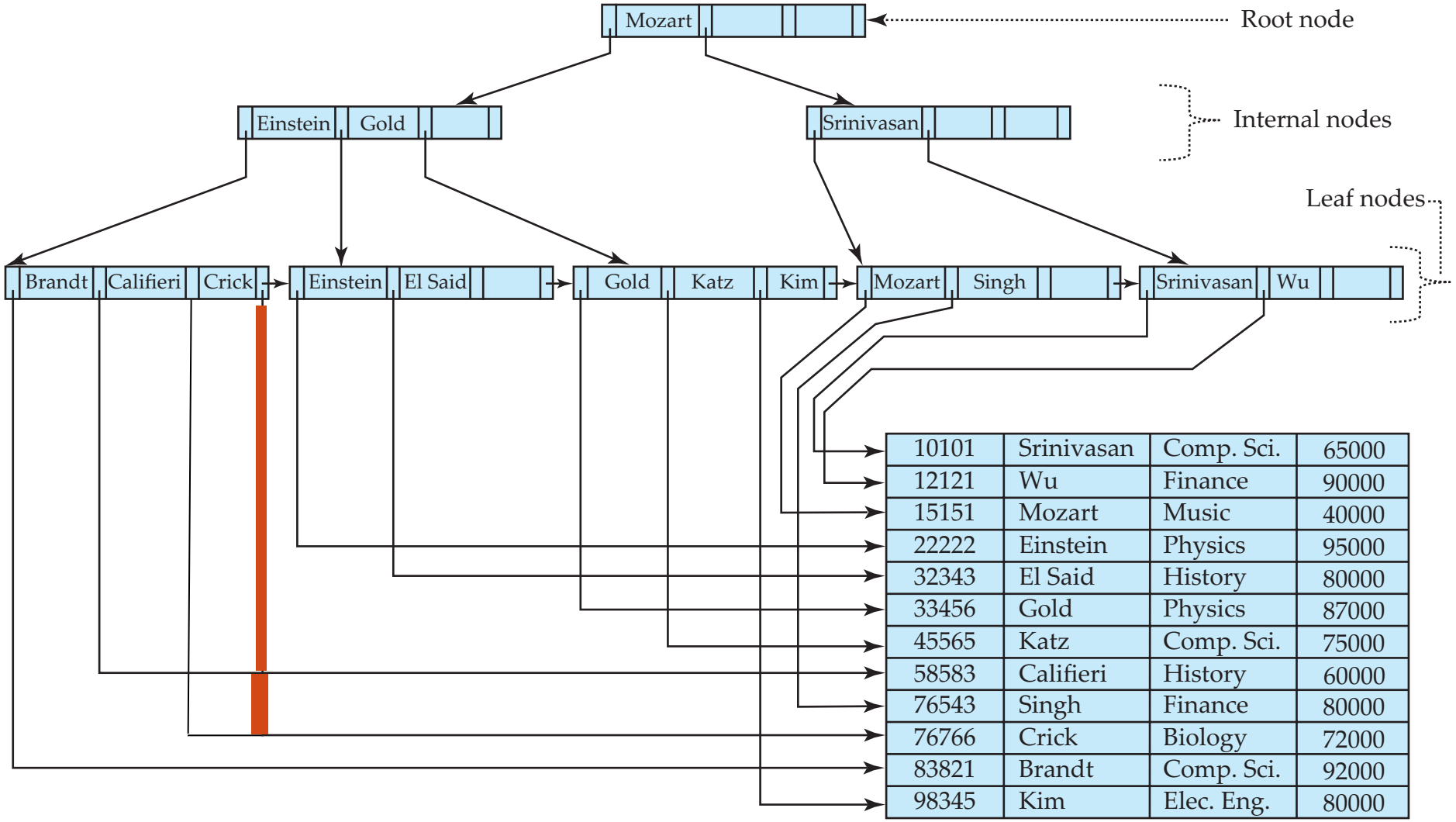
Typical node



- K_i are the search-key values
- P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).

- The search-keys in a node are ordered
  - $K_1 < K_2 < K_3 < \ldots < K_{n-1}$
- Usually the size of a node is that of a block
- **A B+ tree is a rooted tree satisfying the following properties:**
  - All paths from root to leaf are of the same length
  - Each node that is not a root or a leaf has between $\lceil m/2 \rceil$ and n children.
  - A leaf node has between $\lceil (m)/2 \rceil$**-1** and m–1 keys
  - Special cases:
    - If the root is not a leaf, it has at least 2 children.
    - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and (m-1) keys.
  - All values must be at leaf node.

Example of B+-Tree

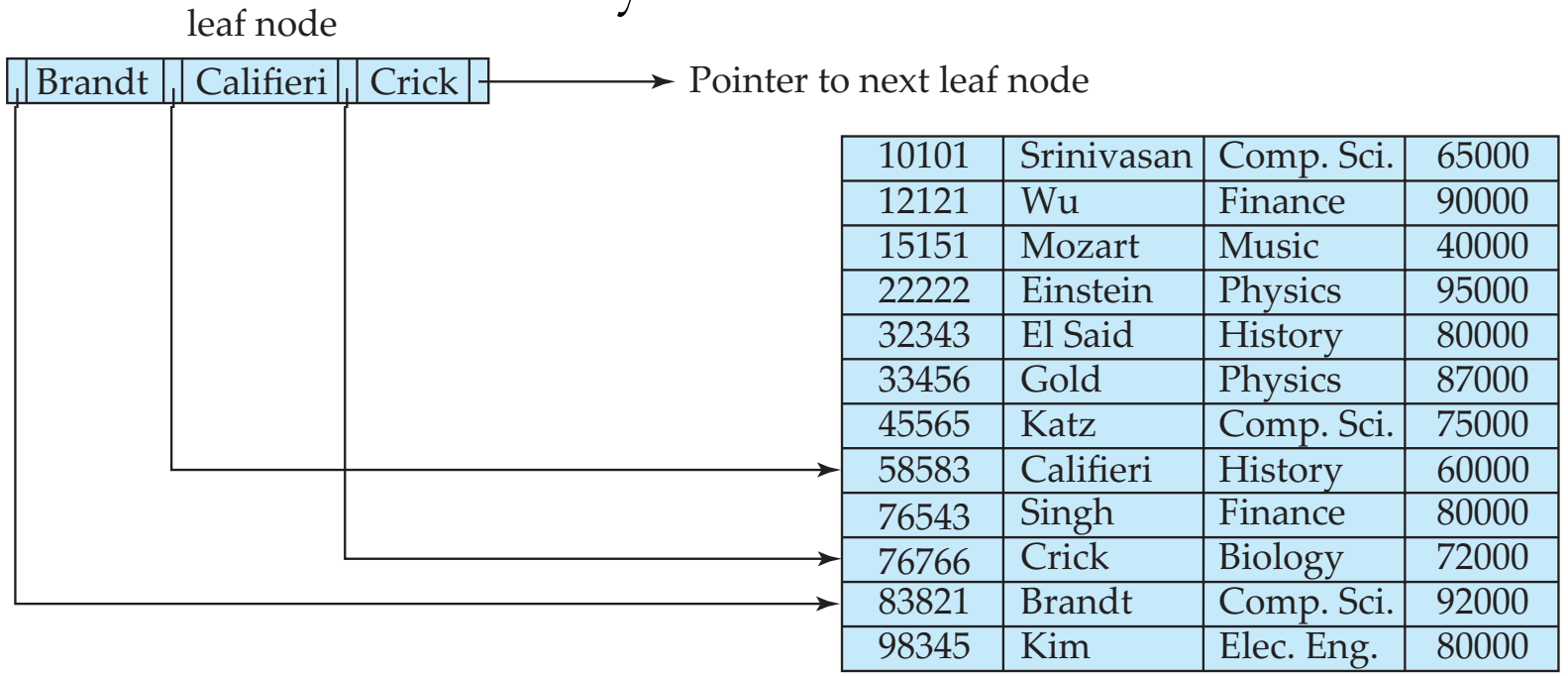


10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Leaf Nodes in B+-Trees

Properties of a leaf node:

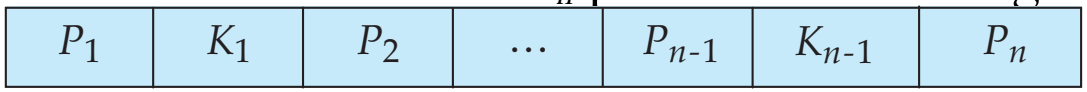
- For $i = 1, 2, \ldots, n-1$, pointer P_i points to a file record with search-key value K_i,
- If L_i, L_j are leaf nodes and $i < j$, L_i's search-key values are less than or equal to L_j's search-key values
- P_n points to next leaf node in search-key order



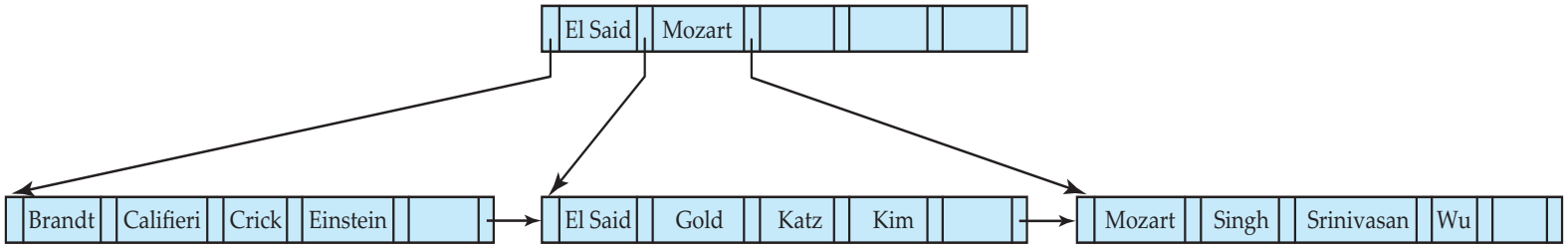
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Non-Leaf Nodes in B+-Trees

- Non leaf nodes form a multi-level sparse index on the leaf nodes. For a non-leaf node with m pointers:
  - All the search-keys in the subtree to which P_1 points are less than K_1
  - For $2 \leq i \leq m-1$, all the search-keys in the subtree to which P_i points have values greater than or equal to K_{i-1} and less than K_i
  - All the search-keys in the subtree to which P_n points have values greater than or equal to K_{n-1}
  - General structure



- Example of B$^+$-tree ($m = 6$)



- Leaf nodes must have between 3 and 5 values ($\lceil (m-1)/2 \rceil$ and $m-1$, with $m = 6$).
- Non-leaf nodes other than root must have between 3 and 6 children ($\lceil m/2 \rceil$ and m with $m = 6$). [*for non leaf node: min no of key* $= \lceil m/2 \rceil - 1$]
- Root must have at least 2 children.

Observations about B+-trees

- Since the inter-node connections are done by pointers, "logically" close blocks need not be "physically" close.
- The non-leaf levels of the B^+-tree form a hierarchy of sparse indices.
- The B^+-tree contains a relatively small number of levels
  - Level below root has at least $2*\lceil m/2 \rceil$ values
  - Next level has at least $2*\lceil m/2 \rceil * \lceil m/2 \rceil$ values
  - .. etc.
  - If there are K search-key values in the file, the tree height is no more than $\lceil \log_{\lceil m/2 \rceil}(K) \rceil$
  - thus searches can be conducted efficiently.
- Insertions and deletions to the main file can be handled efficiently, as the index can be restructured in logarithmic time.

Updates on B+-Trees: Insertion

1. Every element is inserted into a leaf node. So, go to the appropriate leaf node.
2. Insert the key into the leaf node in increasing order only if there is no overflow. If there is an overflow follow the steps to maintain the B+ Tree properties.

Properties for insertion B+ Tree

- **Case 1: Overflow in leaf node**
  1. Split the leaf node into two nodes.
  2. First node contains **ceil((m-1)/2)** values.
  3. Second node contains the remaining values.
  4. Copy the smallest search key value from second node to the parent node.(Right biased)
- **Case 2: Overflow in non-leaf node**
  1. Split the non leaf node into two nodes.
  2. First node contains **ceil(m/2)-1** values.
  3. Move the smallest among remaining to the parent.
  4. Second node contains the remaining keys.

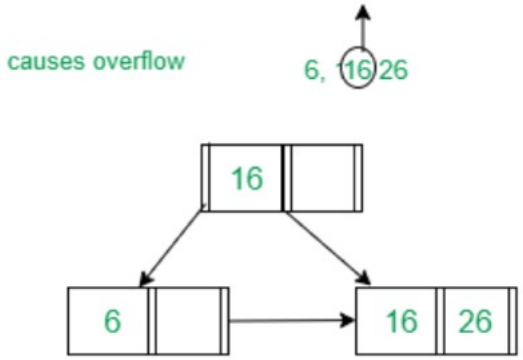
Updates on B+-Trees: Insertion

- Insert the following key values 6, 16, 26, 36, 46 on a B+ tree with order = 3.
- Here Key = m-1 = 3-1 =2 and Maximum no of childs = 3 (order is 3)

Insert 6, 16

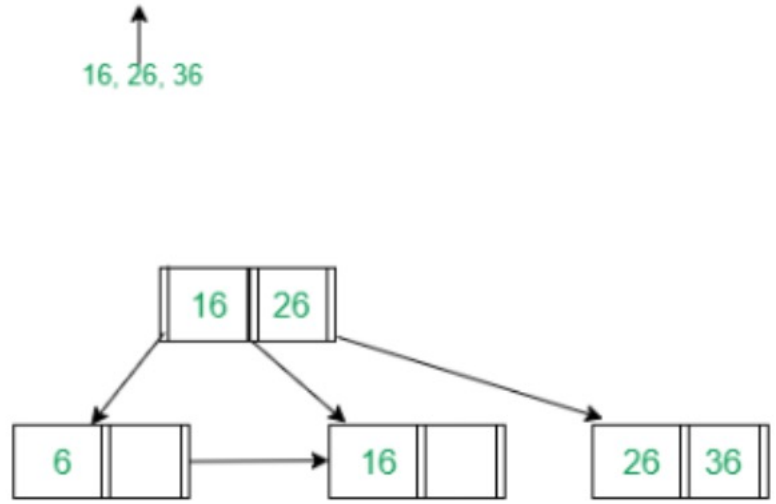


Insert 26



Inserting **26** cause overflow in the leaf node, so split it

Insert 36



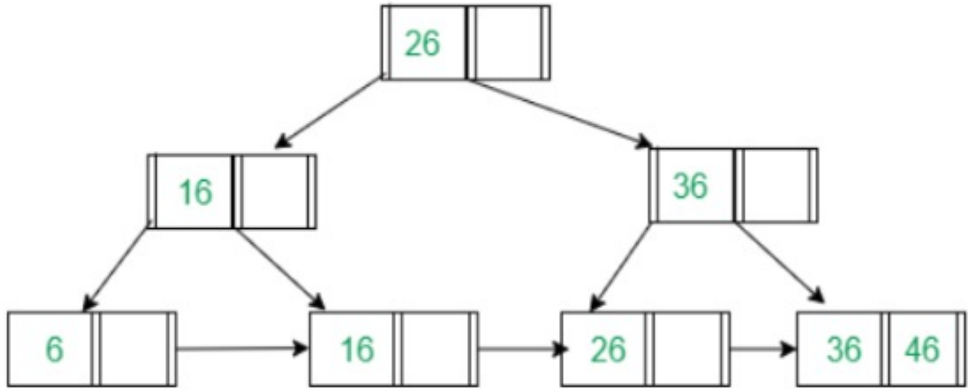
Inserting **36** also cause overflow in the leaf node, so split it

Updates on B+-Trees: Insertion

Insert 46

26, 36, 46 in leaf node

16, 26, 36 in root node

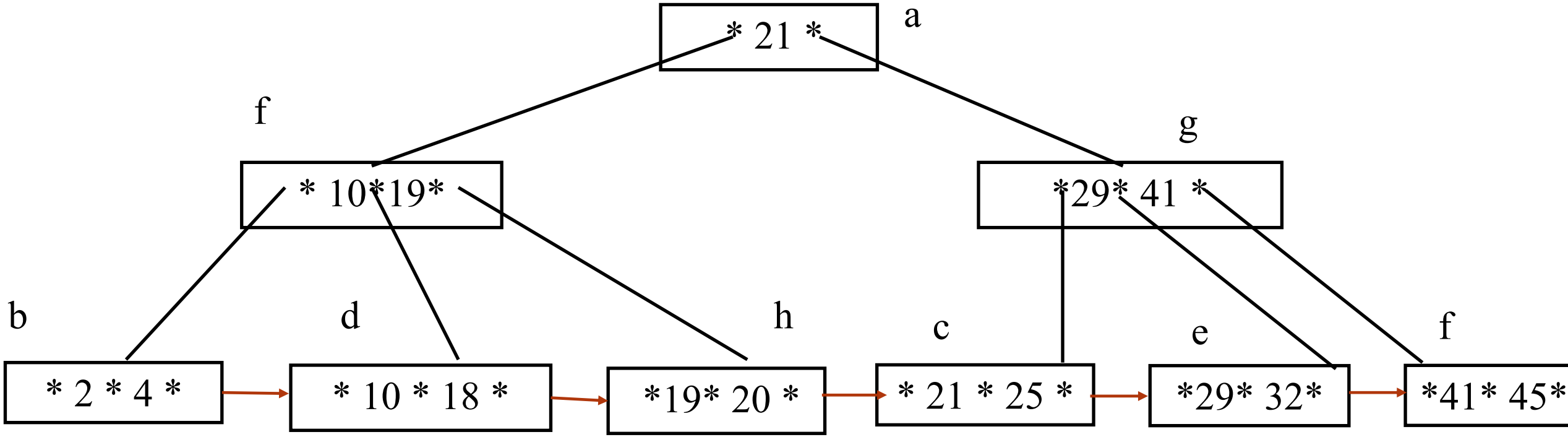


Inserting **46 again cause an overflow** in the leaf node. So, Split it and move the second part into its parents node which again cause an overflow in non leaf node. Apply the rule of splitting in non leaf node and maintain B+ tree property.

Try yourself

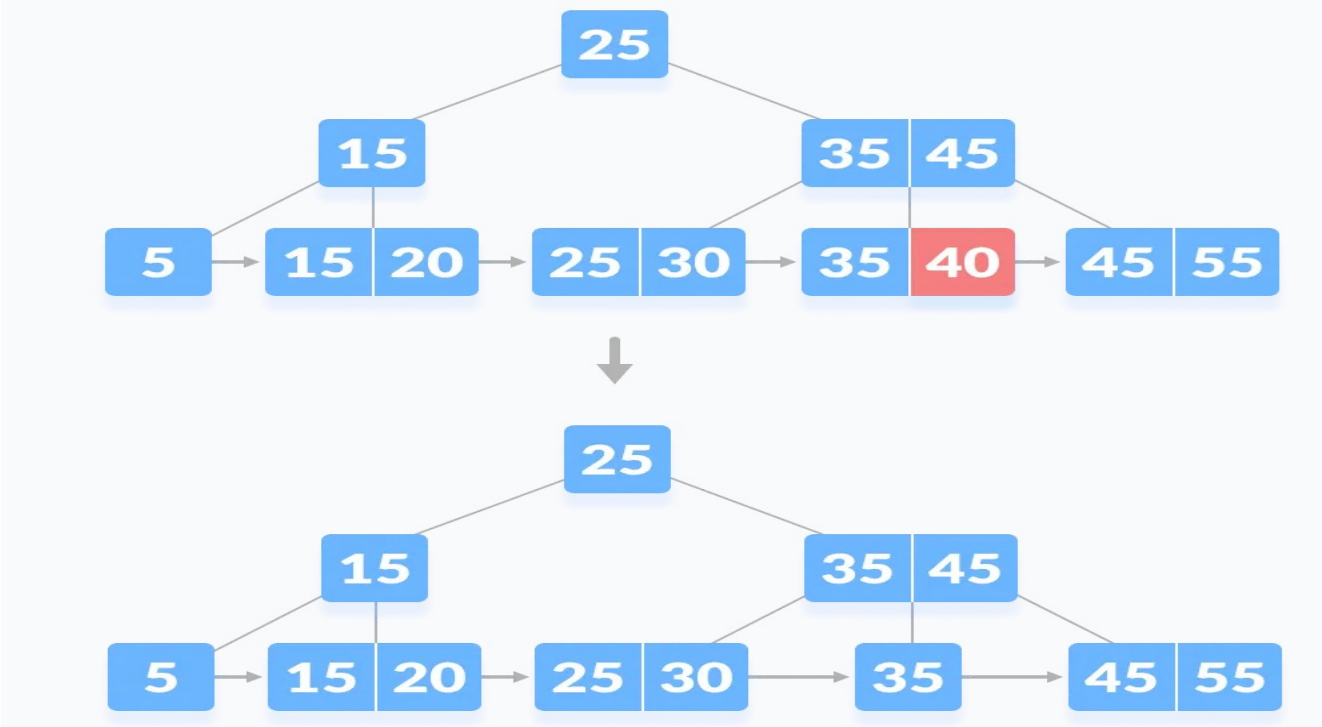
Create a B+ tree of order 4 for following numbers. 2, 4, 10, 18, 21, 32, 25, 19, 20, 29, 41,45

Step 1: Sort the data: 2,4,10,18,19,20,21,25,29,32,41,45



Updates on B+-Trees: Deletion

- Delete a key from the leaf node and maintain B+ tree properties:
- **Case I :** The key to be deleted is present only at the leaf node not in the indexes (or internal nodes).
- There are two cases for it:
  1. There is more than the minimum number of keys in the node. Simply delete the key.

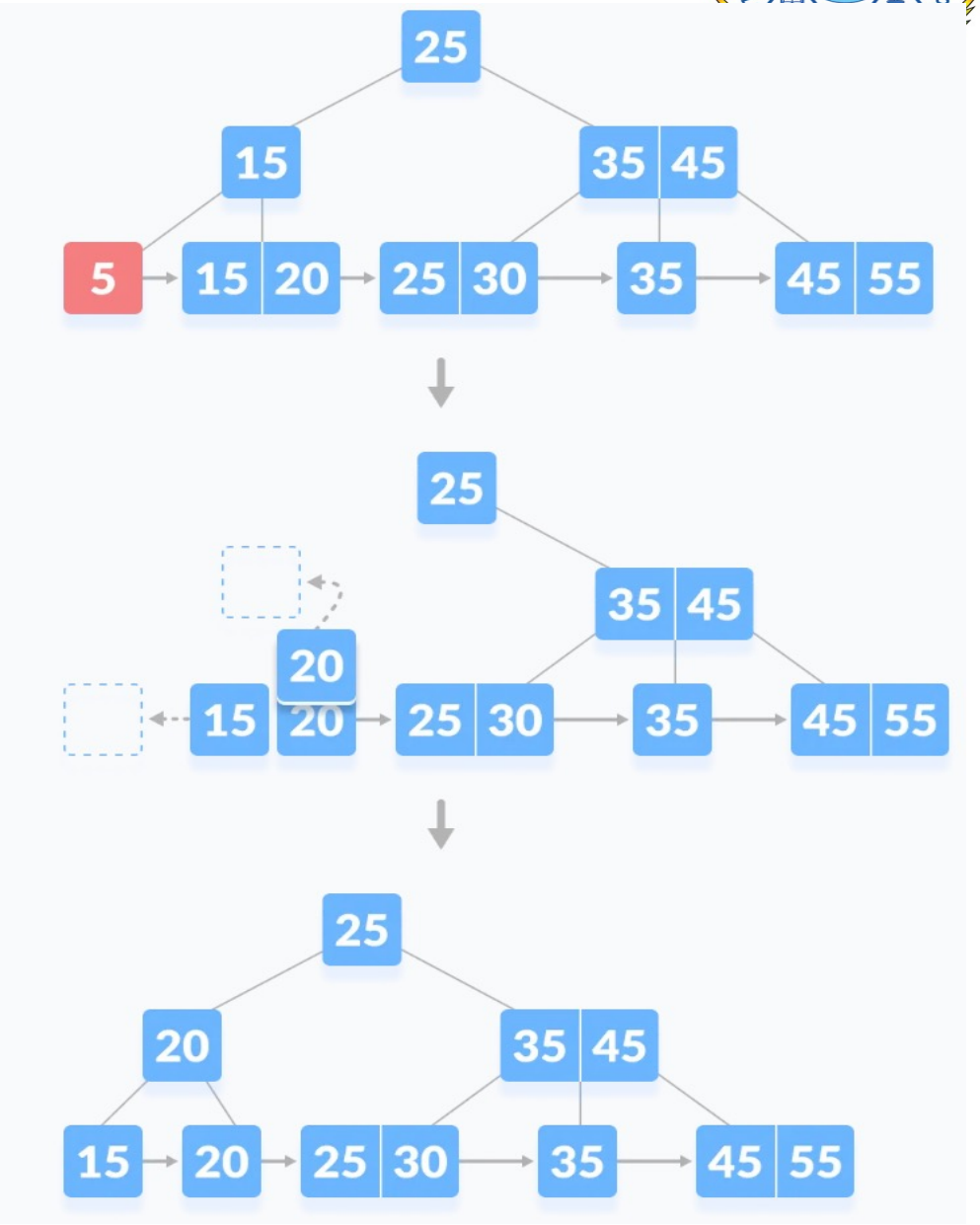


Deleting 40 from B-tree

Updates on B+-Trees: Deletion

2. There is an exact minimum number of keys in the node. Delete the key and borrow a key from the immediate sibling. Add the median key of the sibling node to the parent.

Delete 5 from B+ tree



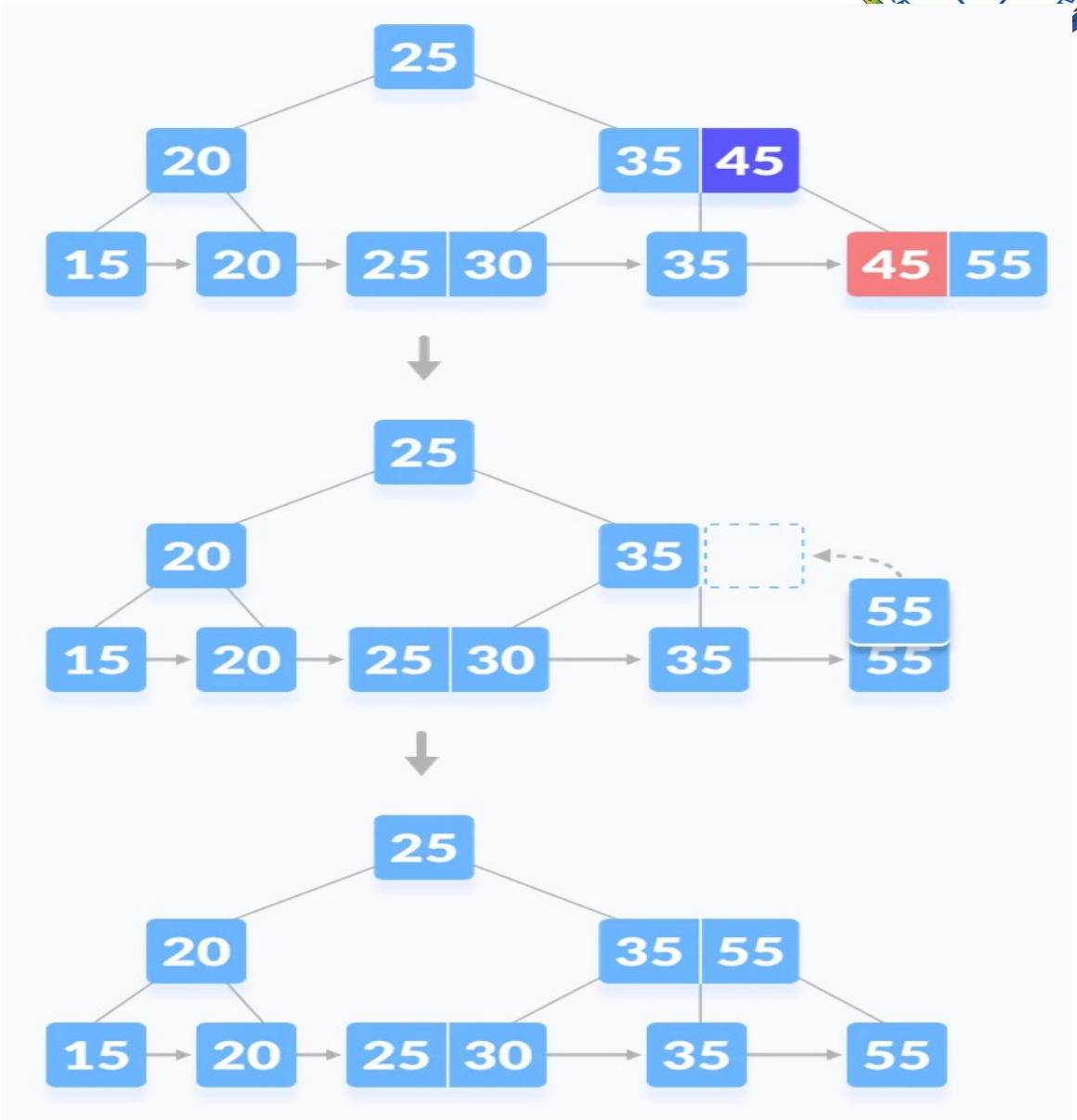
Updates on B+-Trees: Deletion

Case II

- The key to be deleted is present in the internal nodes as well. Then we have to remove them from the internal nodes as well.

1. If there is more than the minimum number of keys in the node, simply delete the key from the leaf node and delete the key from the internal node as well.
   Fill the empty space in the internal node with the inorder successor.

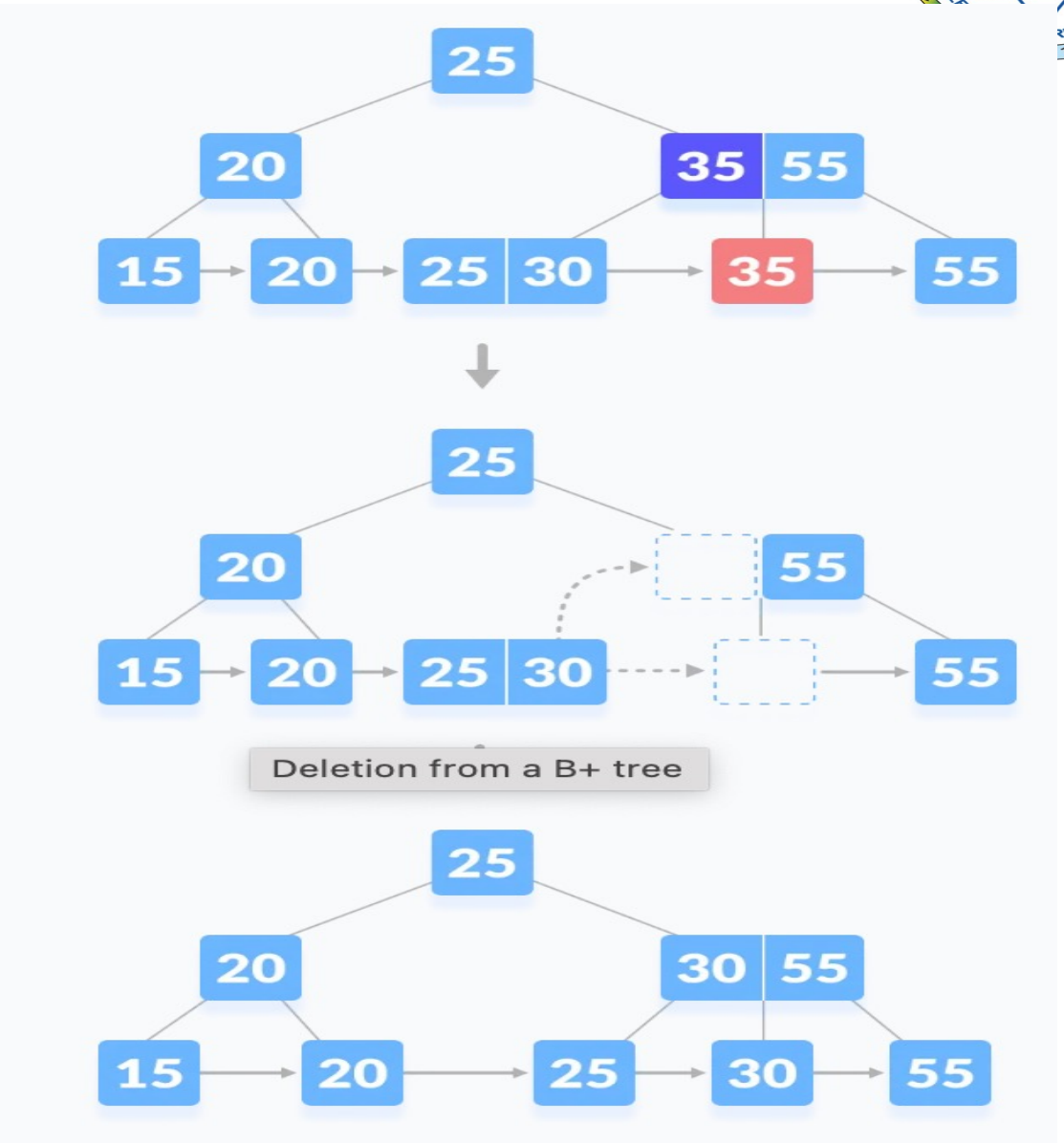
- Delete 45 from B+ Tree



Updates on B+-Trees: Deletion

2. If there is an exact minimum number of keys in the node, then delete the key and borrow a key from its immediate sibling (through the parent).
Fill the empty space created in the index (internal node) with the borrowed key.

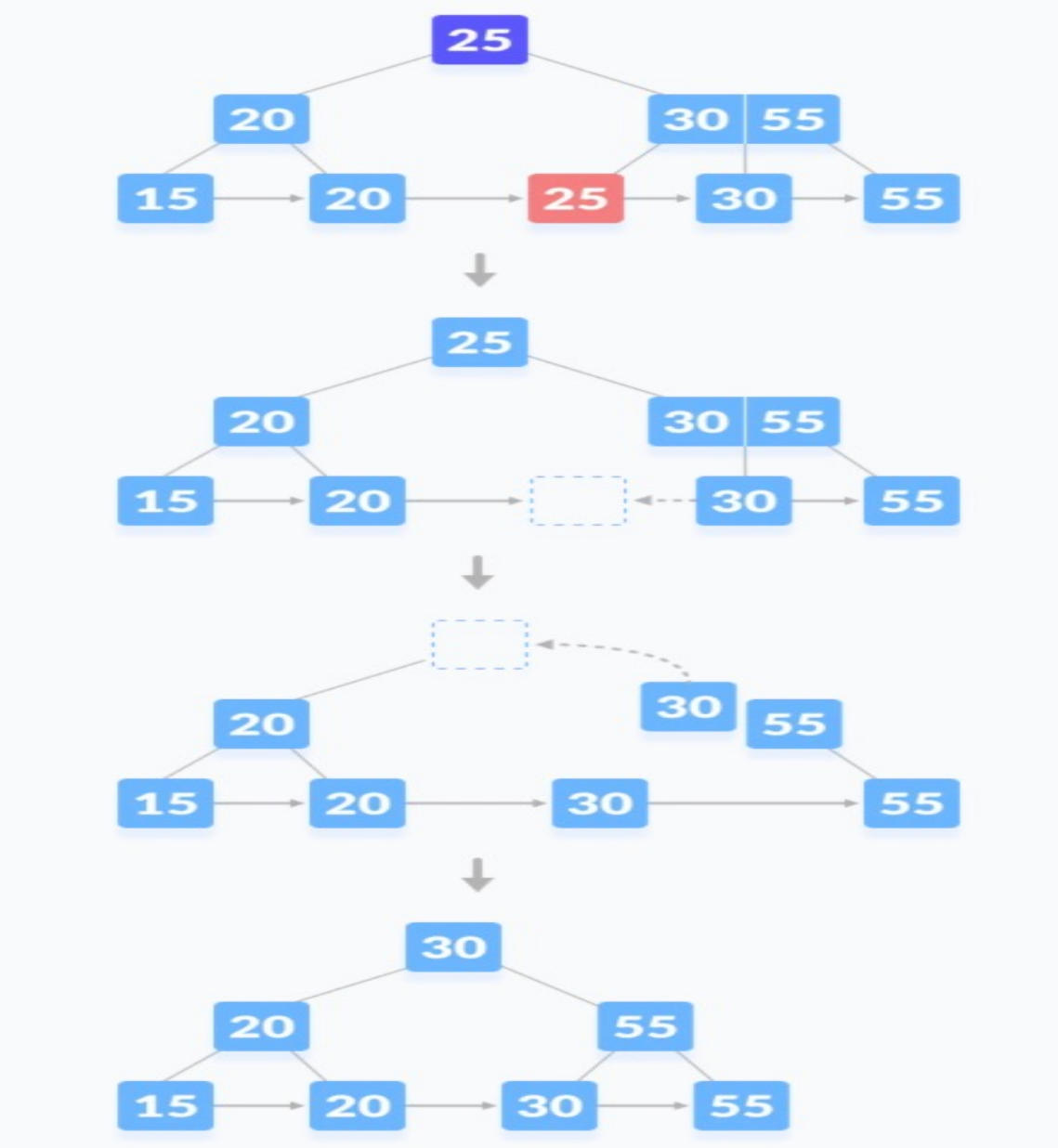
Delete 35 from B+ tree



Updates on B+-Trees: Deletion

3. This case is similar to Case II(1) but here, empty space is generated above the immediate parent node.
After deleting the key, merge the empty space with its sibling.
Fill the empty space in the grandparent node with the inorder successor.

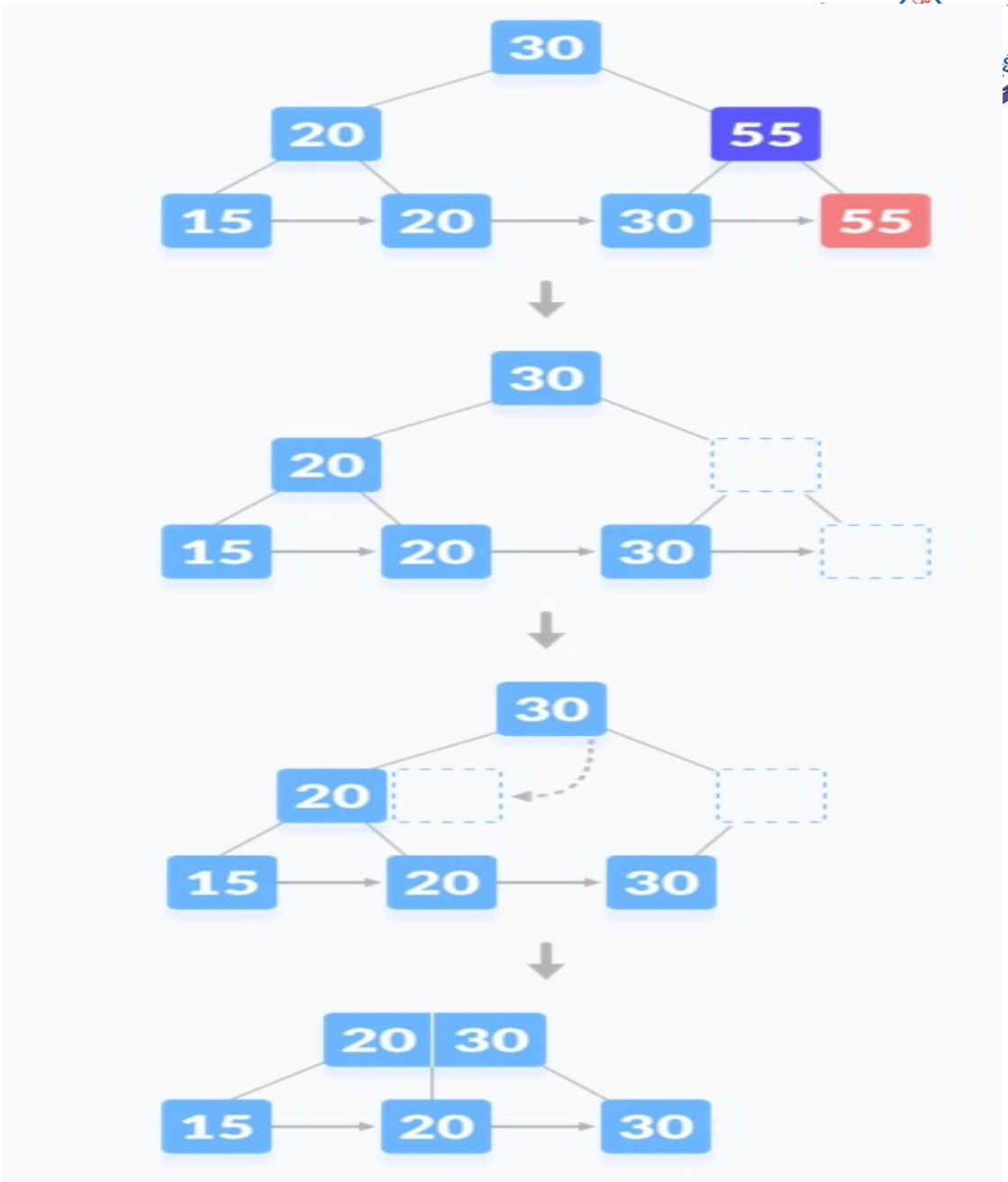
Delete 25 from B+ tree



Updates on B+-Trees: Deletion

Case III: In this case, the height of the tree gets shrinked. It is a little complicated. Deleting 55 from the tree below leads to this condition.

Delete 55 from B+ tree



Complexity of Updates

- Cost (in terms of number of I/O operations) of insertion and deletion of a single entry proportional to height of the tree
  - With K entries and maximum fanout(**maximum number of child pointers**) of m, worst case complexity of insert/delete of an entry is $\mathbf{O(\log_{\lceil m/2 \rceil}(K))}$
- In practice, number of I/O operations is less:
  - Internal nodes tend to be in buffer
  - Splits/merges are rare, most insert/delete operations only affect a leaf node
- Average node occupancy depends on insertion order
  - 2/3^{rds} with random, ½ with insertion in sorted order

Basis of Comparison	B tree	B+ tree
Pointers	All internal and leaf nodes have data pointers	Only leaf nodes have data pointers
Search	Since all keys are not available at leaf, search often takes more time.	All keys are at leaf nodes, hence search is faster and more accurate.
Redundant Keys	No duplicate of keys is maintained in the tree.	Duplicate of keys are maintained and all nodes are present at the leaf.
Insertion	Insertion takes more time and it is not predictable sometimes.	Insertion is easier and the results are always the same.
Deletion	Deletion of the internal node is very complex and the tree has to undergo a lot of transformations.	Deletion of any node is easy because all node are found at leaf.
Leaf Nodes	Leaf nodes are not stored as structural linked list.	Leaf nodes are stored as structural linked list.
Access	Sequential access to nodes is not possible	Sequential access is possible just like linked list
Height	For a particular number nodes height is larger	Height is lesser than B tree for the same number of nodes
Application	B-Trees used in Databases, Search engines	B+ Trees used in Multilevel Indexing, Database indexing

Complexity Comparison of B tree and B+ tree

Operation	B-Tree Complexity	B+ Tree Complexity
Search	$O(\log_m K)$	$O(\log_m K)$
Insertion	$O(\log_m K)$	$O(\log_m K)$
Deletion	$O(\log_m K)$	$O(\log_m K)$
Range Query	$O(\log_m K+t)$	$O(\log_m K+t)$
Height	$O(\log_m K)$	$O(\log_m K)$

K: Number of keys in the tree.

m: Fanout (maximum number of children per node).

t: Number of keys returned in a range query.

When to Use B-tree vs B+ tree

Use B-tree:

1. When **space utilization** is critical (less redundancy).
2. For smaller datasets where range queries are infrequent.
3. When you need to minimize pointer overhead since data can exist in internal nodes.

Use B+ tree:

1. For **databases** and **file systems** where range queries and sequential access are common.
2. When disk I/O optimization is crucial due to linked leaf nodes.
3. For very large datasets, as search time and space are optimized.

Outline

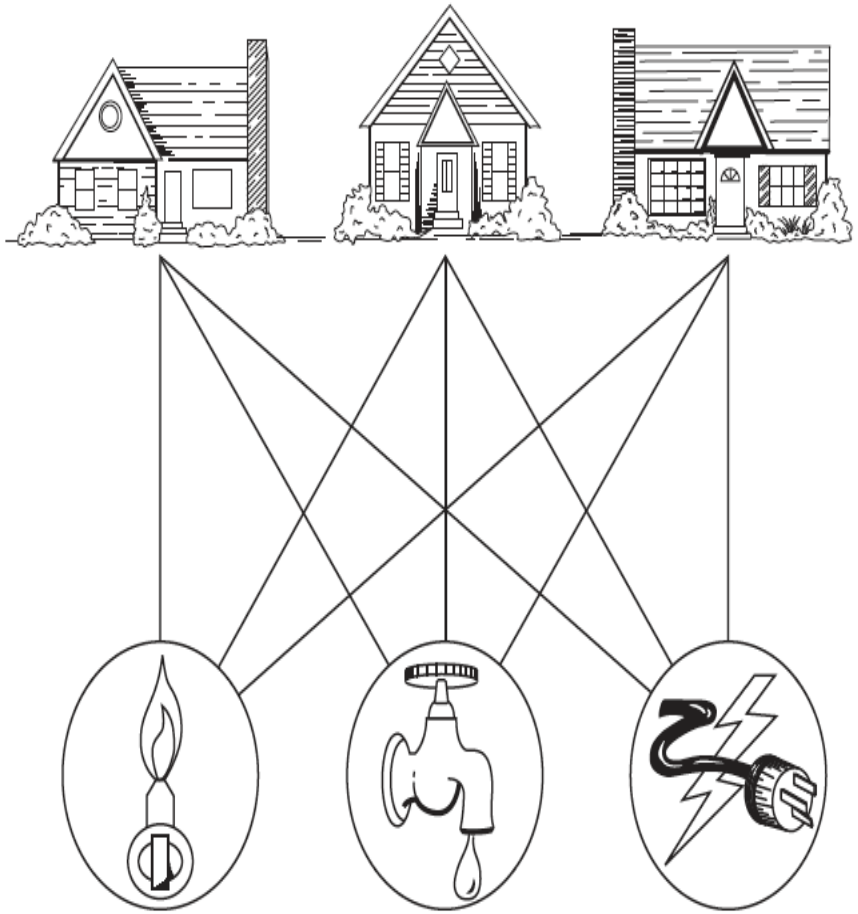
1. Advanced Trees
   - B and B+ Trees
2. Advanced Graph
   - Planar Graphs and its applications in Geographic and Circuit Layout Design
3. Network Flow Algorithms
   - Ford-Fulkerson Algorithm
   - Edmonds-Karp Algorithm
4. Advanced Shortest Path Algorithms
   - Bellman-Ford Algorithm
   - Johnson's algorithm
5. Graphical Data Structures
   - Quadtrees, KD Trees, R-Trees
6. Computational Geometry
   - Convex Hull and Voronoi Diagrams
7. Case Studies in Red-Black Tree and Its Applications, Applications of Directed Acyclic Graph, Applications of Minimum Spanning Tree in Network Design

Advanced Graph

Advanced Graph: Planar Graphs and its applications in Geographic and Circuit Layout Design

Planar Graphs

- Consider the problem of joining three houses to each of three separate utilities as in Figure. Is it possible to join these houses and utilities so that none of the connections cross?



- There are always many ways to represent a graph. When is it possible to find at least one way to represent this graph in a plane without any edges crossing?

Planar Graphs

1. A graph is called **planar** if it can be drawn in the plane without any edges crossing (where a crossing of edges is the intersection of the lines or arcs representing them at a point other than their common endpoint). Such a drawing is called a planar representation of the graph.
2. A graph may be planar even if it is usually drawn with crossings, because it may be possible to draw it in a different way without crossings.
3. A planar representation of a graph splits the plane into regions, including an unbounded region.
4. A complete graph of five vertices is non planar.

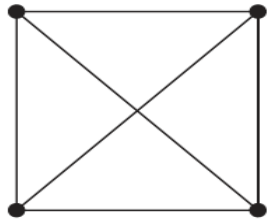


Fig: The Graph K4.

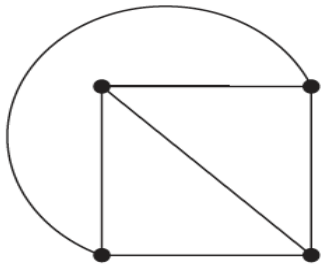


Fig: K4 Drawn with No Crossings.

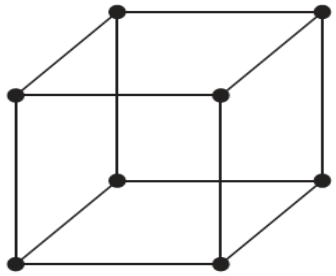


Fig: The Graph Q3.

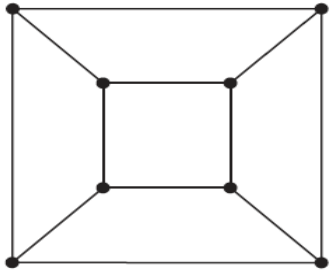
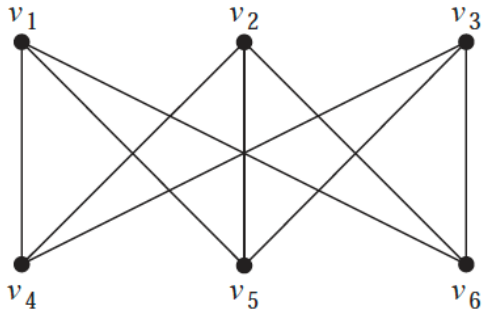
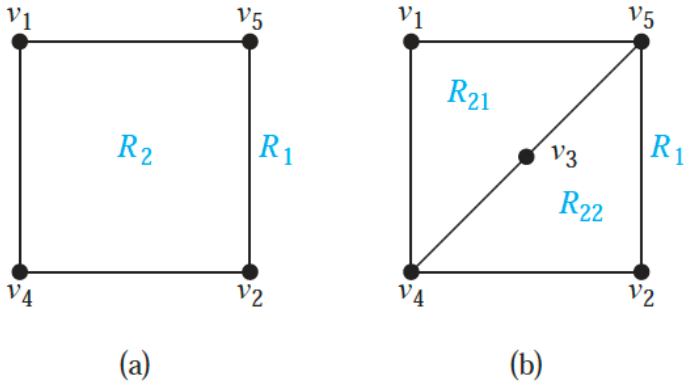


Fig: A Planar Representation of Q3.

Is $K_{3,3}$ graph(in figure shown) is planar?





- Any attempt to draw $K_{3,3}$ in the plane with no edges crossing is doomed.
- In any planar representation of $K_{3,3}$, the vertices v_1 and v_2 must be connected to both v_4 and v_5. These four edges form a closed curve that splits the plane into two regions, R_1 and R_2, as shown in Figure (a). The vertex v_3 is in either R_1 or R_2. When v_3 is in R_2, the inside of the closed curve, the edges between v_3 and v_4 and between v_3 and v_5 separate R_2 into two subregions, R_{21} and R_{22}, as shown in Figure (b).

Planar Graphs

Applications of Planar Graph

- Planarity of graphs plays an important role in the design of electronic circuits. We can model a circuit with a graph by representing components of the circuit by vertices and connections between them by edges. We can print a circuit on a single board with no connections crossing if the graph representing the circuit is planar. When this graph is not planar, we must turn to more expensive options.
- For example, we can partition the vertices in the graph representing the circuit into planar subgraphs. We then construct the circuit using multiple layers.
- We can construct the circuit using insulated wires whenever connections cross. In this case, drawing the graph with the fewest possible crossings is important.
- The planarity of graphs is also useful in the design of road networks.
- Suppose we want to connect a group of cities by roads. We can model a road network connecting these cities using a simple graph with vertices representing the cities and edges representing the highways connecting them.
- We can built this road network without using underpasses or overpasses if the resulting graph is planar.

Assignment

Do some research and write a synopsis paper including case studies where planar graph can be use for Circuit layout design and GIS. Submit it in Google classroom.

Network Flow Algorithms

Network Flow

- Network flow is a type of network optimization problem
  - Arise in many different contexts :
    - Networks: routing as many packets as possible on a given network
    - Transportation: sending as many trucks as possible, where roads have limits on the number of trucks per unit time
    - Bridges: destroying some bridges to disconnect source(s) from sink(t), while minimizing the cost of destroying the bridges
- Max flow problem is represented by a directed graph with a source node ***s*** and a sink node ***t***, and each edge has a capacity that represents the maximum amount of flow that can be sent through it.
- The goal is to find the maximum amount of flow that can be sent from source (***s***) to sink(***t***), while respecting the capacity constraints on the edges.

Network Flow Algorithms

Advantages:

1. The max flow problem is a flexible and powerful modeling tool that can be used to represent a wide variety of real-world situations.
2. The max flow problem has many interesting theoretical properties and connections to other areas of mathematics, such as linear programming and combinatorial optimization.

Disadvantages:

1. In some cases, the max flow problem can be difficult to solve efficiently, especially if the graph is very large or has complex capacity constraints.
2. The max flow problem may not always provide a unique or globally optimal solution, depending on the specific problem instance and algorithm used.
3. Maximum flow problems involve finding a feasible flow through a single-source, single-sink flow network that is maximum.

Network Flow Algorithms

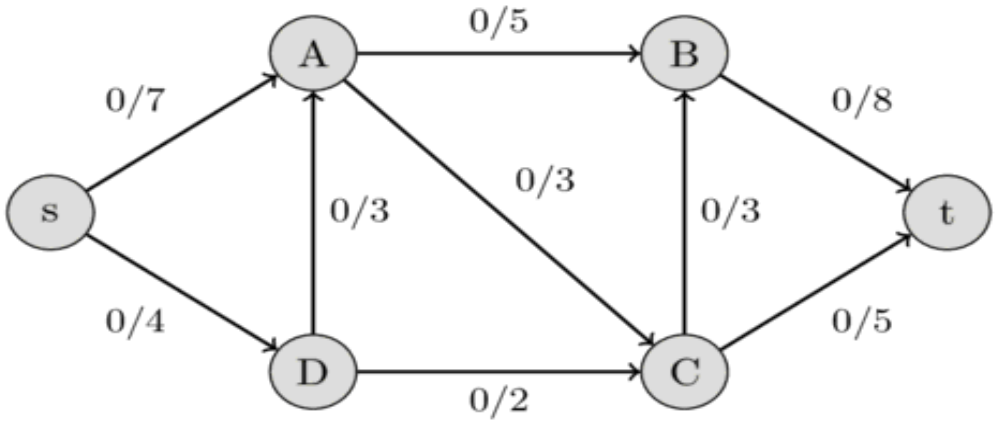
- A **network** is a directed graph G with vertices V and edges E combined with a function c, which assigns each edge $e \in E$ a non-negative integer value, the **capacity** of e. Such a network is called a **flow network**, if we additionally label two vertices, one as **source** and one as **sink**.
- A **flow** in a flow network is function f, that again assigns each edge e a non-negative integer value, namely the flow. The function has to fulfill the following two conditions:
  - The flow of an edge cannot exceed the capacity. $f(e) \leq c(e)$
  - And the sum of the incoming flow of a vertex u has to be equal to the sum of the outgoing flow of u except in the source and sink vertices. $\sum_{(v,u)\in E} f((v,u)) = \sum_{(u,v)\in E} f((u,v))$
- The source vertex s only has an outgoing flow, and the sink vertex t has only incoming flow.

$$\sum_{(s,u)\in E} f((s,u)) = \sum_{(u,t)\in E} f((u,t))$$

Network Flow Algorithms

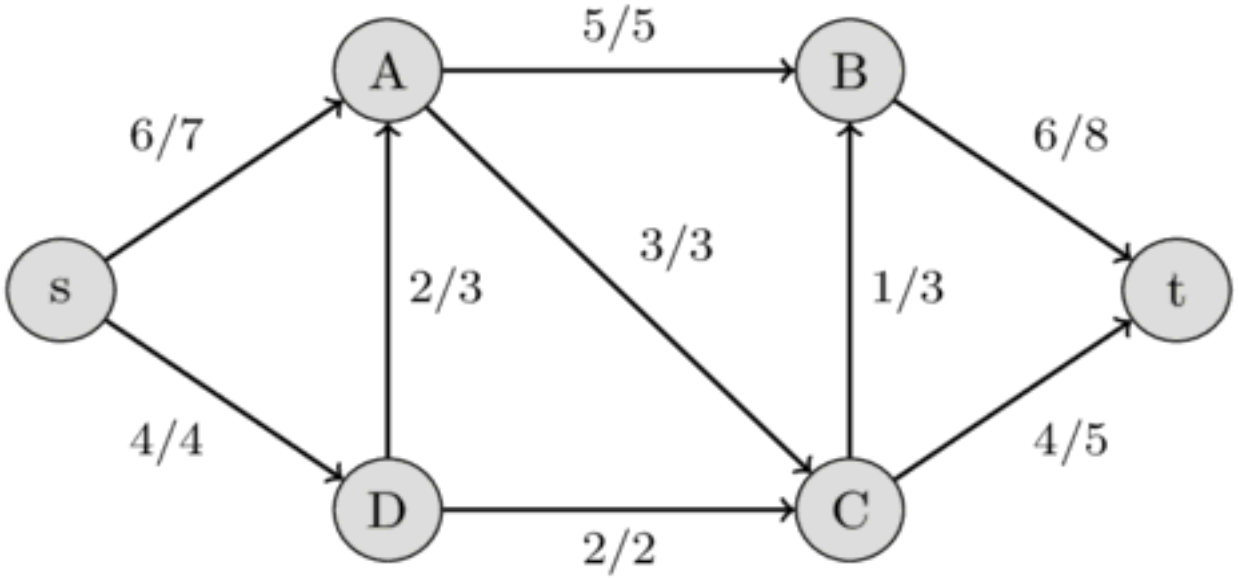
Analogy

- We represent edges as water pipes, the capacity of an edge is the maximal amount of water that can flow through the pipe per second, and the flow of an edge is the amount of water that currently flows through the pipe per second. This motivates the first flow condition. There cannot flow more water through a pipe than its capacity. The vertices act as junctions, where water comes out of some pipes, and then, these vertices distribute the water in some way to other pipes. This also motivates the second flow condition. All the incoming water has to be distributed to the other pipes in each junction. It cannot magically disappear or appear. The source s is origin of all the water, and the water can only drain in the sink t.
- Example of *flow network*. The first value of each edge represents the flow, which is initially 0, and the second value represents the capacity.



Network Flow Algorithms

- The value of the flow of a network is the sum of all the flows that get produced in the source s, or equivalently to the sum of all the flows that are consumed by the sink t.
- A **maximal flow** is a flow with the maximal possible value. Finding this maximal flow of a flow network is the problem that we want to solve.
- In the visualization with water pipes, the problem can be formulated in the following way: how much water can we push through the pipes from the source to the sink?
- The following image shows the maximal flow in the flow network.



Network Flow Algorithms

Ford-Fulkerson Algorithm

- The Ford-Fulkerson algorithm works by looking for a path with available capacity from the source to the sink (called an *augmented path*), and then sends as much flow as possible through that path.
- The Ford-Fulkerson algorithm continues to find new paths to send more flow through until the maximum flow is reached.
- **Residual graph**: A graph that represents the remaining capacity on each edge after the current flow has been sent through the network. [*original capacity – flow sent*]
- **Augmenting path:** A simple path from the source node to the sink node in the residual graph that has available capacity on all its edges. Augmenting path can be done in two ways:
  i. Non- Full forward edges
  ii. Non-Empty backward edges
- **Bottleneck capacity:** The minimum capacity of all the edges in an augmenting path.

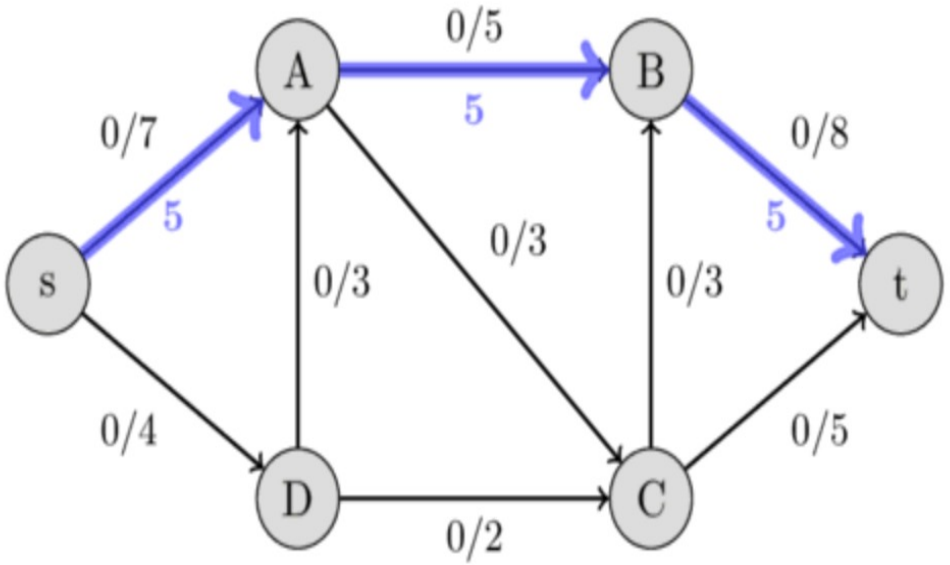
Network Flow Algorithms

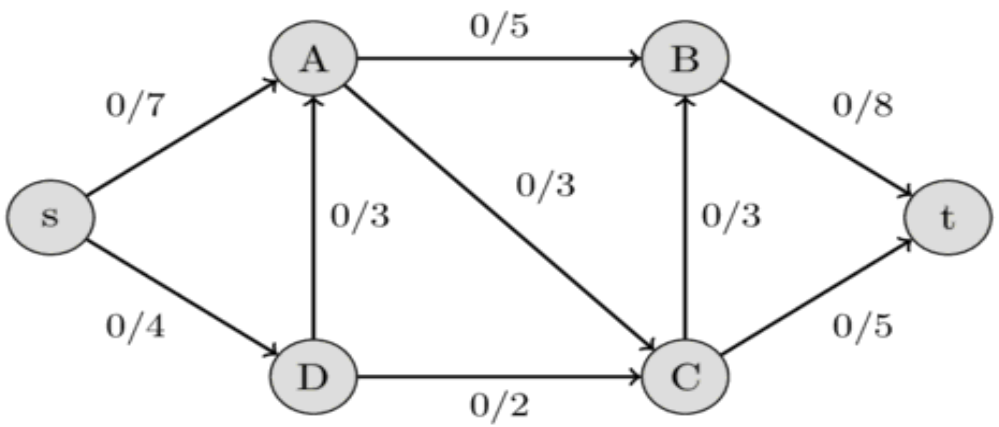
Ford-Fulkerson Algorithm

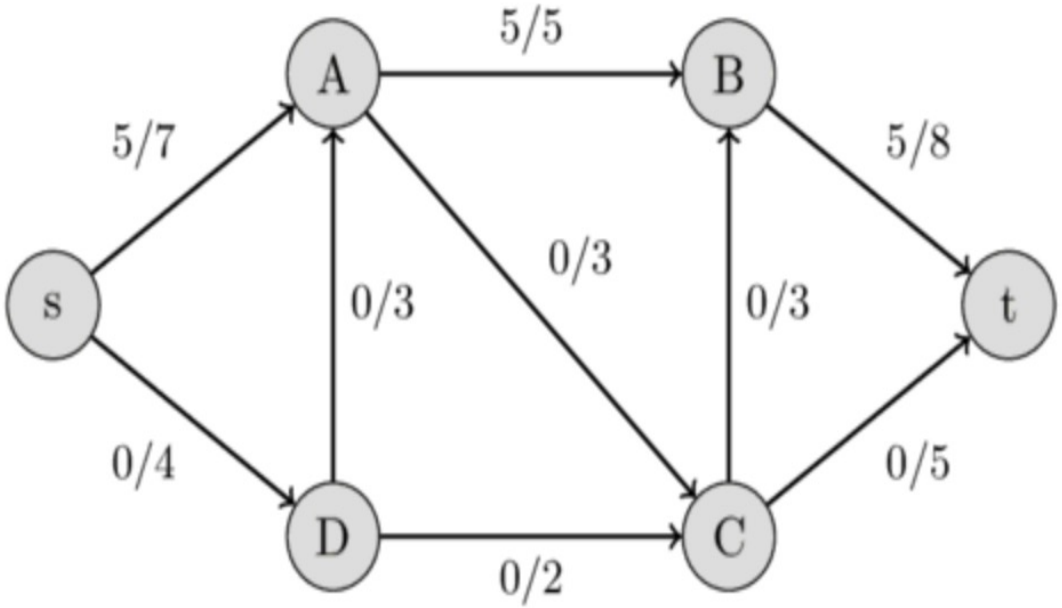
- **Formal Definition**: A **residual capacity** of a directed edge is the capacity minus the flow. It should be noted that if there is a flow along some directed edge (u, v), then the reversed edge has capacity 0 and we can define the flow of it as $f((v, u)) = -f((u, v))$. This also defines the residual capacity for all the reversed edges.
- We can create a **residual network** from all these edges, which is just a network with the same vertices and edges, but we use the residual capacities as capacities.
- Working Principle of Ford-Fulkerson Method:
  - First, we set the flow of each edge to zero. Then we look for an **augmenting path** from s to t.
  - An augmenting path is a simple path in the residual graph where residual capacity is positive for all the edges along that path. If such a path is found, then we can increase the flow along these edges. We keep on searching for augmenting paths and increasing the flow. Once an augmenting path doesn't exist anymore, the flow is maximal.
  - Let C be the smallest residual capacity of the edges in the path. Then we increase the flow in the following way: we update $f((u, v)) \mathrel{+}= C$ and $f((v, u)) \mathrel{-}= C$ for every edge (u, v) in the path.

Example: Ford-Fulkerson

Augmenting path s - A - B - t with the residual capacities 7, 5, and 8. Their minimum is 5, therefore we can increase the flow along this path by 5. This gives a flow of 5 for the network.



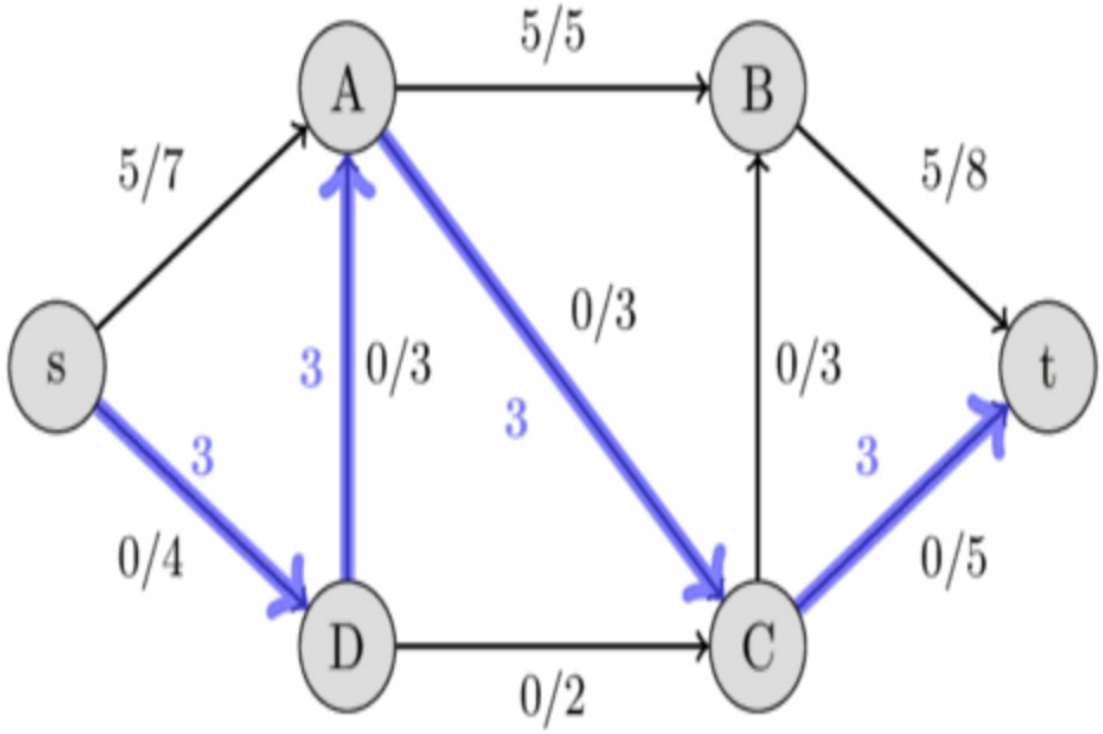


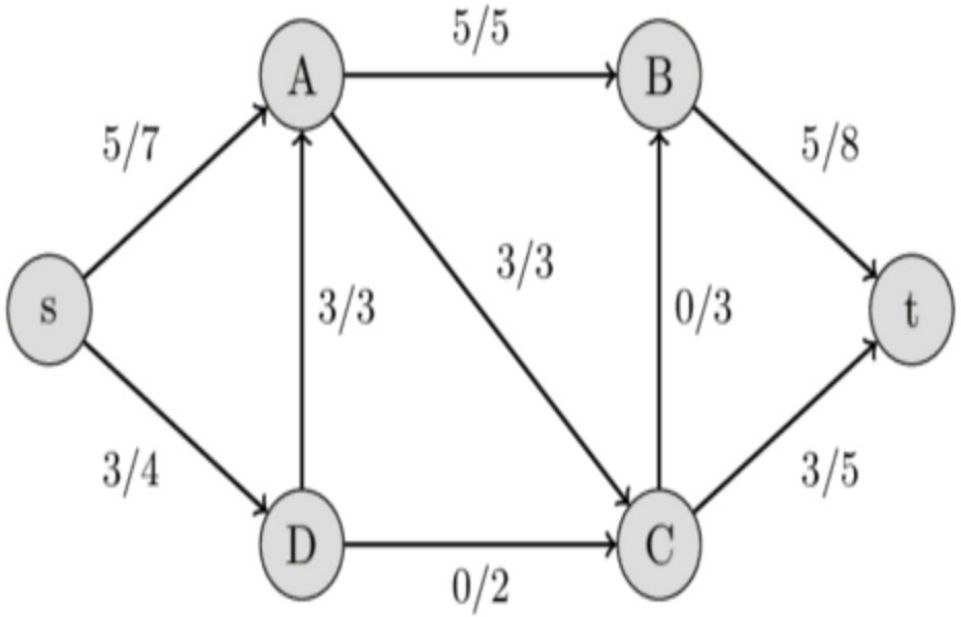


Network Flow Algorithms

Example: Ford-Fulkerson

Another augmenting path: *s-D-A-C-t* with the residual capacities 4, 3, 3, and 5.
Therefore we can increase the flow by 3 and we get a flow of 8 for the network.

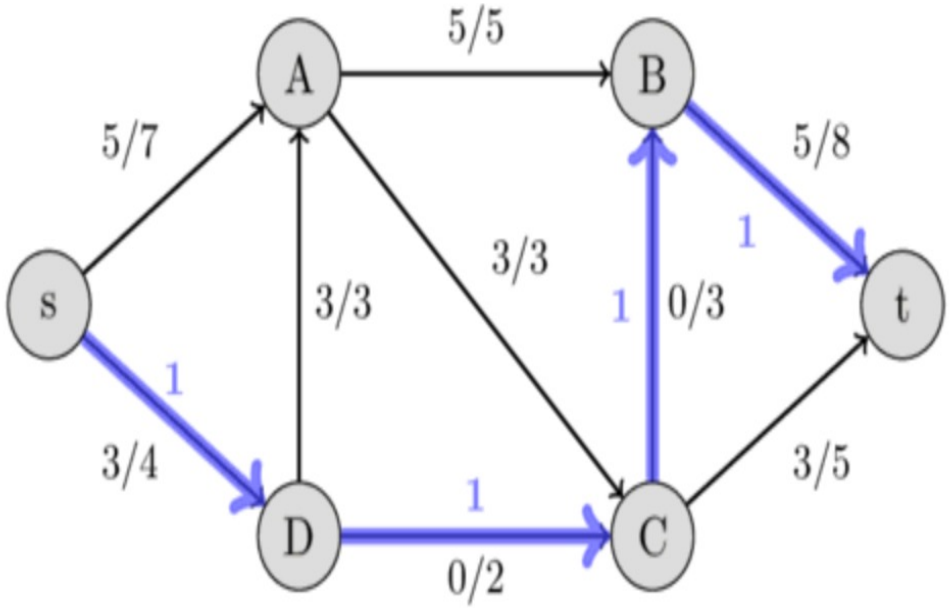


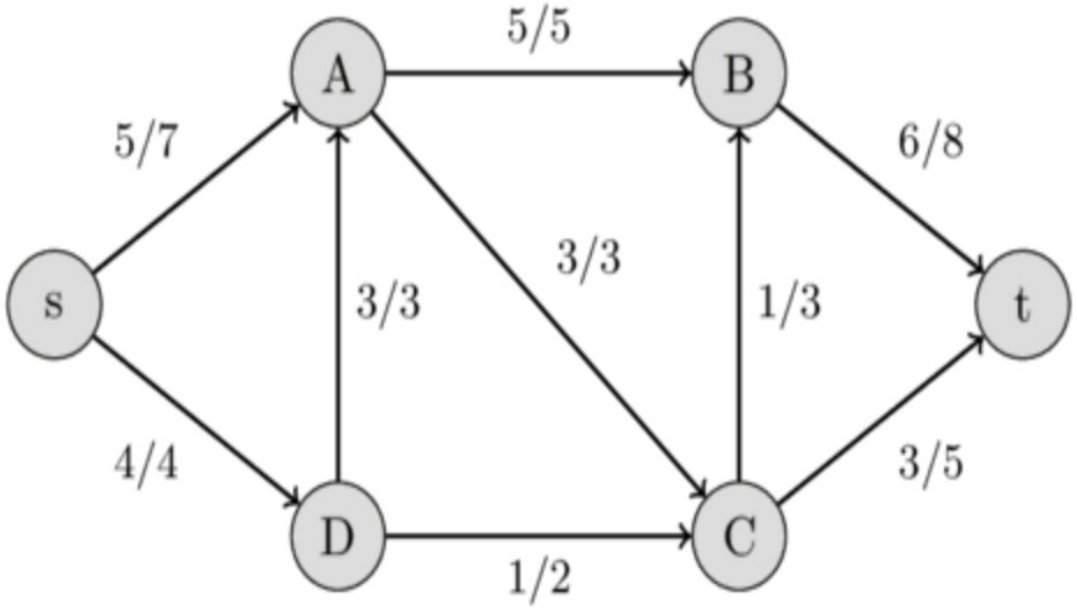


Network Flow Algorithms

Example: Ford-Fulkerson

Another Augmenting path *s-D-C-B-t* with the residual capacities 1, 2, 3, and 3, and hence, we increase the flow by 1. Flow = 5+3+1 = 9

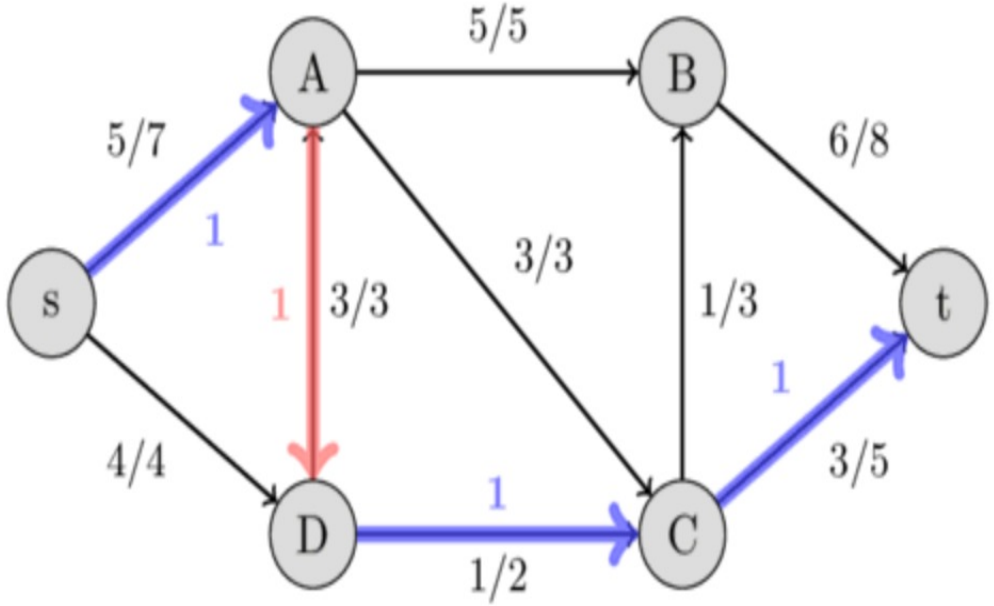


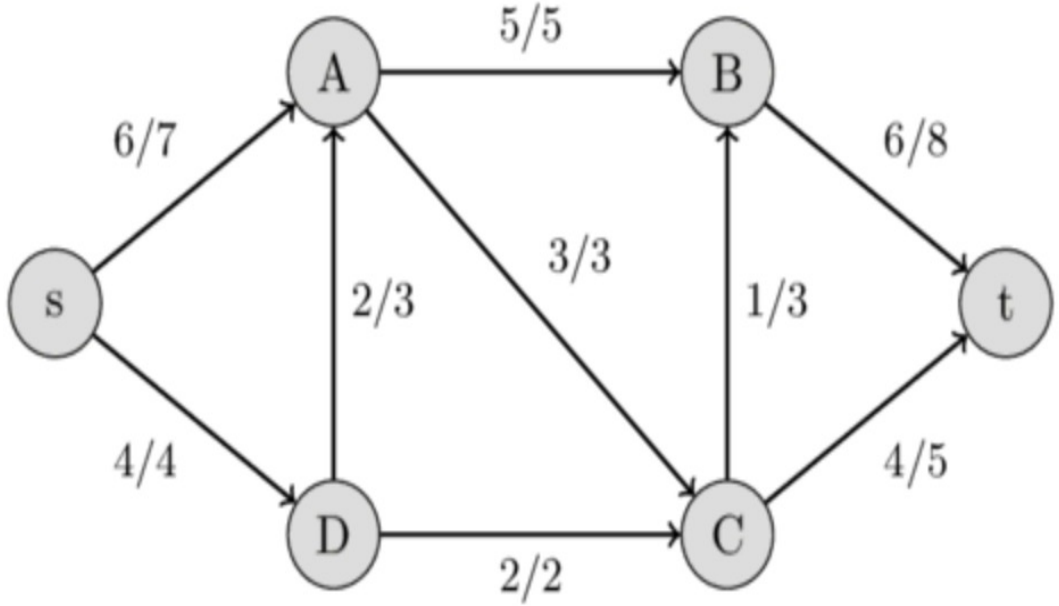


Network Flow Algorithms

Example: Ford-Fulkerson

This time we find the augmenting path *s-A-D-C-t* with the residual capacities 2, 3, 1, and 2. We can increase the flow by 1. But this path is very interesting. It includes the reversed edge *(A, D)*. In the original flow network, we are not allowed to send any flow from *A* to *D*. But because we already have a flow of 3 from *D* to *A*, this is possible. The intuition of it is the following: Instead of sending a flow of 3 from *D* to *A*, we only send 2 and compensate this by sending an additional flow of 1 from *s* to *A*, which allows us to send an additional flow of 1 along the path *D-C-t*. **Flow = 5+3+1+1=10**





Network Flow Algorithms

Example: Ford-Fulkerson (Conclusion)

Now, it is impossible to find an augmenting path between s and t, therefore this flow of 10 is the maximal possible.

It should be noted, that the Ford-Fulkerson method doesn't specify a method of finding the augmenting path. Possible approaches are using DFS or BFS which both work in $O(E)$.
If all the capacities of the network are integers, then for each augmenting path the flow of the network increases by at least 1. So DFS runs **F** times.
Therefore, the complexity of Ford-Fulkerson is $O(EF)$, where **F** is the maximal flow of the network.

Pseudocode of Fold-Fulkerson Method

```
flow = 0
for each edge (u, v) in G:
        flow(u, v) = 0
while there is a path, p, from s -> t in residual network G_f:
        residual_capacity(p) = min(residual_capacity(u, v) : for (u, v) in p)
        flow = flow + residual_capacity(p)
        for each edge (u, v) in p:
                if (u, v) is a forward edge:
                        flow(u, v) = flow(u, v) + residual_capacity(p)
                else:
                        flow(u, v) = flow(u, v) - residual_capacity(p)
return flow
```

Network Flow Algorithms

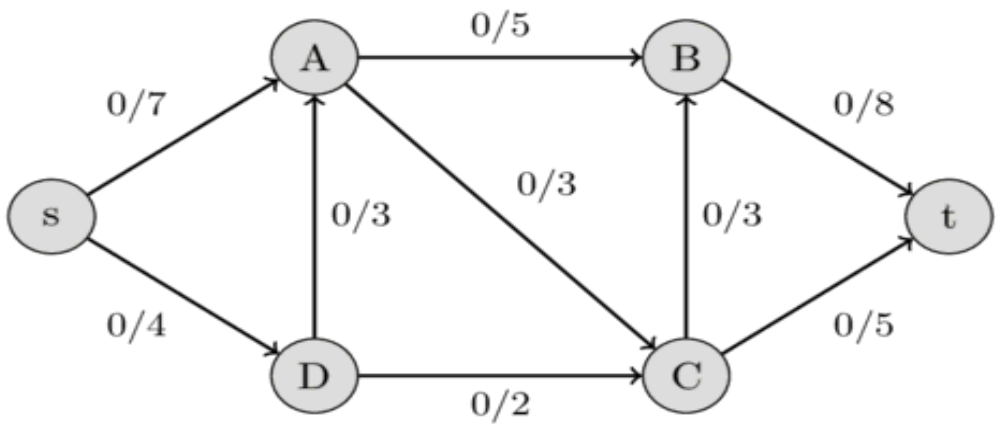
Edmonds-Karp algorithm

- Edmonds-Karp algorithm is just an implementation of the Ford-Fulkerson method that uses BFS for finding augmenting paths.
- The algorithm was first published by Yefim Dinitz in 1970, and later independently published by Jack Edmonds and Richard Karp in 1972.
- The complexity can be given independently of the maximal flow.
- The algorithm runs in $O(VE^2)$ time.
- This means Edmonds-Karp does not depend on the maximum flow, like Ford-Fulkerson does, but on how many vertices and edges we have.
- The intuition is, that every time we find an augmenting path one of the edges becomes saturated, and the distance from the edge to s will be longer if it appears later again in an augmenting path.
- The length of the simple paths is bounded by V.

Network Flow Algorithms

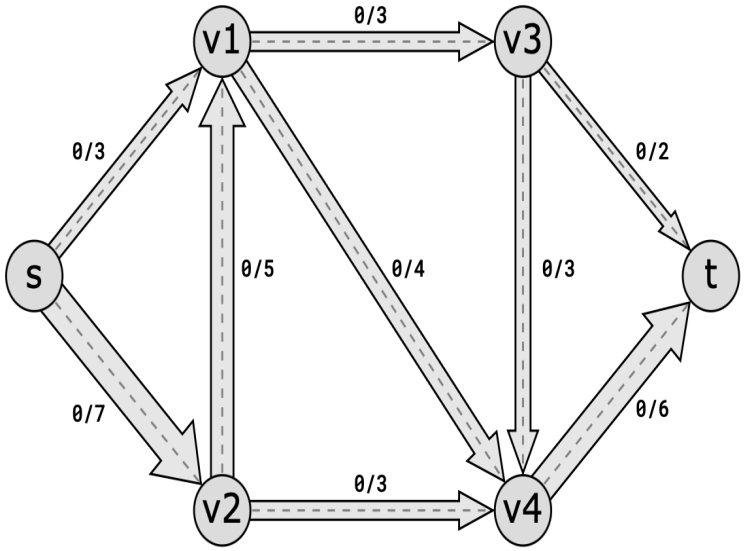
Example: Edmonds-Karp algorithm

Augmenting Path	Bottleneck Capacity
s-A-B-t	5
s-D-C-t	2
s-A-C-t	2
s-D-A-C-t	1
Total Flow	10

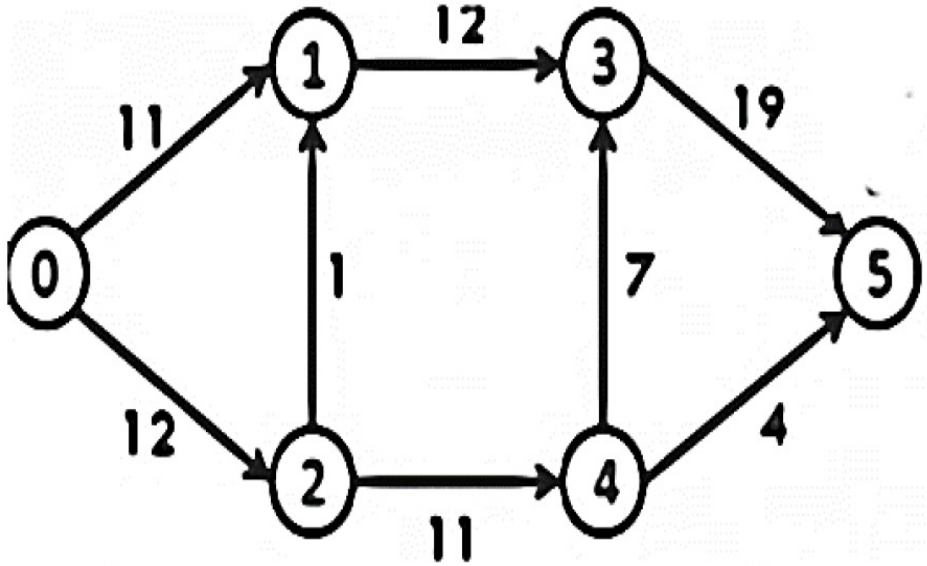


Try Yourself

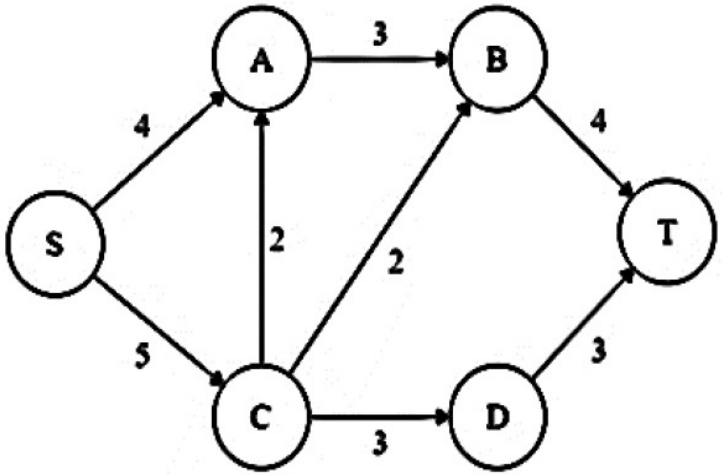
Find the maximum flow in a given network using Ford-Fulkerson and Edmond Karp Algorithm



Ans : 8



Ans : 23

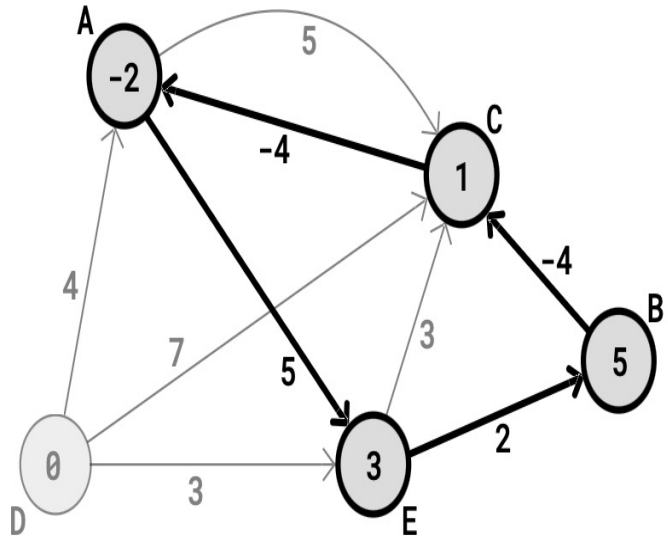


Ans : 7

Try Python Implementation of Edmond Karp Algorithm from :
https://www.w3schools.com/dsa/dsa_algo_graphs_edmondskarp.php

Shortest Path Algorithm

- Algorithms that find the shortest paths, like Dijkstra's algorithm or the Bellman-Ford algorithm, find the shortest paths from one start vertex to all other vertices.
- Dijkstra's algorithm can only find the shortest paths in graphs where all the edges are positive. We can think of the edges between vertices as distances between locations.
- We can represent negative edge for money lost situation.
- Finding the shortest paths becomes impossible if a graph has negative cycles.
- Having a negative cycle means that there is a path where you can go in circles, and the edges that make up this circle have a total path weight that is negative.
- In the graph below, the path A->E->B->C->A is a negative cycle because the total path weight is 5+2-4-4=-1.
- The reason why it is impossible to find the shortest paths in a graph with negative cycles is that it will always be possible to continue running an algorithm to find even shorter paths.



Shortest Path Algorithm

Bellman-Ford Algorithm

- The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge weights may be negative.
- Given a weighted, directed graph G = (V, E) with source vertex *s* and weight function w:E⟶ R, the Bellman-Ford algorithm returns a boolean value indicating whether there is a negative-weight cycle that is reachable from the source.
- If there is such a cycle, the algorithm indicates that no solution exists. If there is no such cycle, the algorithm produces the shortest paths and their weights.

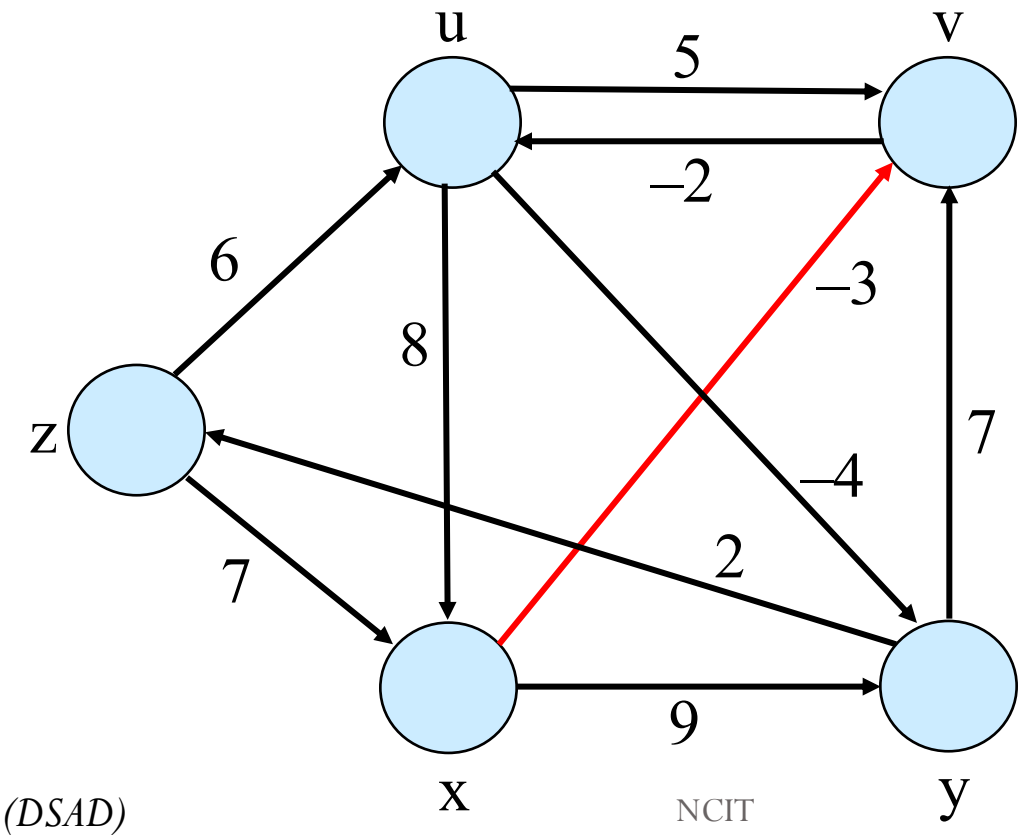
Single-Source Shortest Paths



- **Given:** A single source vertex in a weighted, directed graph.
- Want to compute a shortest path for each possible destination.
  - Similar to BFS.
- We will assume either
  - no negative-weight edges, or
  - no reachable negative-weight cycle that is reachable from source.
- Algorithm will compute a shortest-path tree.
  - Similar to BFS tree.
  - If there is a negative weight cycle, the algorithm indicates that no solution exists.
  - If there is no such cycle, the algorithm produces the shortest paths and their weights.

Negative Edge

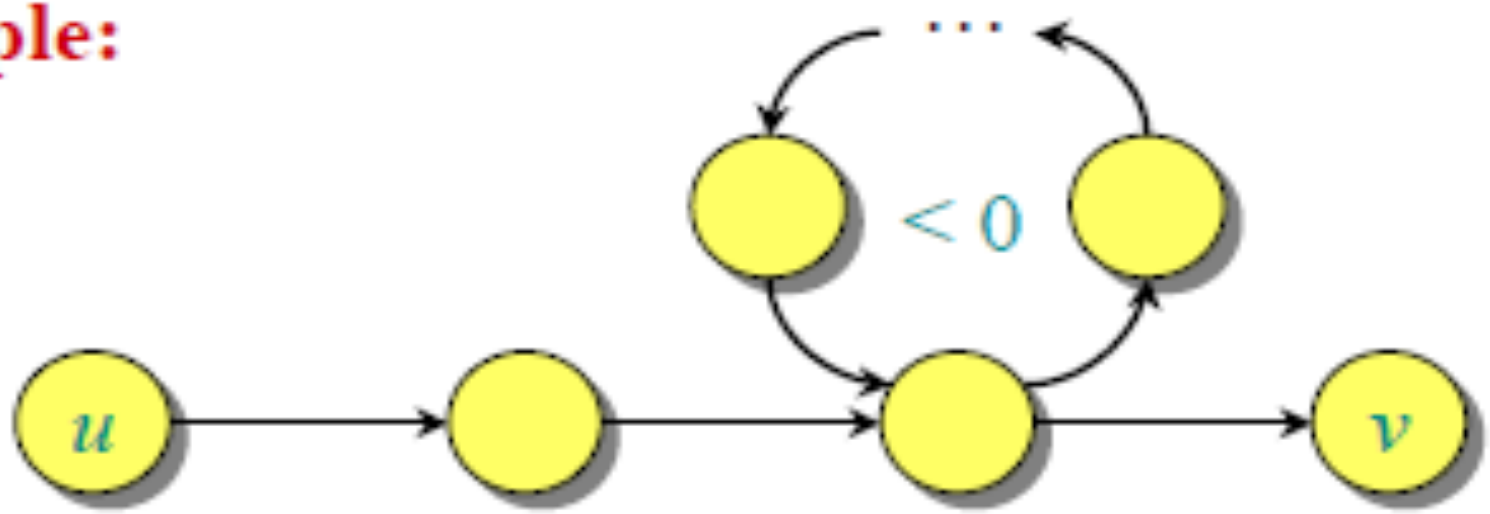
- A negative edge is simply an edge having a negative weight.
  - For example, the edge X-V in the graph is a negative edge. For a negative weight you could simply perform the calculation as you would have done for positive weight edges.



Negative-weight cycles

Recall: If a graph $G = (V, E)$ contains a negative-weight cycle, then some shortest paths may not exist.

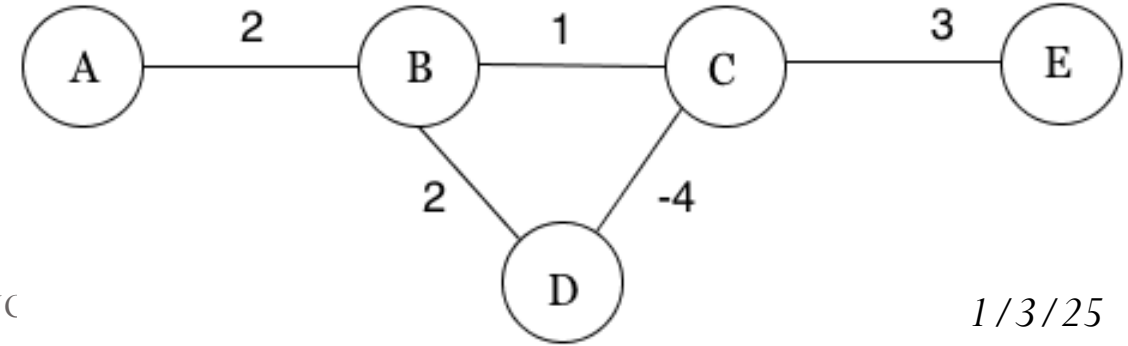
Example:



Bellman-Ford algorithm: Finds all shortest-path lengths from a ***source*** $s \in V$ to all $v \in V$ or determines that a negative-weight cycle exists.

Negative Weight Cycle..

- What is the shortest path between A and E?
  - Initially, we can see that ABCE costing 6 (2+1+3). But actually, taking a deeper look, we would observe a negative cycle, which is BCD. The weight of BCD is 1+(-4)+2 = (-1).
  - While traversing from A to E, it is possible to looping around inside BCD to reduce cost by 1 each time. Like, the path A(BCD)BCE costs 5 (2+(-1)+1+3).
  - Now repeating the cycle infinite times would keep reducing the cost by 1 each time. We could achieve a negative infinite shortest path between A and E.
  - The problem is evident for any negative cycle in a graph. Hence, whenever a negative cycle is present, the minimum weight is not defined or is negative infinity.



Bellman-Ford Algorithm

Bellman-Ford returns a compact representation of the set of shortest paths from s to all other vertices in the graph reachable from s. This is contained in the predecessor subgraph.

If Bellman-Ford has not converged after V(G) - 1 iterations, then there cannot be a shortest path tree, so there must be a negative weight cycle.

Bellman-Ford algorithm

$d[s] \leftarrow 0$
for each $v \in V - \{s\}$
  do $d[v] \leftarrow \infty$

} initialization

for $i \leftarrow 1$ **to** $|V| - 1$ **do**
  for each edge $(u, v) \in E$ **do**
    if $d[v] > d[u] + w(u, v)$ **then**
      $d[v] \leftarrow d[u] + w(u, v)$
      $\pi[v] \leftarrow u$

} *relaxation step*

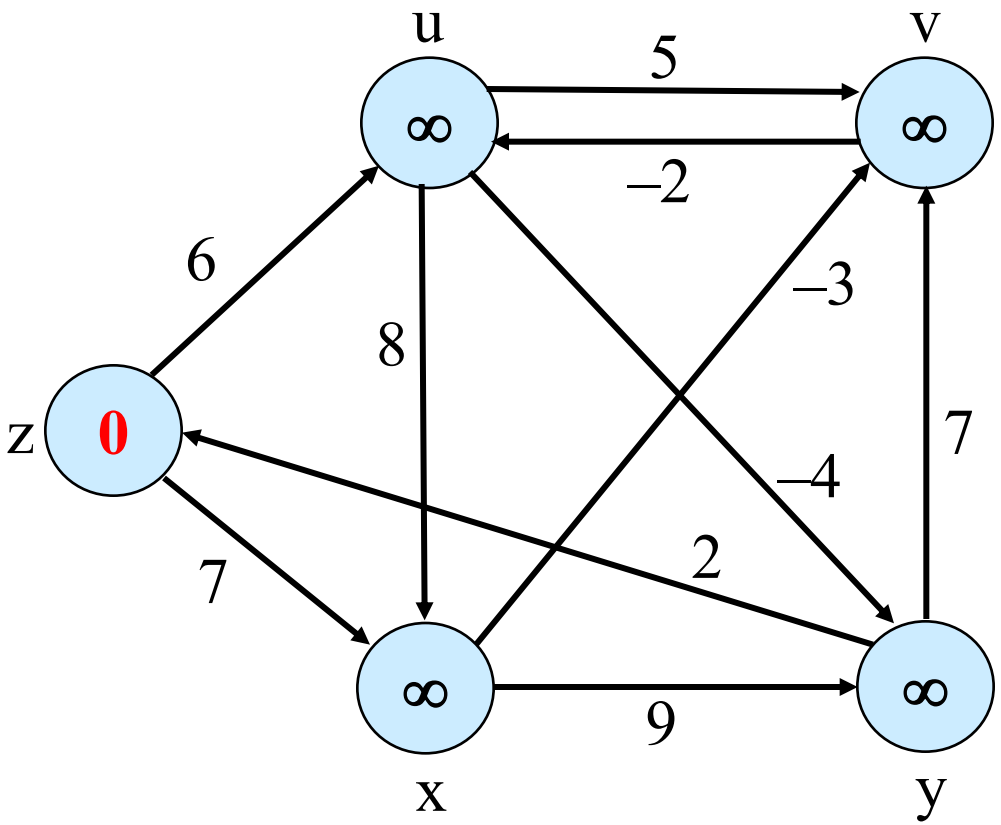
for each edge $(u, v) \in E$
  do if $d[v] > d[u] + w(u, v)$
    then report that a negative-weight cycle exists

Can have negative-weight edges.
Will "detect" reachable negative-weight cycles.

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

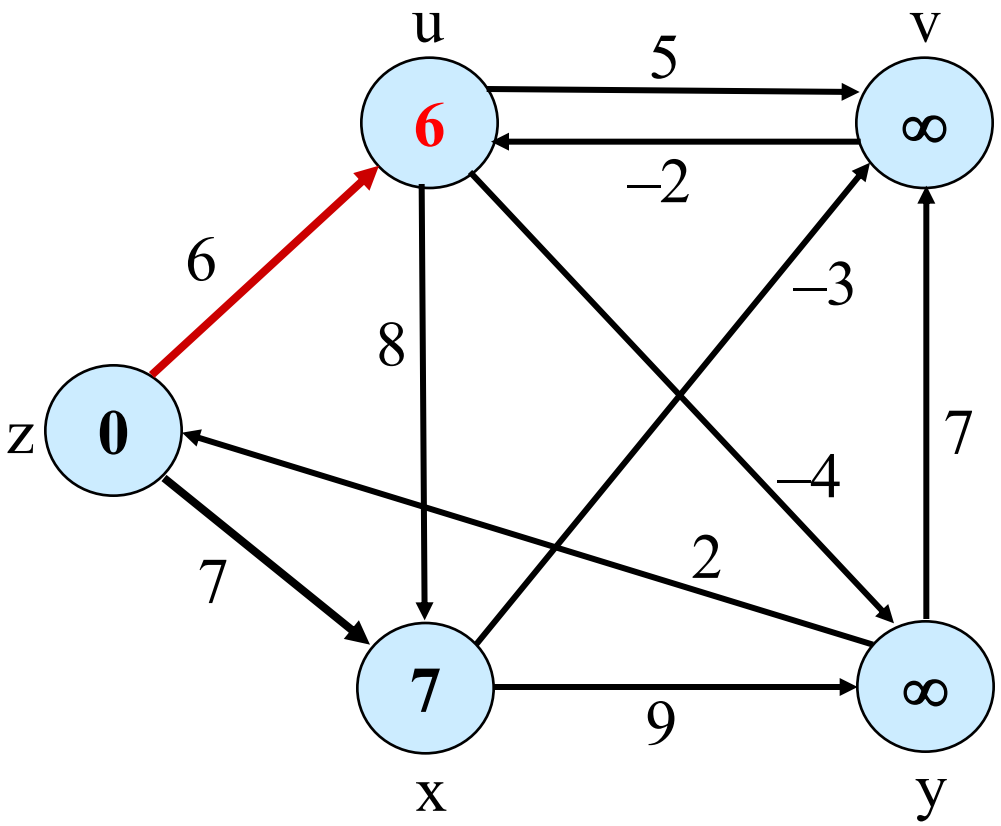


Vertex	D[v]	Π[v]
Z	0	Nil
U	∞	Nil
V	∞	Nil
X	∞	Nil
Y	∞	Nil

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

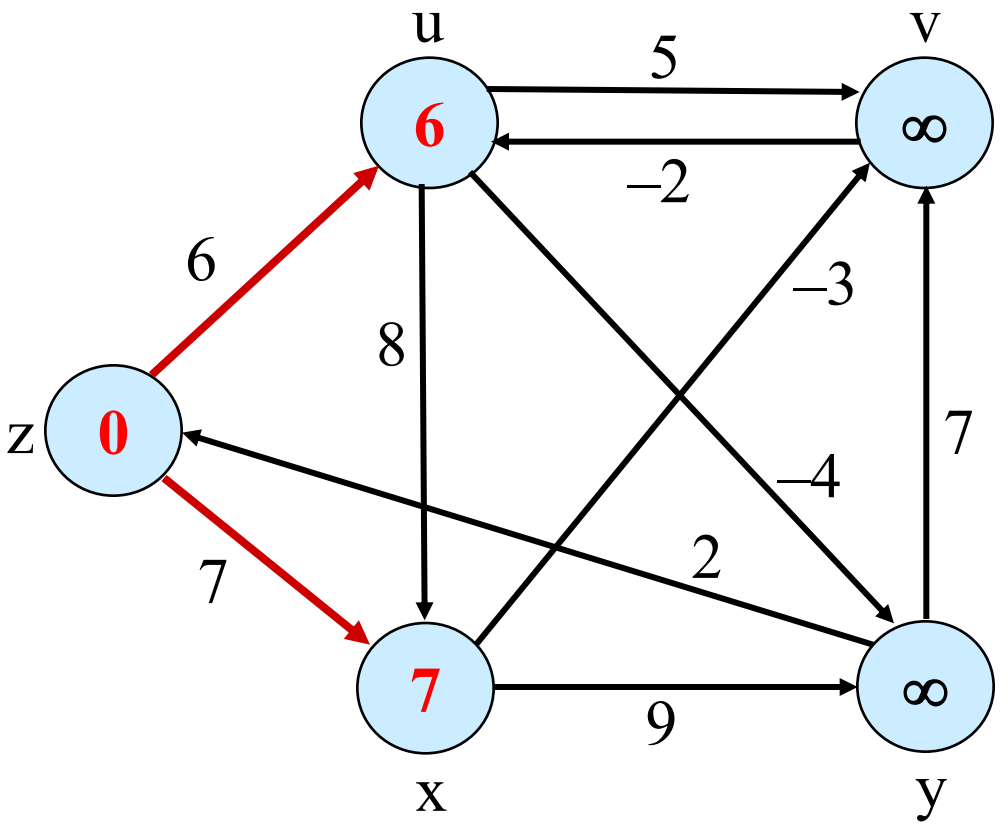


Vertex	D[v]	Π[v]
Z	0	Nil
U	6	z
V	∞	Nil
X	∞	Nil
Y	∞	Nil

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)



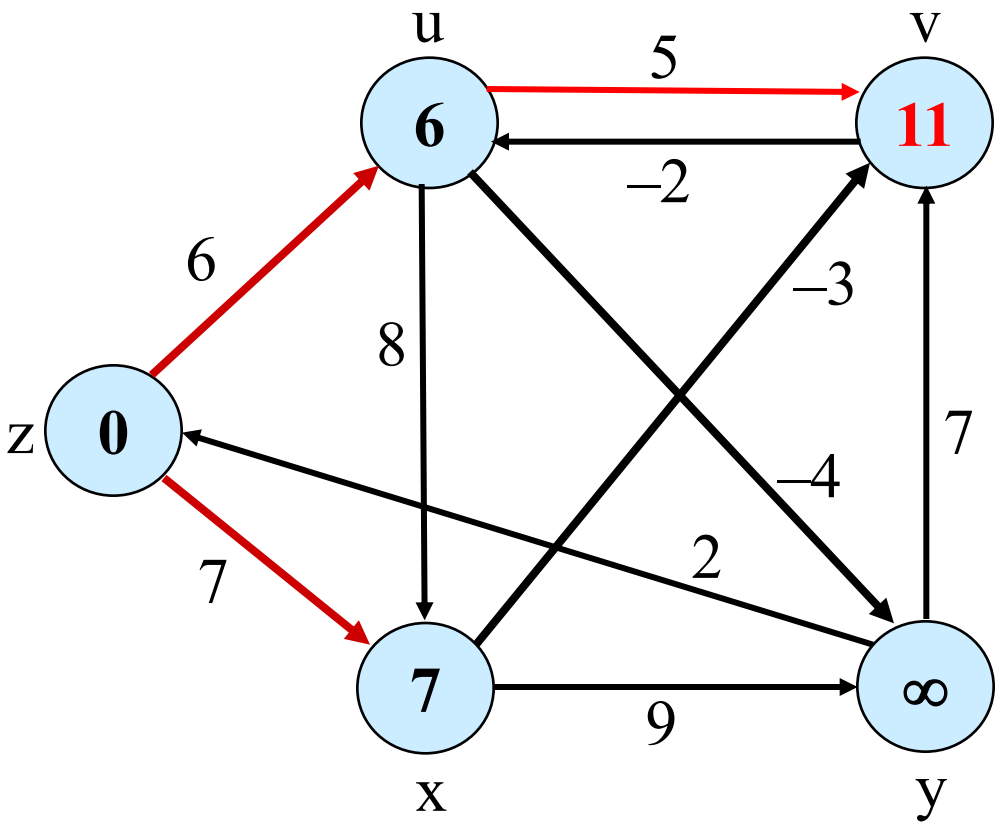
Vertex	D[v]	Π[v]
Z	0	Nil
U	6	z
V	∞	Nil
X	7	z
Y	∞	Nil

Bellman-Ford Algorithm

Example

Order of Edges

(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)



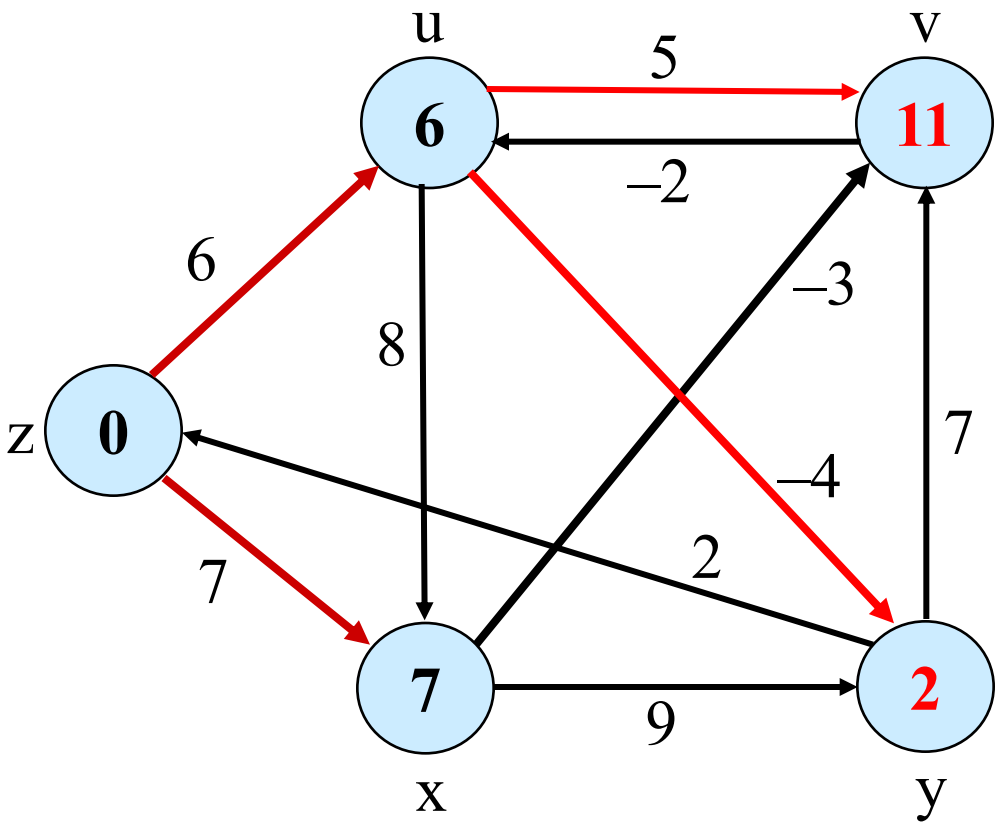
Vertex	D[v]	Π[v]
Z	0	Nil
U	6	z
V	11	U
X	7	z
Y	∞	Nil

Bellman-Ford Algorithm

Example

Order of Edges

(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

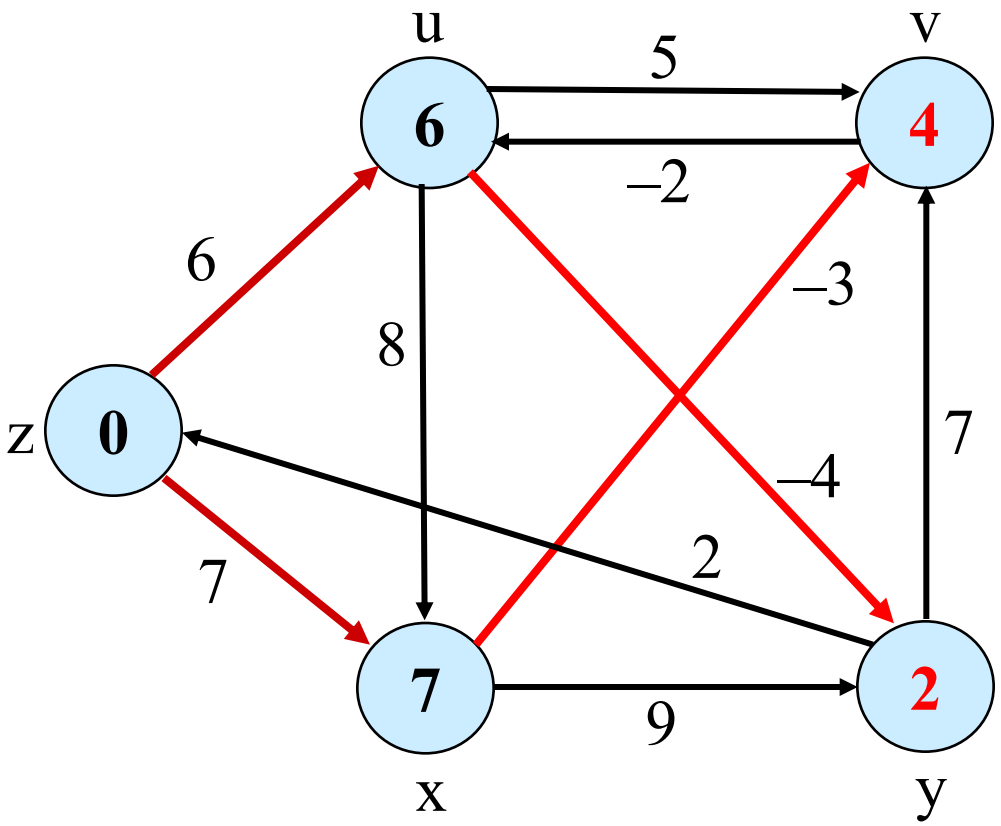


Vertex	D[v]	Π[v]
Z	0	Nil
U	6	z
V	11	U
X	7	z
Y	2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

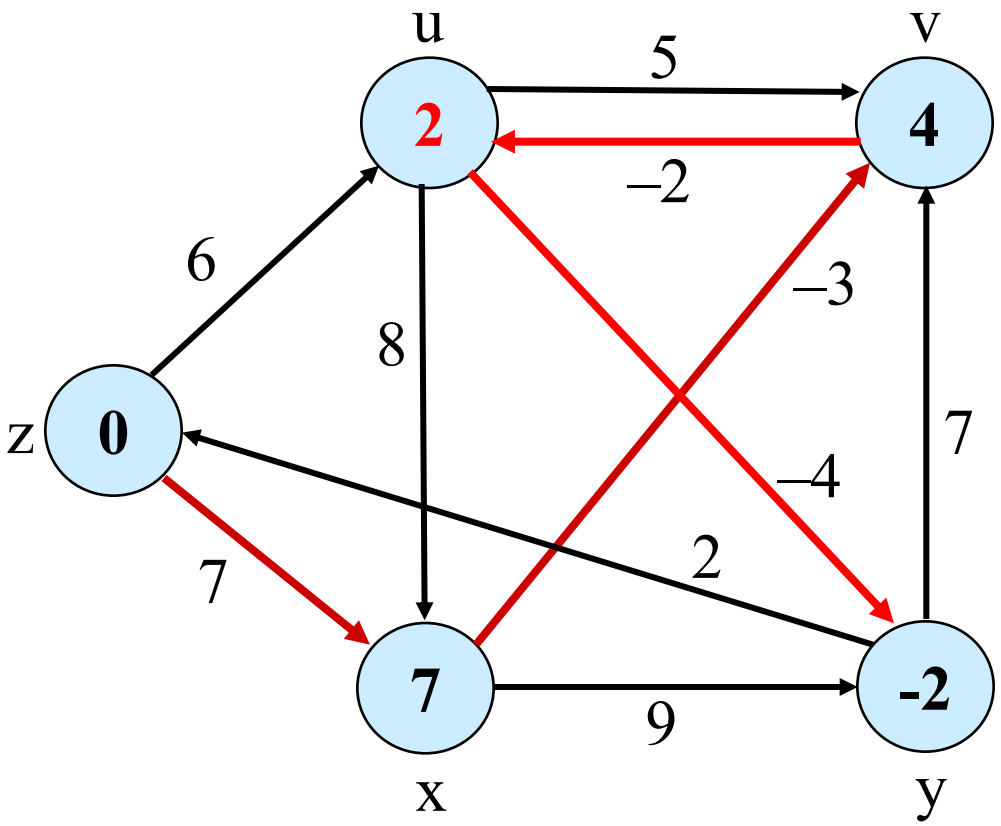


Vertex	D[v]	Π[v]
Z	0	Nil
U	6	z
V	4	x
X	7	z
Y	2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

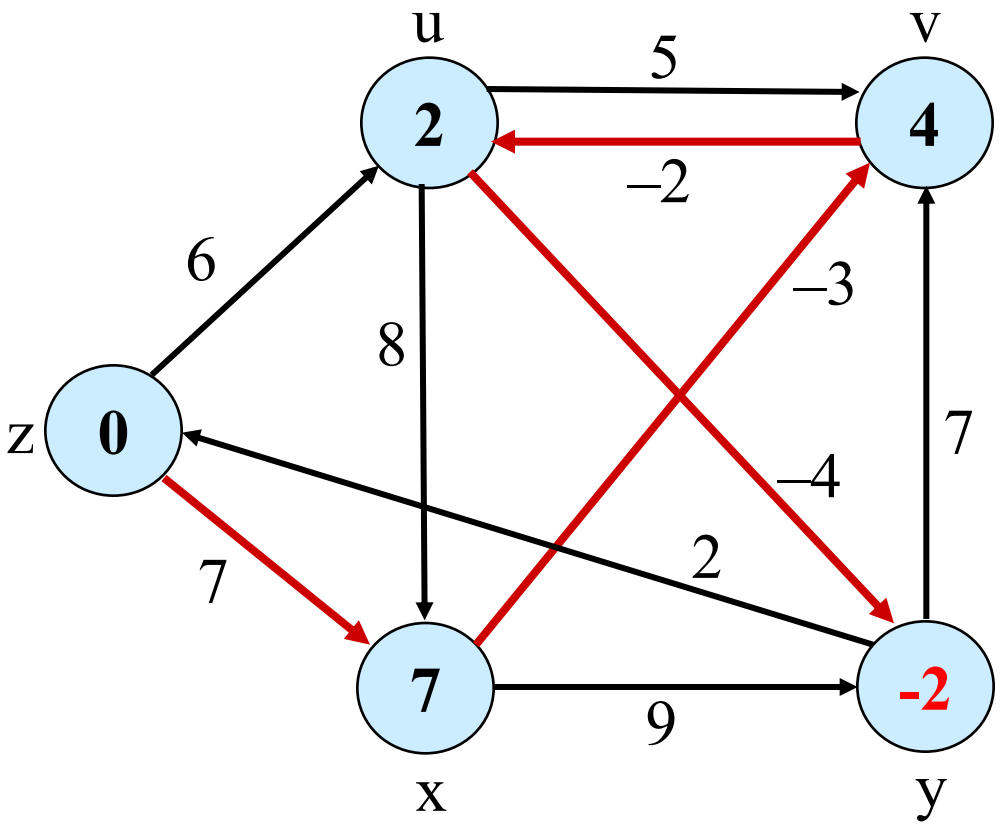


Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)



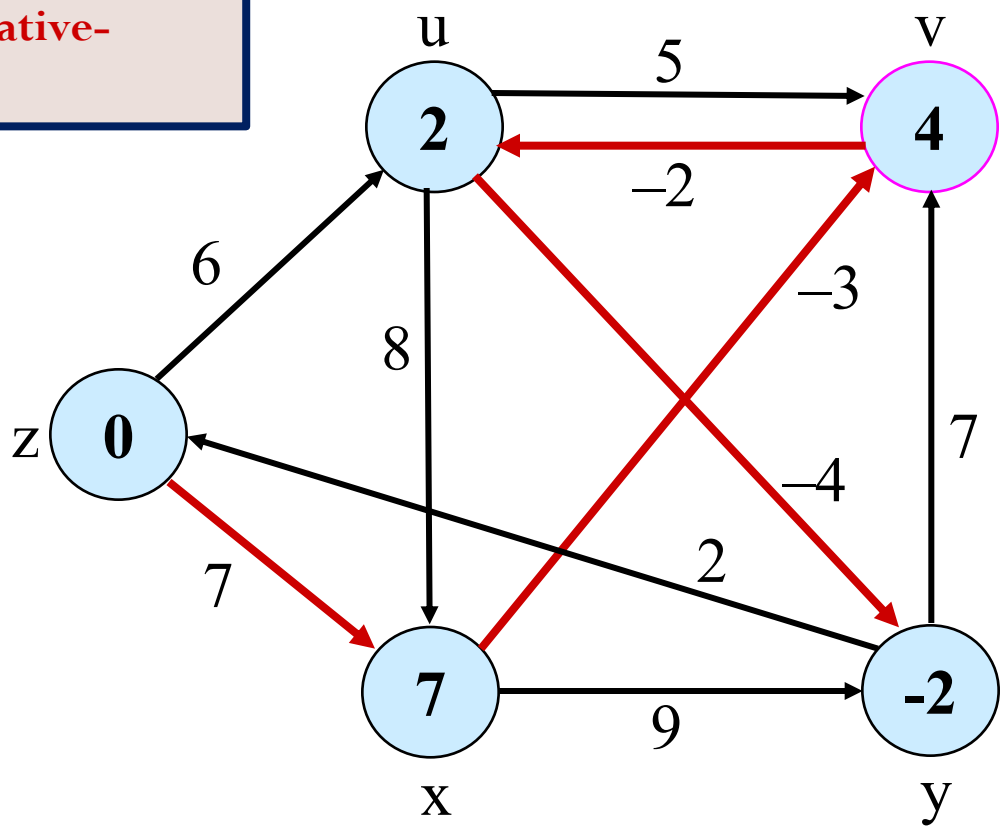
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



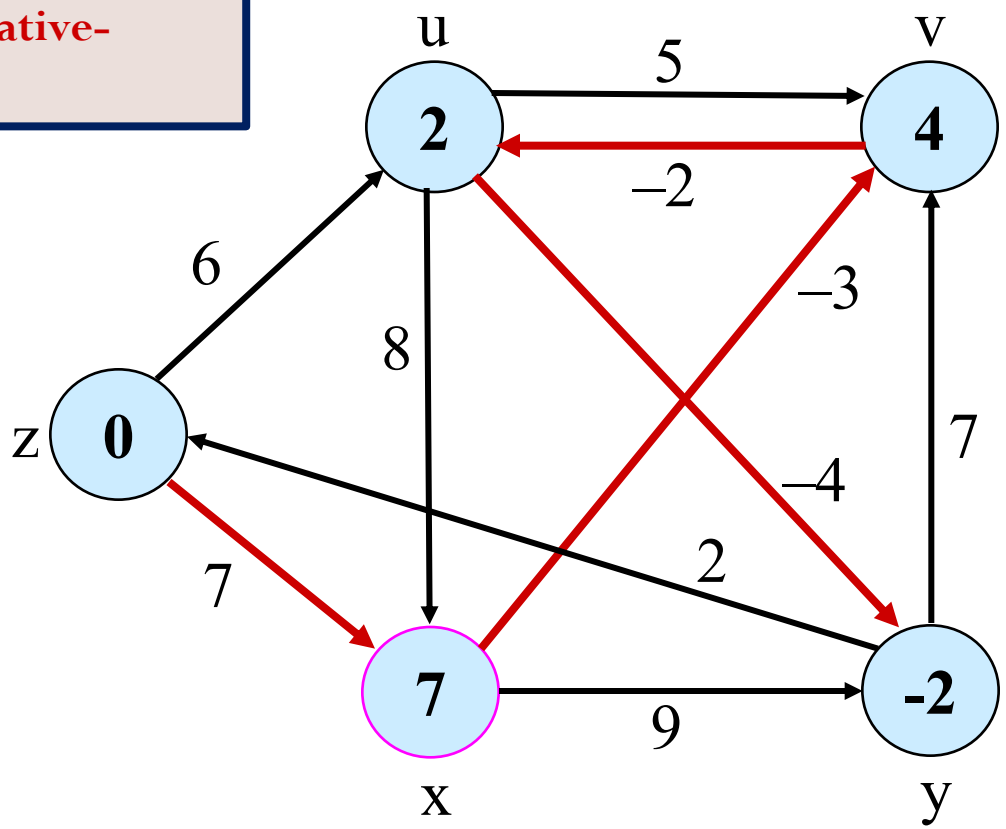
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



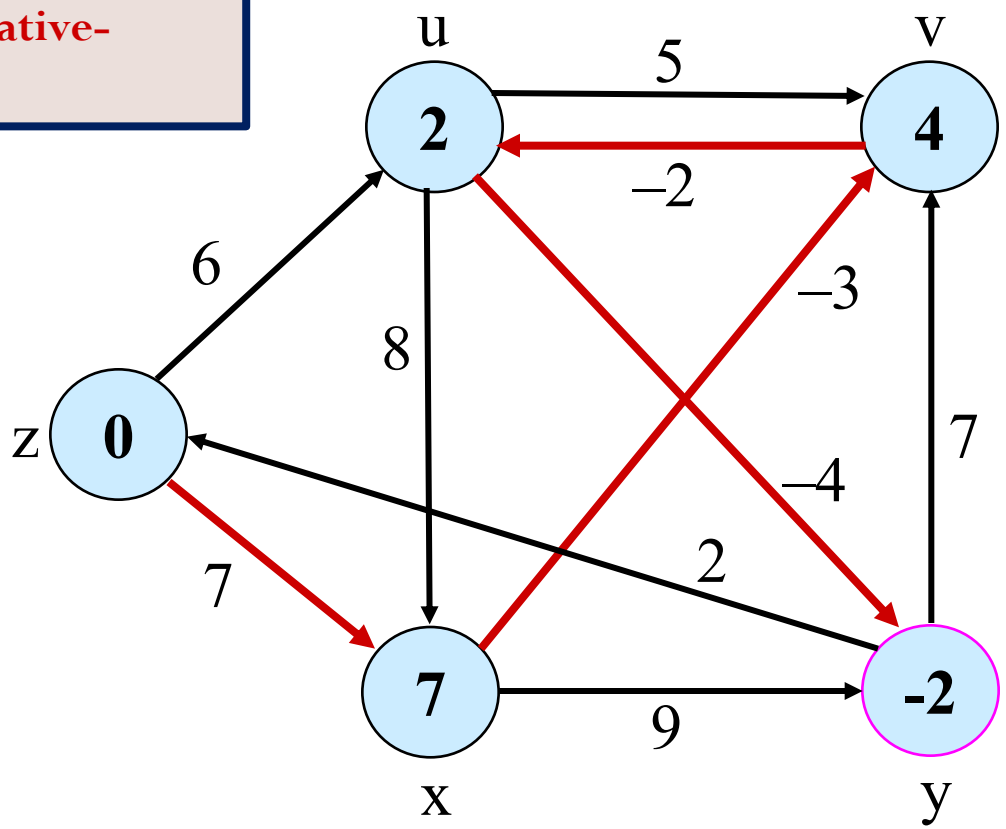
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



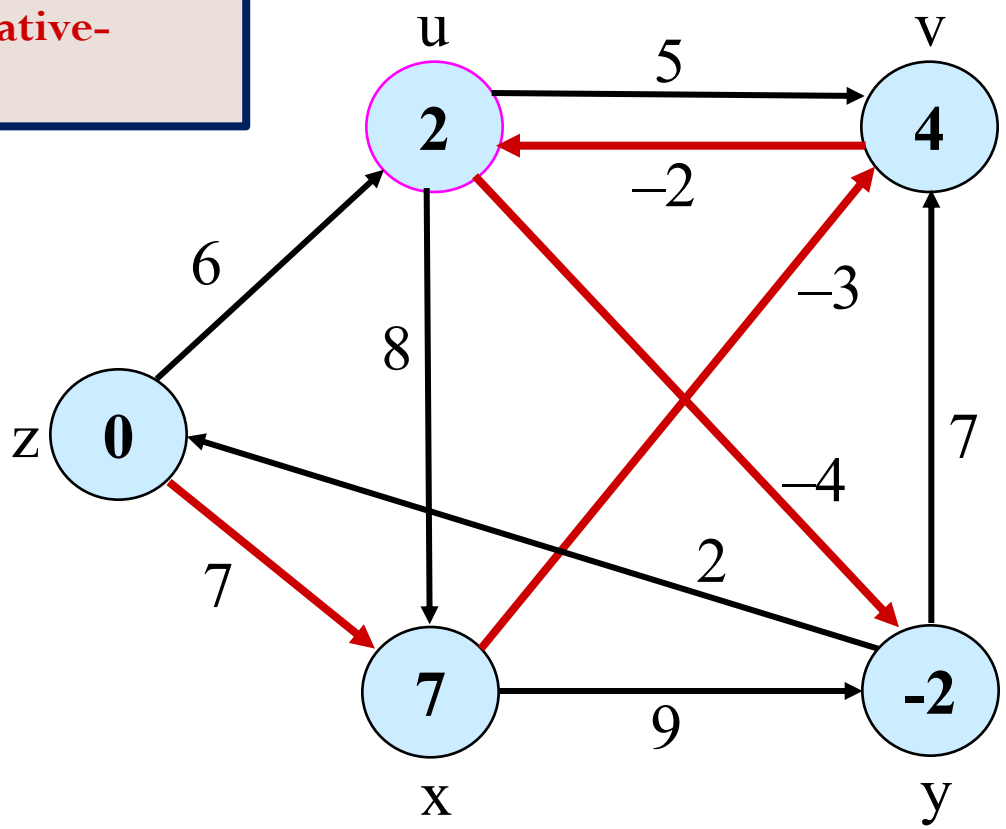
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



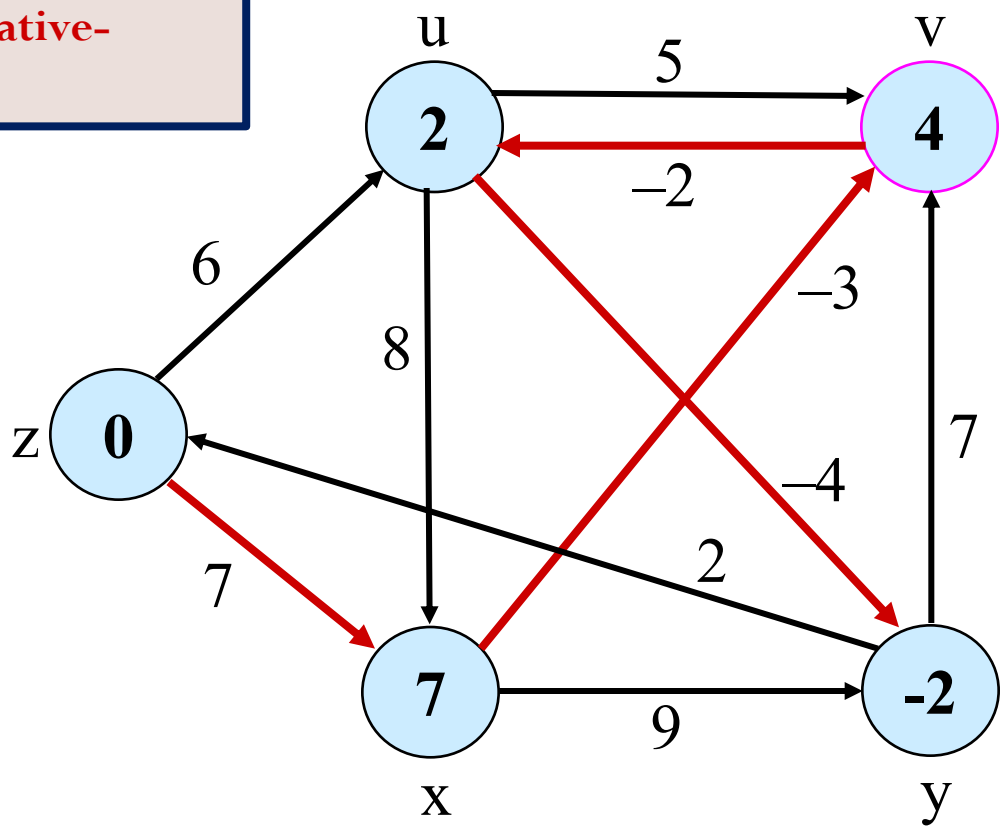
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



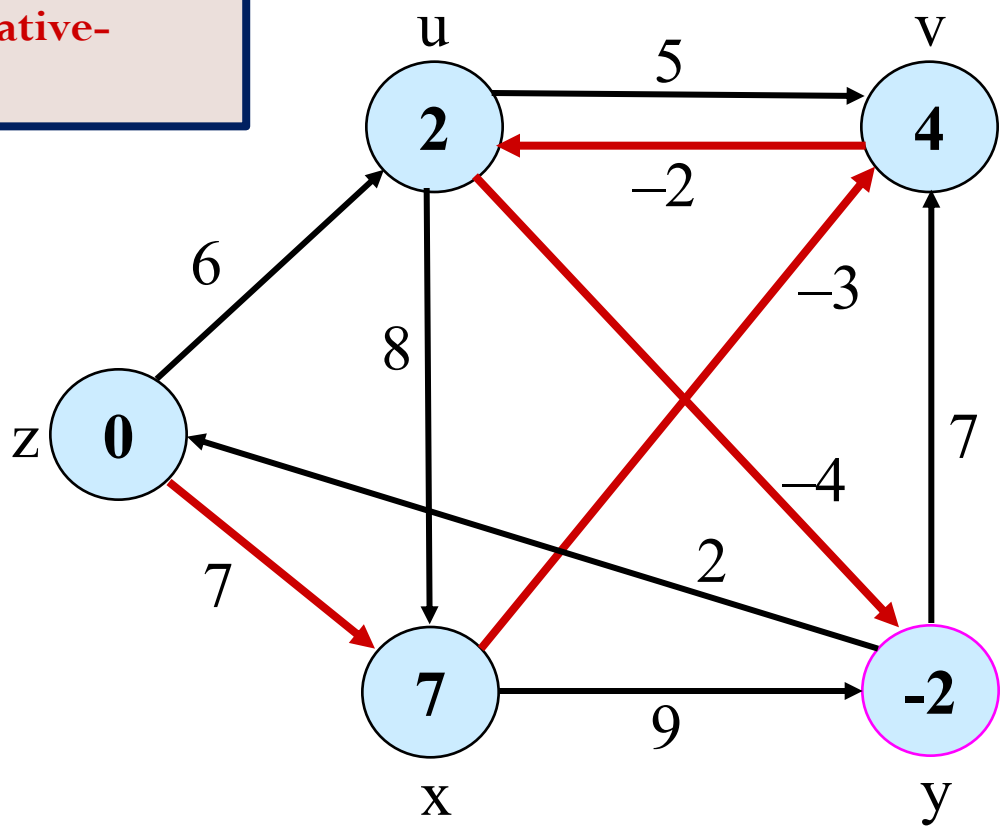
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



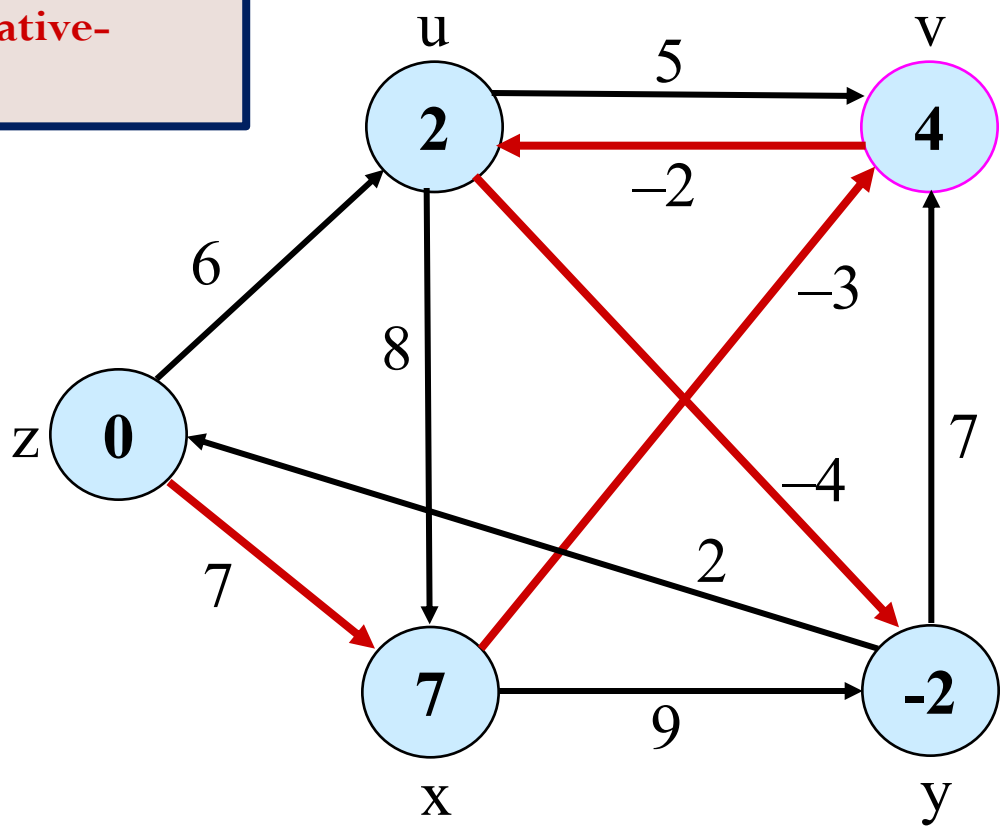
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



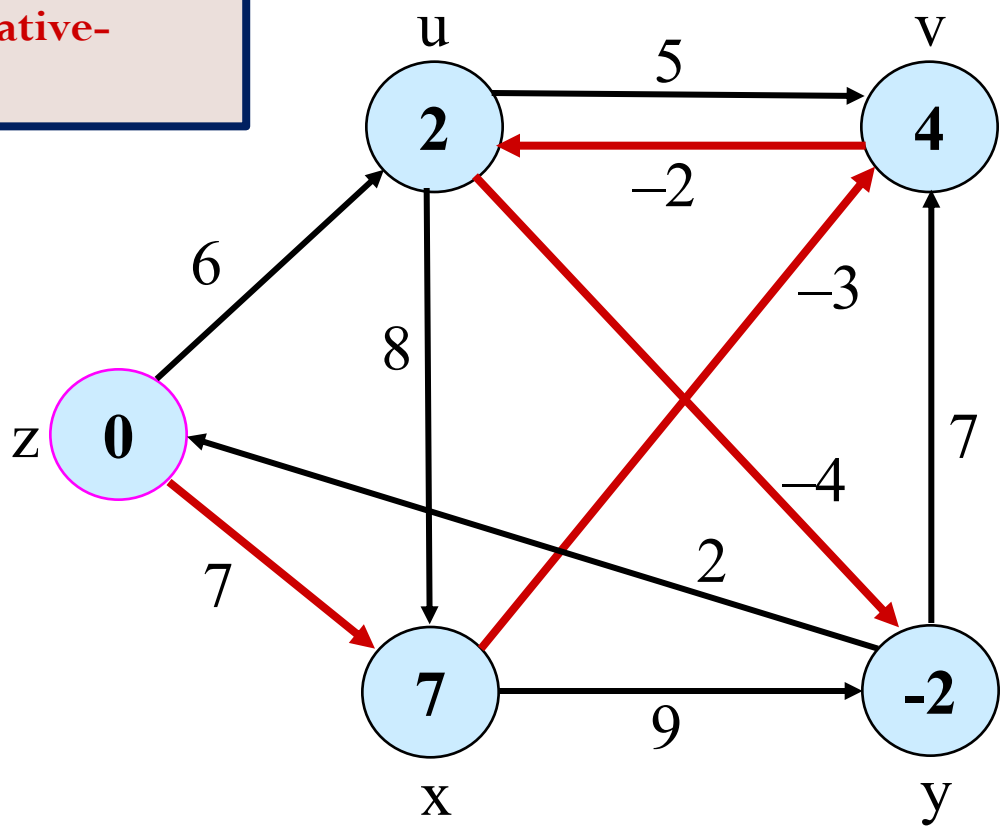
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



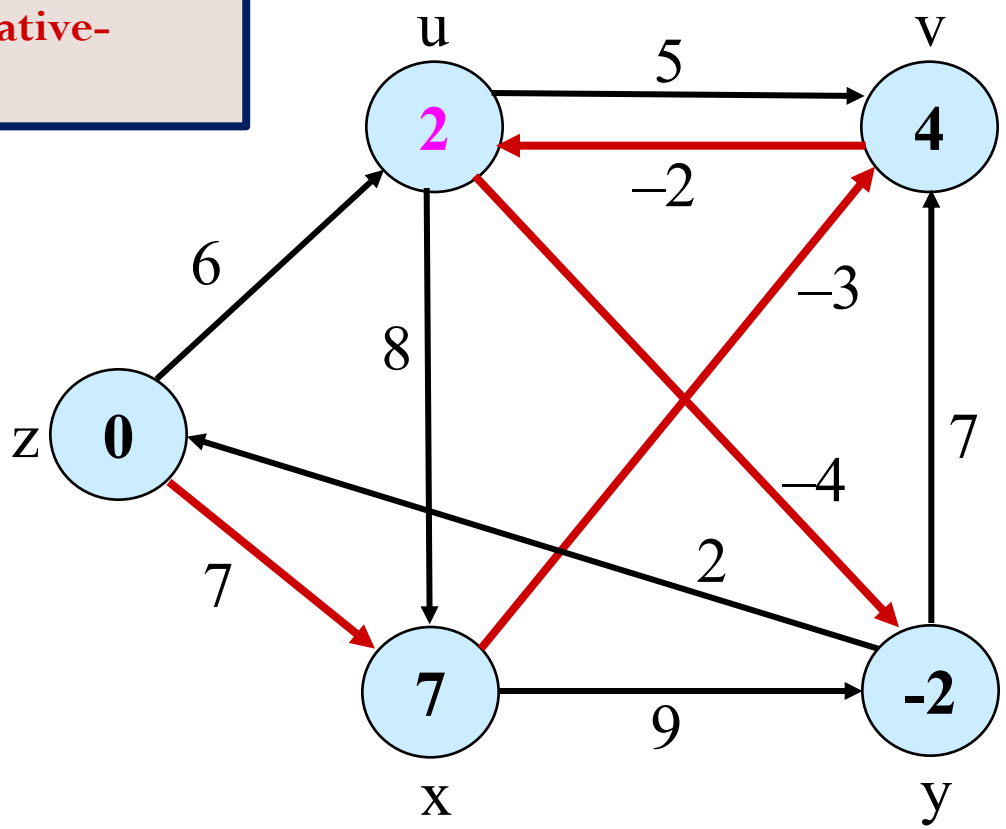
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



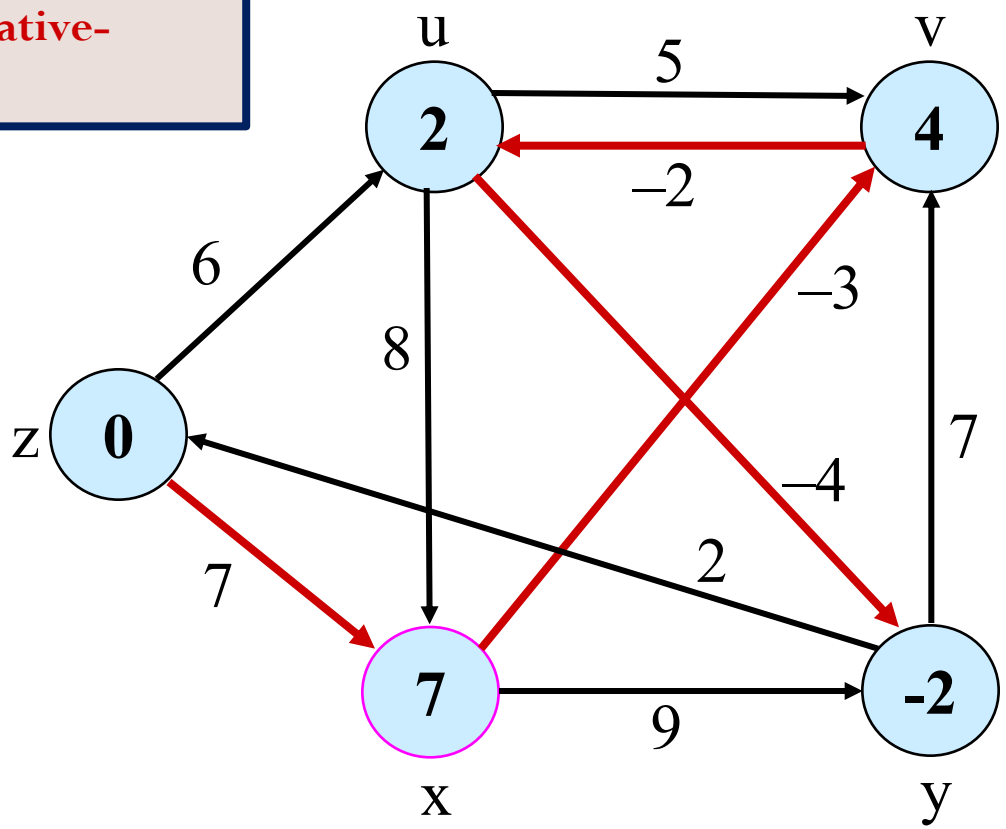
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

"detect" reachable negative-weight cycles.



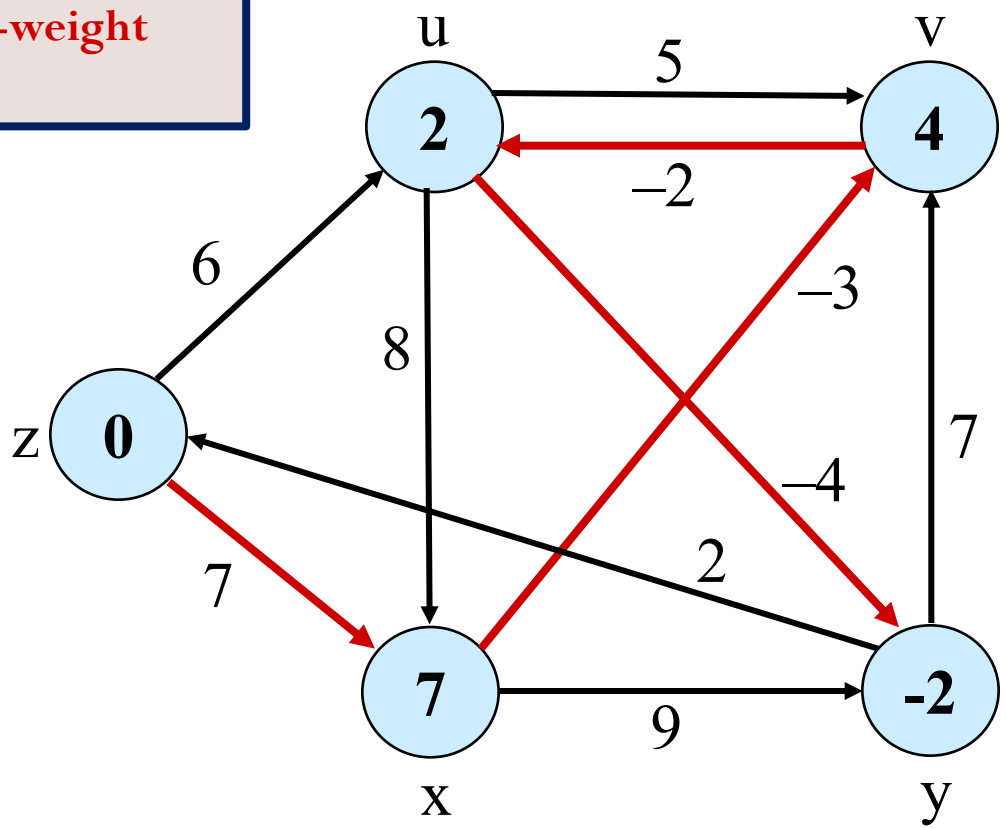
Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman-Ford Algorithm

Example

Order of Edges
(U, V), (U,X), (U,Y), (V,U), (X,V), (X,Y), (Y,V), (Y,Z), (Z,U), (Z,X)

No reachable negative-weight cycles.



Vertex	D[v]	Π[v]
Z	0	Nil
U	2	v
V	4	x
X	7	z
Y	-2	U

Bellman Ford Algorithm Analysis

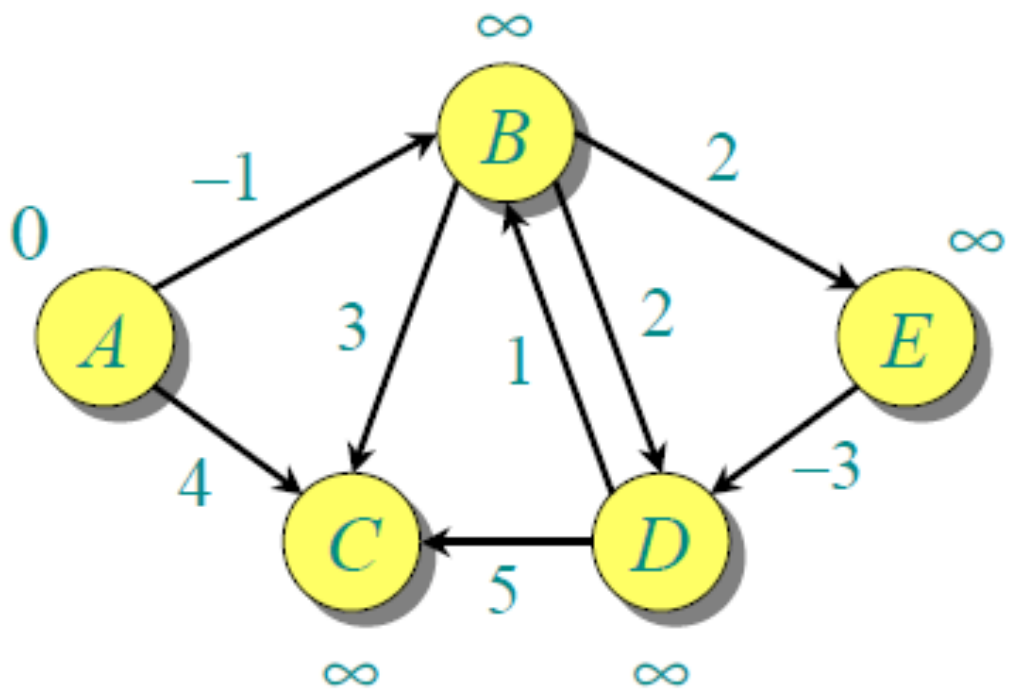
```
d[s] ← 0
for each v ∈ V − {s}                                   } O(V)
    do d[v] ← ∞

for i ← 1 to |V| − 1 do                                                  } O(VE)
   for each edge (u, v) ∈ E do                                           } O(E)
      if d[v] > d[u] + w(u, v) then                                      } O(1)
            d[v] ← d[u] + w(u, v)
            π[v] ← u
for each edge (u, v) ∈ E                                                 } O(E)
    do if d[v] > d[u] + w(u, v)
          then report that a negative-weight cycle exists
```

At the end, $d[v] = \delta(s, v)$. Time = $O(|V|\,|E|)$.

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*



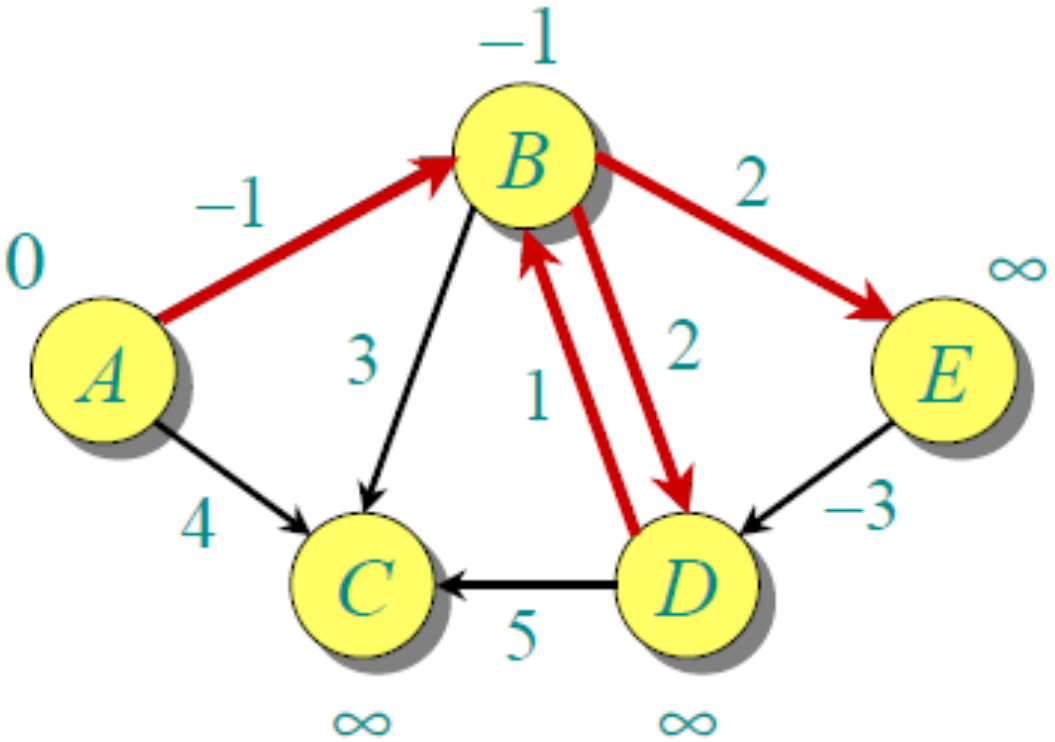
A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞

Repeat *V-1* times: relax all *E* edges.

Initialize

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

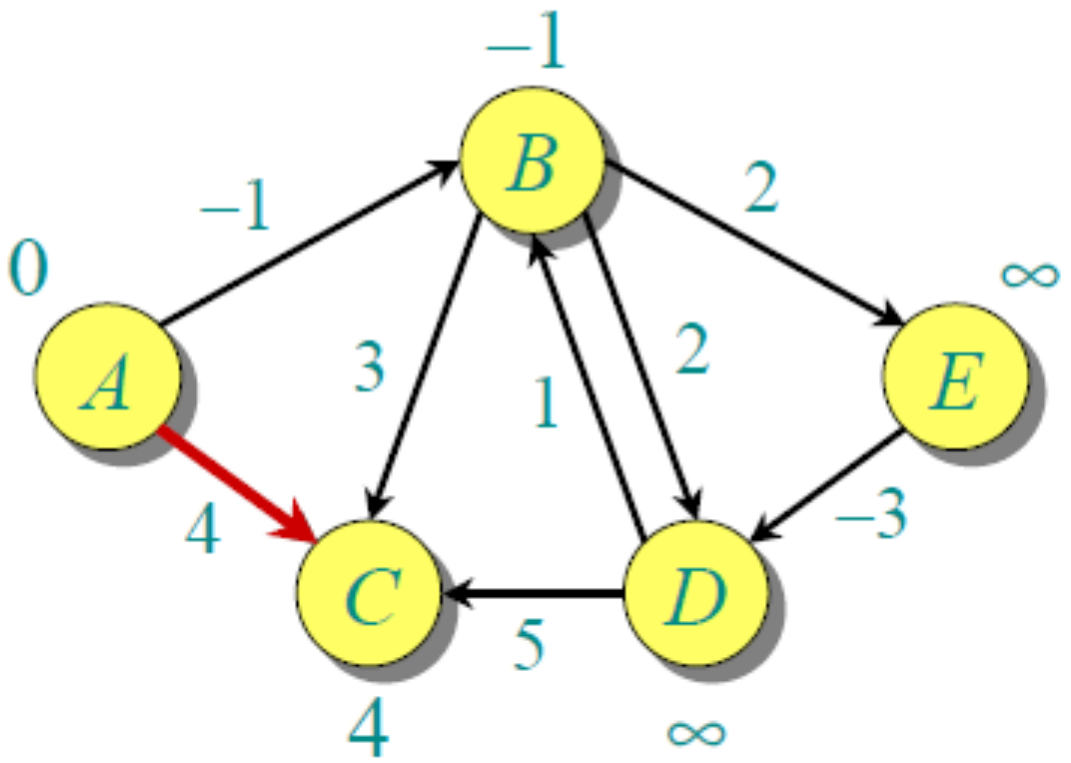


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞

Pass 1

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

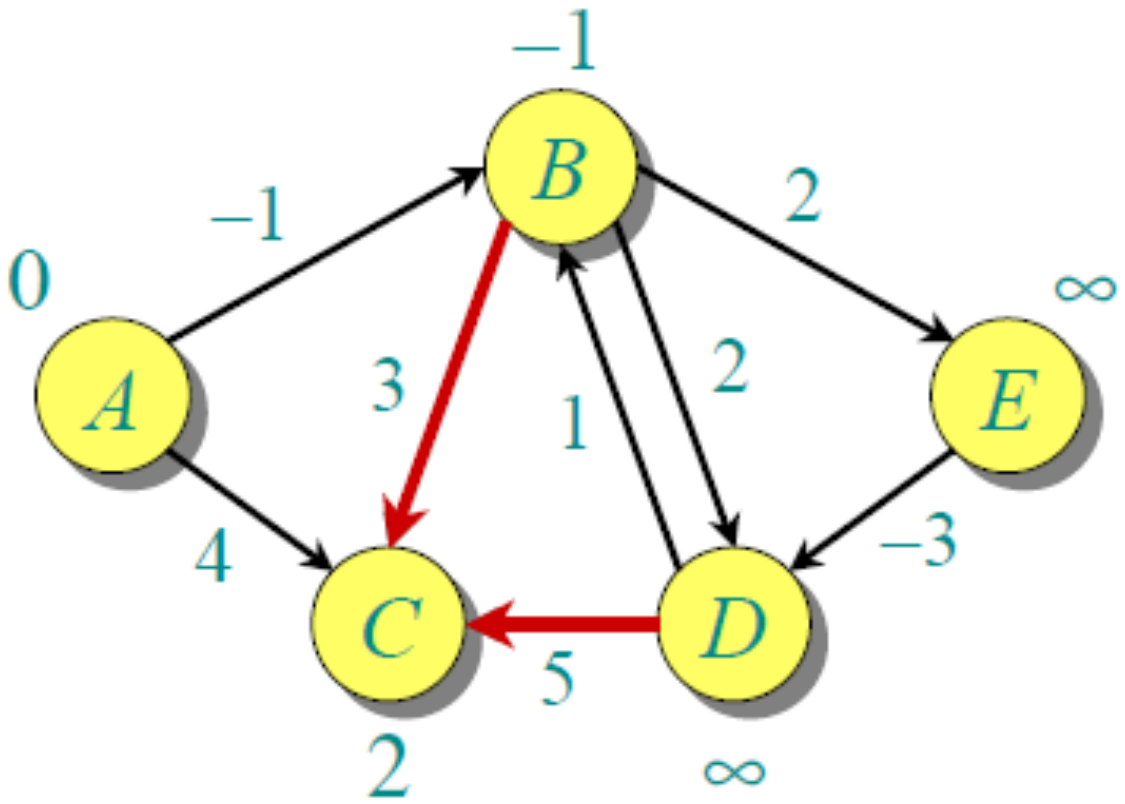


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞

Pass 1

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

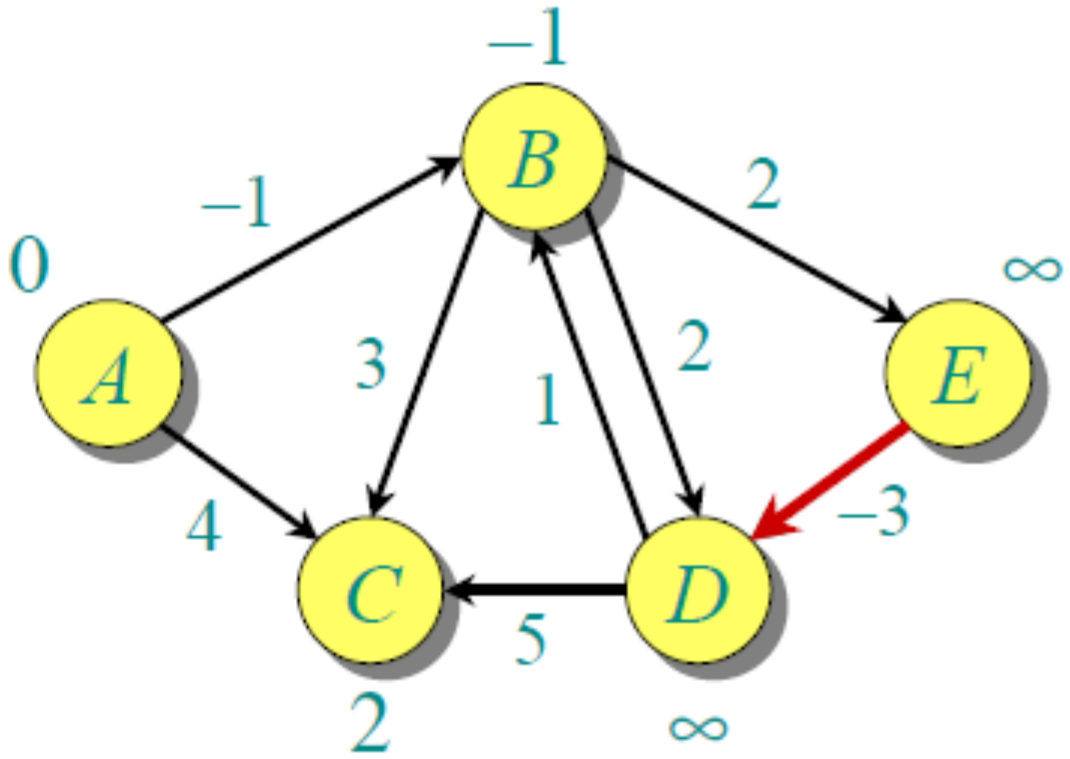


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞

Pass 1

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

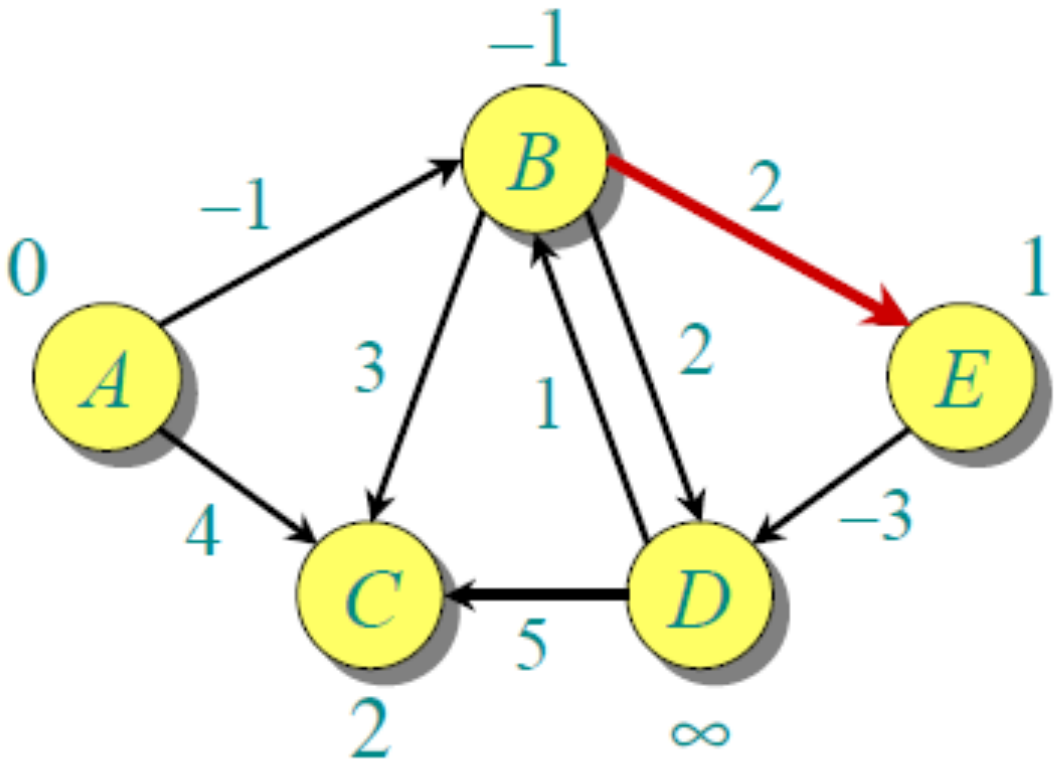


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞

Pass 1

Example of Bellman-Ford

Order of edges: (B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)

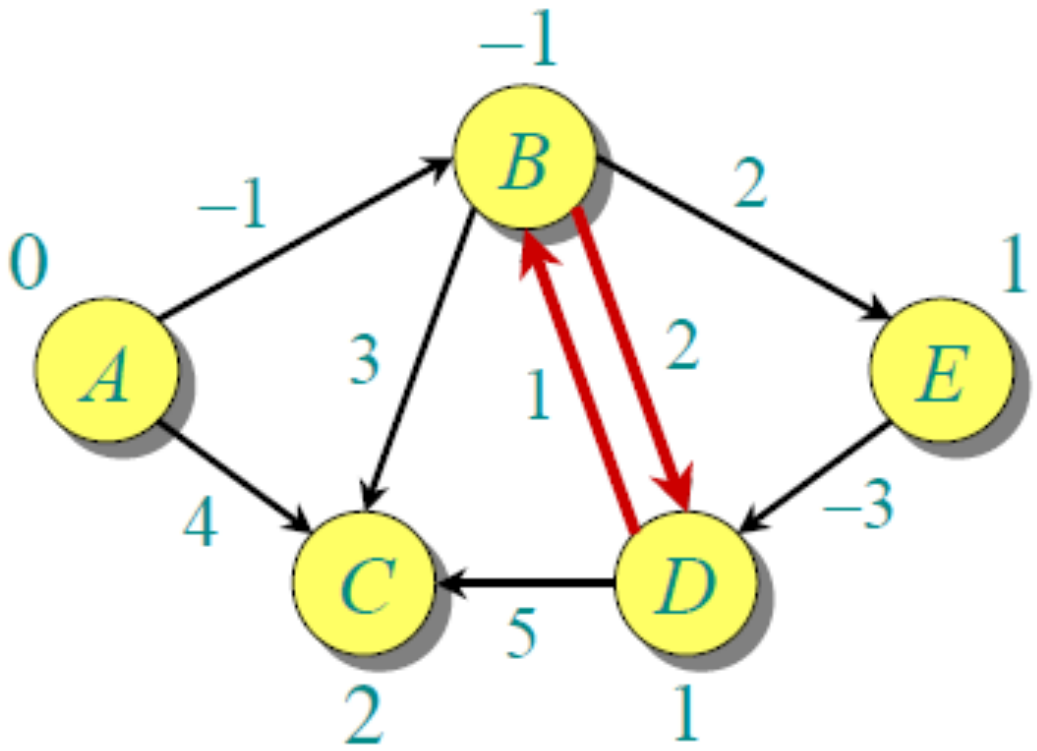


A	B	C	D	E
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1

Pass 2

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

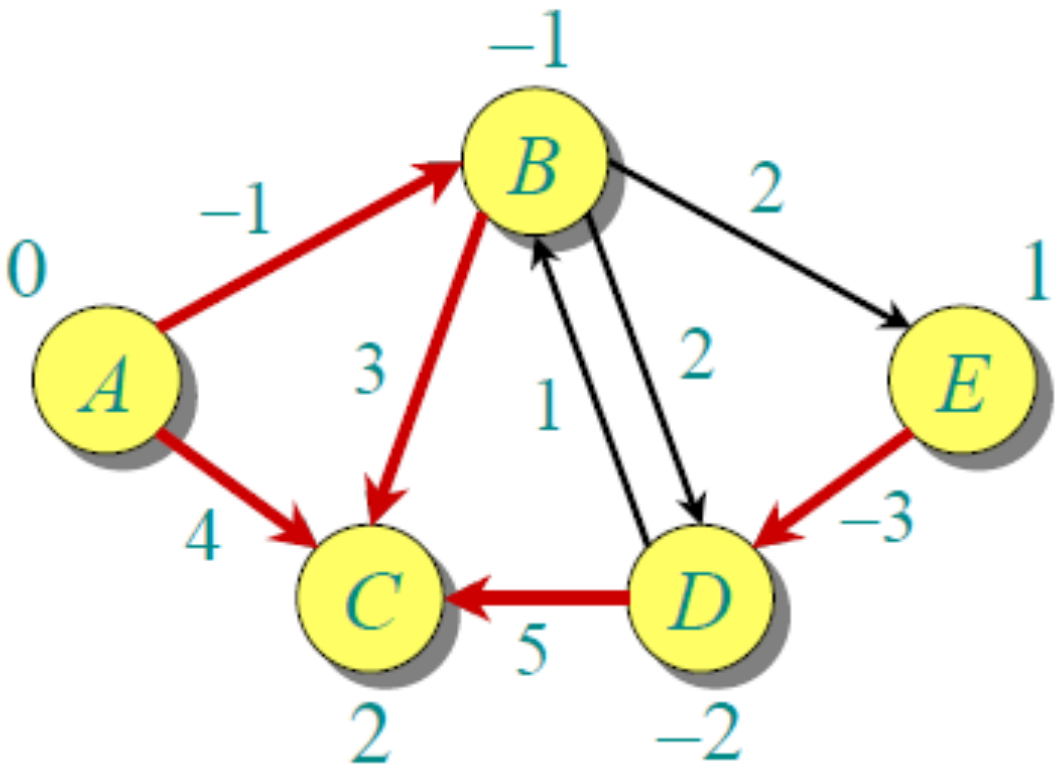


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1
0	−1	2	1	1

Pass 2

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*

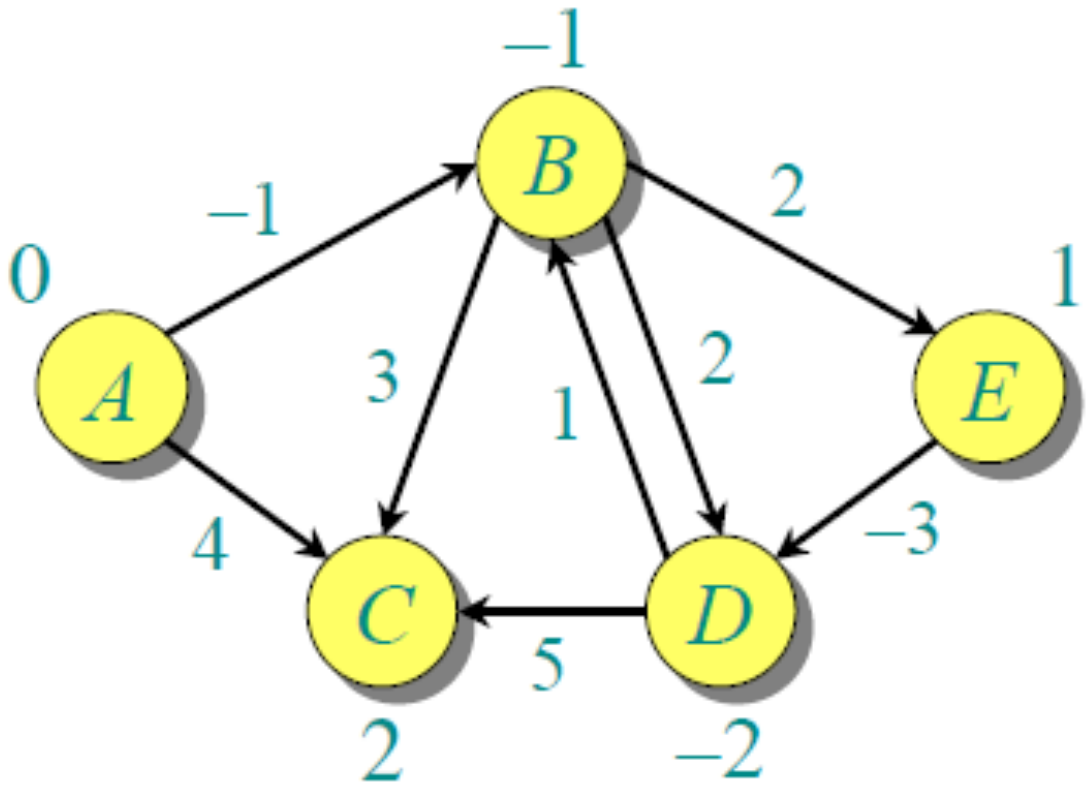


A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1
0	−1	2	1	1
0	−1	2	−2	1

Pass 2

Example of Bellman-Ford

Order of edges: *(B,E), (D,B), (B,D), (A,B), (A,C), (D,C), (B,C), (E,D)*



A	*B*	*C*	*D*	*E*
0	∞	∞	∞	∞
0	−1	∞	∞	∞
0	−1	4	∞	∞
0	−1	2	∞	∞
0	−1	2	∞	1
0	−1	2	1	1
0	−1	2	−2	1

Note: Values decrease monotonically.

Pass 3

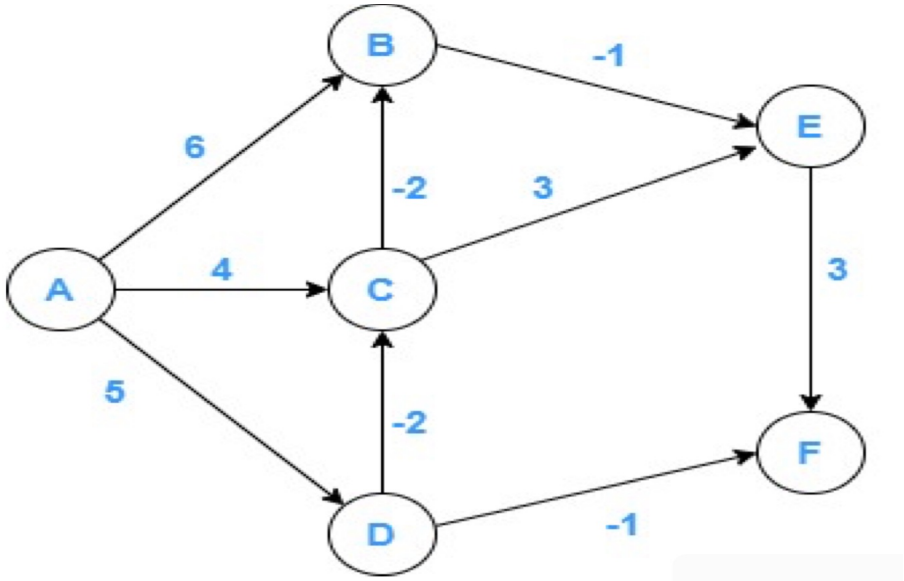
Bellman-Ford Algorithm

Try Yourself

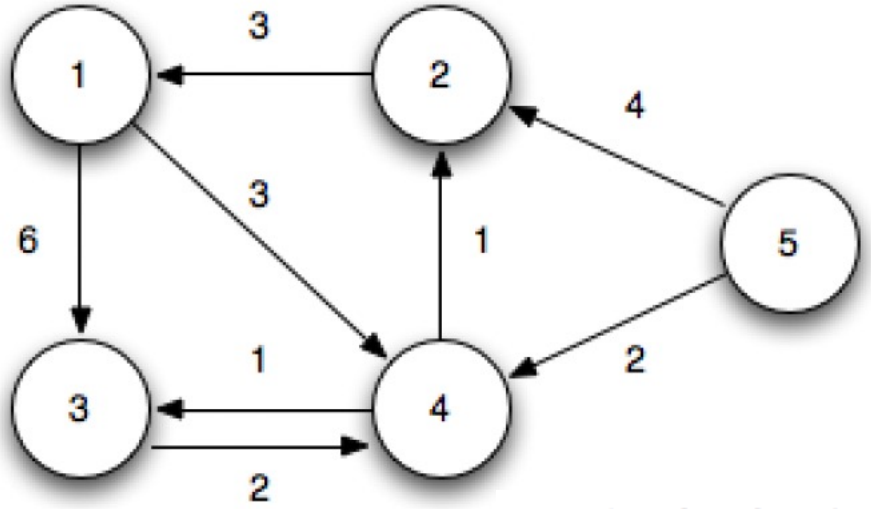
Find the shortest distance from source vertex to other vertex using Bellman-ford algorithm.

Answer

A- A: 0
A- B: 1
A- C: 3
A- D: 5
A- E: 0
A- F: 3



Source = A



Source = 5

	1	2	3	4	5
d	6	3	3	2	0
π	2	4	4	5	/

Advanced Shortest Path Algorithms

1. **Bellman-Ford Algorithm**
2. **Johnson's algorithm**

Shortest Path Algorithms

Johnson's algorithm (All Pair Shortest Path)

- Johnson's algorithm is a all pair shortest path algorithm i,e, it find the shortest path between every pair of vertices in that graph.
- It is similar to the Floyd-Warshall algorithm which is most effective for dense graphs (many edges), while Johnson's algorithm is most effective for sparse graphs (few edges).
- The reason that Johnson's algorithm is better for sparse graphs is that its time complexity depends on the number of edges in the graph. Johnson's algorithm runs in $\mathbf{O(V^2 \cdot \log(V) + |V| \cdot |E|)}$ time. So, if the number of edges is small (i.e. the graph is sparse), it will run faster than the $O(V^3)$ runtime of Floyd-Warshall.
- Johnson's algorithm uses two other shortest path algorithms as subroutines.
  - It uses Bellman-Ford in order to reweight the input graph to eliminate negative edges and detect negative cycles.
  - With the new graph, it then uses Dijkstra's shortest path algorithm to calculate the shortest path between all pairs of vertices.
  - The output of the algorithm is then the set of shortest paths in the original graph.

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Johnson's algorithm has three main steps.

1. A new vertex is added to the graph, and it is connected by edges of zero weight to all other vertices in the graph.
2. All edges go through a reweighting process that eliminates negative weight edges.
3. The added vertex from step 1 is removed and Dijkstra's algorithm is run on every node in the graph.

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting

- Given a Weighted, Directed Graph $G = (V, E)$ with weight function $w : E \rightarrow R$, construct a new Graph $G' = (V', E')$ where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$.
- The nonnegative weights $\hat{w}(u, v)$ are computed as follows:
  1. Extend the weight function w so that $w(s, v) = 0$ for all $v \in V$.
  2. Compute $h(v) = \delta(s, v)$ for all $v \in V'$.
  3. Compute $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$.

$$h(v) \leq h(u) + w(u, v) \quad \text{(Triangle Inequality)} \quad (3.1)$$

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v) \quad \text{(Lemma)} \quad (3.2)$$

(3.1) and (3.2) $\rightarrow \hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$.

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...

1. $w(s, v) = 0$ for all $v \in V$.

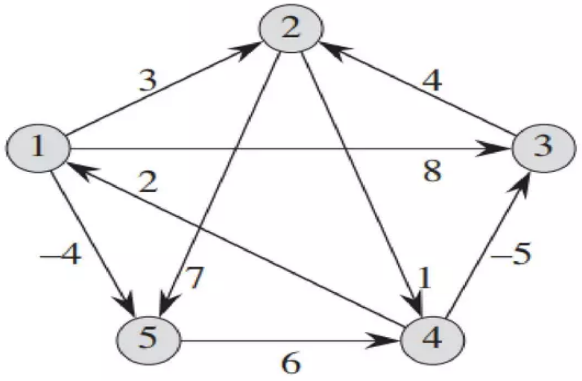


Fig (a): Graph G

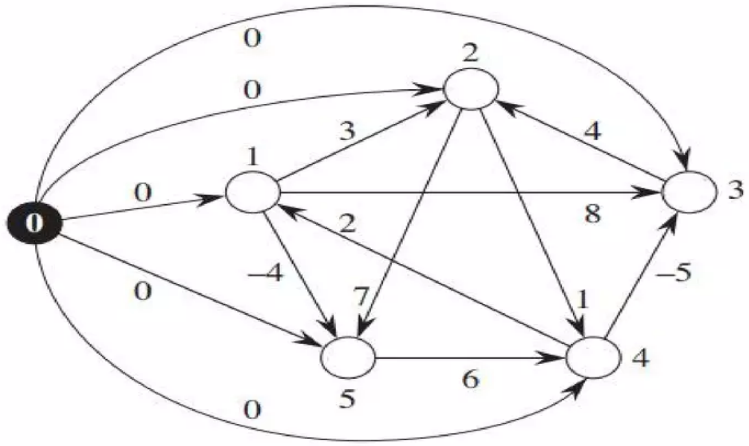


Fig (b): Graph G'

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...

2. $h(v) = \delta(s, v)$ for all $v \in V'$ (J1)

$\delta(s, v)$ is computed using Bellman-Ford Algorithm

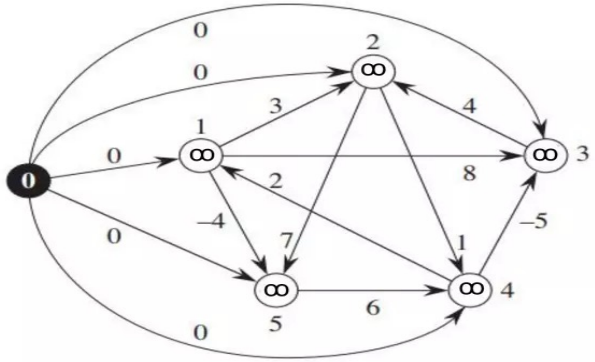
- The edges are relaxed in the following order:

  (0, 1); (0, 2); (0, 3); (0, 4); (0, 5); (1, 2); (1, 3); (1, 5); (2, 4); (2, 5); (3, 2); (4, 1); (4, 3); (5, 4);

- The Bellman-Ford Algorithm runs for 5 passes since there are 6 vertices in the graph G'
- At the end of 5th pass, $\delta(s, v)$, the shortest paths from s to all the vertices $v \in V'$ would be computed if G' does not consist a negative cycle.

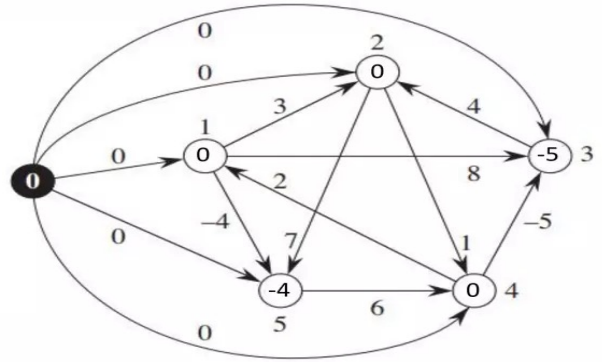
Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...



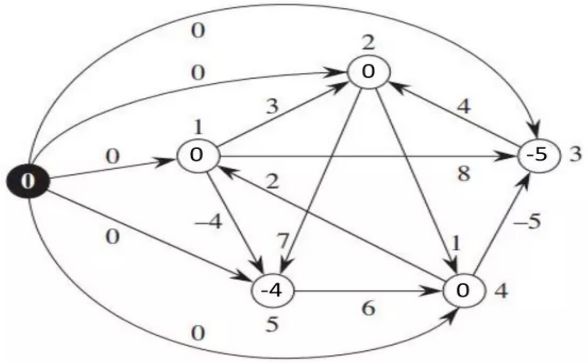
Relax (0, 1): $1.d = \min(1.d, 0.d + w(0, 1)) = (\infty, 0 + 0) = 0$
Relax (0, 2): $2.d = \min(2.d, 0.d + w(0, 2)) = (\infty, 0 + 0) = 0$
Relax (0, 3): $3.d = \min(3.d, 0.d + w(0, 3)) = (\infty, 0 + 0) = 0$
Relax (0, 4): $4.d = \min(4.d, 0.d + w(0, 4)) = (\infty, 0 + 0) = 0$
Relax (0, 5): $5.d = \min(5.d, 0.d + w(0, 5)) = (\infty, 0 + 0) = 0$
Relax (1, 2): $2.d = \min(2.d, 1.d + w(1, 2)) = (0, 0 + 3) = 0$
Relax (1, 3): $3.d = \min(3.d, 1.d + w(1, 3)) = (0, 0 + 8) = 0$

Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (0, 0 + (-4)) = -4$
Relax (2, 4): $4.d = \min(4.d, 2.d + w(2, 4)) = (0, 0 + 1) = 0$
Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (-4, 0 + 7) = -4$
Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (0, 0 + 4) = 0$
Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (0, 0 + 2) = 0$
Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (0, 0 + -5) = -5$
Relax (5, 4): $4.d = \min(4.d, 5.d + w(5, 4)) = (0, (-4) + 6) = 0$



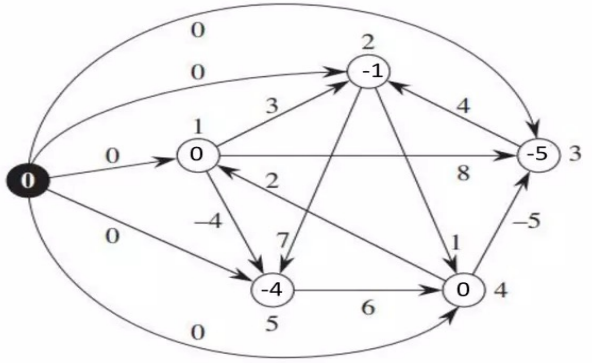
Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...



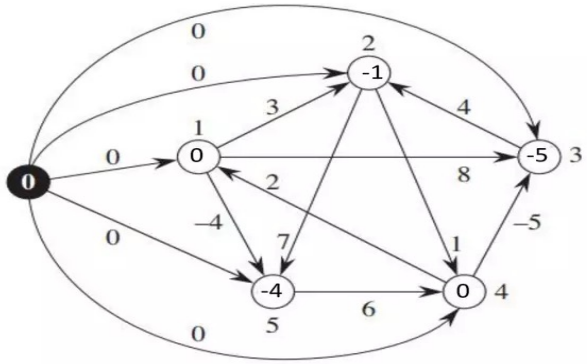
Relax (0, 1): $1.d = \min(1.d, 0.d + w(0, 1)) = (0, 0 + 0) = 0$
Relax (0, 2): $2.d = \min(2.d, 0.d + w(0, 2)) = (0, 0 + 0) = 0$
Relax (0, 3): $3.d = \min(3.d, 0.d + w(0, 3)) = (-5, 0 + 0) = -5$
Relax (0, 4): $4.d = \min(4.d, 0.d + w(0, 4)) = (0, 0 + 0) = 0$
Relax (0, 5): $5.d = \min(5.d, 0.d + w(0, 5)) = (-4, 0 + 0) = -4$
Relax (1, 2): $2.d = \min(2.d, 1.d + w(1, 2)) = (0, 0 + 3) = 0$
Relax (1, 3): $3.d = \min(3.d, 1.d + w(1, 3)) = (-5, 0 + 8) = -5$

Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (-4, 0 + (-4)) = -4$
Relax (2, 4): $4.d = \min(4.d, 2.d + w(2, 4)) = (0, 0 + 1) = 0$
Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (-4, 0 + 7) = -4$
Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (0, (-5) + 4) = -1$
Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (0, 0 + 2) = 0$
Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (-5, 0 + -5) = -5$
Relax (5, 4): $4.d = \min(4.d, 5.d + w(5, 4)) = (0, (-4) + 6) = 0$



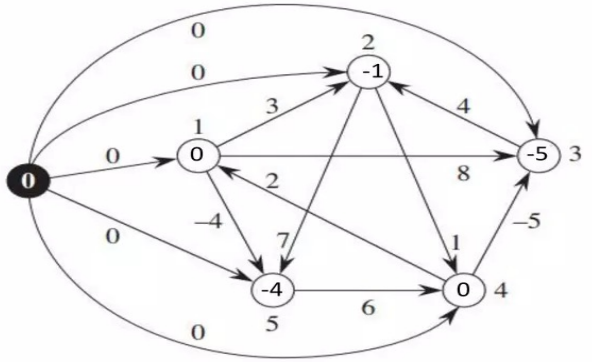
Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...



Relax (0, 1): $1.d = \min(1.d, 0.d + w(0, 1)) = (0, 0 + 0) = 0$
Relax (0, 2): $2.d = \min(2.d, 0.d + w(0, 2)) = (-1, 0 + 0) = -1$
Relax (0, 3): $3.d = \min(3.d, 0.d + w(0, 3)) = (-5, 0 + 0) = -5$
Relax (0, 4): $4.d = \min(4.d, 0.d + w(0, 4)) = (0, 0 + 0) = 0$
Relax (0, 5): $5.d = \min(5.d, 0.d + w(0, 5)) = (-4, 0 + 0) = -4$
Relax (1, 2): $2.d = \min(2.d, 1.d + w(1, 2)) = (-1, 0 + 3) = 0$
Relax (1, 3): $3.d = \min(3.d, 1.d + w(1, 3)) = (-5, 0 + 8) = -5$

Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (-4, 0 + (-4)) = -4$
Relax (2, 4): $4.d = \min(4.d, 2.d + w(2, 4)) = (-1, 0 + 1) = -1$
Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (-4, (-1) + 7) = -4$
Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (-1, (-5) + 4) = -1$
Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (0, 0 + 2) = 0$
Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (-5, 0 + (-5)) = -5$
Relax (5, 4): $4.d = \min(4.d, 5.d + w(5, 4)) = (0, (-4) + 6) = 0$



Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting…

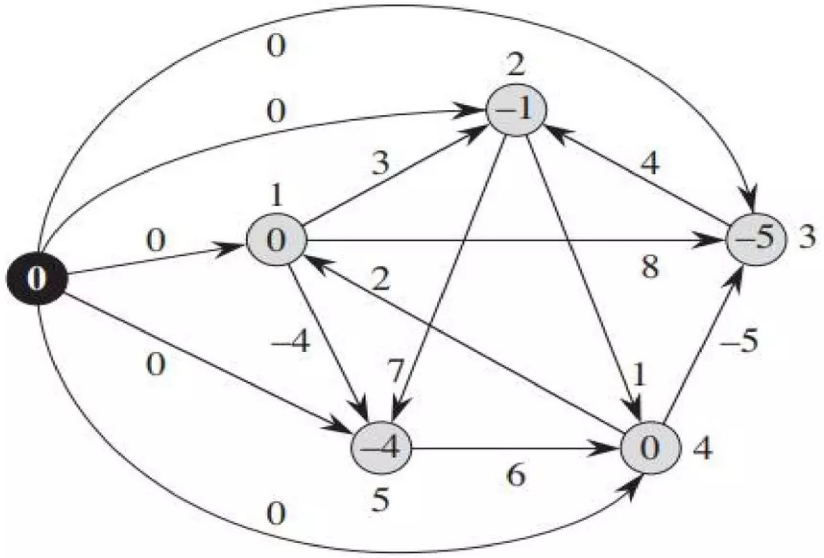


Fig (*c*): *h* (*v*) computed for *G* ′ of Fig (*b*)

Vertex, *v*	*h* (*v*)
1	0
2	-1
3	-5
4	0
5	-4

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Producing Nonnegative Weights by Reweighting...

3. $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$

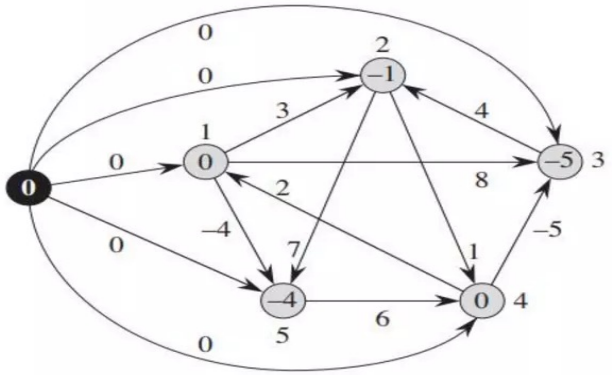


Fig (c): $h(v)$ computed for G'

$\hat{w}(5,4) = w(5,4) + h(5) - h(4)=6-4-0=2$
$\hat{w}(2,4) = w(2,4) + h(2) - h(4)=1-1-0=0$
$\hat{w}(4,3) = w(4,3) + h(4) - h(3)=-5+0+5=0$
$\hat{w}(3,2) = w(3,2) + h(3) - h(2)=4-5+1=0$

$\hat{w}(1,2) = w(1,2) + h(1) - h(2)=3+0+1=4$
$\hat{w}(1,3) = w(1,3) + h(1) - h(3)=8+0+5=13$
$\hat{w}(4,1) = w(4,1) + h(4) - h(1)=2+0-0=2$
$\hat{w}(1,5) = w(1,5) + h(1) - h(5)=-4+0+4=0$
$\hat{w}(2,5) = w(2,5) + h(2) - h(5)=7-1+4=10$

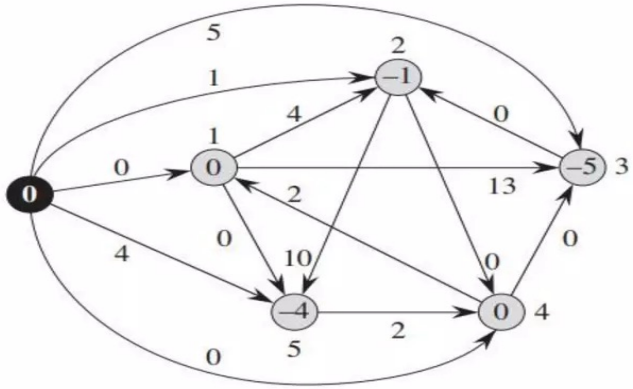


Fig (d): G' with Reweighted edges

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Computing All-Pairs Shortest Path

- Dijkstra's algorithm is used to find the shortest paths.
- The algorithm is run once with each vertex $v \in V$ as source.
- It takes the reweighted graph G' as input and computes $\hat{\delta}$ for all the vertex $v \in V$.
- Finally $\delta(u, v)$ is computed as: $\delta(u, v) = \hat{\delta}(u, v) + h(v) - h(u)$
- After V - 1 passes the shortest paths between all pairs of vertices would be computed.

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Algorithm

- Johnson's algorithm uses as subroutines both the Bellman-Ford algorithm and the Dijkstra's algorithm.
- At first, it extends the given weighted directed graph G to produce graph G'.
- Runs Bellman-Ford algorithm to compute $h(v)$ if the graph G' does not contain negative cycles.
- Reweights the graph G' to get reweighted edges E'.
- Runs Dijkstra's algorithm $V - 1$ times to get All-Pairs Shortest Path of G', and correspondingly computes the shortest paths of the graph G using: $\delta(u, v) = \hat{\delta}(u, v) + h(v) - h(u)$
- It returns a $|V| \times |V|$ matrix, $D = d_{uv}$, where $d_{uv} = \delta(u, v)$.

Running Time:

- If the min-priority queue in Dijkstra's algorithm is implemented by a Fibonacci heap, Johnson's algorithm runs in $O(V^2 \lg V + VE)$ time.
- Binary min heap implementation yields a running time of $O(VE \lg V)$.

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Algorithm...

JOHNSON (G, w)
Compute G', where $G'.V = G.V \cup \{s\}$, $G'.E = G.E \cup \{(s, v): v \in G.V\}$ and $w(s, v) = 0$ for all $v \in G.V$
If (BELLMAN-FORD (G', w, s) == *FALSE*)
    print "Graph contains Negative-Weight Cycle"
Else For (Each Vertex $v \in G'.V$)
    Set $h(v)$ to the value of $\delta(s, v)$ computed by BELLMAN-FORD algorithm
    For each edge $(u, v) \in G'.E$
        $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$
    Let $D = (d_{uv})$ be a new n x n matrix
    For (Each Vertex $u \in G.V$)
        Run DIJKSTRA (G, $\hat{w}$, u) to compute $\hat{\delta}(u, v)$ for all $v \in G.V$
        For (Each Vertex $v \in G.V$)
            $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$
    Return D

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

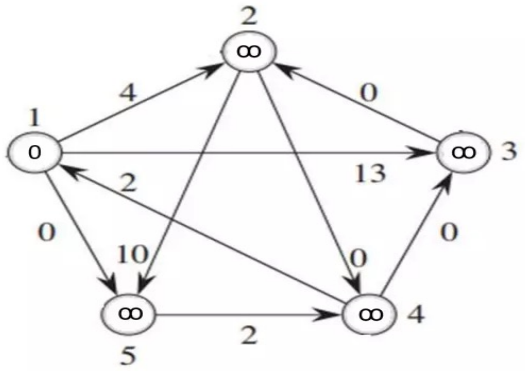


Fig (*e*): Reweighted Graph

Example

$\hat{\delta}$ (*u*, *v*) from Source 1

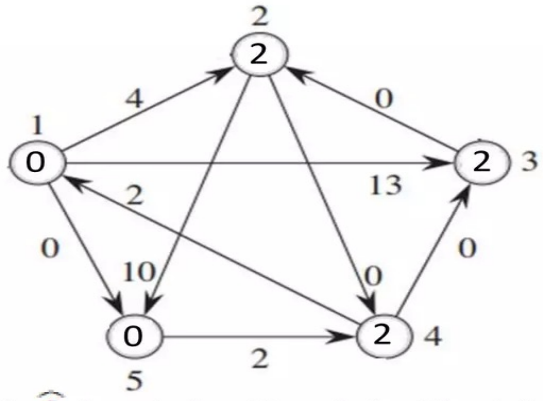


Fig (*f*): $\hat{\delta}$ (*u*, *v*) for Graph in Fig (*e*) from Source 1using Dijkstra's Algorithm

S	*V*	Relax (*u*, *v*, *w*)	*Q*	*u*	Cost
			1,2,3,4,5	1	0
1	2 3 5	Relax (1,2): 2.*d* = min (2.*d*, 1.*d* + *w* (1, 2)) = (∞, 0 + 4) = 4 Relax (1,3): 3.*d* = min (3.*d*, 1.*d* + *w* (1, 3)) = (∞, 0 + 13) = 13 Relax (1,5): 5.*d* = min (5.*d*, 1.*d* + *w* (1, 5)) = (∞, 0 + 0) = 0	2,3,4,5	5	0
1,5	4	Relax (5,4): 4.*d* = min (4.*d*, 5.*d* + *w* (5, 4)) = (∞, 0 + 2) = 2	2,3,4	4	2
1,5,4	3	Relax (4,3): 3.*d* = min (3.*d*, 4.*d* + *w* (4, 3)) = (13, 2 + 0) = 2	2,3	3	2
1,5,4,3	2	Relax (3,2): 2.*d* = min (2.*d*, 3.*d* + *w* (3, 2)) = (4, 2 + 0) = 2	2	2	2

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...

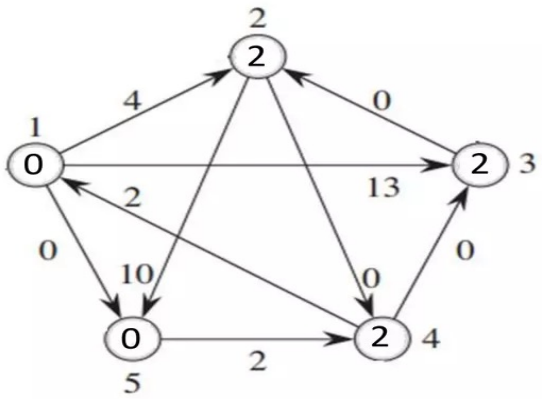


Fig (*f*): $\hat{\delta}(u, v)$ from Source 1

$\delta(u, v)$ from Source 1

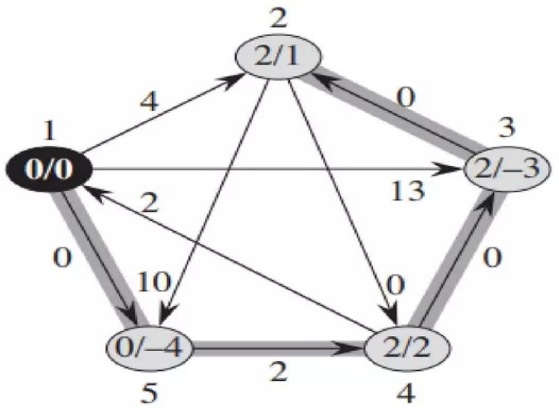


Fig (*g*): $\delta(u, v)$ of Graph in Fig (*f*) from source 1 using $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$
($\hat{\delta}$ (u,v), δ (u,v)) – Values within each vertex.

$$\delta(1, 2) = \hat{\delta}(1,2) + h(2) - h(1) = 2 - 1 - 0 = 1$$
$$\delta(1, 3) = \hat{\delta}(1,3) + h(3) - h(1) = 2 - 5 - 0 = -3$$
$$\delta(1, 4) = \hat{\delta}(1,4) + h(4) - h(1) = 2 + 0 - 0 = 2$$
$$\delta(1, 5) = \hat{\delta}(1,5) + h(5) - h(1) = 0 - 4 - 0 = -4$$

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

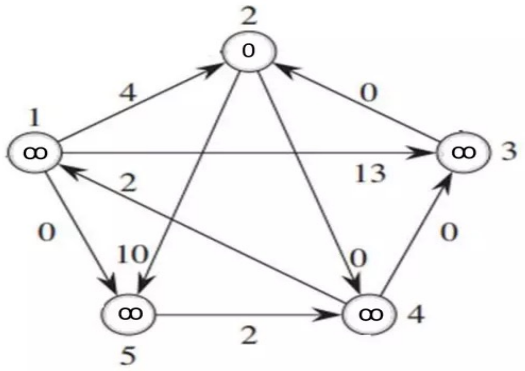


Fig (*e*): Reweighted Graph

$\hat{\delta}$ (*u*, *v*) from Source 2

Example...

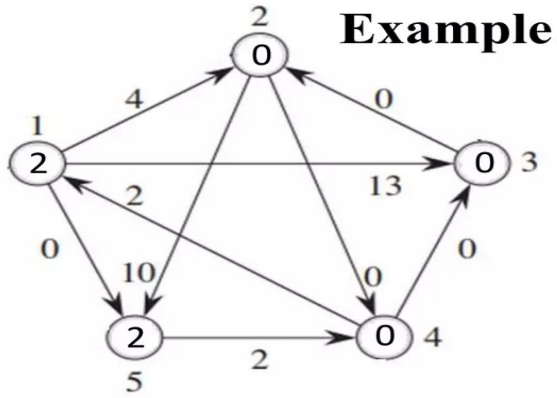


Fig (*h*): $\hat{\delta}$ (*u*, *v*) for Graph in Fig (*e*) from Source 2 using Dijkstra's Algorithm

S	*V*	Relax (*u*, *v*, *w*)	*Q*	*u*	Cost
			1,2,3,4,5	2	0
2	4 5	Relax (2, 4): $4.d = \min(4.d, 2.d + w(2, 4)) = (\infty, 0 + 0) = 0$ Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (\infty, 0 + 10) = 10$	1,3,4,5	4	0
2,4	1 3	Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (\infty, 0 + 2) = 2$ Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (\infty, 0 + 0) = 0$	1,3,5	3	0
2,4,3	-		1,5	1	2
2,4,3,1	5	Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (10, 2 + 0) = 2$	5	5	2

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...

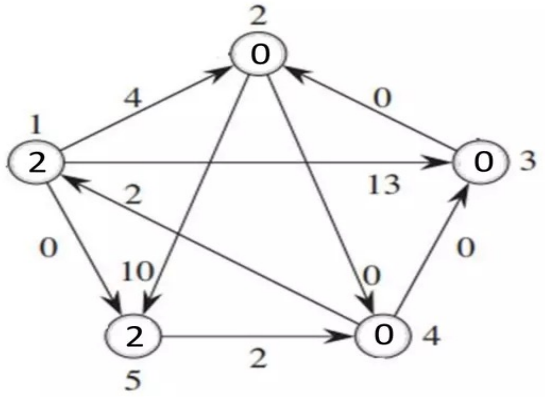


Fig (*h*): $\hat{\delta}$ (*u*, *v*) from Source 2

δ (*u*, *v*) from Source 2

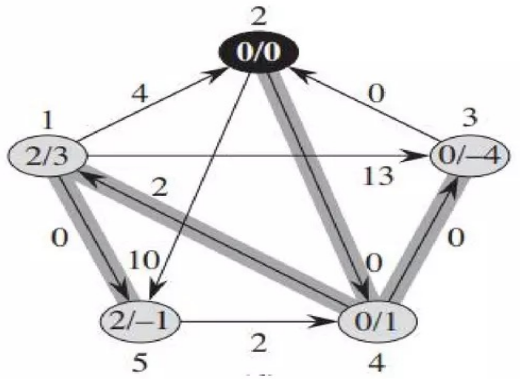


Fig (*i*): δ (*u*, *v*) of Graph in Fig (*f*) from source 2 using $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$
($\hat{\delta}$ (u,v), δ (u,v)) – Values within each vertex.

$$\delta(2, 1) = \hat{\delta}(2, 1) + h(1) - h(2) = 2 + 0 + 1 = 3$$
$$\delta(2, 3) = \hat{\delta}(2, 3) + h(3) - h(2) = 0 - 5 + 1 = -4$$
$$\delta(2, 4) = \hat{\delta}(2, 4) + h(4) - h(2) = 0 + 0 + 1 = 1$$
$$\delta(2, 5) = \hat{\delta}(2, 5) + h(5) - h(2) = 2 - 4 + 1 = -1$$

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

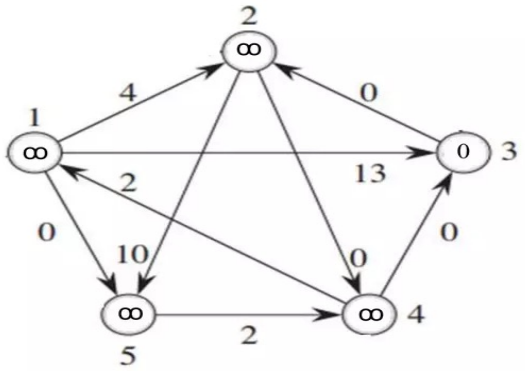


Fig (*e*): Reweighted Graph

$\widehat{\delta}$ (*u*, *v*) from Source 3

Example...

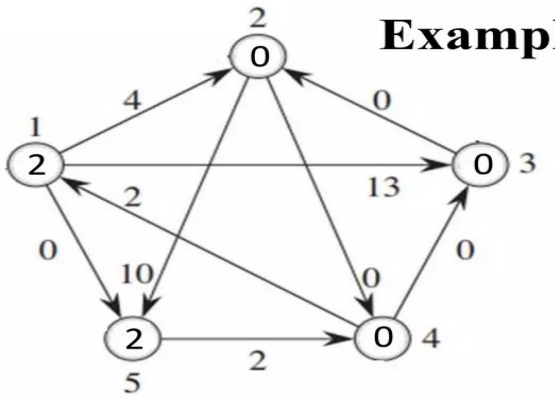
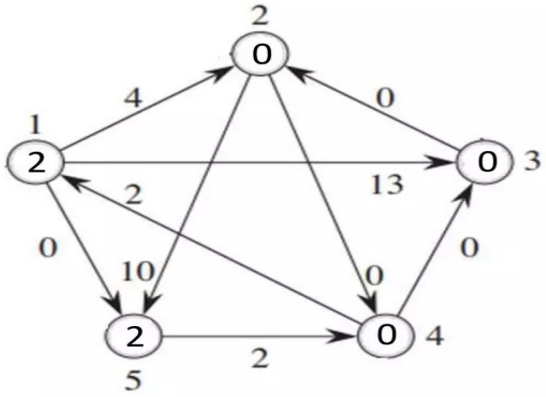


Fig (*j*): $\widehat{\delta}$ (*u*, *v*) for Graph in Fig (*e*) from Source 3 using Dijkstra's Algorithm

S	*V*	Relax (*u*, *v*, *w*)	*Q*	*u*	Cost
			1,2,3,4,5	3	0
3	2	Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (\infty, 0 + 0) = 0$	1,2,4,5	2	0
3,2	4 5	Relax (2, 4): $4.d = \min(4.d, 2.d + w(2, 4)) = (\infty, 0 + 0) = 0$ Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (\infty, 0 + 10) = 10$	1,4,5	4	0
3,2,4	1	Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (\infty, 0 + 2) = 2$	1,5	1	2
3,2,4,1	5	Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (10, 2 + 0) = 2$	5	5	2

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...



$\delta\,(u, v)$ from Source 3

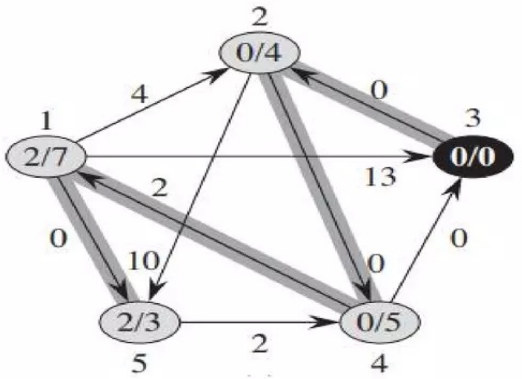


Fig (*j*): $\hat{\delta}\,(u, v)$ from Source 3

Fig (*k*): $\delta\,(u, v)$ of Graph in Fig (*f*) from source 3 using $d_{uv} = \hat{\delta}\,(u, v) + h\,(v) - h\,(u)$
($\hat{\delta}$ (u,v), δ (u,v)) – Values within each vertex.

$$\delta\,(3, 1) = \hat{\delta}\,(3, 1) + h\,(1) - h\,(3) = 2 + 0 + 5 = 7$$
$$\delta\,(3, 2) = \hat{\delta}\,(3, 2) + h\,(2) - h\,(3) = 0 - 1 + 5 = 4$$
$$\delta\,(3, 4) = \hat{\delta}\,(3, 4) + h\,(4) - h\,(3) = 0 + 0 + 5 = 5$$
$$\delta\,(3, 5) = \hat{\delta}\,(3, 5) + h\,(5) - h\,(3) = 2 - 4 + 5 = 3$$

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

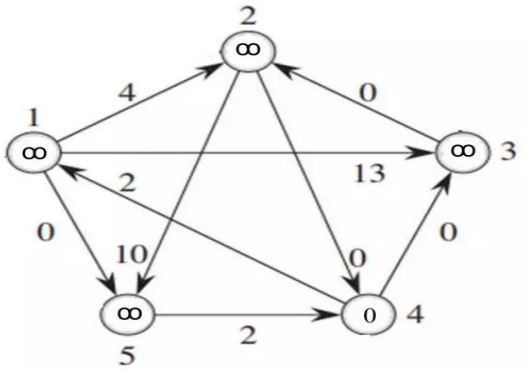


Fig (*e*): Reweighted Graph

$\hat{\delta}$ (*u*, *v*) from Source 4

Example...

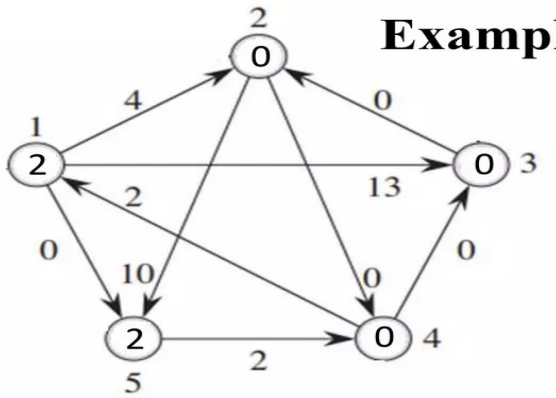
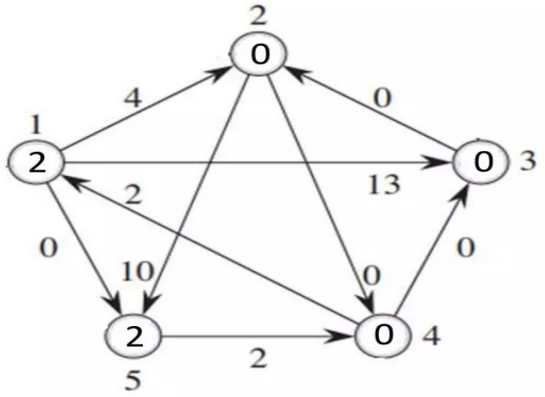


Fig (*l*): $\hat{\delta}$ (*u*, *v*) for Graph in Fig (*e*) from Source 4 using Dijkstra's Algorithm

S	*V*	Relax (*u*, *v*, *w*)	*Q*	*u*	Cost
			1,2,3,4,5	4	0
4	1 3	Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (\infty, 0 + 2) = 2$ Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (\infty, 0 + 0) = 0$	1,2,3,5	3	0
4,3	2	Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (\infty, 0 + 0) = 0$	1,2,5	2	0
4,3,2	5	Relax (2, 5): $5.d = \min(5.d, 2.d + w(2, 5)) = (\infty, 0 + 10) = 10$	1,5	1	2
4,3,2,1	5	Relax (1, 5): $5.d = \min(5.d, 1.d + w(1, 5)) = (10, 2 + 0) = 2$	5	5	2

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...



$\delta(u, v)$ from Source 4

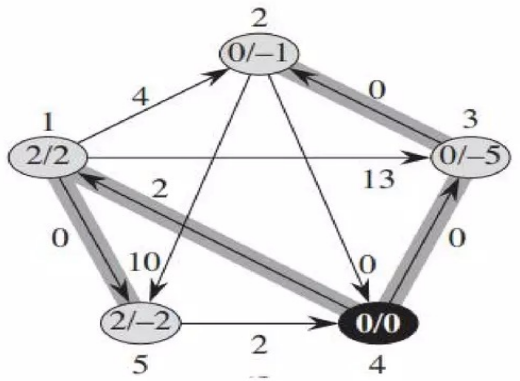


Fig (*l*): $\widehat{\delta}(u, v)$ from Source 4

Fig (*m*): $\delta(u, v)$ of Graph in Fig (*f*) from source 4 using $d_{uv} = \widehat{\delta}(u, v) + h(v) - h(u)$
($\widehat{\delta}$ (u,v), δ (u,v)) – Values within each vertex.

$$\delta(4, 1) = \widehat{\delta}(4, 1) + h(1) - h(4) = 2 + 0 - 0 = 2$$
$$\delta(4, 2) = \widehat{\delta}(4, 2) + h(2) - h(4) = 0 - 1 - 0 = -1$$
$$\delta(4, 3) = \widehat{\delta}(4, 3) + h(3) - h(4) = 0 - 5 - 0 = -5$$
$$\delta(4, 5) = \widehat{\delta}(4, 5) + h(5) - h(4) = 2 - 4 - 0 = -2$$

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

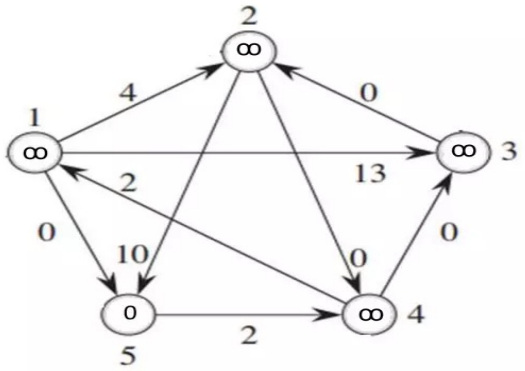


Fig (*e*): Reweighted Graph

$\hat{\delta}$ (*u*, *v*) from Source 5

Example...

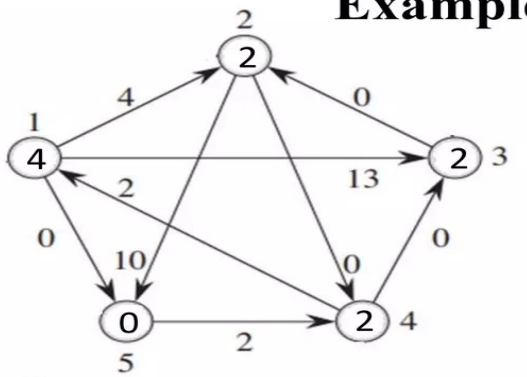
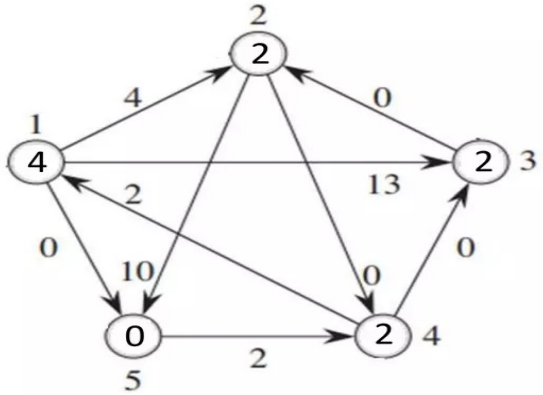


Fig (*n*): $\hat{\delta}$ (*u*, *v*) for Graph in Fig (*e*) from Source 5 using Dijkstra's Algorithm

S	*V*	Relax (*u*, *v*, *w*)	*Q*	*u*	Cost
			1,2,3,4,5	5	0
5	4	Relax (5, 4): $4.d = \min(4.d, 5.d + w(5, 4)) = (\infty, 0 + 2) = 2$	1,2,3,4	4	2
5,4	1 3	Relax (4, 1): $1.d = \min(1.d, 4.d + w(4, 1)) = (\infty, 2 + 2) = 4$ Relax (4, 3): $3.d = \min(3.d, 4.d + w(4, 3)) = (\infty, 2 + 0) = 2$	1,2,3	3	2
5,4,3	2	Relax (3, 2): $2.d = \min(2.d, 3.d + w(3, 2)) = (\infty, 2 + 0) = 2$	1,2	2	2
5,4,3,2	-		1	1	4

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...



$\delta(u, v)$ from Source 5

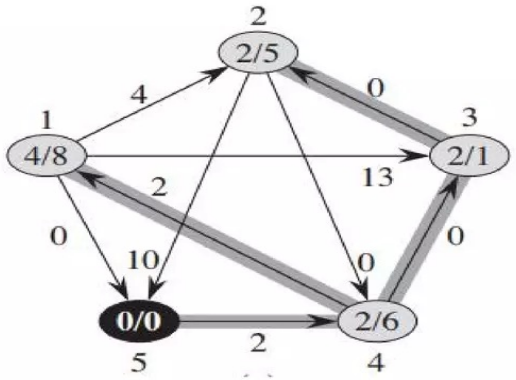


Fig (*n*): $\widehat{\delta}(u, v)$ from Source 5

Fig (*o*): $\delta(u, v)$ of Graph in Fig (*f*) from source 5 using $d_{uv} = \widehat{\delta}(u, v) + h(v) - h(u)$
($\widehat{\delta}$ (u,v), δ (u,v)) – Values within each vertex.

$$\delta(5, 1) = \widehat{\delta}(5, 1) + h(1) - h(5) = 4 + 0 + 4 = 8$$
$$\delta(5, 2) = \widehat{\delta}(5, 2) + h(2) - h(5) = 2 - 1 + 4 = 5$$
$$\delta(5, 3) = \widehat{\delta}(5, 3) + h(3) - h(5) = 2 - 5 + 4 = 1$$
$$\delta(5, 4) = \widehat{\delta}(5, 4) + h(4) - h(5) = 2 + 0 + 4 = 6$$

Johnson's algorithm (All Pair Shortest Path for Sparse Graph)

Example...

Output:

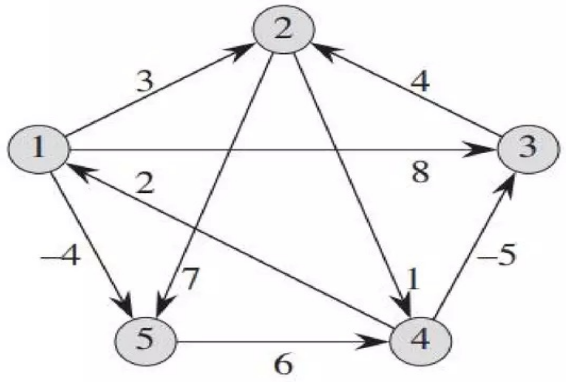


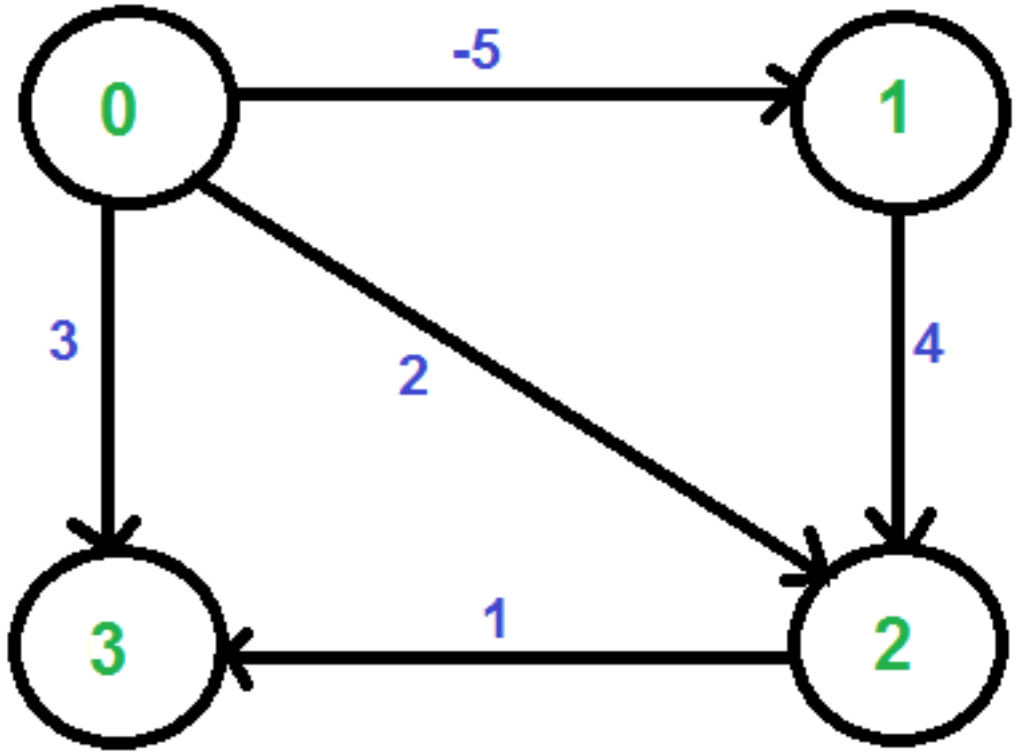
Fig (a): Input Graph G

All-Pairs Shortest Path Matrix d_{uv}

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

Try yourself

- Find the all pair shortest path using Johnson Algorithm for below figure



Answer:

	0	1	2	3
0	0	-5	-1	0
1	INF	0	4	5
2	INF	INF	0	1
3	INF	INF	INF	0

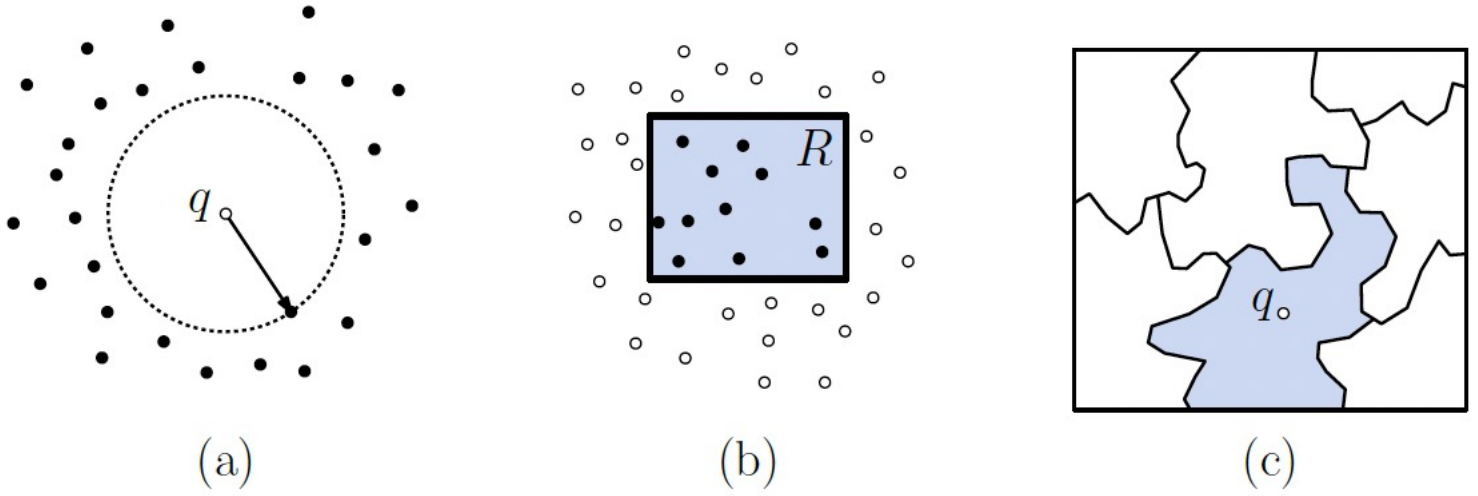
Graphical/Spatial Data Structure

- **Spatial data structures**, are specialized data structures used for representing, organizing, and querying spatial or geometric data.
- Geometric data structures are fundamental to the efficient processing of data sets arising from myriad applications(technology, science, general life, business, engineering etc) including spatial databases, automated cartography (maps) and navigation, computer graphics, robotics and motion planning, solid modeling and industrial engineering, particle and fluid dynamics, molecular dynamics and drug design in computational biology, machine learning, image processing and pattern recognition, computer vision.
- To store a large datasets consisting of geometric objects (e.g.,points, lines and line segments, simple shapes (such as balls, rectangles, triangles), and complex shapes such as surface meshes) in order to answer queries on these data sets efficiently.

Graphical/Spatial Data Structure

Types of Query

- **Nearest-Neighbor Searching**: Store a set of points so that given a query point q, it is possible to find the closest point of the set (or generally the closest k objects) to the query point (see Fig. 1(a)).
- **Range Searching**: Store a set of points so that given a query region R (e.g., a rectangle or circle), it is possible to report (or count) all the points of the set that lie inside this region (see Fig. 1(b)).
- **Point location**: Store the subdivision of space into disjoint regions (e.g., the subdivision of the globe into countries) so that given a query point q, it is possible determine the region of the subdivision containing this point efficiently (see Fig. 1(c)).

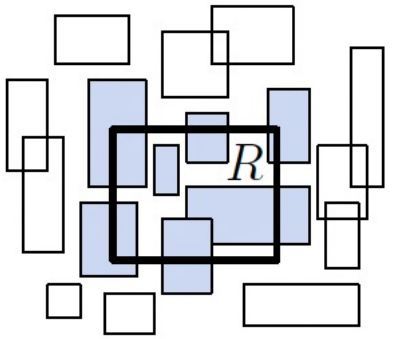


Graphical/Spatial Data Structure

Types of Query

- **Intersection Searching**: Store a collection of geometric objects (e.g., rectangles), so that given a query consisting of an object R of this same type, it is possible to report (or count) all of the objects of the set that intersect the query object (see Fig. 1(d)).
- **Ray Shooting**: Store a collection of object so that given any query ray, it is possible to determine whether the ray hits any object of the set, and if so which object does it hit first.

- If the size n of the set is huge, consisting for example of millions of objects, and the objective is to answer the query in time that is significantly smaller than n, ideally O(log n).
- It is not always possible to achieve efficient query times with storage that grows linearly with n. In such instances, we would like the storage to slowly, for example, O(n log n).



(d)

Graphical/Spatial Data Structure

- Spatial Data Structure are are particularly useful in applications like computer graphics, geographic information systems (GIS), computer-aided design (CAD), gaming, simulations, and collision detection.

1. Quadtrees

- Primarily used to partition a two-dimensional space by recursively subdividing it into four quadrants or regions.
- **Applications:** GIS for region-based data, image compression, and efficient range queries in 2D space.

2. Octrees

- An extension of quadtrees to three-dimensional space, recursively subdividing space into eight octants.
- **Applications:** 3D computer graphics, spatial indexing in 3D environments, and volumetric data representation.

Graphical/Spatial Data Structure

3. KD-Trees (k-dimensional Trees)

- A space-partitioning data structure for organizing points in a k-dimensional space.
- **Applications:** Nearest neighbor search, range queries, and dimensional indexing.

4. R-Trees

- Used to store and query rectangles or multidimensional objects in databases.
- **Applications:** Efficient indexing and querying of spatial data, such as polygons or rectangles in GIS and spatial databases.

5. Voronoi Diagrams

- Partition a plane based on the distance to a specified set of points, resulting in regions known as Voronoi cells.
- **Applications:** Spatial analysis, nearest neighbor search, and cellular networks.

6. Etc.

Spatial Data Structure

- Search trees such as **BSTs**, **AVL trees**, **splay trees**, **2-3 Trees**, **B-trees**, and **tries** are designed for searching on a one-dimensional key.
- A typical example is an integer key, whose one-dimensional range can be visualized as a number line.
- These various tree structures can be viewed as dividing this one-dimensional number line into pieces.
- Some databases require support for multiple keys. In other words, records can be searched for using any one of several key fields, such as name or ID number.
- Typically, each such key has its own one-dimensional index, and any given search query searches one of these independent indices as appropriate.

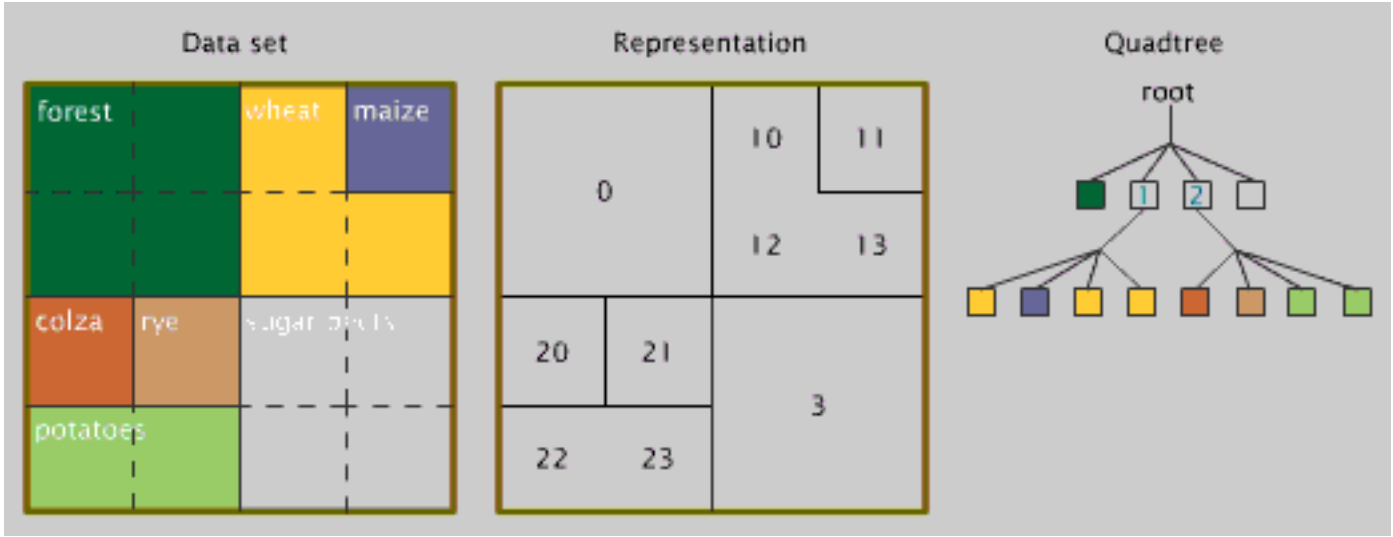
Spatial Data Structure

Multdimensional Keys

- Suppose that we have a database of city records, where each city has a name and an *xy* coordinate. A BST or splay tree provides good performance for searches on city name, which is a one-dimensional key.
- Search on one of the two coordinates is not a natural way to view search in a two-dimensional space.
- Another option is to combine the xy coordinates into a single key, say by concatenating the two coordinates, and index cities by the resulting key in a BST. That would allow search by coordinate, but would not allow for an efficient two-dimensional **range query** such as searching for all cities within a given distance of a specified point.
- The problem is that the BST only works well for one-dimensional keys, while a coordinate is a two-dimensional key where neither dimension is more important than the other.
- To implement spatial applications efficiently requires the use of a **spatial data structure**.
- Spatial data structures store data objects organized by position and are an important class of data structures used in geographic information systems, computer graphics, robotics, and many other fields.
- A number of spatial data structures are used for storing point data in two or more dimensions.
- The **kd tree** is a natural extension of the BST to multiple dimensions.

Quadtree

- A **quadtree** is a tree data structure in which each internal node has exactly four children.
- In Quadtrees two-dimensional space are partitioned by recursively subdividing it into four quadrants or regions.
- The data associated with a leaf cell varies by application, but the leaf cell represents a "unit of interesting spatial information".



Quadtree

Types of Quadtree

1. Point Quadtree

- **Purpose:** Stores individual points in 2D space.
- **Structure:** Each node represents a point, and the space is recursively divided into four quadrants around that point.
- **Applications:**
  - Efficient point location.
  - Nearest neighbor search.
- **Limitation:** Not efficient for uniformly distributed points due to unbalanced tree structures.

Quadtree

Types of Quadtree

2. Region Quadtree

- **Purpose:** Represents a 2D space divided into regions or blocks, often for images or terrains.
- **Structure:** Each node represents a square region, and it is recursively subdivided into quadrants until each leaf is homogeneous.
- **Applications:**
  - Image compression.
  - Region-based queries in GIS.
- **Example:** Used for storing binary images (black and white).

Quadtree

Types of Quadtree

3. PR Quadtree (Point-Region Quadtree)

- **Purpose:** Handles spatial point data by dividing the 2D space into quadrants regardless of point positions.
- **Structure:**
  - Each node corresponds to a specific region of space.
  - Subdivides space into quadrants until each leaf contains at most one point.
- **Applications:**
  - Spatial indexing.
  - Geographic information systems (GIS).
- **Key Feature:** Fixed spatial decomposition regardless of data distribution.

Quadtree

Types of Quadtree

4. MX Quadtree (Matrix Quadtree)

- **Purpose:** Specialized for fixed-grid data, such as pixels in an image.
- **Structure:**
  - Similar to region quadtrees but operates on a matrix-like uniform grid.
  - Subdivision occurs until each region contains homogeneous data.
- **Applications:**
  - Image processing.
  - Terrain modeling.

Quadtree

Types of Quadtree

5. Edge Quadtree

- **Purpose:** Designed for representing linear features, such as lines or curves, within a 2D space.
- **Structure:**
  - The quadtree is used to approximate and store line segments by recursively subdividing space.
- **Applications:**
  - Vector graphics storage.
  - Pathfinding and navigation.
- **Key Feature:** Supports efficient rendering of complex edges.

Quadtree

Types of Quadtree

6. Adaptive Quadtree

- **Purpose:** Dynamically adapts to the density of data in space.
- **Structure:**
  - Regions are subdivided only where necessary (e.g., regions with dense data are further divided).
- **Applications:**
  - Simulation of physical systems.
  - Dynamic terrain or scene representation.
- **Key Feature:** Reduces unnecessary subdivisions and memory usage.

Quadtree

Types of Quadtree

7. Loose Quadtree

- **Purpose:** Allows objects to overlap boundaries by expanding node boundaries slightly.
- **Structure:**
  - Nodes have a tolerance margin that accommodates objects spanning multiple quadrants.
- **Applications:**
  - Collision detection in gaming and simulations.
- **Key Feature:** Reduces the number of unnecessary splits and simplifies spatial relationships.

Comparison of Key Quadtree Types

Type	Primary Focus	Common Applications
Point Quadtree	Storing point data	Nearest neighbor, GIS
Region Quadtree	Dividing 2D space	Image compression, GIS
PR Quadtree	Fixed spatial divisions	Spatial indexing, GIS
MX Quadtree	Matrix-based representation	Image processing
Edge Quadtree	Linear feature representation	Vector graphics, GIS
Adaptive Quadtree	Dynamic data density	Terrain modeling, physics
Loose Quadtree	Flexible boundaries	Collision detection

Point quadtree

- Let us first consider a natural way of generalizing unbalanced binary trees in the 1-dimensional case to a d-dimensional context. Suppose that we wish to store a set $P = \{P_1, P_2, \ldots P_n\}$ of n points in d-dimensional space.
- In binary trees, each point naturally splits the real line in two. In two dimensions if we run a vertical and horizontal line through the point, it naturally subdivides the plane into four quadrants about this point.
- To simplify the presentation, assume that we are working in 2-dimensional space. The resulting data structure is called a point quadtree. (In dimension three, the corresponding structure is naturally called an octree).
- As the dimension grows, it is too complicated to figure out the proper term for the number of children, and so the term quadtree is often used in arbitrary dimensions, even though the outdegree of each node is 2^d, not four.)
- Each node has four (possibly null) children, corresponding to the four quadrants defined by the 4-way subdivision and label these according to the compass directions, as NW, NE, SW, and SE.
- In terms of implementation, we can think of assigning these the values 0, 1, 2, 3, and use them as indices to a 4-element array of children pointers.

Point quadtree

Consider the insertion of the following points: (35; 40); (50; 10); (60; 75); (80; 65); (85; 15); (5; 45); (25; 35); (90; 5)

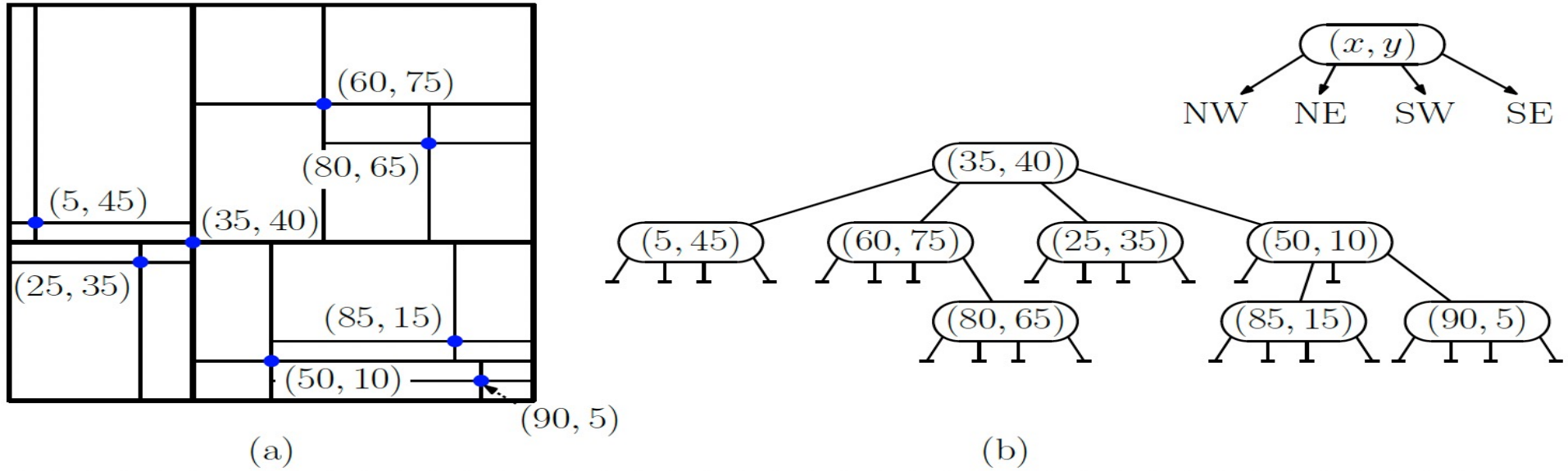


Fig: Point Quadtree (a) Subdivision of space (b) tree structure

- Each node in the tree is naturally associated with a rectangular region of space, which we call its cell.
- Note that some rectangles are special in that they extend to infinity.
- Algorithm will be discuss in **kd-trees** which is a similar algorithm for quadtrees.

Point-Region quadtree

- In the **Point-Region quadtree** (**PR quadtree**) each node either has exactly four children or is a leaf.
- The PR quadtree is a full four-way branching (4-ary) tree in shape.
- The PR quadtree represents a collection of data points in two dimensions by decomposing the region containing the data points into four equal quadrants, subquadrants, and so on, until no leaf node contains more than a single point.
- If a region contains zero or one data points, then it is represented by a PR quadtree consisting of a single leaf node.
- If the region contains more than a single data point, then the region is split into four equal quadrants.
- The corresponding PR quadtree then contains an internal node and four subtrees, each subtree representing a single quadrant of the region, which might in turn be split into subquadrants.
- Each internal node of a PR quadtree represents a single split of the two-dimensional region.
- The four quadrants of the region (or equivalently, the corresponding subtrees) are designated (in order) NW, NE, SW, and SE.
- Each quadrant containing more than a single point would in turn be recursively divided into subquadrants until each leaf of the corresponding PR quadtree contains at most one point.

Point-Region quadtree

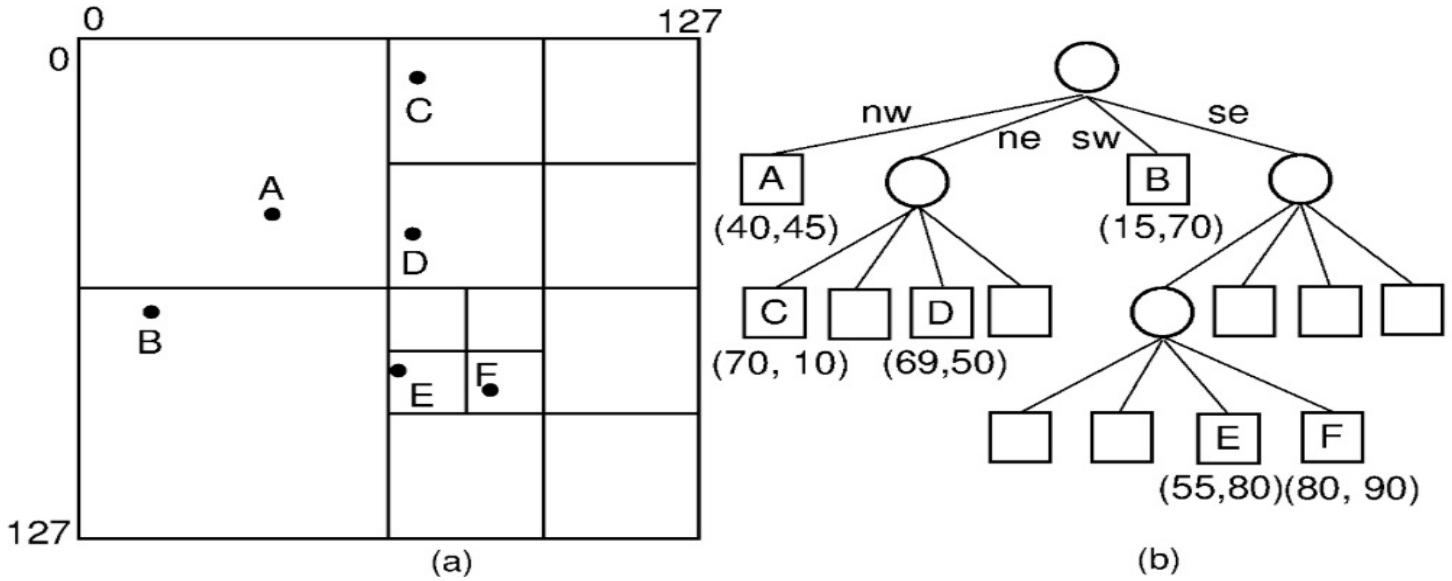


Fig: Example of a PR quadtree.(a) A map of data points. (b) The PR quadtree for the points in (a). (a) also shows the block decomposition imposed by the PR quadtree for this region.

- Consider the region of Fig (a) and the corresponding PR quadtree in Fig (b). The decomposition process demands a fixed key range. In eg,the region is assumed to be of size 128×128. Note that the internal nodes of the PR quadtree are used solely to indicate decomposition of the region; internal nodes do not store data records.

Point-Region quadtree

- Search for a record matching point Q in the PR quadtree is straightforward. Beginning at the root, we continuously branch to the quadrant that contains Q until our search reaches a leaf node. If the root is a leaf, then just check to see if the node's data record matches point Q. If the root is an internal node, proceed to the child that contains the search coordinate.
- For example, the NW quadrant of Fig(a) contains points whose x and y values each fall in the range 0 to 63. The NE quadrant contains points whose x value falls in the range 64 to 127, and whose y value falls in the range 0 to 63.
- If the root's child is a leaf node, then that child is checked to see if Q has been found.
- If the child is another internal node, the search process continues through the tree until a leaf node is found.
- If this leaf node stores a record whose position matches Q then the query is successful; otherwise Q is not in the tree.

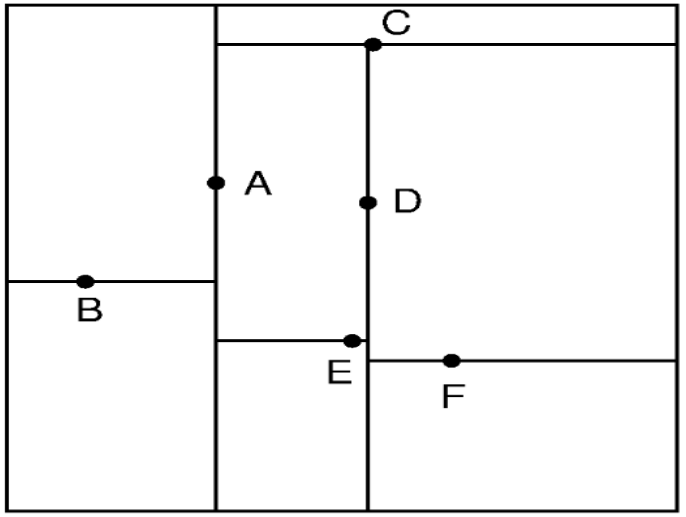
KD Trees

KD Trees

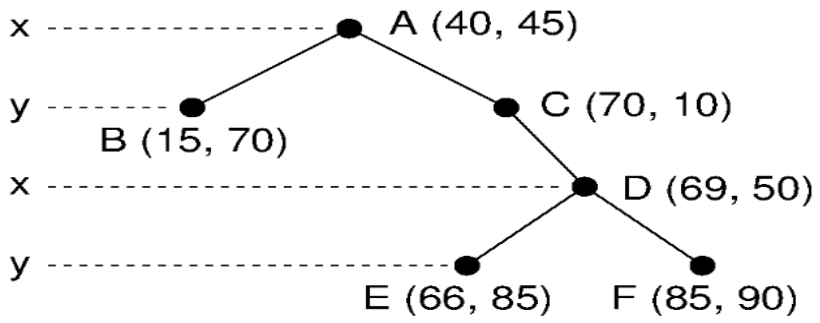
- The **kd tree** is a modification to the **BST** that allows for efficient processing of **multi-dimensional search keys**.
- The **kd tree** differs from the BST in that each level of the **kd tree** makes branching decisions based on a particular search key associated with that level, called the **discriminator or cutting dimension**.
- In principle, the **kd tree** could be used to unify key searching across any arbitrary set of keys (like name and zipcode).
- But in practice, it is nearly always used to support search on multi-dimensional coordinates, such as locations in 2D or 3D space.
- **Discriminator/Cutting Dimension** at level i: $i \bmod k$ for k dimensions.
- Eg: If data organized by xy-coordinates (2D): $k = 2$.
  - with the x-coordinate field arbitrarily designated ***key 0***, and the y-coordinate field designated ***key 1***.

KD Trees

- At each level, the discriminator/cutting dimension alternates between *x* and *y*.
- A node N at *level 0* (the root)
  - Its left subtree only nodes whose *x* values are less than N_x (because *x* is search key 0, and *0 mod 2=0*).
  - The right subtree would contain nodes whose *x* values are greater than N_x.
- A node M at level 1 would have in its left subtree only nodes whose *y* values are less than M_y. (key = 1, *1 mod 2=1*)
- There is no restriction on the relative values of M_x and the *x* values of M's descendants, because branching decisions made at M are based solely on the *y* coordinate.



(a)



(b)

Fig KD(1): Example of a kd tree. (a) The kd tree decomposition for a 128×128-unit region (b) The kd tree for the region of (a).

KD Trees

- In above figure, the region containing the points is restricted to a 128×128 square, and each internal node splits the search space.
- Each split is shown by a line, vertical for nodes with *x* discriminators and horizontal for nodes with *y* discriminators.
- The root node splits the space into two parts; its children further subdivide the space into smaller parts.
- The children's split lines do not cross the root's split line. Thus, each node in the ***kd tree*** helps to decompose the space into rectangles that show the extent of where nodes can fall in the various subtrees.

KD Trees

```
class KDNode { // node in a kd-tree
        Point point; //splitting point
        int cutDim; //cutting dimension
        KDNode left; // children
        KDNode right;

KDNode(Point point, int cutDim){//
        constructor
        this.point = point;
        this.cutDim = cutDim;
        left = right = null;
}
        boolean inLeftSubtree(Point x){//is
                x in left subtree?
        return x[cutDim] < point[cutDim];
        }
}
```

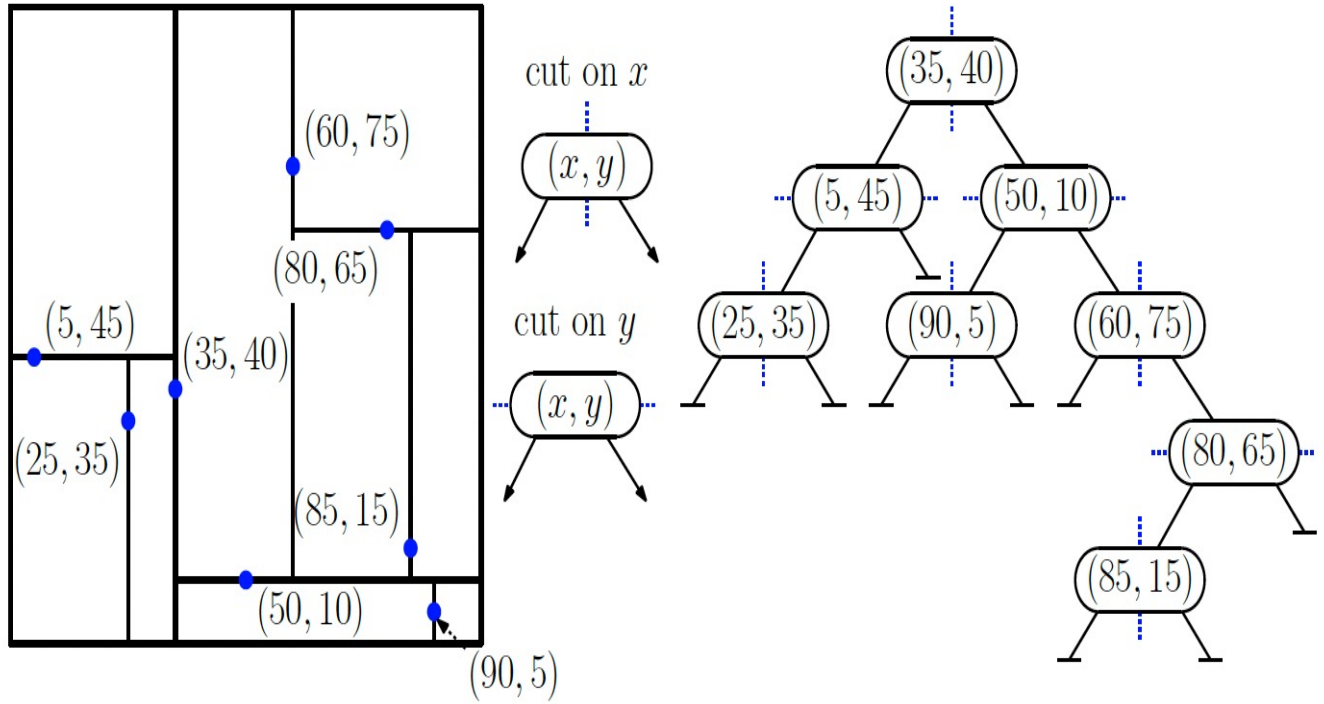
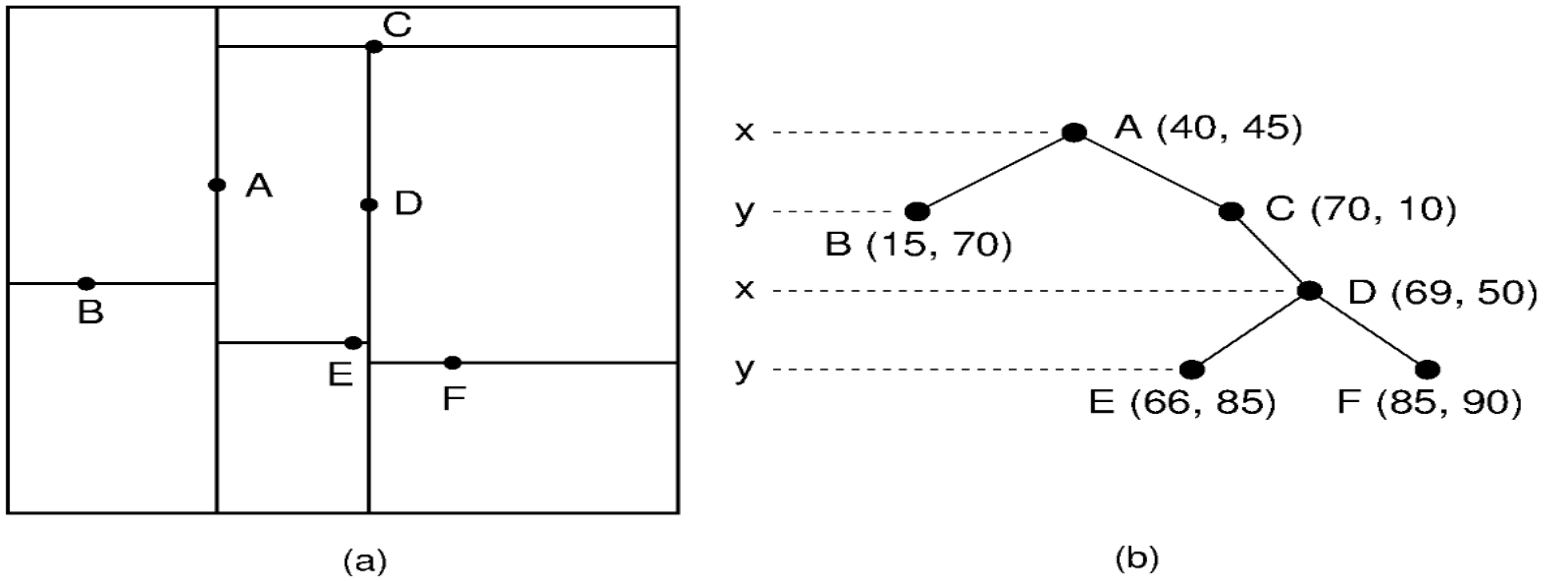


Fig KD(2): Point kd-tree decomposition.

Code: Representation of kd tree

The contents of the left child of a node contain the points x such that *x[cutDim]* < *point[cutDim]* and for the right child *x[cutDim]* >= *point[cutDim]*.



Search Demo (Next Slide)

KD Trees (Searching)

- Consider searching the **kd tree** for a record located at P=(69,50).
- First compare P with the point stored at the root (record A in Fig KD(1),previous slide). If P matches the location of A, then the search is successful.
- In example: it do not match (A(40,45) is not same as (69, 50),so the search must continue.
- The *x* value of A is compared with that of P to determine in which direction to branch. Because A_x's value of 40 is less than P's *x* value of 69, we branch to the right subtree.
- A_y does not affect the decision on which way to branch at this level.
- At the second level, P does not match record C's position, so another branch must be taken.
- At this level we branch based on the relative *y* values of point P and record C(since *1%2=1*, which corresponds to the y-coordinate).
- C_y's value of 10 is less than P_y's value of 50, we branch to the right.
- At this point, P is compared against the position of D. A match is made and the search is successful.
- If the search process reaches a NULL pointer, then that point is not contained in the tree.

KD Trees

```
Point findMin(KDNode p, int i) { // get min point along dim i
    if (p == null) { // fell out of tree?
        return null;
    }
    if (p.cutDim == i) { // cutting dimension matches i?
        if (p.left == null) // no left child?
                return p.point; // use this point
        else
                return findMin(p.left, i); // get min from left subtree
    }
    else { // it may be in either side
        Point q = minAlongDim(p.point, findMin(p.left, i), i);
        return minAlongDim(q, findMin(p.right, i), i);
    }
}
Point minAlongDim(Point p1, Point p2, int i) { // return smaller point on dim i
    if (p2 == null || p1[i] <= p2[i]) // p1[i] is short for p1.get(i)
        return p1;
    else
        return p2;
}
```

Code: Find the minimum point in subtree along i^{th} coordinate (same function can use for deletion as well)

KD Trees (Insertion)

- Insertion operates as it would for a regular binary search tree.
- We descend the tree until falling out, and then we create a node containing the point and assign its cutting dimension by whatever policy is used by the tree.
- The principal utility function is presented in the below.
- The function takes three arguments, the point *x* being inserted, the current node *p*, and the cutting dimension of the newly created node.
- The initial call is *root = insert(x, root, 0)*.
- An example is shown in ***Fig KD(3)***, where we insert the point (50, 90) into the kd-tree of **Fig KD(2).**
- We descend the tree until we fall out on the left subtree of node (60, 75).
- We create a new node at this point, and the cutting dimension cycles from the parent's x-cutting dimension (cutDim = 0) to a y-cutting dimension (cutDim = 1).

KD Trees (Insertion)

```
KDNode insert(Point x, KDNode p, int cutDim) {
    if (p == null) { // fell out of tree
        p = new KDNode(x, cutDim); // create new leaf
    }
    else if (p.point.equals(x)) {
        throw Exception("duplicate point"); // duplicate data point!
    }
    else if (p.inLeftSubtree(x)) { // insert into left subtree
        p.left = insert(x, p.left, (p.cutDim + 1) % x.getDim());
    }
    else { // insert into right subtree
        p.right = insert(x, p.right, (p.cutDim + 1) % x.getDim());
    }
        return p;
}
```

Note: In some references, we can replace with new point if p.point.equals(x)

KD Trees (Insertion)

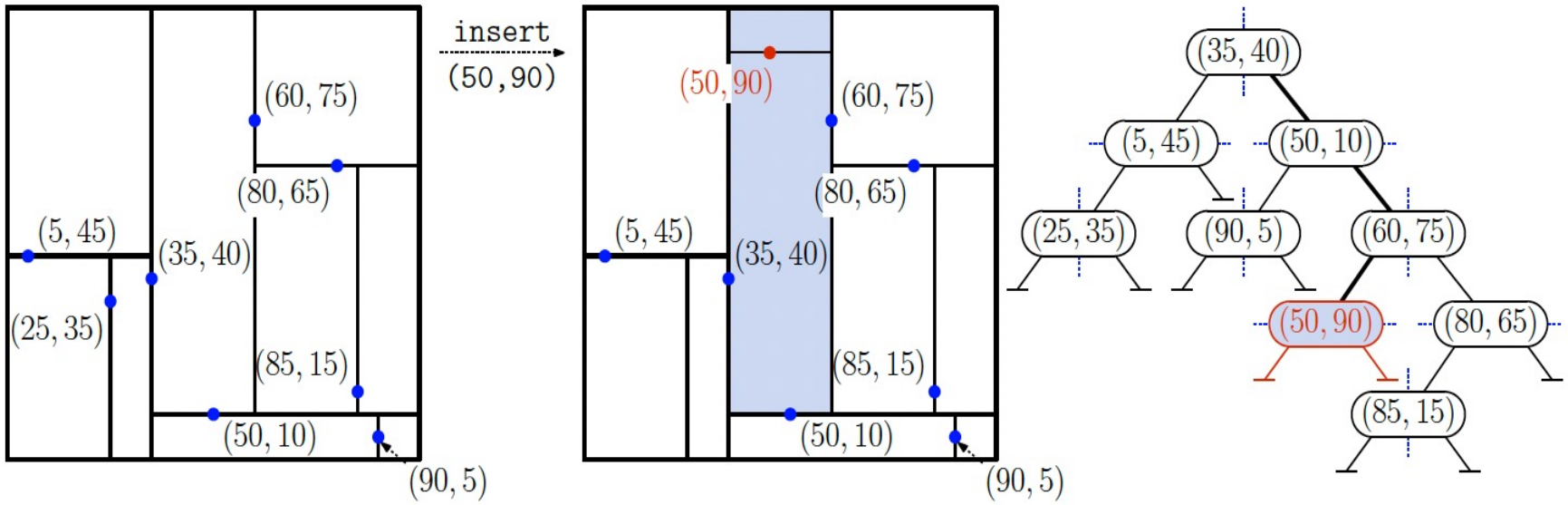


Fig KD(3): Insertion into the point kd-tree of **Fig KD(2)**

Another Example KD tree Insertion Demo from Professor Joshua Hug (Berkeley, California, United States)

KD Trees (Deletion and Replacement)

- With deletion in standard binary search trees, an issue that will arise when deleting a point from the middle of the tree is what to use in place of this node, that is, the replacement point.
- It is not as simple as selecting the next point in an inorder traversal of the tree, since we need a point that satisfies the necessary geometric conditions.
- Suppose that we wish to delete a point in some node p, and suppose further that $p.cutDim == 0$, that is, the x-coordinate. An appropriate choice for the replacement point is the point of $p.right$ that has the smallest x-coordinate.
- In a procedure *findMin(p, i, cutDim)* that computes the point in the subtree rooted at p that has the smallest i^{th} coordinate. The procedure operates recursively. When it arrive at a node p, if the cutting dimension matches i, then it is known that the subtrees are ordered according the the ith coordinate.
- If the left subtree is nonempty, then the desired point is the minimum from this subtree. Otherwise, it return p's associated point.

KD Trees (Deletion and Replacement)

- If *p*'s cutting dimension differs from *i* then it cannot infer which subtree may contain the point with the minimum i^{th} coordinate. In that case try the right subtree, the left subtree, and the point at this node.
- A function *minAlongDim(p1,p2, i)*, which returns whichever point p_1 or p_2 is smallest along coordinate *i*.
- `Function : Point findMin(KDNode p, int i)`
- `Function : Point minAlongDim(Point p1, Point p2, int i)`
- Figure below (next slide) presents an example of the execution of this algorithm to find the point with the minimum *x*-coordinate in the subtree rooted at (55, 40).
- Since this node splits horizontally, we need to visit both of its subtrees to find their minimum *x* values. (These will be (15, 10) for the left subtree and (10, 65) for the right subtree.)
- These values are then compared with the point at the root to obtain the overall *x*-minimum point, namely (10, 65).
- The subtrees at (45, 20) and (35, 75) both split on the *x*-coordinate, and we are looking for the point with the minimum *x*-coordinate, we do not need to search their right subtrees.
- The nodes visited in the search are shaded in blue.

KD Trees (Deletion and Replacement)

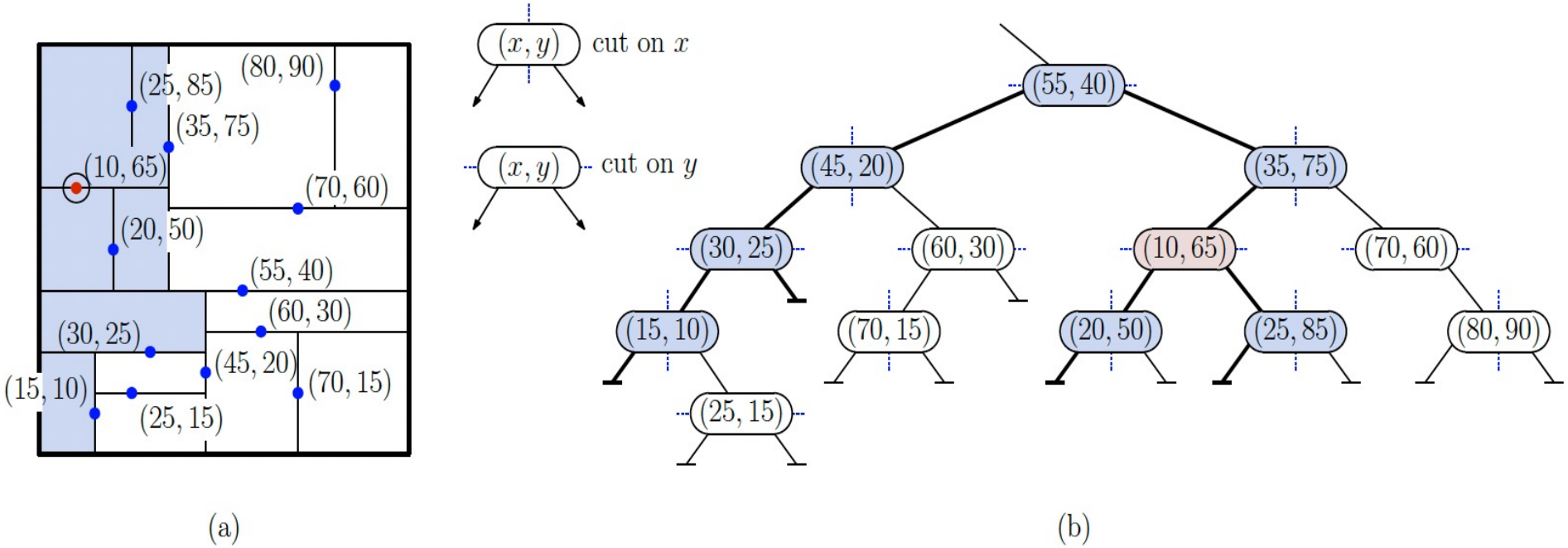


Fig KD(4) : Example of findMin when i = 0 (the x-coordinate) on the subtree rooted at (55, 40). The function returns (10, 65).

KD Trees (Deletion)

Deletion from a kd-tree:

- In 1D BST case we needed to consider a number of different cases.
- If the node is a leaf we just delete the node. Otherwise, its deletion would result in a hole in the tree.
- We need to find an appropriate replacement element. In 1D case, we were able to simplify this if the node has a single child (by making this child the new child of our parent).
- This would move the child from an even level to an odd level, or vice versa (this would violate the assumption of cutting dimensions cycle among the coordinates. (Given procedure will not alter a node's cutting dimension)
- Assume that the right subtree is non-empty. In 1D case, the replacement key was taken to be the smallest key from the right child, and after using the replacement to fill the hole, it recursively deleted the replacement.
- In Multi-dimentional case ,the thing to do is to find the point whose coordinate along the current cutting dimension is minimum. Thus, if the cutting dimension is the x-axis, then the replacement key is the point with the smallest x-coordinate in the right subtree.
- We use the findMin() function to do this.

KD Trees (Deletion)

- What if the right subtree is empty?
  - At first, it might seem that the right thing to do is to select the maximum node from the left subtree. (It should be noted that the points whose coordinates are equal to the cutting dimension are stored in the right subtree. If we select the replacement point to be the point with the maximum coordinate from the left subtree, and if there are other points with the same coordinate value in this subtree, then it violates our invariant).
  - Trick: For the replacement element we will select the minimum (not maximum) point from the left subtree, and we move the left subtree over and becomes the new right subtree.
  - The left child pointer is set to null.

KD Trees (Deletion)

```
KDNode delete(Point x, KDNode p) {
    if (p == null) { // fell out of tree?
        throw Exception("point does not exist");
    }
    else if (p.point.equals(x)) { // found it
        if (p.right != null) { // take replacement from right
                p.point = findMin(p.right, p.cutDim);
                p.right = delete(p.point, p.right);
        }
        else if (p.left != null) { // take replacement from left
                p.point = findMin(p.left, p.cutDim);
                p.right = delete(p.point, p.left); // move left subtree to right!
                p.left = null; // left subtree is now empty
        }
        else { // deleted point in leaf
                p = null; // remove this leaf
        }
    } else if (p.inLeftSubtree(x)) {
        p.left = delete(x, p.left); // delete from left subtree
    }
    else { // delete from right subtree
        p.right = delete(x, p.right);
    }
    return p;
}
```

KD Trees

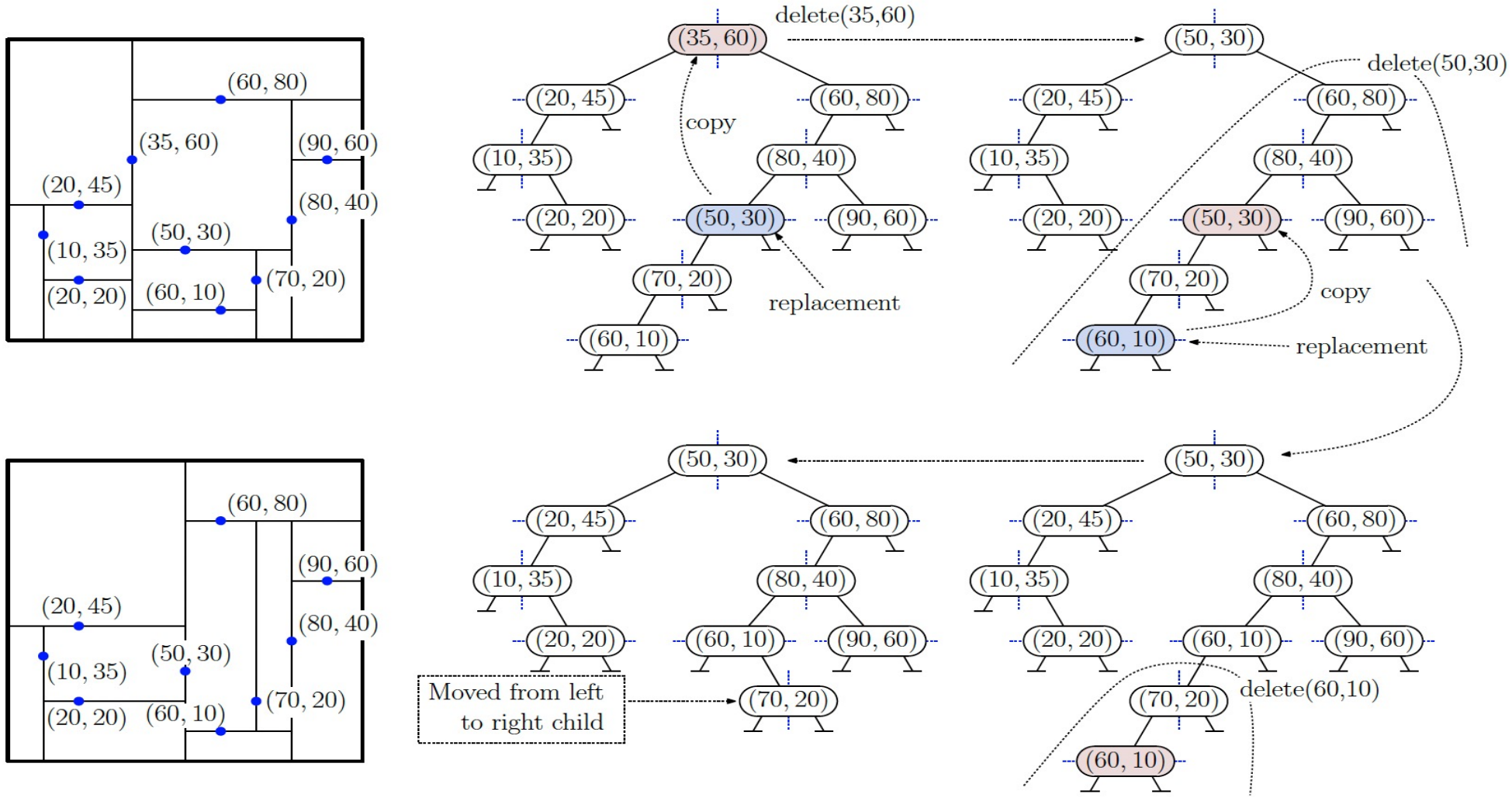


Fig KD(5): Deletion from a kd-tree. **Delete (35,60)**

KD Trees

Description: Objective is to delete the point (35,60)

- It is at the root of the tree. Because the cutting dimension is vertical, search its right subtree to find the point with the minimum *x*-coordinate, which is (50, 30).
- The point is copied to the root, and then recursively delete (50, 30) from the root's right subtree. This recursive call then seeks the node p containing (50, 30).
- This node has no right child, unlike standard binary search trees, we cannot simply unlink it from the tree. It is observe that its cutting dimension is horizontal, and we search for the point with the minimum *y*-coordinate in *p*'s left subtree, which is (60, 10).
- Copy (60, 10) to *p*, and then recursively delete (60,10) from *p*'s left subtree.
- It is a leaf, so it may simply be unlinked from the tree.
- Finally move this left subtree *p* over to become *p*'s right subtree.

KD Trees (Nearest Neighbor Search)

Steps:

1. **Start at the Root Node**
   - Begin with the root of the KD-tree and compare the query point with the splitting dimension (e.g., x, y, etc.) of the current node.
   - Decide whether to go to the **left** or **right** child based on the comparison .

2. **Traverse Down the Tree**
   - Recursively move down the tree following these rules:
     - If the query point's value in the splitting dimension is less than the node's value, go to the left child.
     - Otherwise, go to the right child.
   - Keep traversing until you reach a leaf node (a node with no children).

KD Trees (Nearest Neighbor Search)

Steps:

3. Assume the Leaf Node is the Nearest Neighbor

- Set the current nearest neighbor (NN) as the point at the leaf node.
- Calculate the distance between the query point and this leaf node (using Euclidean distance : $\mathbf{sqrt((x_2-x_1)^2+(y_2-y_1)^2))}$
- Store this distance as the **current best distance**.

4. Backtrack and Explore Other Branches

- During backtrack up the tree, check if the splitting hyperplane (the dividing line or plane for the current node) could have points closer to the query point.
- **How to check:** Compare the best distance so far with the distance to the splitting hyperplane. If the hyperplane is closer, the other branch might contain a closer point.
- If the other branch could contain a closer point, **explore it recursively**.

KD Trees (Nearest Neighbor Search)

Steps:

5. Update the Nearest Neighbor

- For each node visited (while backtracking or exploring the other branch):
  - Calculate the distance between the query point and the current node.
  - If this distance is smaller than the current best distance, update the nearest neighbor and the best distance.

6. Continue Until the Root is Revisited

- Keep backtracking and checking all possible branches that might contain closer points.
- When returning to the root node and finish exploring all branches, the **current nearest neighbor** will be the final result.

KD Trees (Nearest Neighbor Search)

Correction (Another Version):

- Many implementations of **KD-Tree Nearest Neighbor Search** do calculate the distance to the root and intermediate nodes **while traversing to the leaf node**. This approach is perfectly valid and can provide some advantages.

Advantages:

- **More Pruning:**
  - Early distance calculations allow branches to be pruned earlier, reducing unnecessary computations.
- **Faster Nearest Neighbor Detection:**
  - A good candidate may be found early, potentially avoiding deeper backtracking.
- **Flexibility in Implementation:**
  - This approach often feels more intuitive and aligns well with depth-first traversal logic.

KD Trees (Nearest Neighbor Search)

Simple Example:

KD-Tree Structure

- Root: A=(2,3)
- Left Child: B=(1,5)
- Right Child: C=(5,4)
- Query Point: Q=(3,4)

Step-by-Step Nearest Neighbor Search

1. Start at the Root Node A(2,3)

- Calculate the distance between Q and A: $d(Q,A)=\text{sqrt}((3-2)^2+(4-3)^2)=1+1=\text{sqrt}(2)$
- Current nearest neighbor = A(2,3), best distance = sqrt(2) .

2. Traverse to the Right Child C(5,4)

- Compare Q[0](3) with A[0](2). Since 3>2, move to the right child.
- Calculate the distance between Q and C(5,4) :$d(Q,C)=\text{sqrt}((5-3)^2+(4-4)2)=\text{sqrt}(4+0)=2$
- Compare d(Q,C)=2 with the current best distance sqrt(2):
  - Sqrt(2)<2, so A(2,3) remains the nearest neighbor.

KD Trees (Nearest Neighbor Search)

3. No Further Right Subtree (Leaf Node Reached)

- Since C(5,4) is a leaf, stop downward traversal.

4. Backtrack to Check the Left Child B(1,5)

- The left child of A(2,3) is B(1,5).
- Calculate the distance between Q and B :$d(Q,B)=\text{sqrt}((3-1)^2+(4-5)^2)=\text{sqrt}(4+1)=\text{sqrt}(5)$
  $d(Q,B)=\text{sqrt}(5)$

- Compare $d(Q,B)=\text{sqrt}(5)$ with the current best distance sqrt(2):
  - Sqrt(2<5) , so A(2,3) remains the nearest neighbor.

- **Conclusion**: The nearest neighbor to Q(3,4) is: A(2,3)
- Distance to the nearest neighbor: sqrt(2) ≈1.412

- Another Example KD tree Nearest Search Demo from Professor Joshua Hug (Berkeley, California, United States)

Complexities in KD tree

Operation	Average Case	Worst Case
Construction	O(nlogn)	$O(n^2)$
Insertion	O(logn)	O(n)
Deletion	O(logn)	O(n)
Exact Match Search	O(logn)	O(n)
Nearest Neighbor Search	O(logn)	O(n)
Range Search	$O(n^{1-1/k}+m)$	O(n)
Space Complexity	O(n)	O(n)

R-Trees

R-Tree

- Efficient indexing and searching of spatial data, such as counties or map objects, pose challenges due to their multi-dimensional nature and non-zero size in space.
- 1D indexing structures like B-trees and hash tables are unsuitable for spatial searches, as they cannot handle the complexity of multi-dimensional range queries.
- This limitation is significant in applications like CAD and geo-data systems, where operations such as finding all objects within a specified distance are common.
- Structures like quad trees, k-d trees, and grid files attempt to address these needs, they often fall short in handling dynamic datasets or large-scale collections efficiently.
- To overcome these challenges, the R-tree is introduced as a robust solution, representing spatial objects using intervals in multiple dimensions and supporting efficient search, insertion, deletion, and updates.

R-Tree

- An R-tree is a height-balanced tree similar to a B-tree with index records in its leaf nodes containing pointers to data object.
- Nodes correspond to disk pages if the index is disk-resident, and the structure is designed so that a spatial search requires visiting only a small number of nodes
- The index is completely dynamic; inserts and deletes can be mixed with searches and no periodic reorganization is required.
- A spatial database consists of a collection of tuples representing spatial objects, and each tuple has a unique identifier which can be used to retrieve it.

R-Tree Structure

- Let P be a set of points. In R-Tree
  - Actual data point is store in the leaf node. (P points).
  - Each leaf node can store up to B points, where B is the size of a disk page (a fixed number of entries that can fit in one page of disk memory).
  - Non-leaf nodes in the tree represent bounding boxes (minimum bounding rectangles, or **MBR**s) that encapsulate the data points or child nodes beneath them.
  - Each non-leaf node can have up to B children, ensuring efficient organization and a balanced tree structure.
  - The root of the tree must have at least 2 children unless it is the only node in the tree. This ensures the tree maintains its hierarchical structure while accommodating small datasets.
  - *Each leaf node has between **0:4B** and **B** data points, where $B \geq 3$ is a parameter.*
  - The use of B as the maximum number of points or children per node aligns with the concept of **disk paging**, where nodes are designed to fit entirely within a single page of memory to minimize disk I/O during queries.
- There is no constraint on how points should be grouped into leaf nodes, and in general, how non-leaf nodes should be grouped into nodes of higher levels. Since each point is stored only once, the entire tree consumes linear space O(N/B), where N is the cardinality of P.

R-Tree (Example)

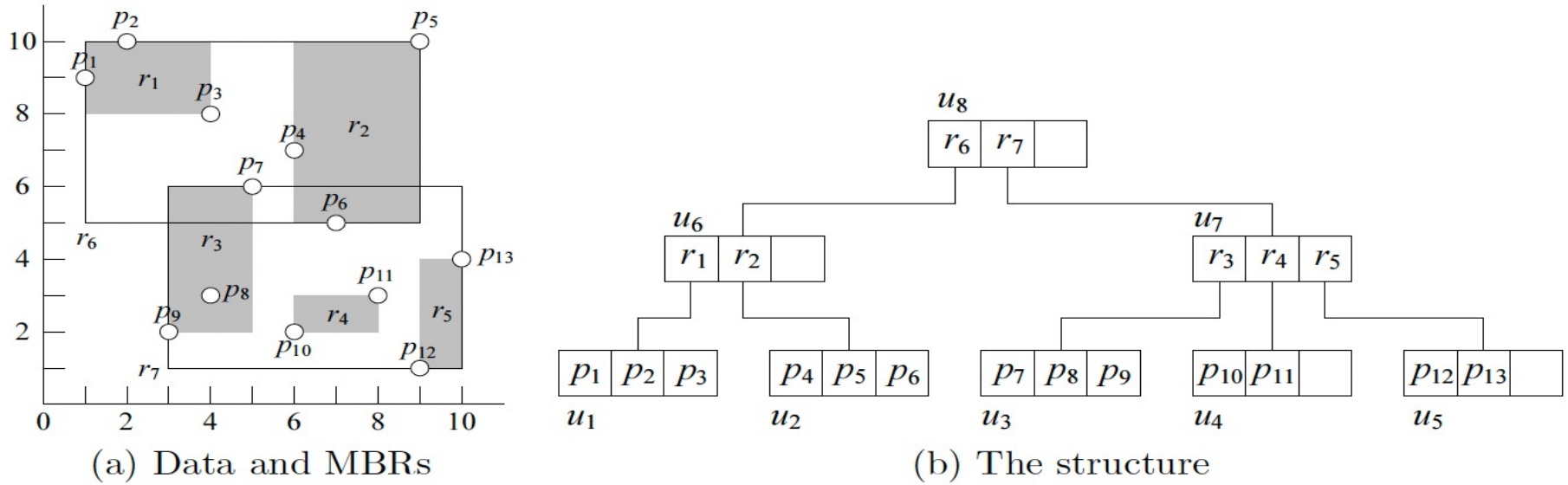
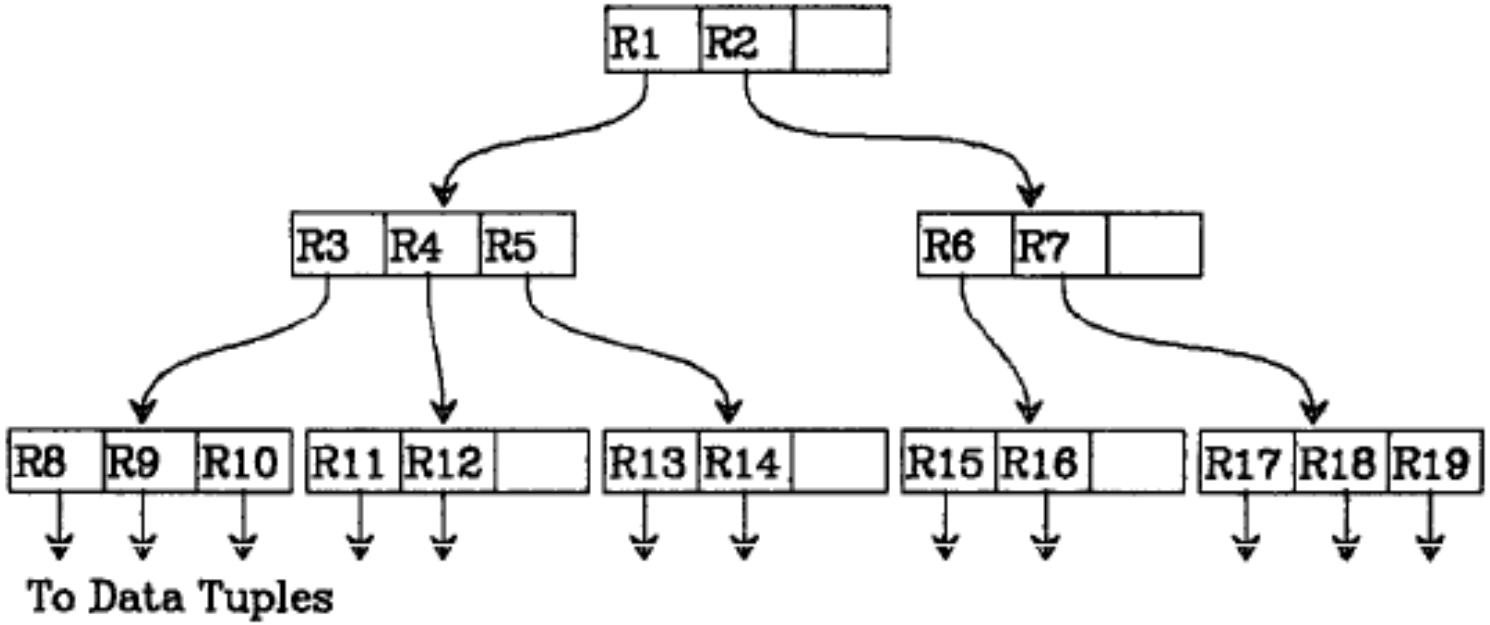


Fig: An Example of R-Tree

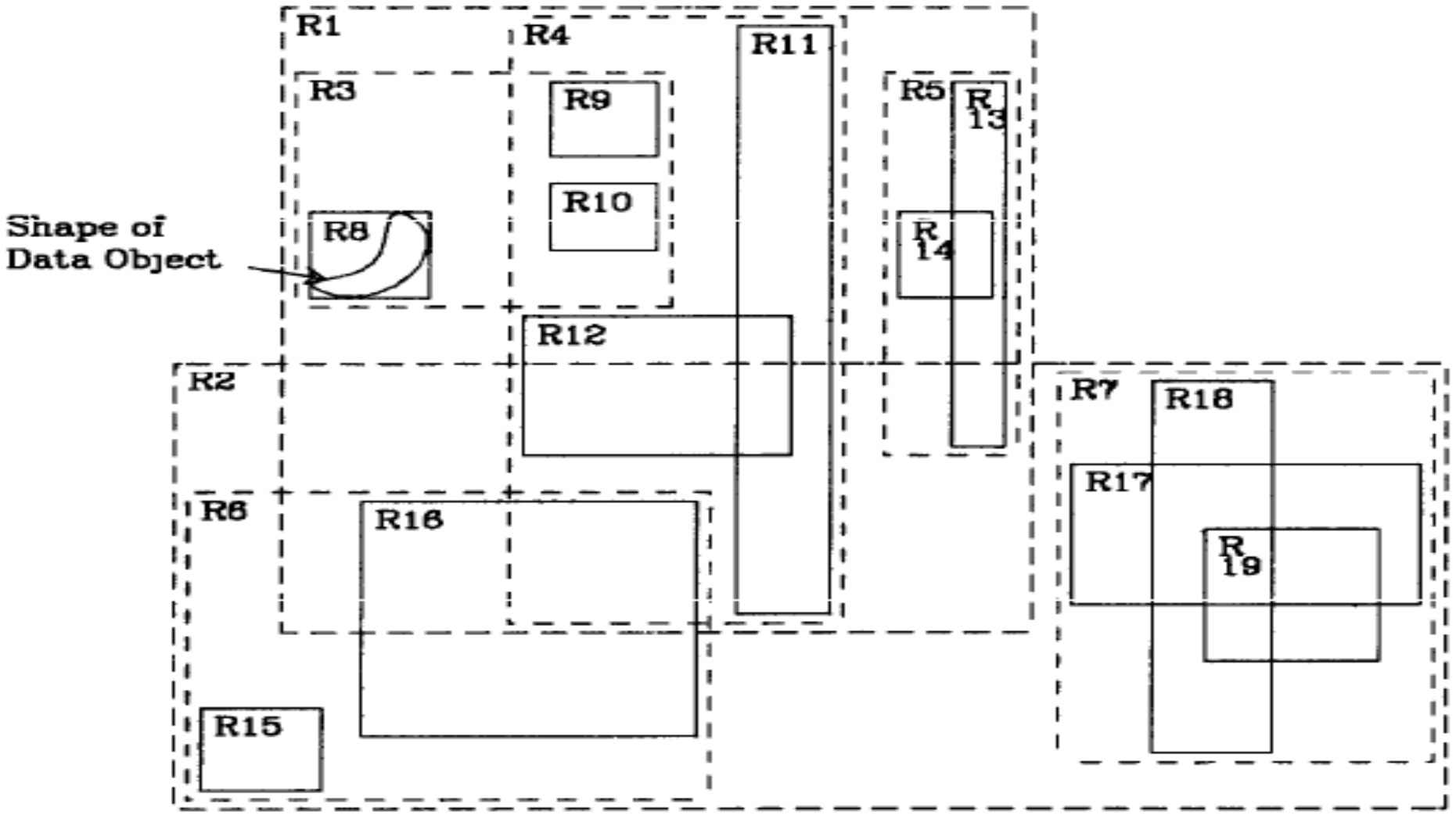
Above fig is an example of R-tree where P has 13 points $p_1, p_2, \ldots, p_{13}$. Points p_1, p_2, p_3, for example, are grouped into leaf node u_1. This leaf is a child of of non-leaf node u_6, which stores an MBR r_1 for u_1. Note that r_1 tightly bounds all the points in u_1.

R-Tree (Example)



Ref: Antonin Guttman. 1984. R-trees: a dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD international conference on Management of data* (SIGMOD '84)

R-Tree (Example)



Ref: Antonin Guttman. 1984. R-trees: a dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD international conference on Management of data* (SIGMOD '84)

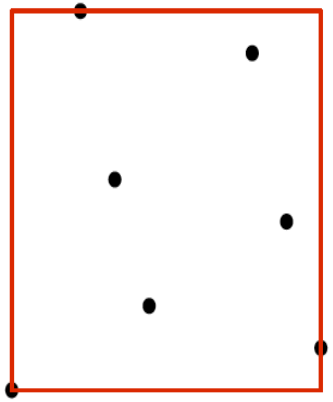
R-Tree

R-Tree

- Each leaf node has between $0.4B$ and B data points, where $B \geq 3$ is a parameter. The only exception applies when the leaf is the root, in which case it is allowed to have between 1 and B points. All the leaf nodes are at the same level.
- Each internal node has between $0.4B$ and B child nodes, except when the node is the root, in which case it needs to have at least 2 child nodes.

R-Tree

For any node u, denote by S_u the set of points in the subtree of u. Consider now u to be an internal node with child nodes $v_1, ..., v_f$ ($f \leq B$). For each v_i ($i \leq f$), u stores the minimum bounding rectangle (MBR) of S_{v_i}, denoted as $MBR(v_i)$.

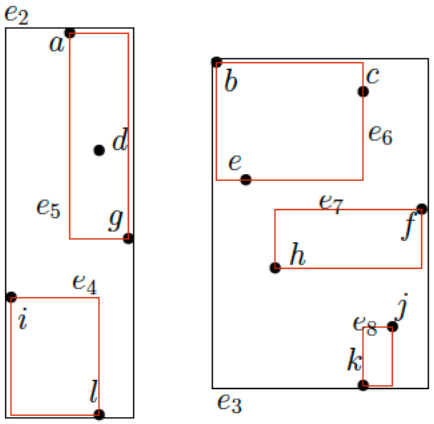


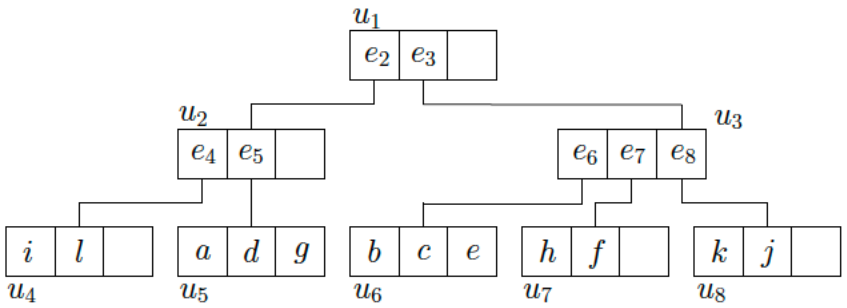
The above is an MBR on 7 points.

R-Tree

Example

Assume $B = 3$.

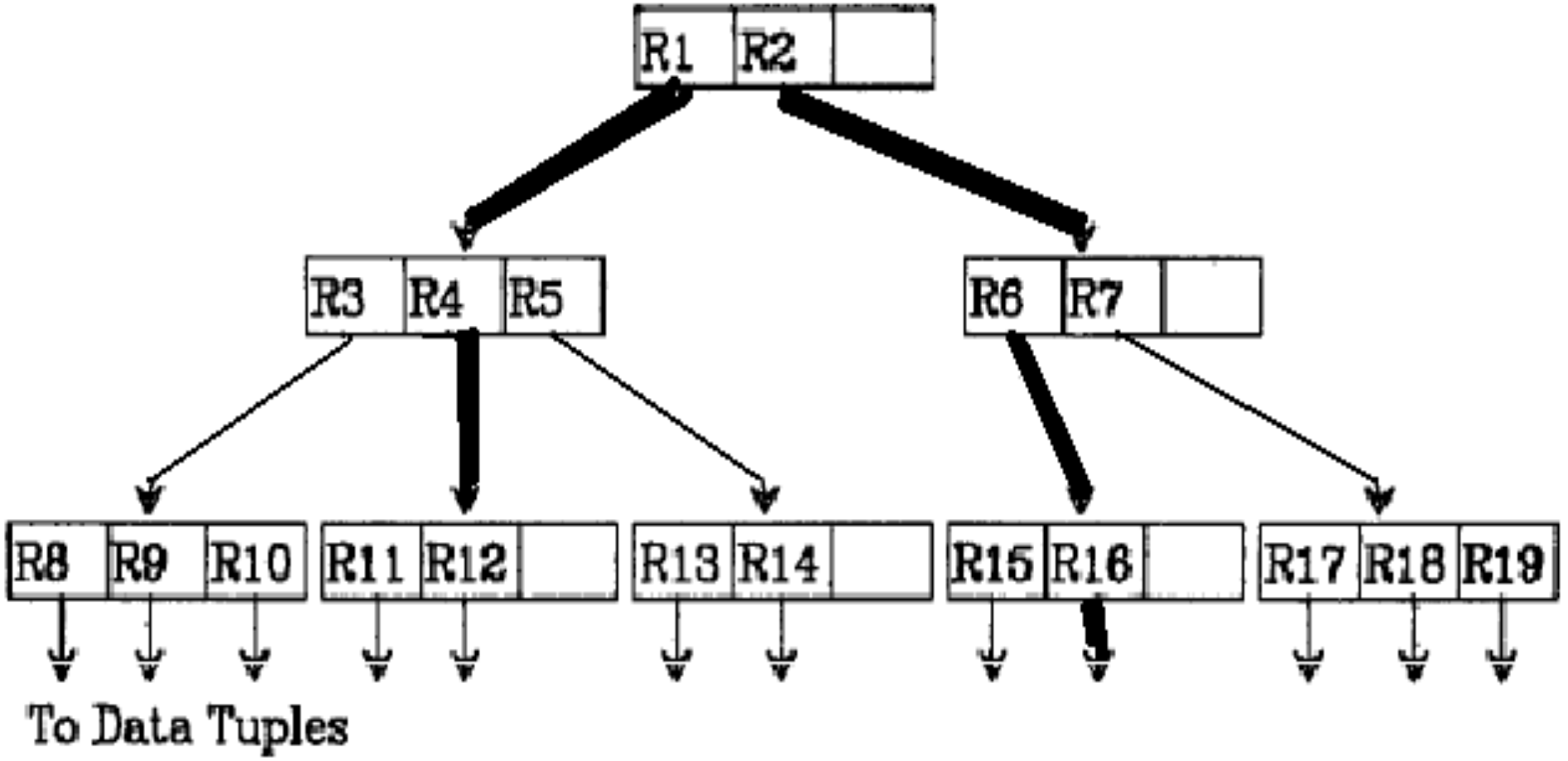




Searching

- Given a search rectangle S …
  1. Start at root and locate all child nodes whose rectangle I intersects S (via linear search).
  2. Search the subtrees of those child nodes.
  3. When you get to the leaves, return entries whose rectangles intersect S.
- Searches may require inspecting several paths.
- Worst case running time is not so good …

S = R16


R1
R2
R3
R4
R5
R6
R7
R8
R9
R10
R11
R12
R13
R14
R15
R16
R17
R18
R19
To Data Tuples

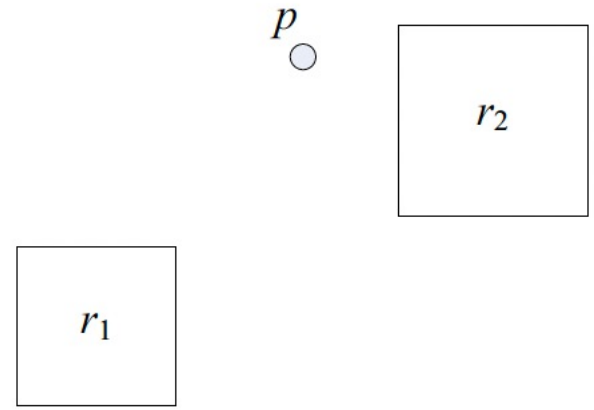
R-Tree (Insertion)

- To insert a point p, use a strategy similar to that of a B-tree. Specifically, we add p to a leaf node u by following a single root-to-leaf path. If u overflows, split it, which creates a new child of parent(u). In case parent(u) overflows, also split it, which propagates upwards in the same manner. Finally, if the root is split, then a new root is created.
- Even though all these sound familiar, however, two important differences.
  - First, although in the B-tree the insertion path is unique this is not true at all for the R-tree.
    - The new point p can be inserted into any leaf, which always results in a legal structure.
    - If a bad leaf is chosen (to contain p), its MBR may need to be enlarged substantially, thus harming the efficiency of the tree.
  - Second, the split algorithm is not as trivial as in a B-tree because now it have to tackle multiple dimensions.

R-Tree (Insertion)

Choosing a subtree to insert.

- We are essentially facing the following problem. Given a non-leaf node *u* with children $v_1, v_2, \ldots, v_f$ for some f = B, we need to pick the best child v* such that the new point *p* is best inserted into the subtree of v*.
- An approach that seems to work well in practice is a greedy one. Specifically, v* can simply be the child v_i whose MBR requires the least increase of perimeter in order to cover *p*. For example, in below figure, both MBRs r_1 and r_2 must be expanded to enclose *p*, but r_2 incurs smaller perimeter increase, and hence, is a better choice.
- It is possible that *p* falls into the overlapping region of multiple MBRs.
- All those MBRs have a tie because none of them needs any perimeter increase to cover *p*.
- In this case, the winner can be decided according to other factors such as picking the MBR having the smallest area.



Fig; MBR r_2 requires smaller perimeter increase to cover p

R-Tree (Insertion)

Node split. The node split problem can be phrased as follows. Given a set S of $B + 1$ points, split it into disjoint subsets S_1 and S_2 with $S_1 \cup S_2 = S$ such that

- $|S_1| \geq \lambda B$, $|S_2| \geq \lambda B$, where constant λ is the *minimum utilization rate* of a node, and
- the sum of the perimeters of $MBR(S_1)$ and $MBR(S_2)$ is small.

- For simplicity we assume that $|S|$ is an even number, and $|S_1| = |S_2| = |S|/2$, i.e., we always aim at an even split.
- Intention is to find the optimal split that minimizes the perimeter sum of MBR(S_1) and MBR(S_2).
- Since an MBR is decided by 4 coordinates (i.e., a pair of opposite corners), it is easy to find the optimal split is significantly improved to $O(B^2)$ time from $O(B^4)$ [***Explain in other paper***]
- Unfortunately, even a quadratic split time is usually excessively long in practice. Therefore, we turn our attention to heuristics that do not guarantee optimality, but usually produce fairly good splits.
- Split algorithm that runs in **O(BlogB)** time, or **O(dBlogB)** time in general d-dimensional space.

R-Tree (Insertion)

Insertion

Let p be the point being inserted. The pseudo-code below should is invoked as `insert`$(root, p)$, where *root* is the root of the tree.

Algorithm `insert`(u, p)

1. **if** u is a leaf node **then**
2. add p to u
3. **if** u overflows **then**
   /* namely, u has $B + 1$ points */
4. `handle-overflow`(u)
5. **else**
6. $v \leftarrow$ `choose-subtree`(u, p)
   /* which subtree under u should we insert p into? */
7. `insert`(v, p)

Overflow Handling

Algorithm `handle-overflow`(u)

1. `split`(u) into u and u'
2. **if** u is the root **then**
3. create a new root with u and u' as its child nodes
4. **else**
5. $w \leftarrow$ the parent of u
6. update $MBR(u)$ in w
7. add u' as a child of w
8. **if** w overflows **then**
9. `handle-overflow`(w)

R-Tree (Insertion)

Splitting a Leaf Node

Algorithm `split`(u)

1. m = the number of points in u
2. sort the points of u on x-dimension
3. **for** $i = \lceil 0.4B \rceil$ to $m - \lceil 0.4B \rceil$
4. $\quad S_1 \leftarrow$ the set of the first i points in the list
5. $\quad S_2 \leftarrow$ the set of the other i points in the list
6. $\quad$ calculate the perimeter sum of $MBR(S_1)$ and $MBR(S_2)$; record it if this is the best split so far
7. Repeat Lines 2-6 with respect to y-dimension
8. **return** the best split found

R-Tree (Insertion)

Heuristic Splitting

Splitting an Internal Node

Let S be a set of $B+1$ rectangles. Divide S into two disjoint sets S_1 and S_2 to minimize the perimeter sum of $MBR(S_1)$ and $MBR(S_2)$, subject to the condition that $|S_1| \geq 0.4B$ and $|S_2| \geq 0.4B$.

Once again, we will settle for an algorithm that is fast but does not always return an optimal split.

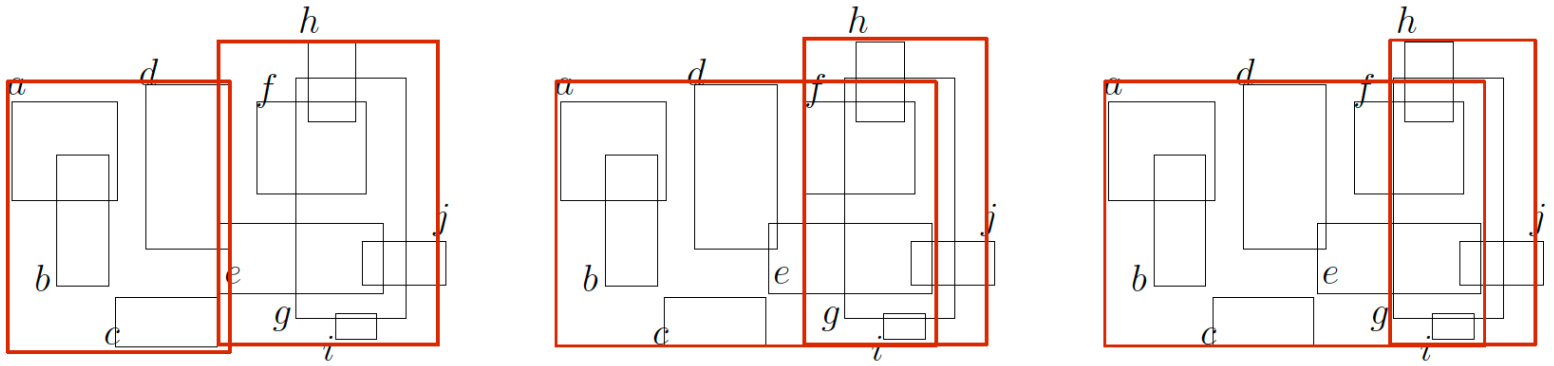
Splitting an Internal Node

Algorithm `split`(u)
/* u is an internal node */

1. m = the number of points in u
2. sort the rectangles in u by their left boundaries on the x-dimension
3. **for** $i = \lceil 0.4B \rceil$ to $m - \lceil 0.4B \rceil$
4. $S_1 \leftarrow$ the set of the first i rectangles in the list
5. $S_2 \leftarrow$ the set of the other i rectangles in the list
6. calculate the perimeter sum of $MBR(S_1)$ and $MBR(S_2)$; record it if this is the best split so far
7. Repeat Lines 2-6 with respect to the right boundaries on the x-dimension
8. Repeat Lines 2-7 w.r.t. the y-dimension
9. **return** the best split found

R-Tree (Insertion)

Example

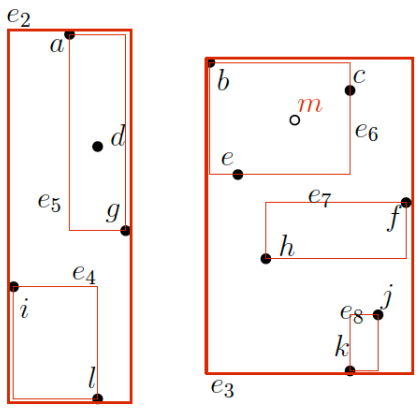


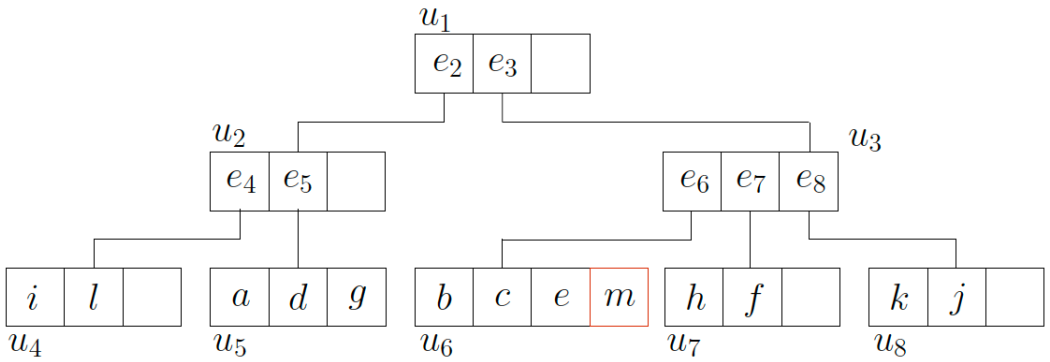
There are 3 possible splits w.r.t. the left boundaries on the x-dimension. Remember that each node must have at least $0.4B = 4$ points (here $B = 10$).

R-Tree (Insertion)

Insertion Example

Assume that we want to insert the white point m. By applying `choose-subtree` twice, we reach the leaf node u_6 that should accommodate m. The node overflows after incorporating m (recall $B = 3$).

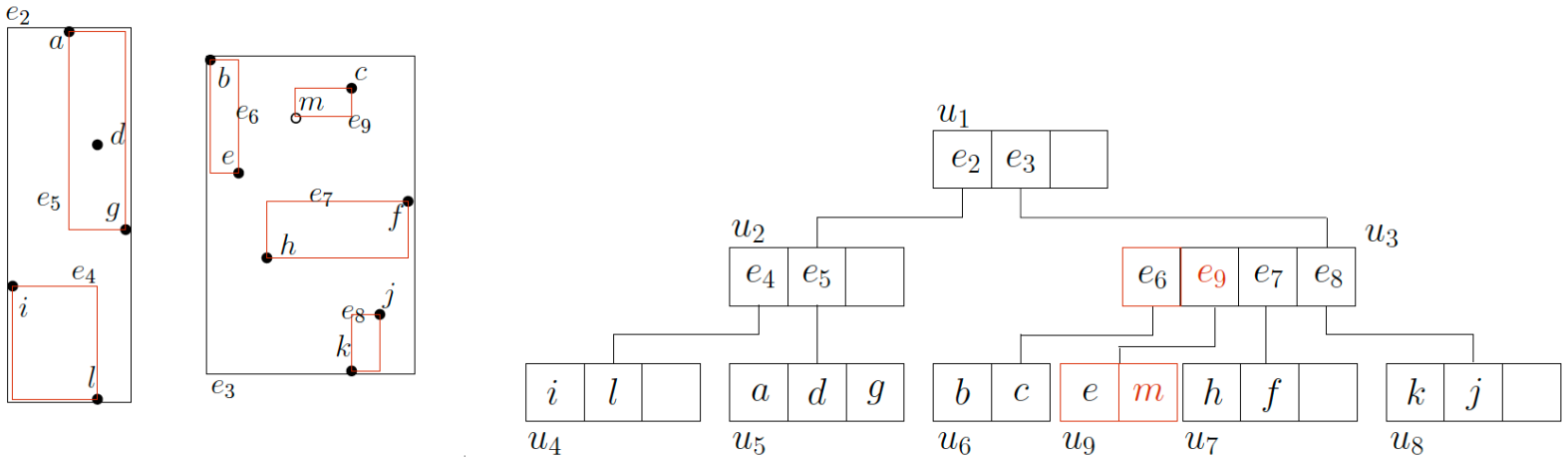




R-Tree (Insertion)

Insertion Example

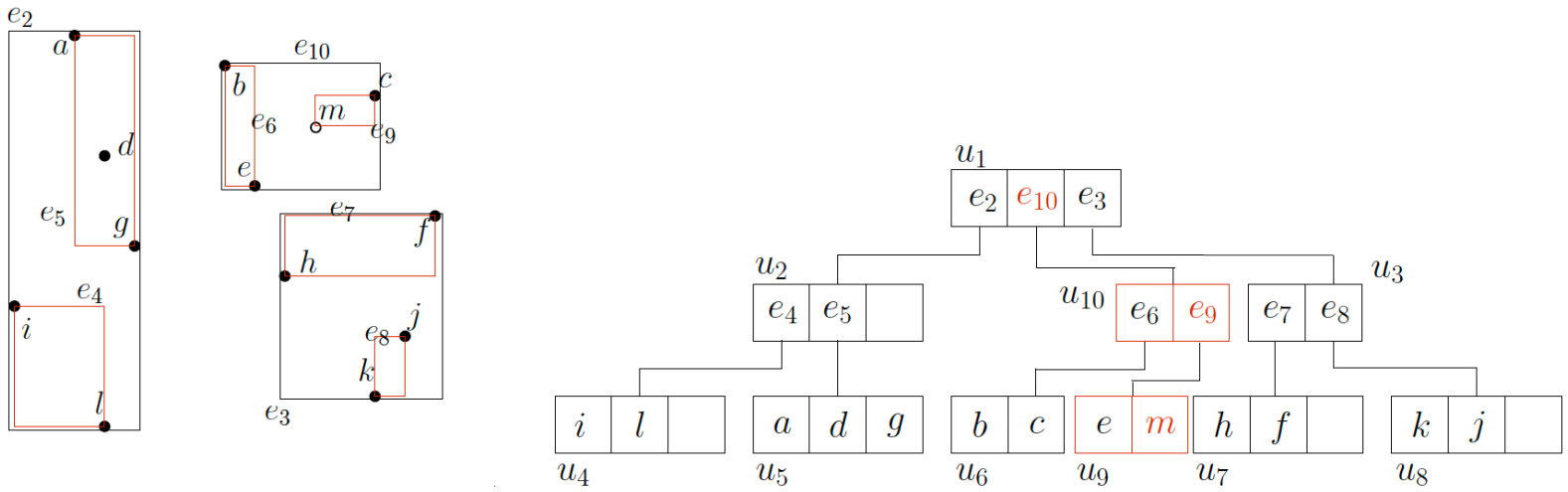
Node u_6 splits, generating u_9. Adding u_9 as a child of u_3 causes u_3 to overflow.



R-Tree (Insertion)

Insertion Example

Node u_3 splits, generating u_{10}. The insertion finishes after adding u_{10} as a child of the root.



R-Tree (Deletion)

- Node underflows are handled in a way that differs considerably from the conventional merging approach as in a B-tree.
- let p be the point to be deleted. First, we need to find the leaf node u where p is stored. This can be achieved with a special range query using p itself as the search region. Then, p is removed from u. The deletion finishes if u still has λB items, where λ denotes the minimum node utilization.
- Otherwise, u underflows, which is handled by first removing u from its parent, and then re-inserting all the remaining points in u (using exactly the insertion algorithm).
- Note that removing u from parent(u) may cause parent(u) to underflow too.
- In general, the underflow of a non-leaf node u' is also handled by re-insertions, with the only difference that the items re-inserted are MBRs, and each MBR is re-inserted to the same level of u'.
- Note that re-insertion actually gives better search performance than merging-based algorithms to handle node underflows.
- This is because the structure of an R-tree is sensitive to the insertion order of the data points. Re-insertion gives the the early inserted points to be inserted in other (better) branches, thus improving the overall structure.

Reference

- Quad Tree and kd trees:
  - https://www.cs.umd.edu/class/spring2022/cmsc420-0101/Lects/lect13-kd-dict.pdf
  - https://opendsa-server.cs.vt.edu/ODSA/Books/CS3/html/PRquadtree.html
  - https://opendsa-server.cs.vt.edu/ODSA/Books/CS3/html/KDtree.html
- K-nearest Search :
  - http://45.33.90.109/courses/cs201/w22/notes/trees-kd.html#sec-2
- R-Tree :
  - Antonin Guttman. 1984. R-trees: a dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD international conference on Management of data* (SIGMOD '84)
  - https://www.cse.cuhk.edu.hk/~taoyf/course/infs4205/lec/rtree.pdf

Computational Geometry

- ***Computational geometry*** is a branch of computer science that focuses on the study of algorithms which can be expressed in terms of geometry.
- It deals with the design and analysis of algorithms for solving geometric problems involving points, lines, polygons, and other geometric objects

- Two main branches of computational geometry:
  1. **Combinatorial Computational Geometry**: This branch deals with geometric objects as discrete entities. It includes problems like finding the convex hull of a set of points, determining the intersections of line segments, and solving the closest pair of points problem.
  - **Numerical Computational Geometry**: Also known as geometric modeling or computer-aided geometric design (CAGD), this branch focuses on representing real-world objects in forms suitable for computer computations, often used in CAD/CAM systems.

Computational Geometry

Applications

1. **Computer Graphics**: Used in rendering scenes, modeling shapes, and animations. Algorithms for hidden surface removal, ray tracing, and mesh generation are based on computational geometry.
2. **Robotics**: Essential for motion planning, collision detection, and pathfinding. Robots use geometric algorithms to navigate environments and avoid obstacles.
3. **Geographic Information Systems (GIS)**: Helps in managing and analyzing spatial data. Applications include map overlay, terrain modeling, and spatial querying.
4. **Computer-Aided Design (CAD)**: Used in designing and manufacturing products. Geometric algorithms help in modeling complex shapes, surface fitting, and tool path generation.
5. **Computer Vision**: Involves 3D reconstruction, object recognition, and image segmentation. Geometric algorithms help in interpreting visual data and reconstructing 3D models from 2D images.

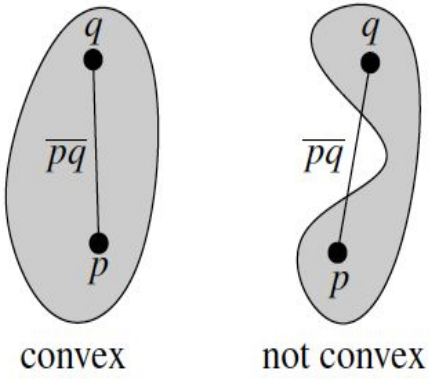
Computational Geometry

Applications

6. **Integrated Circuit Design**: Used in the layout and verification of circuits. Algorithms help in optimizing the placement of components and routing of connections.
7. **Virtual Reality (VR) and Augmented Reality (AR)**: Geometric algorithms are used to create immersive environments and overlay digital information onto the real world.
8. **Medical Imaging**: Helps in reconstructing 3D images from 2D scans, such as CT or MRI scans, and in planning surgical procedures.

Computational Geometry (Convex Hull)

- The **convex hull** of a set of points is the smallest convex polygon that can enclose all the points.
- A subset S of the plane is called convex if and only if for any pair of points $p,q \in S$ the line segment $\overline{pq}$ is completely contained in S. The convex hull $CH(S)$ of a set S is the smallest convex set that contains S. To be more precise, it is the intersection of all convex sets that contain S.
- Lets visualize what the convex hull looks like by a thought experiment.
- Imagine that the points are nails sticking out of the plane, take an elastic rubber band, hold it around the nails, and let it go. It will snap around the nails, minimizing its length.
- The area enclosed by the rubber band is the convex hull of P.
- This leads to an alternative definition of the convex hull of a finite set P of points in the plane: it is the unique convex polygon whose vertices are points from P and that contains all points of P.



Computational Geometry (Convex Hull)

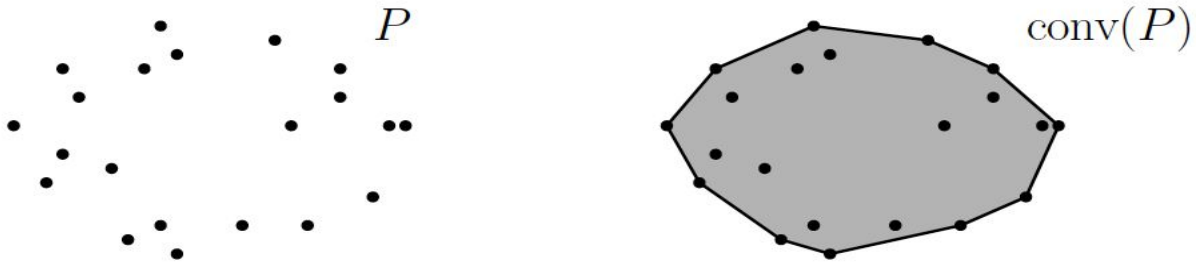


Fig: Point sets and a convex hull

Properties of Convex hull

Convexity: The convex hull is always convex, meaning any line segment between two points inside it lies entirely within the hull.

Minimality: The convex hull is the smallest convex shape that completely contains the given points.

Uniqueness: For a set of points, there is only one convex hull.

Extreme Points: The vertices (corners) of the convex hull are the "outermost" points of the set,

which cannot be formed by combining other points.

Affine Invariance: If the points are transformed (e.g., rotated, scaled, or shifted), the convex hull of

Computational Geometry (Convex Hull)

Finding convex hull using Graham scan Algorithm

- **Input:** A set $P=\{p_1,p_2,\ldots,p_n\}$, where each p_i is a point in the plane (x_i,y_i).
- **Output:** The points forming the convex hull in counterclockwise order.

Steps:

1. **Find the Point with the Lowest Y-coordinate**:
    1. Identify the point p_0 with the lowest *y-coordinate* (and the lowest *x-coordinate* in case of ties). This point is guaranteed to be part of the convex hull.
2. **Sort Points by Polar Angle**:
    1. Sort the remaining points $\{p_1,p_2,\ldots,p_n\}$(excluding p_0) based on the **polar angle** they make with p_0.
        1. **Formula for Polar Angle**: $\theta=arctan(y-y_0/x-x_0)$ where (x,y) is the current point and (x_0,y_0) is p_0.
    2. If two points have the same angle, keep only the farthest point from p_0 using the Euclidean $d = \sqrt{(x - x_0)^2 + (y - y_0)^2}$

Computational Geometry (Convex Hull)

3. **Construct the Hull**:
   - Start with p_0 and the first two points in the sorted list as the initial hull (Stack).
4. **Check Orientation Using Cross Product**:
   - For each point p_i in the sorted list:
     - Compute the **orientation** of the triplet of points $(p_{prev}, p_{curr}, p_i)$:Cross Product: $(x_{curr} - x_{prev}) \cdot (y_i - y_{curr}) - (y_{curr} - y_{prev}) \cdot (x_i - x_{curr})$ OR

       $$\text{Cross Product} = (x_2 - x_1) \cdot (y_3 - y_2) - (y_2 - y_1) \cdot (x_3 - x_2)$$

       - If the result > 0: Left turn (valid for convex hull).
       - If the result < 0: Right turn (remove p_{curr} from the hull).
       - If the result = 0: Collinear points (retain the farthest point).
     - Keep removing the last point from the hull until the orientation forms a left turn.
5. **Close the Hull**:
   - After processing all points, the final list of points in the stack forms the convex hull.

Computational Geometry (Convex Hull)

GRAHAM_SCAN(P):

1. Find the point p_0 with the lowest y-coordinate (and smallest x-coordinate if there are ties).
2. Sort the points in P by polar angle with respect to p_0:
   a. For each point pi in P:
      - Compute the polar angle $\theta = \arctan(y_i - y_0, x_i - x_0)$.
      - If angles are the same, keep the point farthest from p_0.
3. Initialize a stack (HULL) to store the vertices of the convex hull.
   - Push p_0, the first sorted point, and the second sorted point onto HULL.
4. For each point p_i in the sorted list (starting from the third point):
   a. While there are at least two points in HULL and the last three points in HULL do not form a left turn:
      - Pop the top point from HULL.
   b. Push p_i onto HULL.
5. The points in HULL now form the convex hull in counterclockwise order.
6. Return HULL.

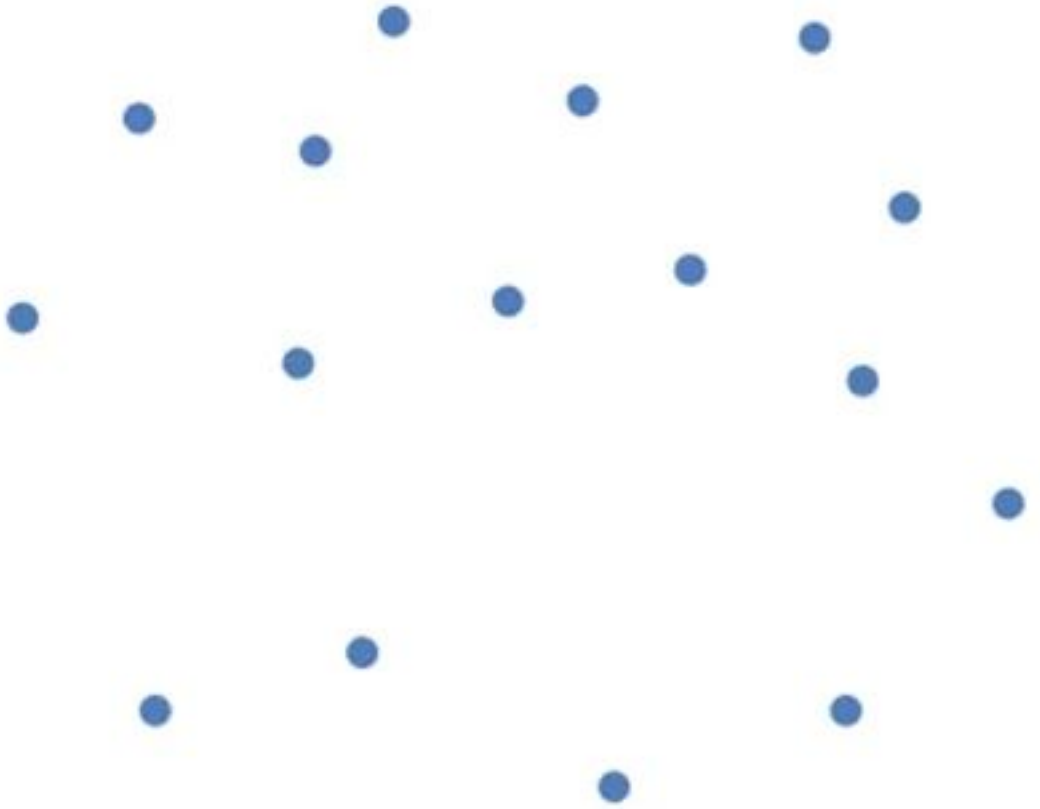
Graham Scan Algorithm Pseudocode

Graham's Scan Algorithm

Input: S, a set of n points in $\Re^2$.
Output: Σ, a stack containing conv(S) with the points in counter-clockwise order.

$p_0 \leftarrow$ the point in S with the minimum y-coordinate.
Sort S - $\{p_0\}$ with the orientations of $\overline{p_0 p_i}$ so that they are ordered $p_1, p_2, \ldots, p_{n-1}$.
PUSH(p_0, Σ)
PUSH(p_1, Σ).
for (i == 2) to $n - 1$ **do**
    while (*orient*(second(Σ), top(Σ), p_i) ≤ 0) **do**
        POP(Σ)
    end-while
    PUSH(p_i, Σ)
end-for
▷▷ Comment: Σ now contains the sequence of points forming conv(S) in counter clockwise order.
return Σ

Graham scan Algorithm(Example)

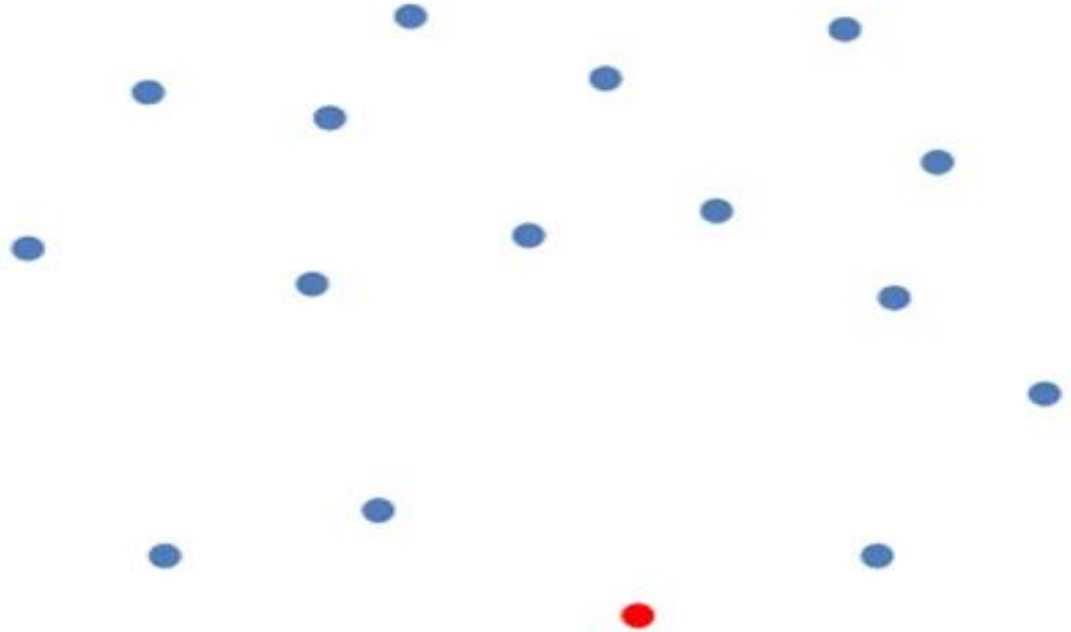




Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

We have a set of points. It's clear which point has the lowest y-coordinate v



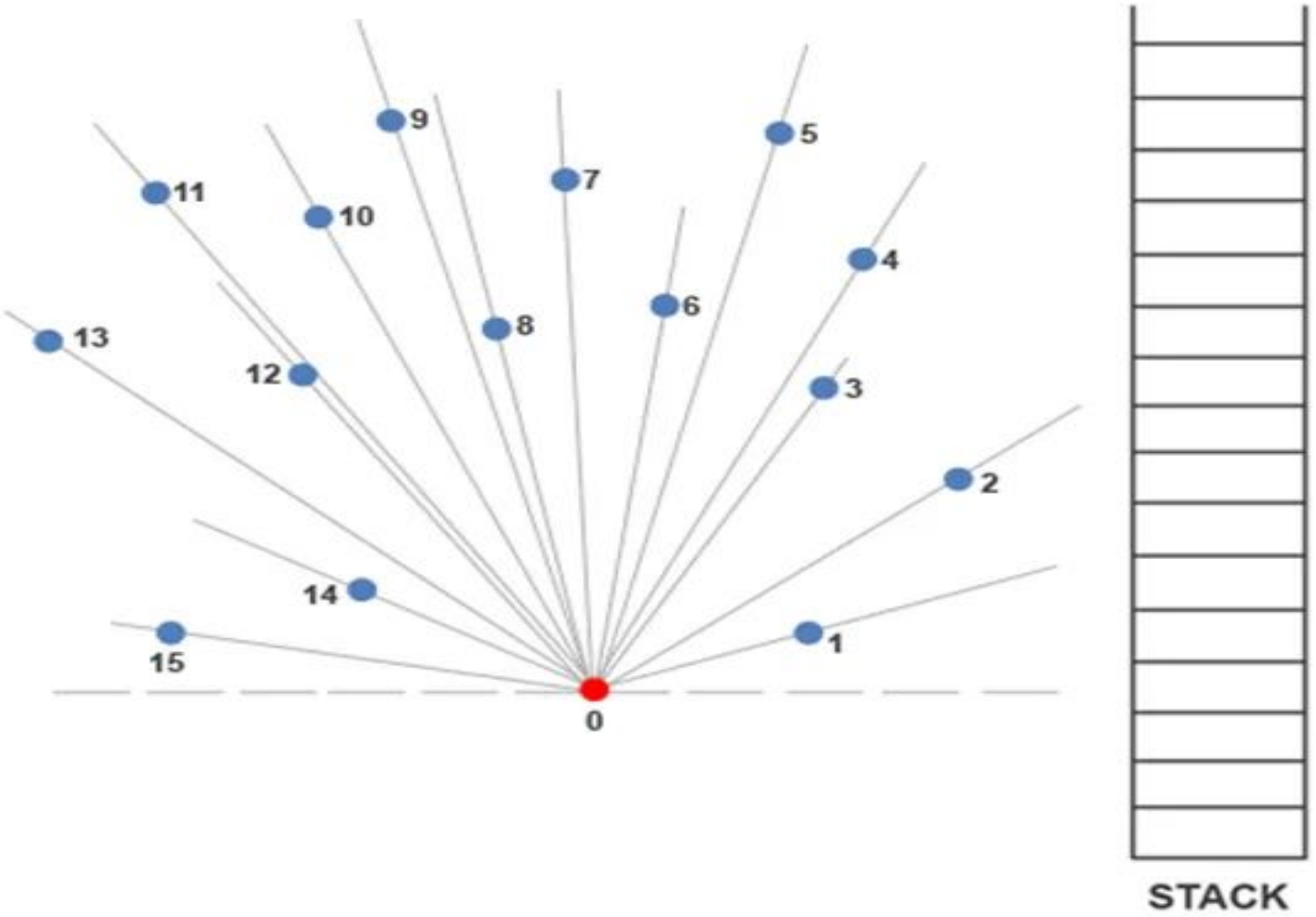


Computational Geometry (Convex Hull)

Graham scan Algorithm

(Example)

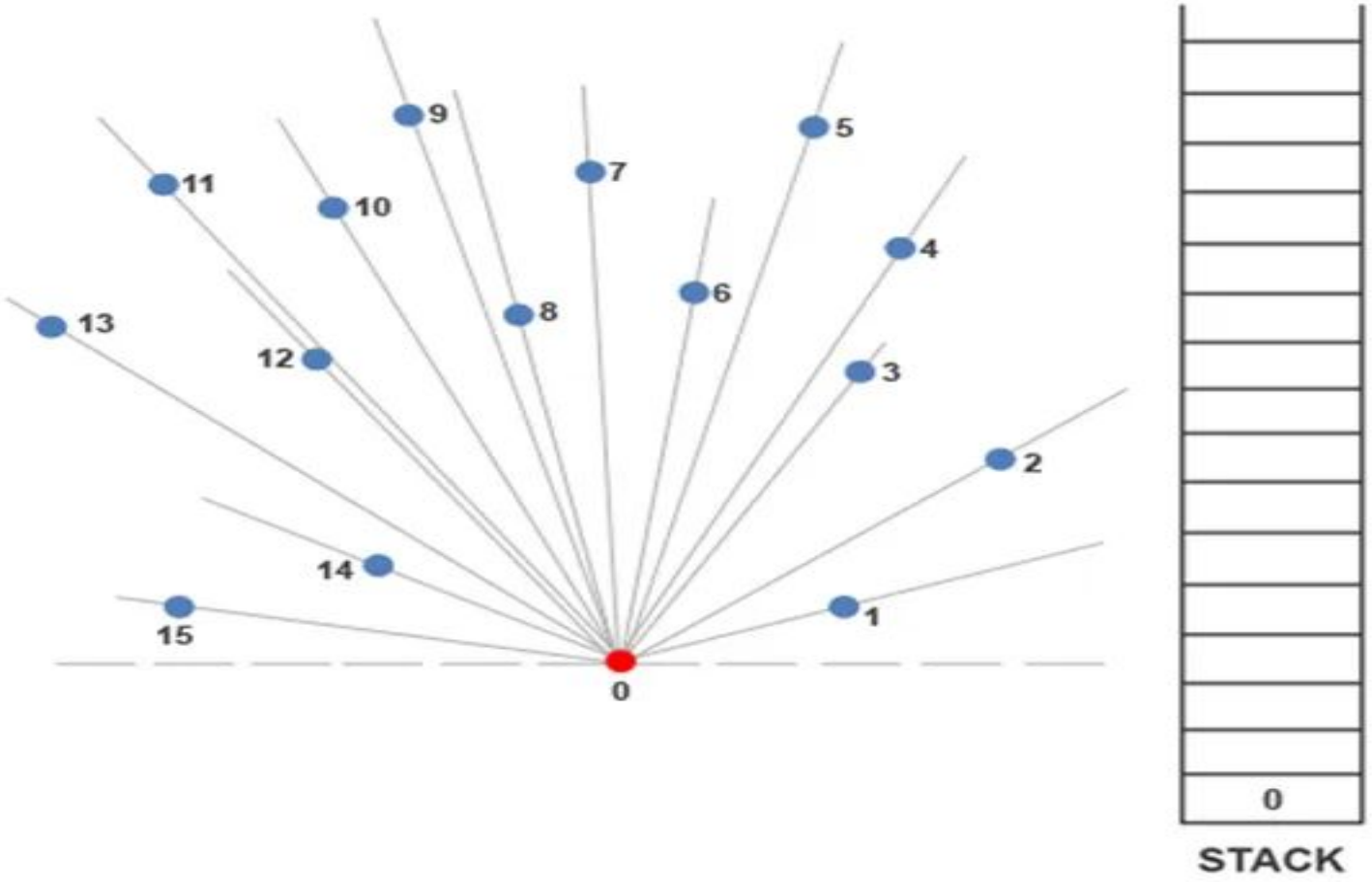
From the plot, points are ordered in increasing angle.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

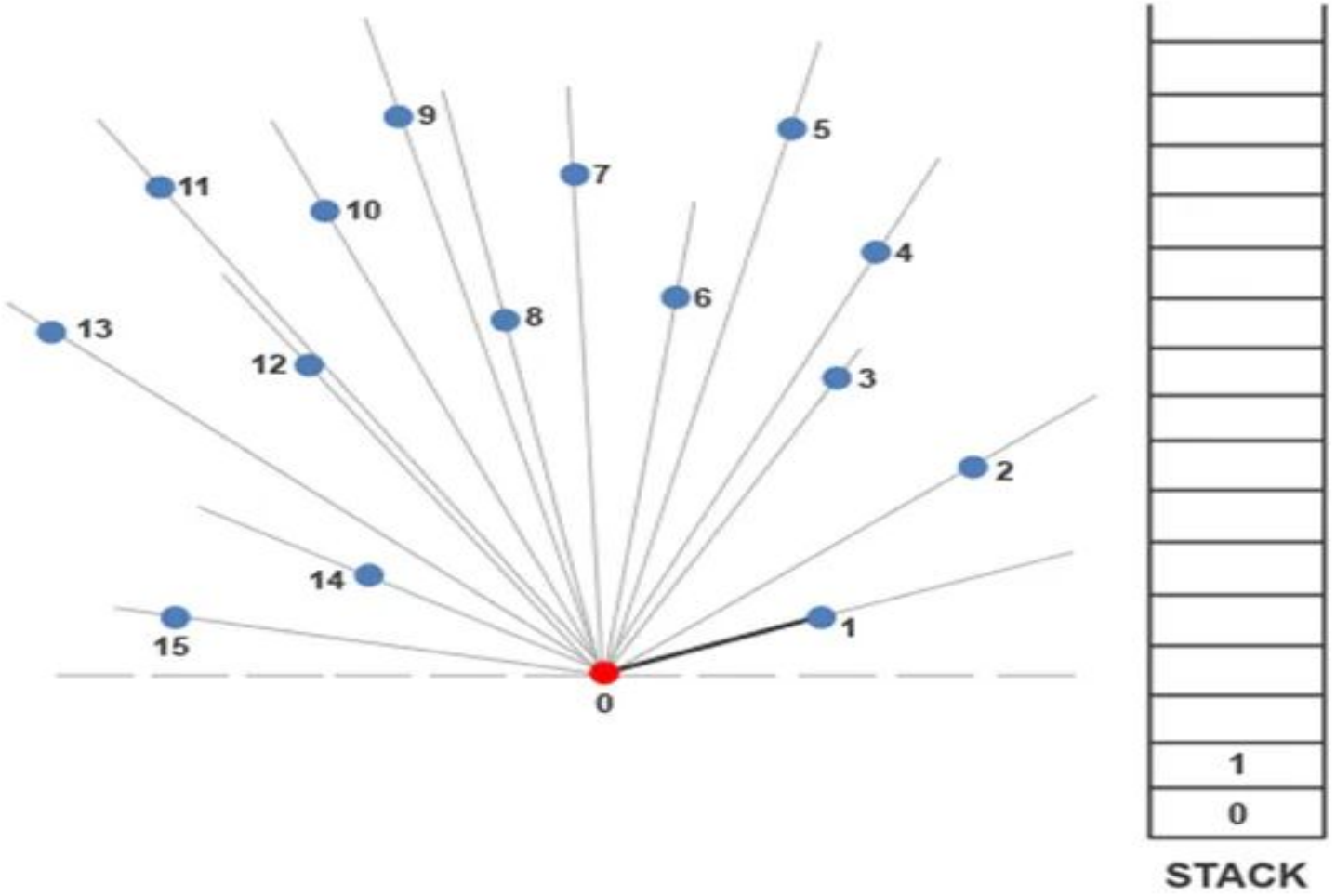
Now we can follow Graham's scan to find out which points create the convex hull. Point 0 is pushed onto the stack.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

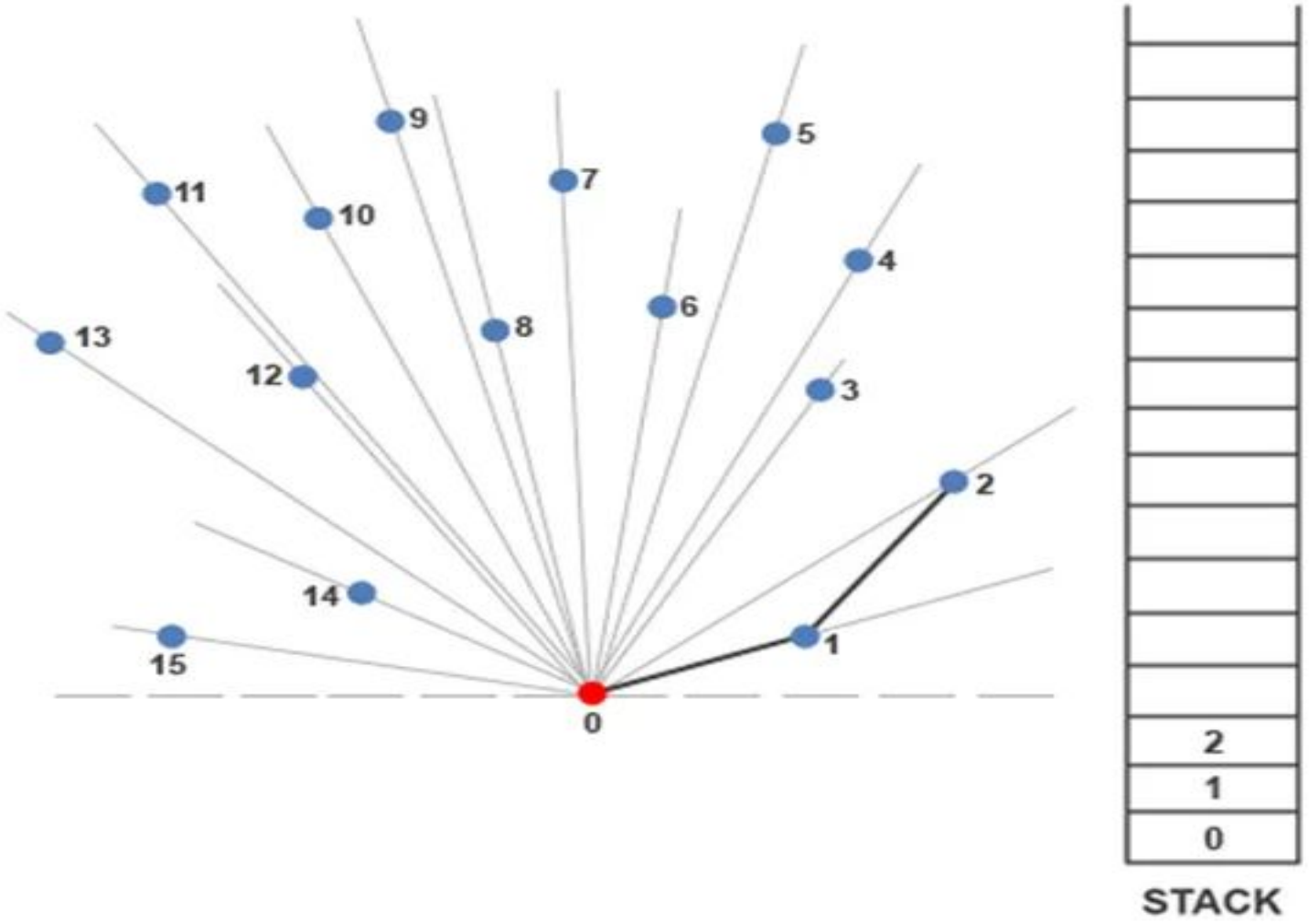
Point 1 is pushed onto the stack immediately after.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

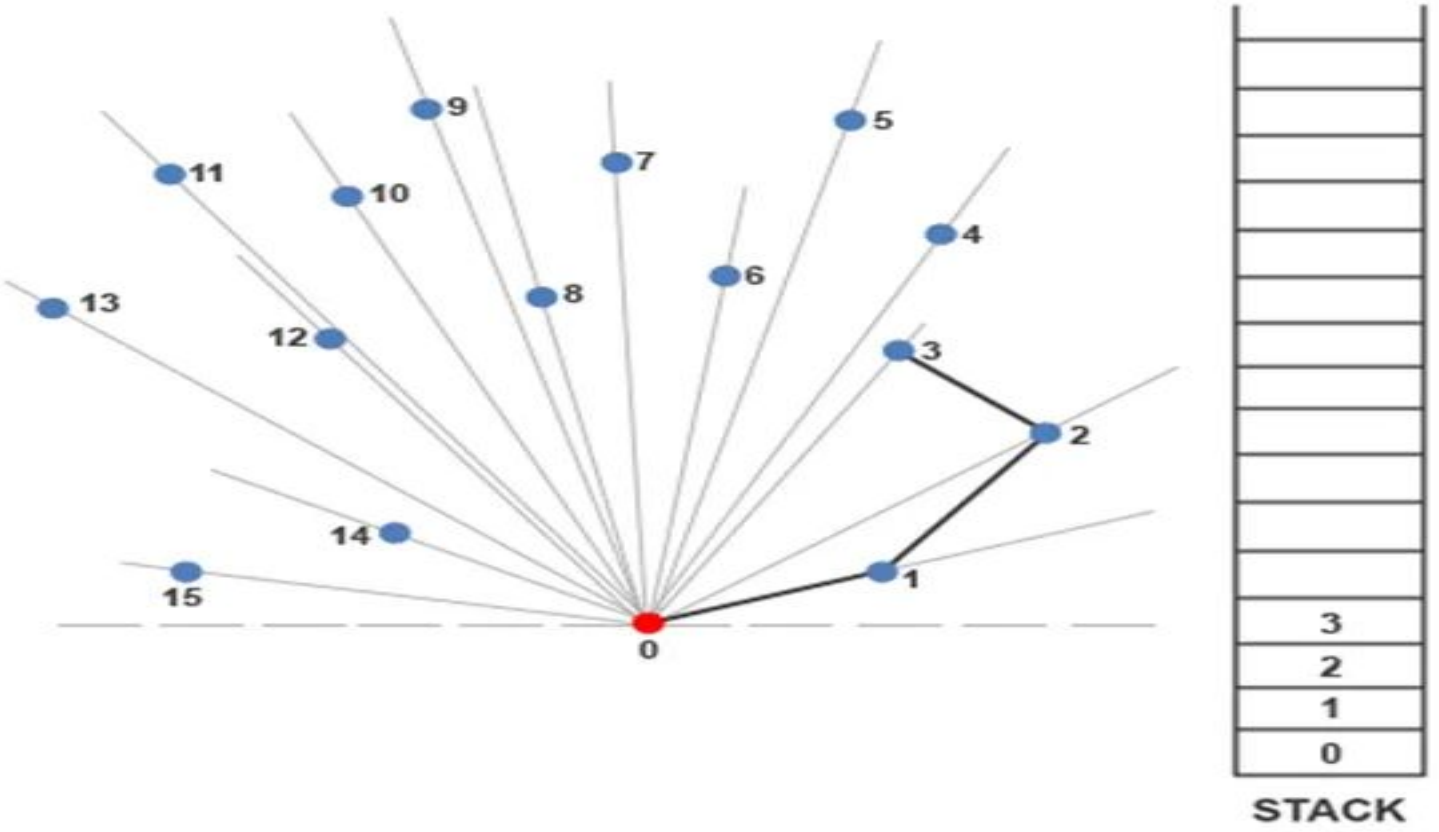
The next point to be added to the stack is 2. A line is made from point 1 to point 2.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

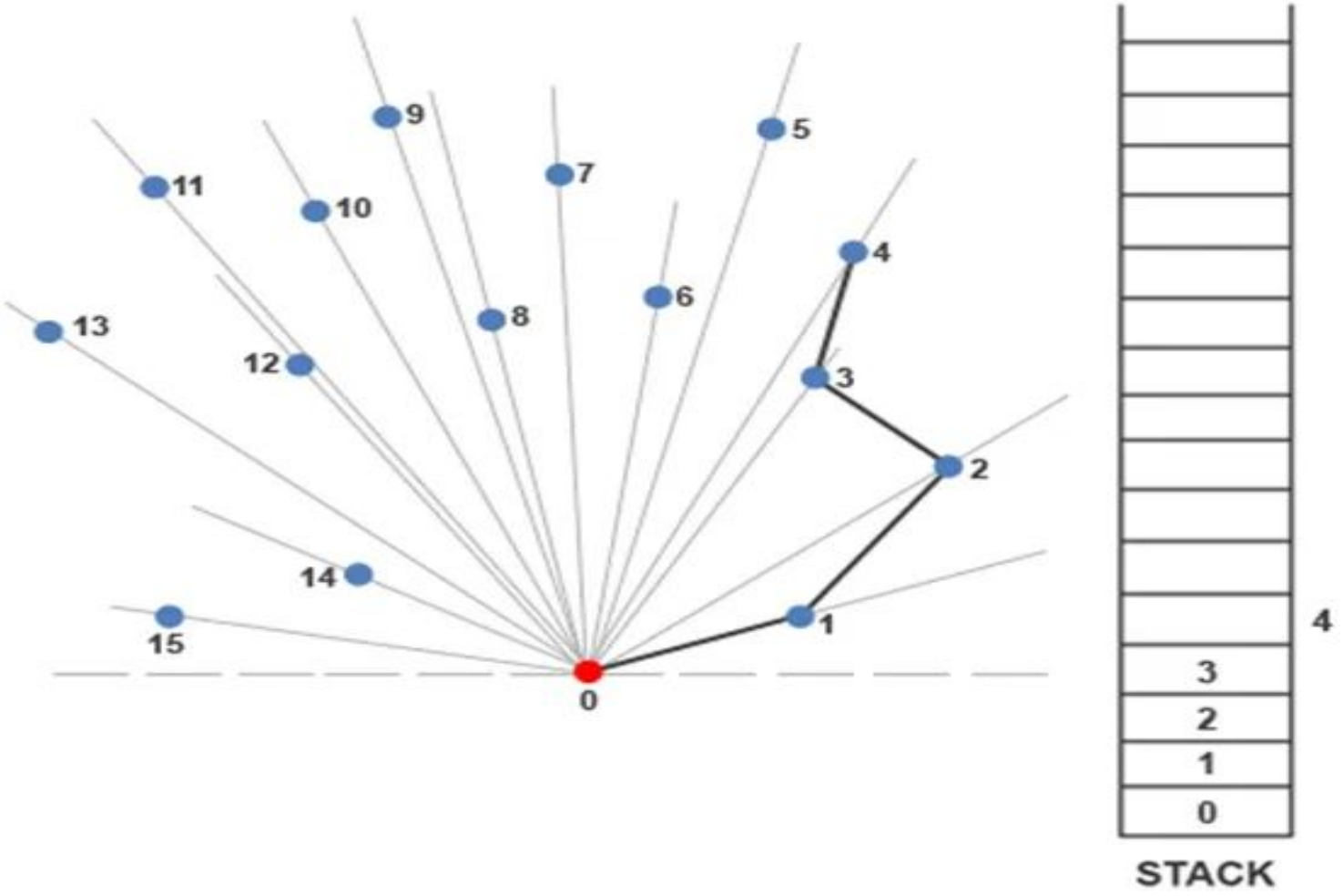
Whenever a left turn is made, the point is presumed to be part of the convex hull. We can clearly see a left turn being made to reach 2 from point 1. To get to point 3, another left turn is made. Currently, point 3 is part of the convex hull. A line segment is drawn from point 2 to 3 and 3 is pushed onto the s



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

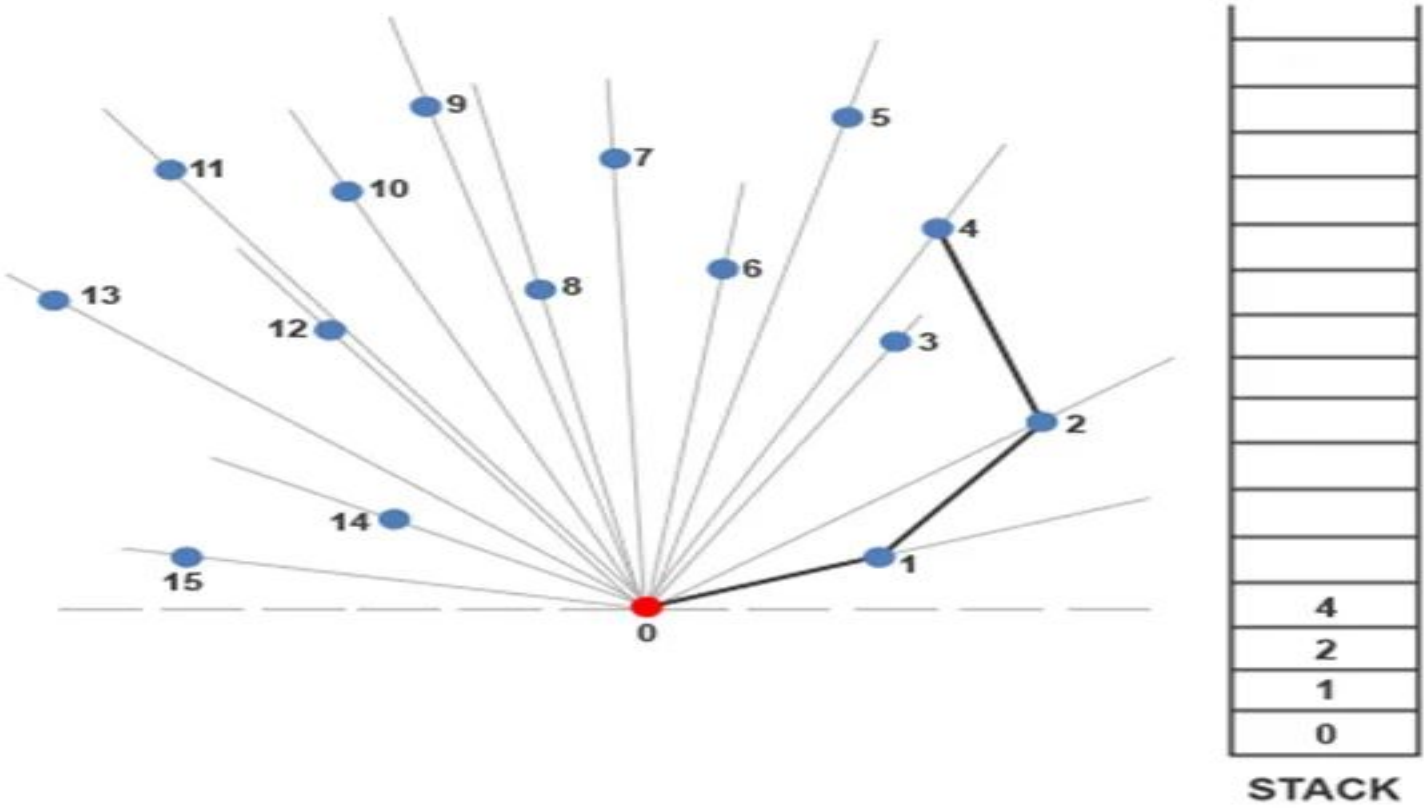
We make a right turn going to point 4. We'll draw the line to point 4 but will not push it onto the stack.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

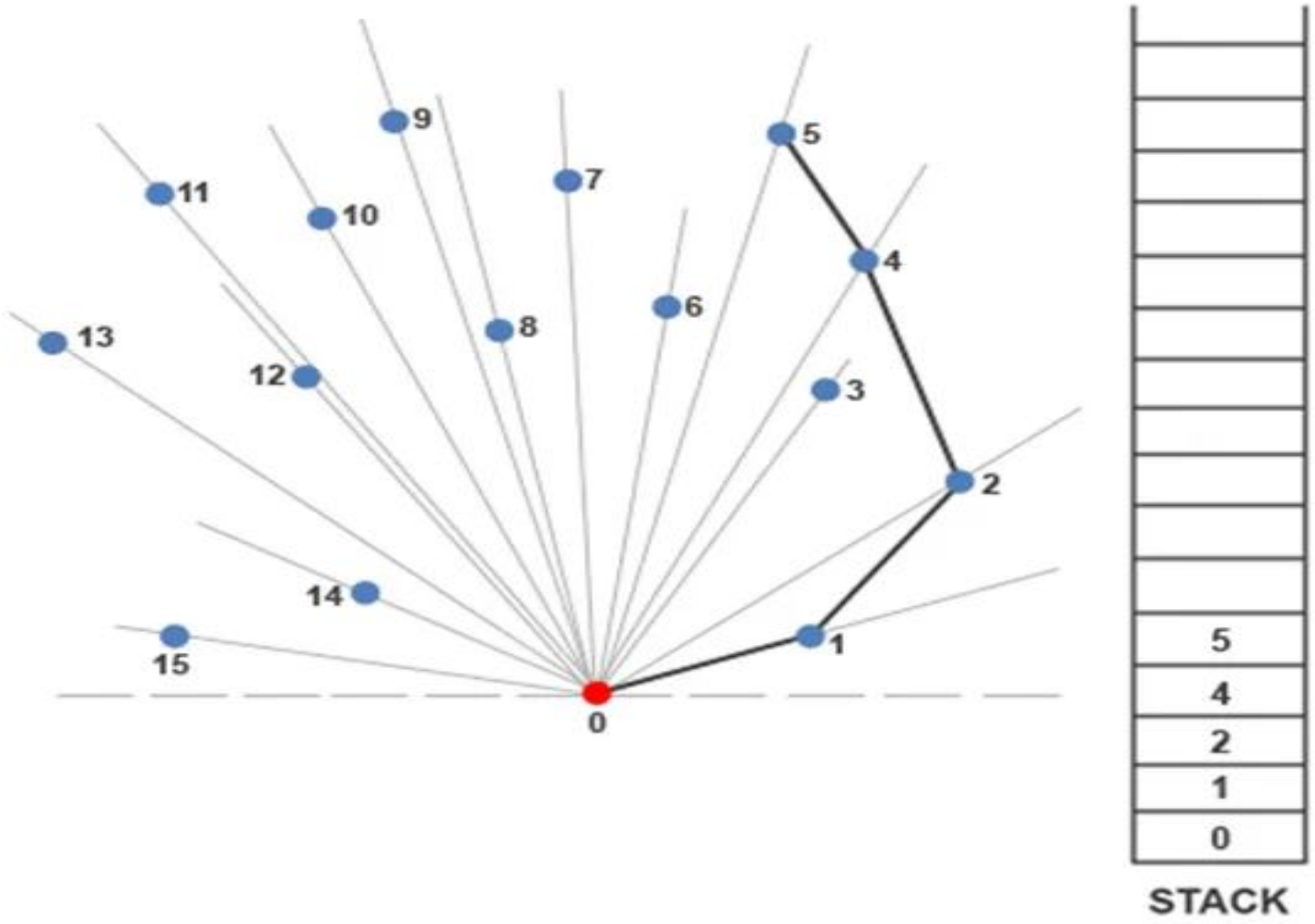
Whenever a right turn is made, Graham's scan algorithm pops the previous value from the stack and compares the new value with the top of the stack again. In this case, we'll pop 3 from the top of the stack and we'll see if going from point 2 to point 4 creates a left bend. In this case it does, so we'll draw a line segment from 2 to 4 and push 4 onto the stack.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

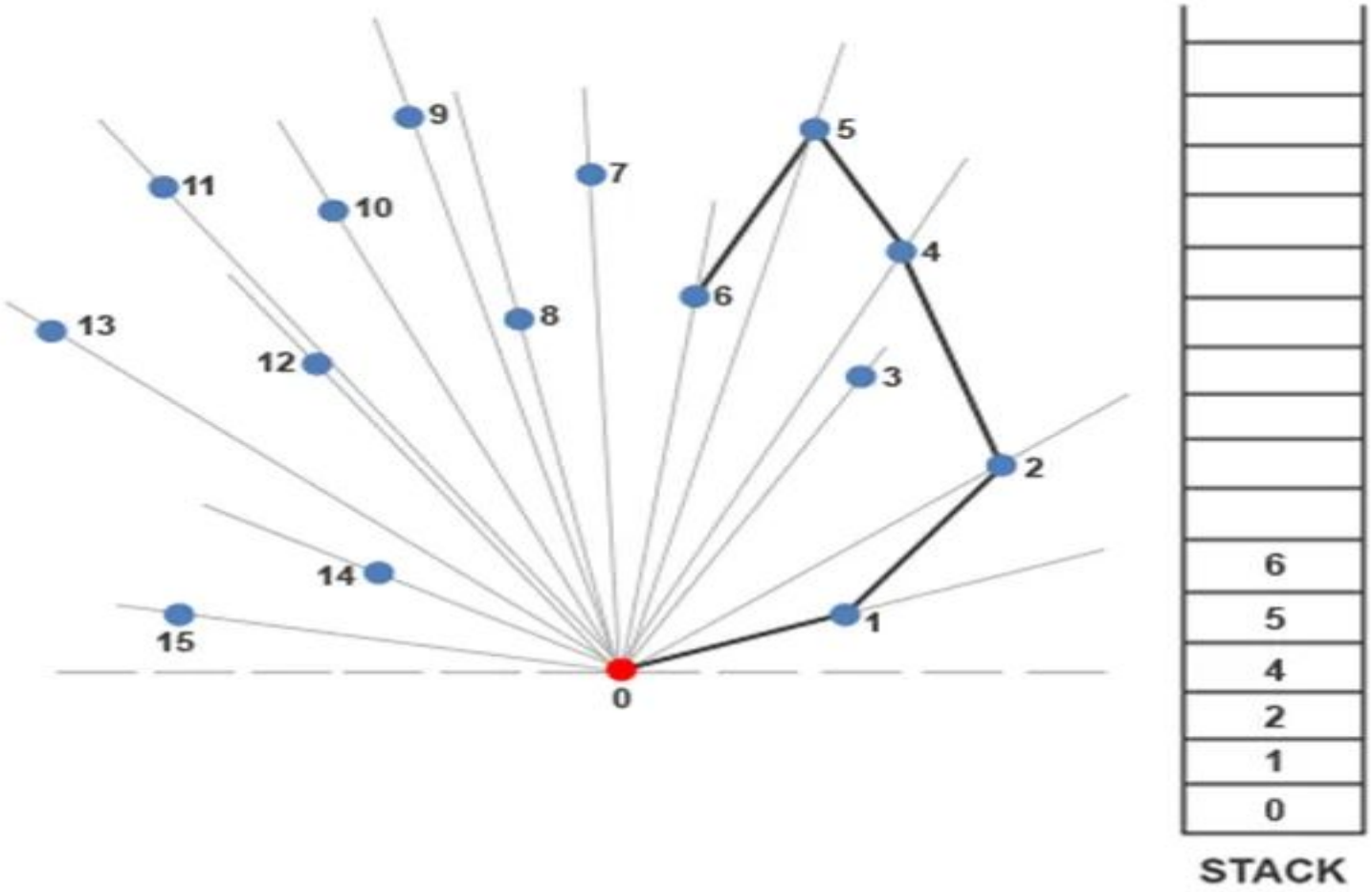
Since going from 4 to 5 creates a left turn, we'll push 5 onto the stack. Point 5 is currently part of the convex hull.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

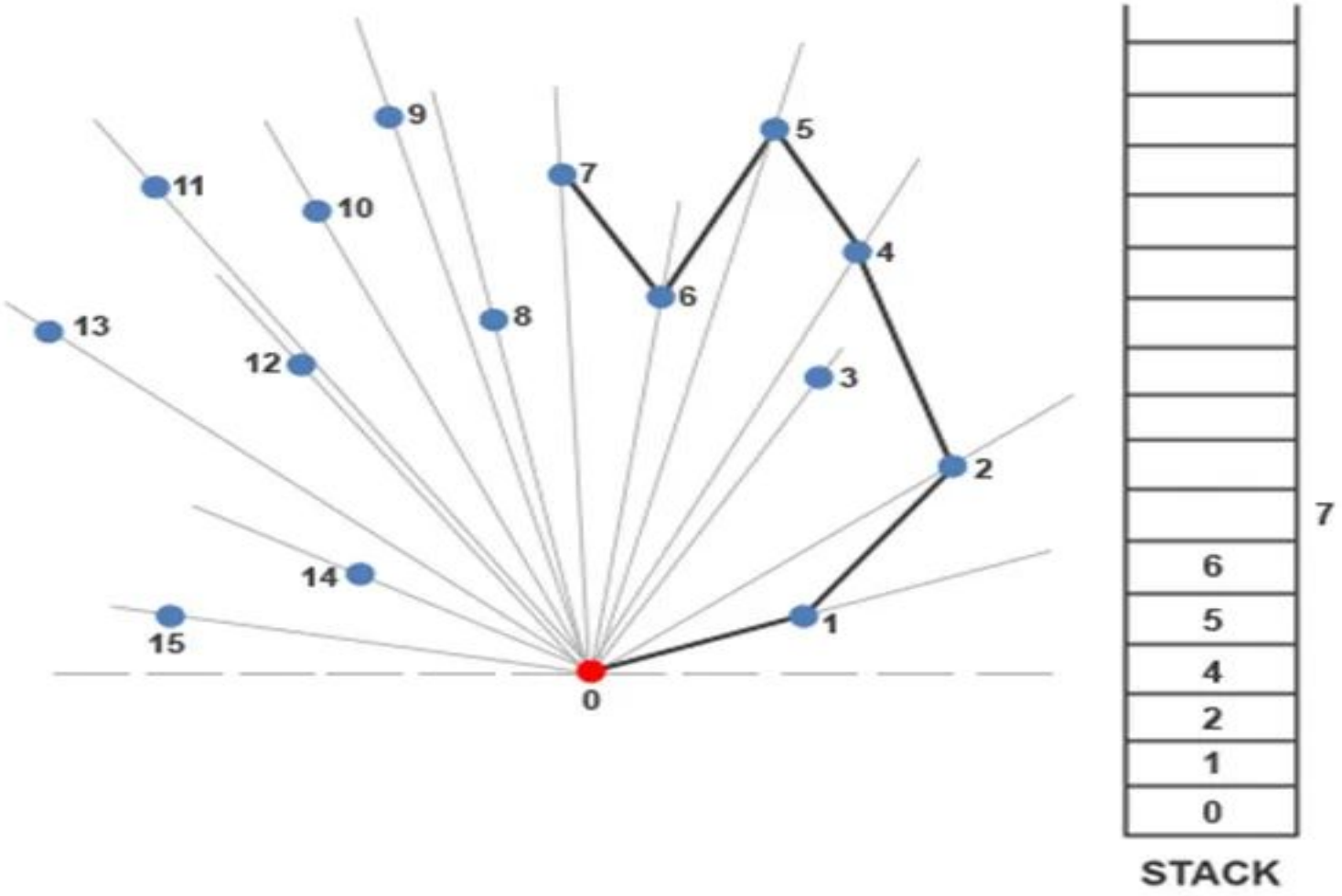
Moving from point 5 to 6 creates a left-hand turn, so we'll push 6 onto the stack. Point 6 is currently part of the convex hull.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

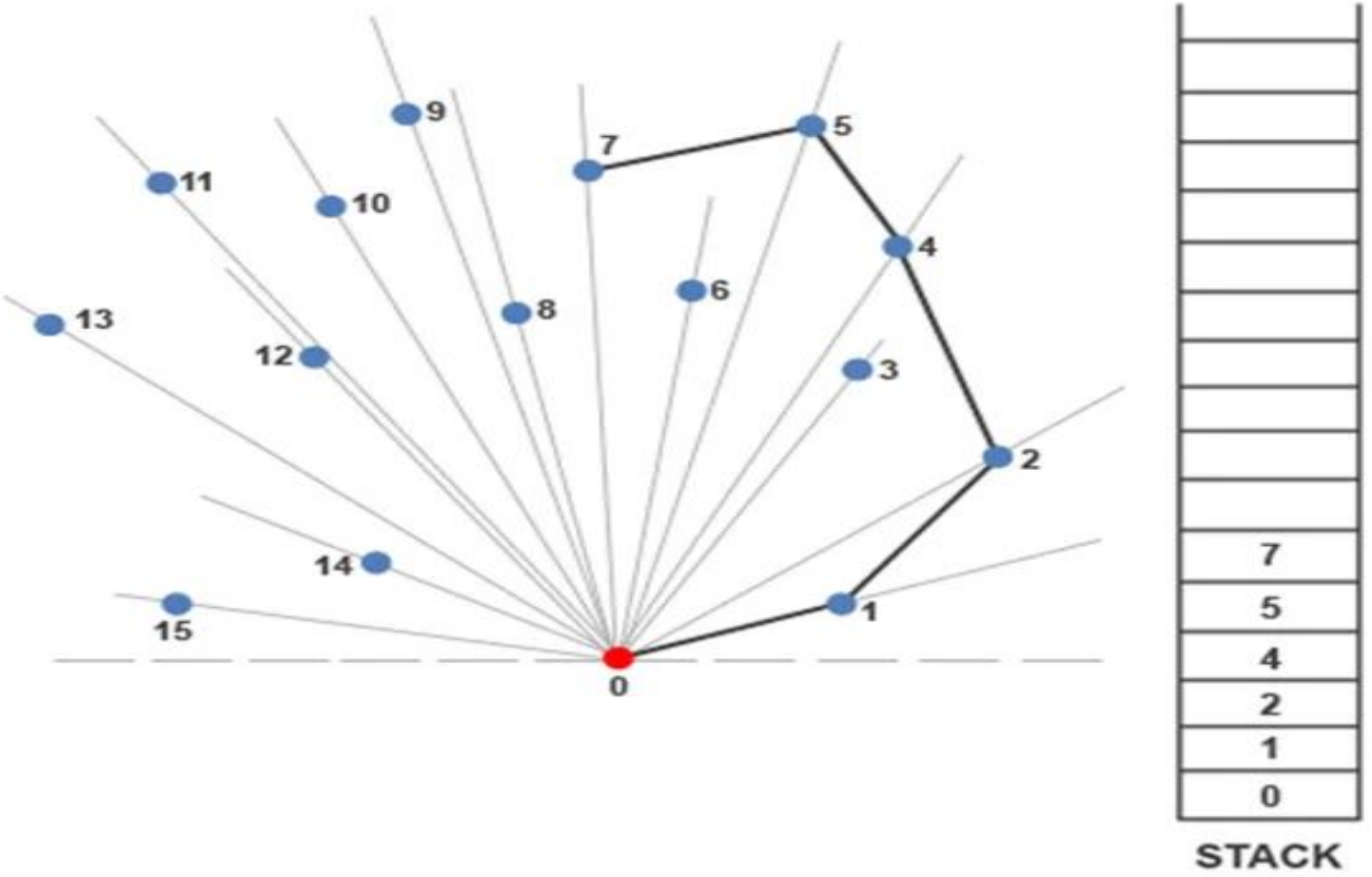
- To get to point 7, we must make a right-hand turn at 6.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

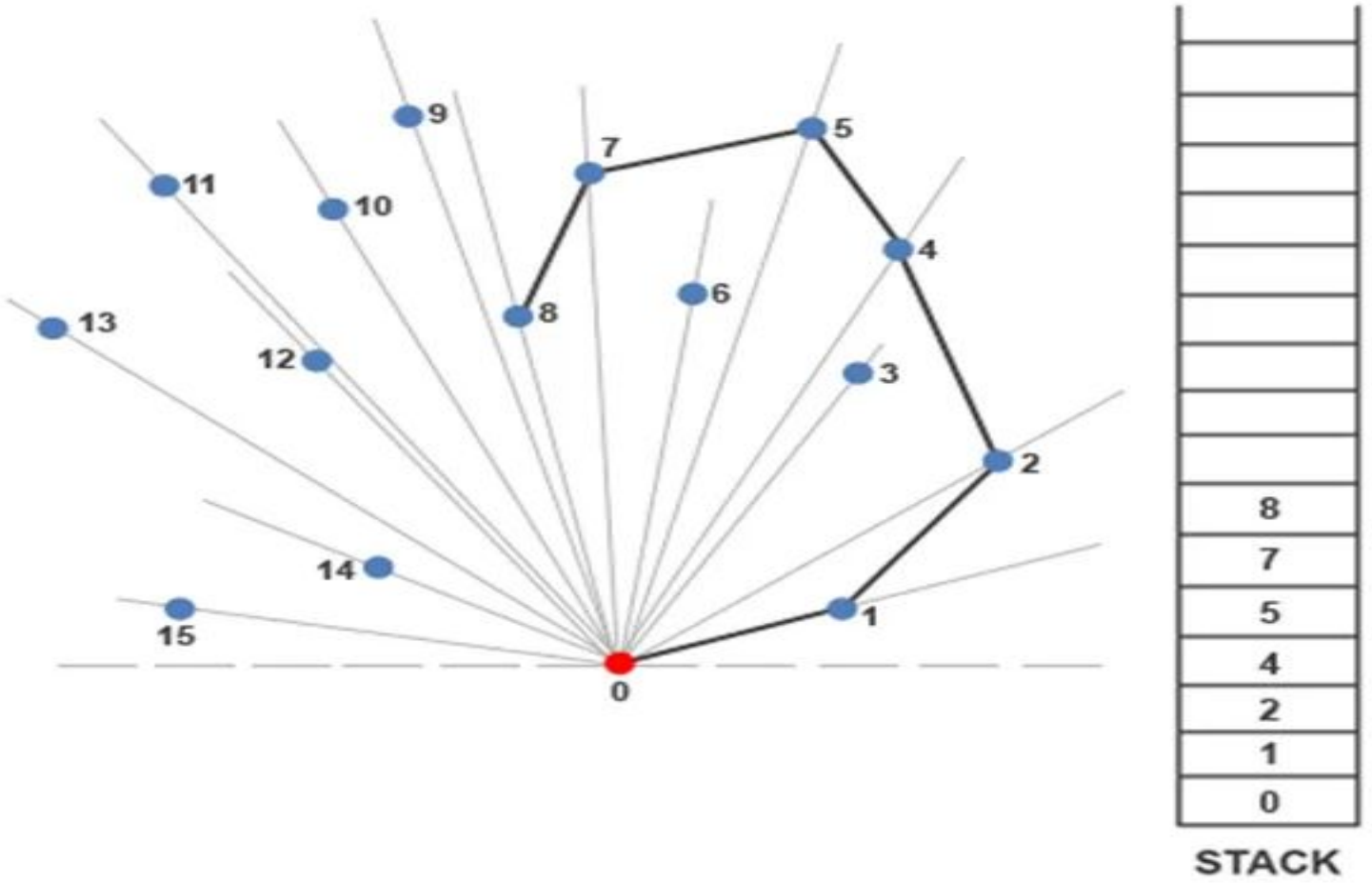
Point 6 is popped from the stack and the turn is examined from point 5 to point 7. Since we make a left hand turn from point 5 to point 7, we push point 7 onto the stack.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

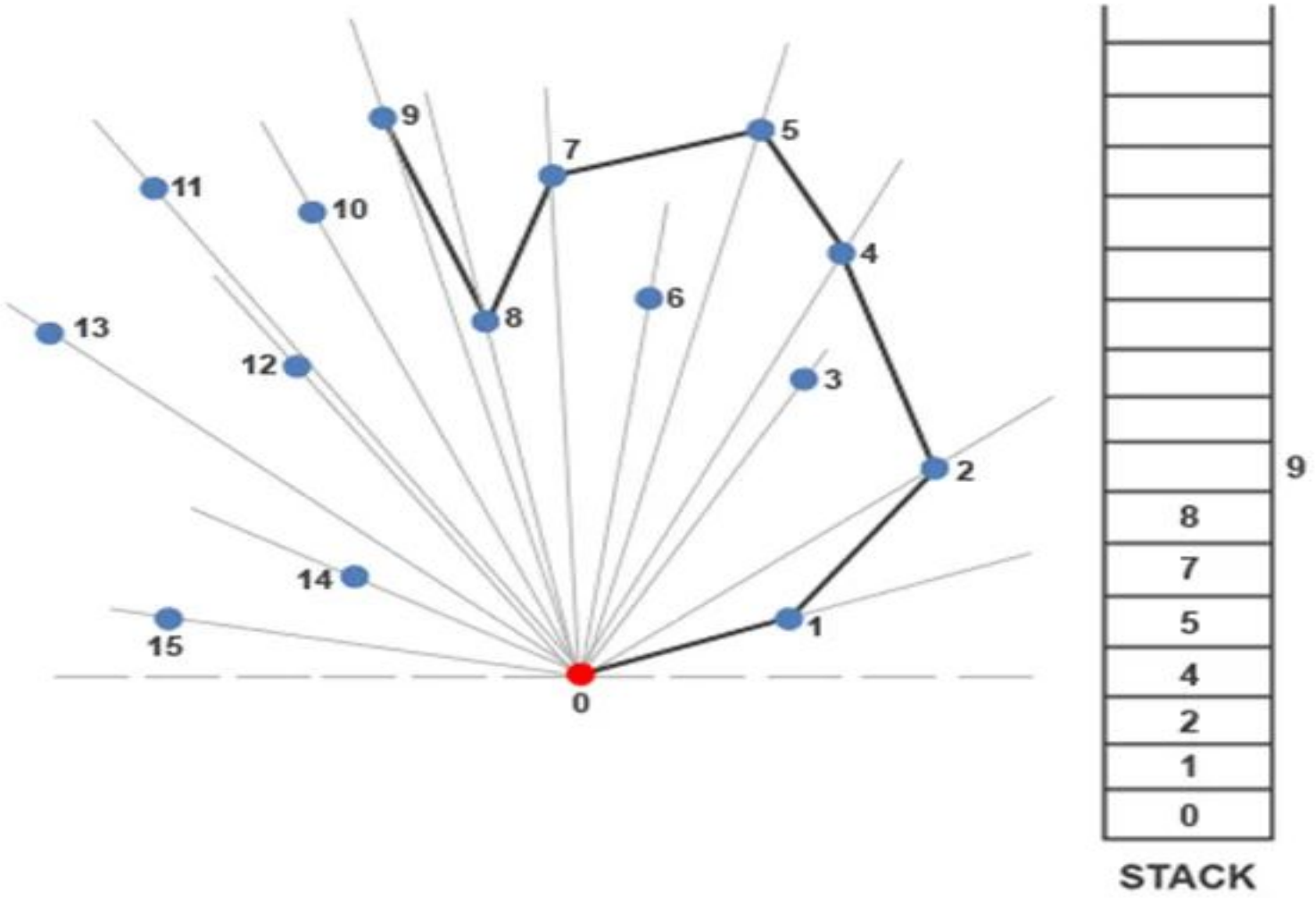
We attempt to push point 8 onto the stack. To get to point 8, we make a left at point 7, therefore point 8 is added to the stack. Point 8 is currently part of the convex hull.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

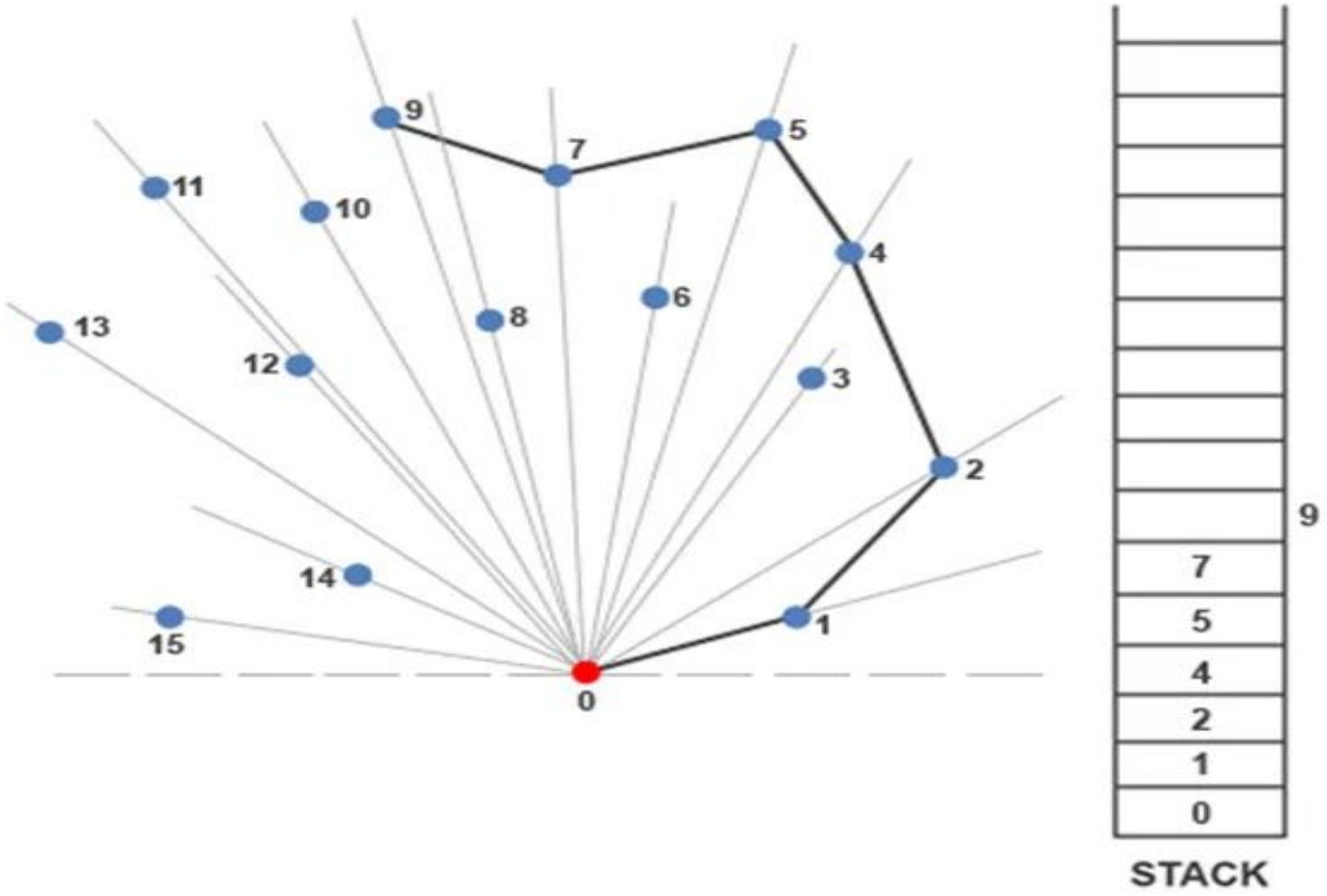
Going to point 9 requires a right-hand turn at point 8.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

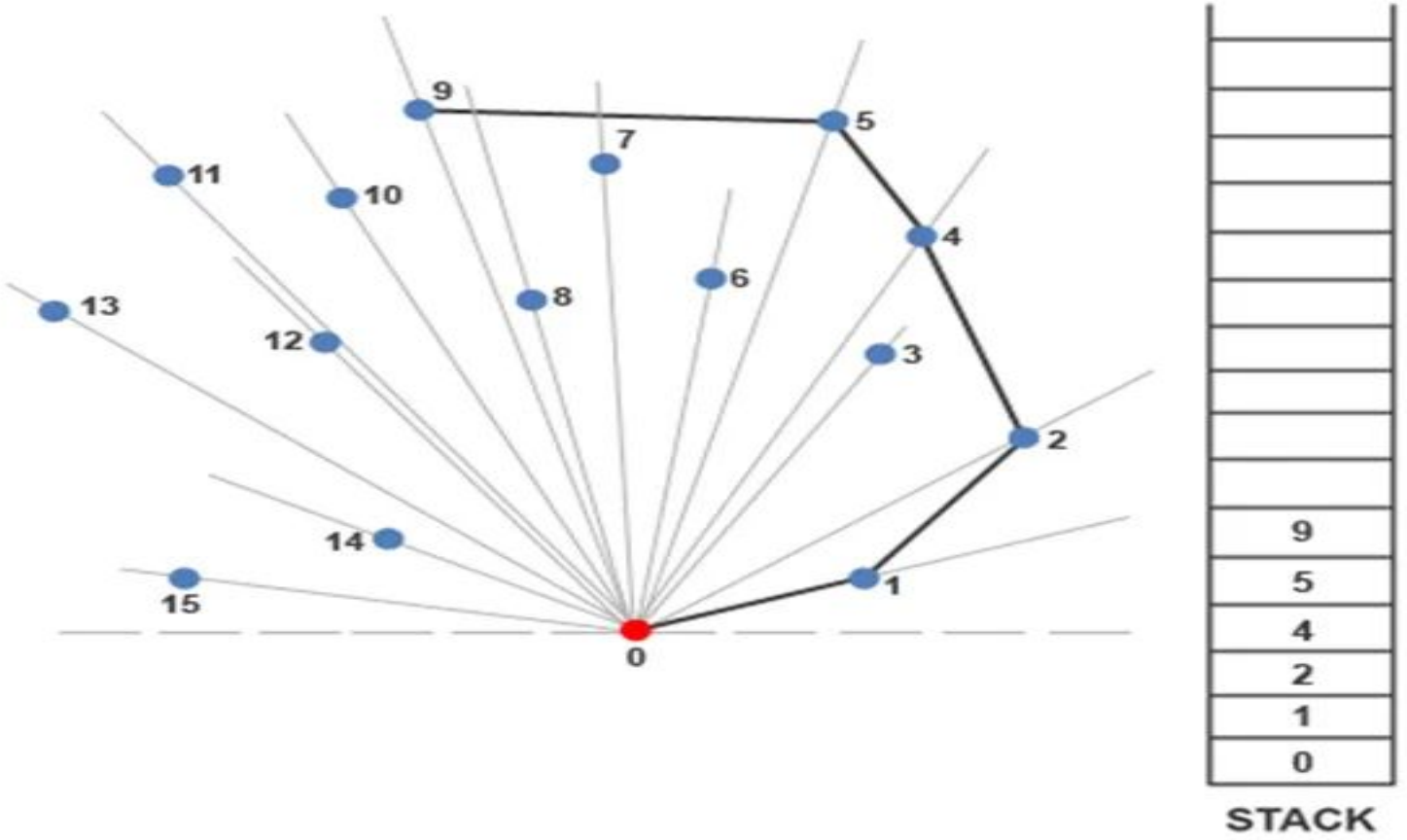
Since there's a right-hand turn, point 8 is popped from the stack and point 9 is compared with point 7.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

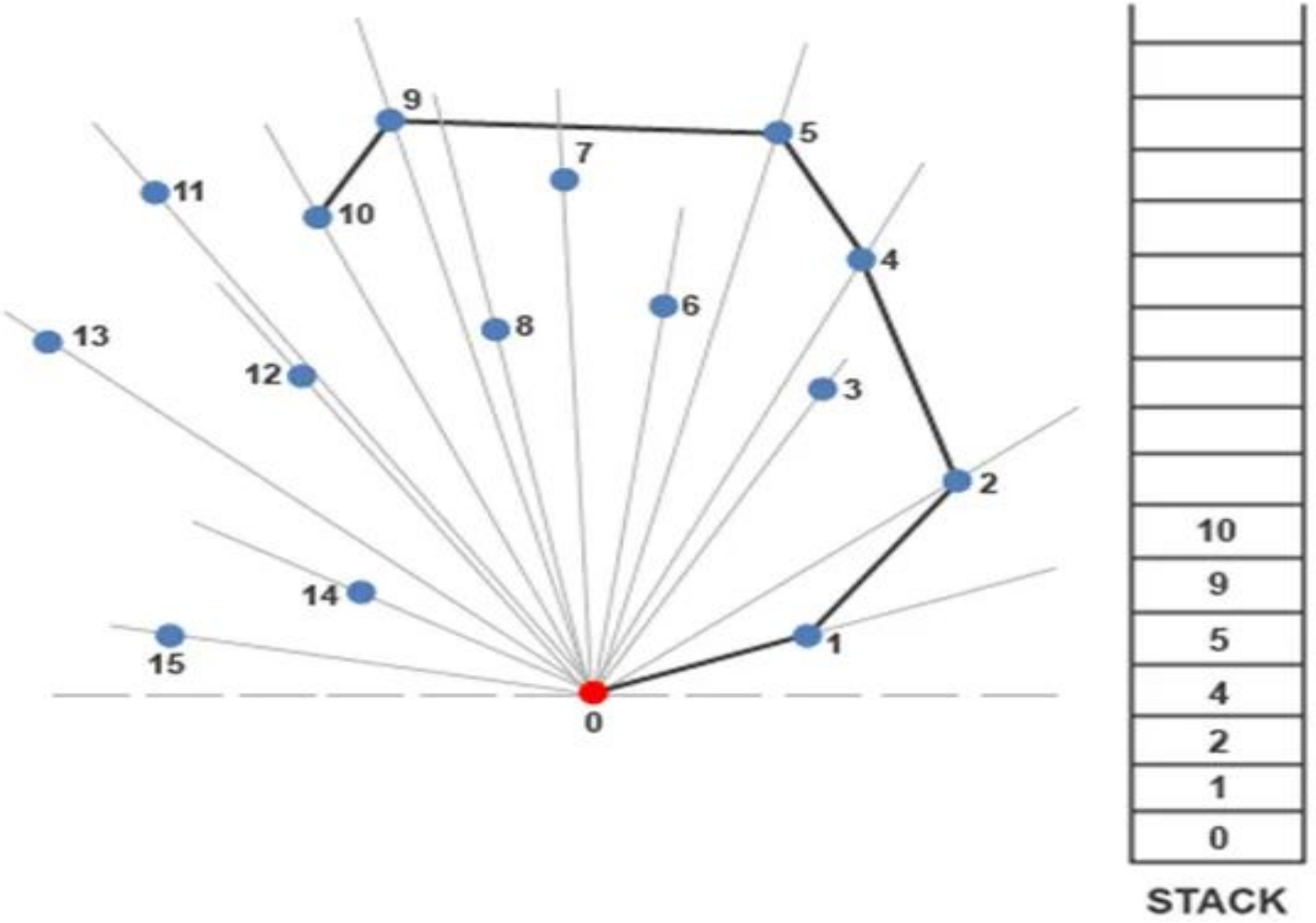
To get to point 9 from point 7 requires another right-turn, so we pop point 7 from the stack too and compare point 9 to point 5. We make a left-hand turn at point 5 to get to point 9, so 9 is pushed onto the stack.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

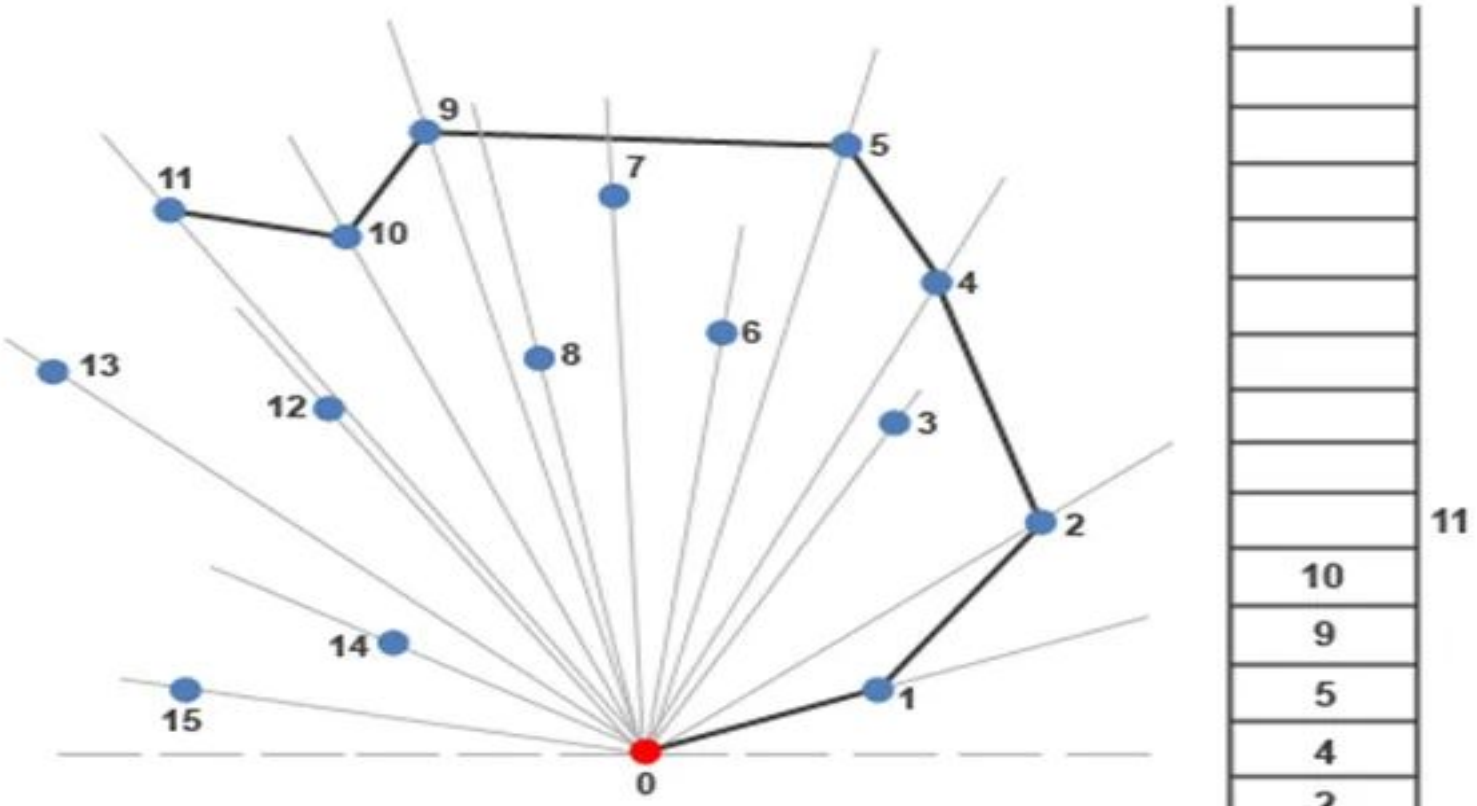
Next, we make a left turn to get to point 10. Point 10 is currently part of the convex hull.

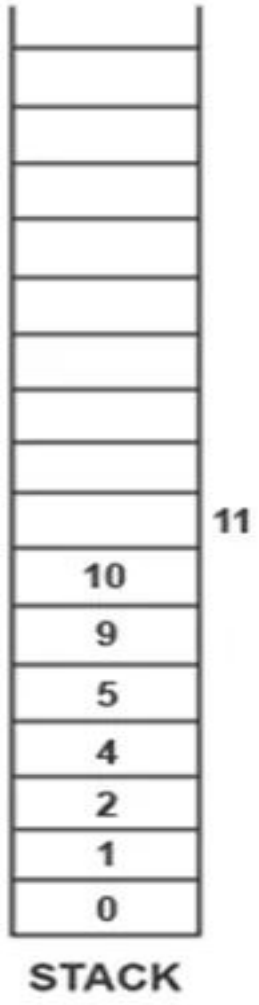


Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

A right turn is required to get to point 11 from point 10.

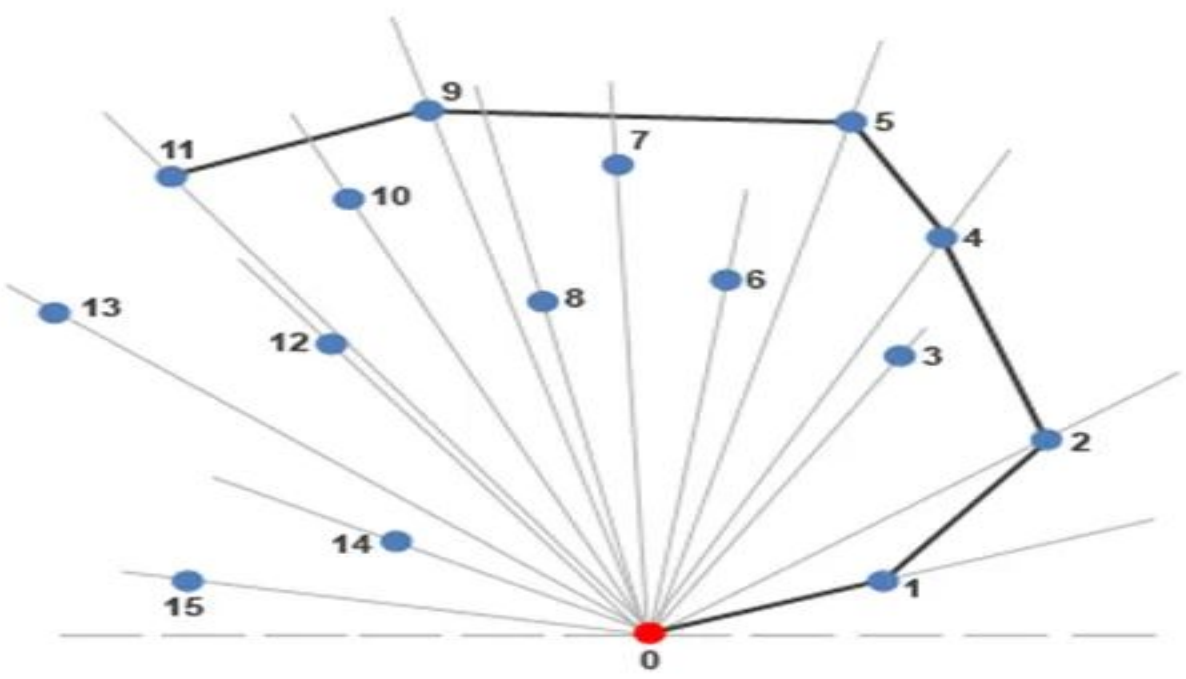


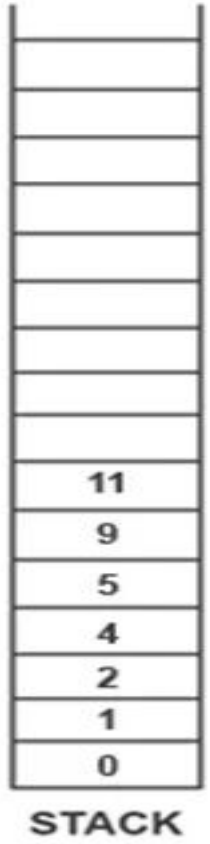


Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

Since a right turn is taken at point 10, point 10 is popped from the stack and the path to point 10 from point 9 is examined. Since a left turn is made at point 9 to get to point 11, point 11 is pushed onto the stack.

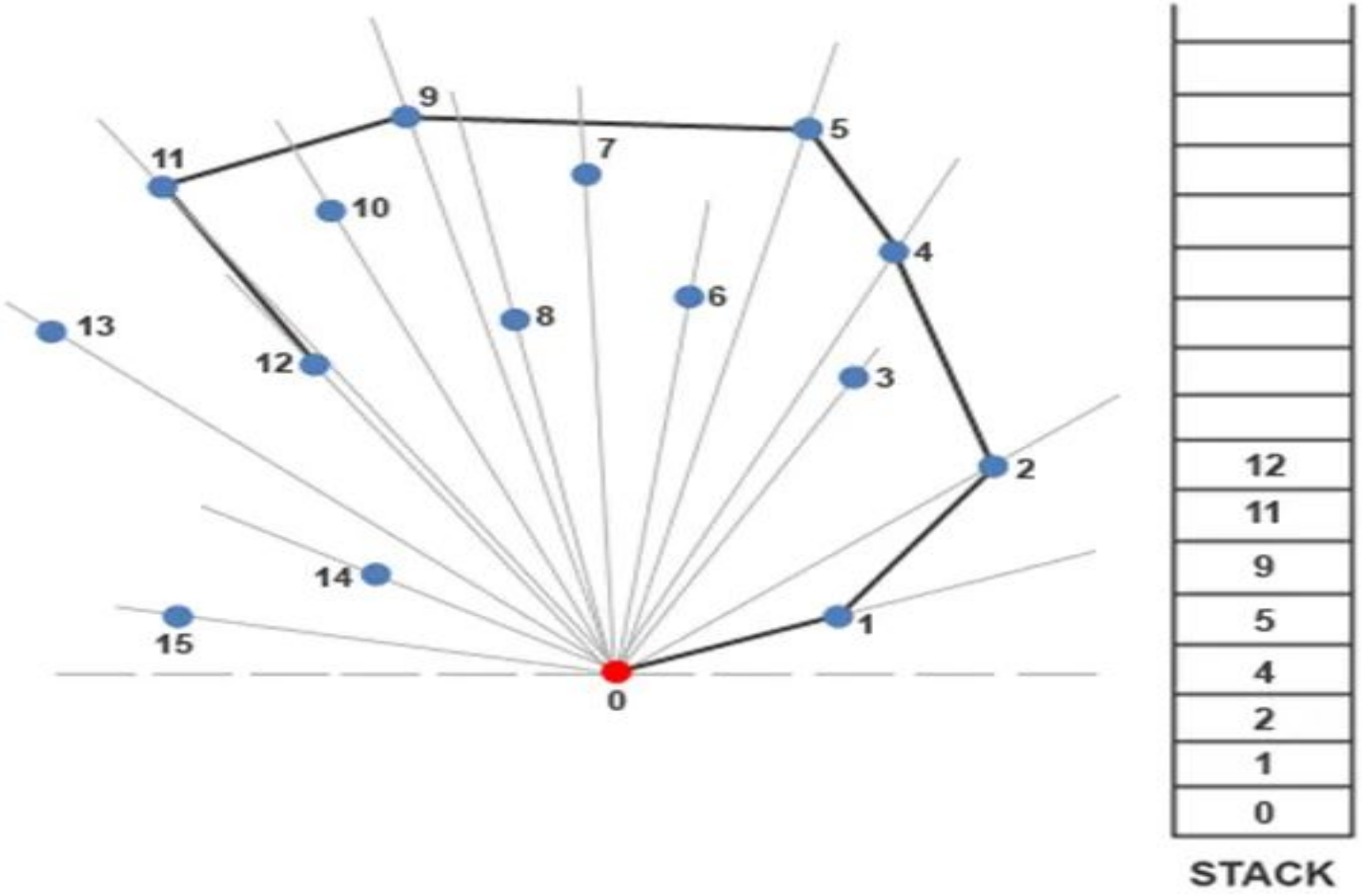




Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

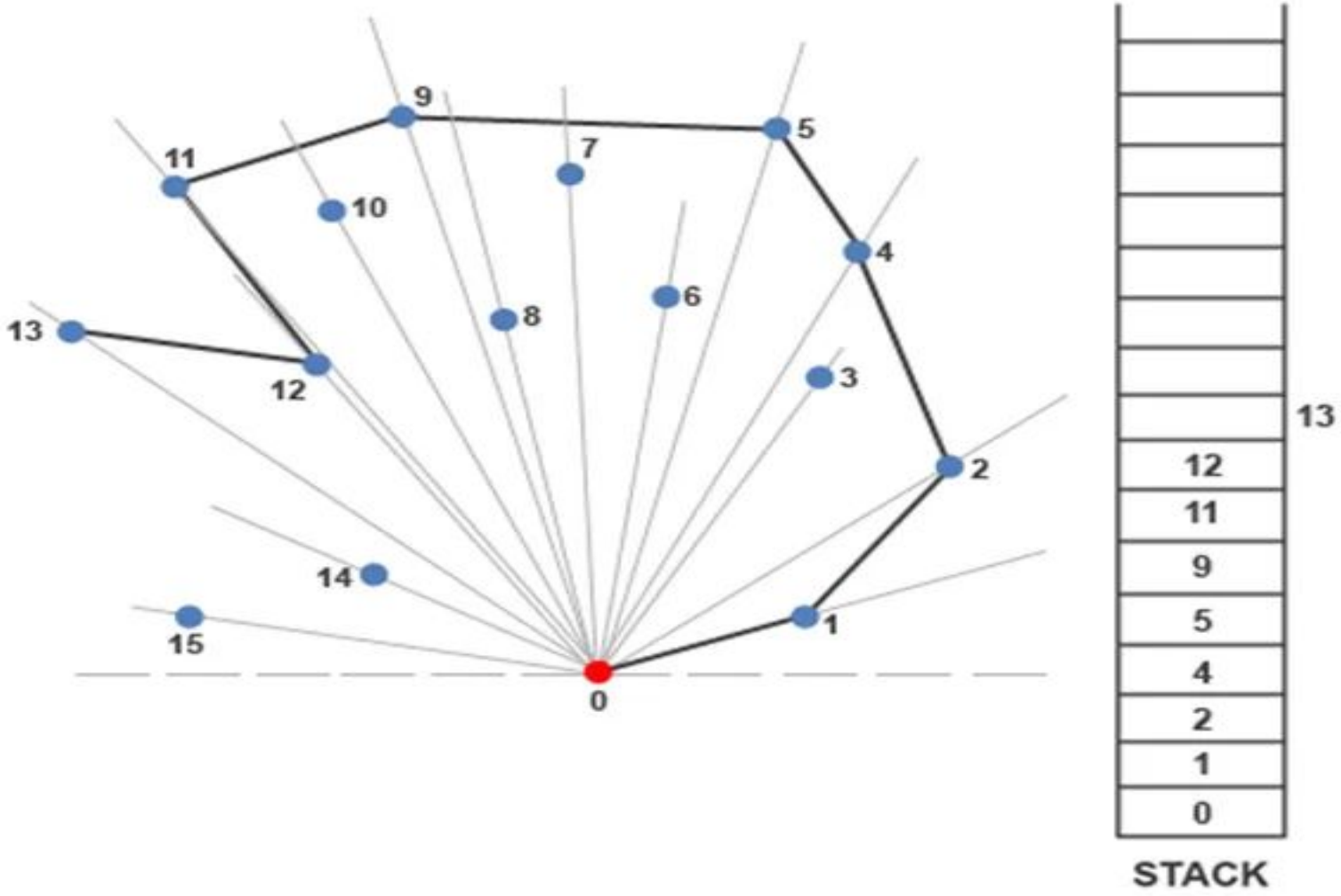
A left turn is made at point 11 to get to point 12. Point 12 is therefore pushed onto the stack and is currently considered part of the convex hull.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

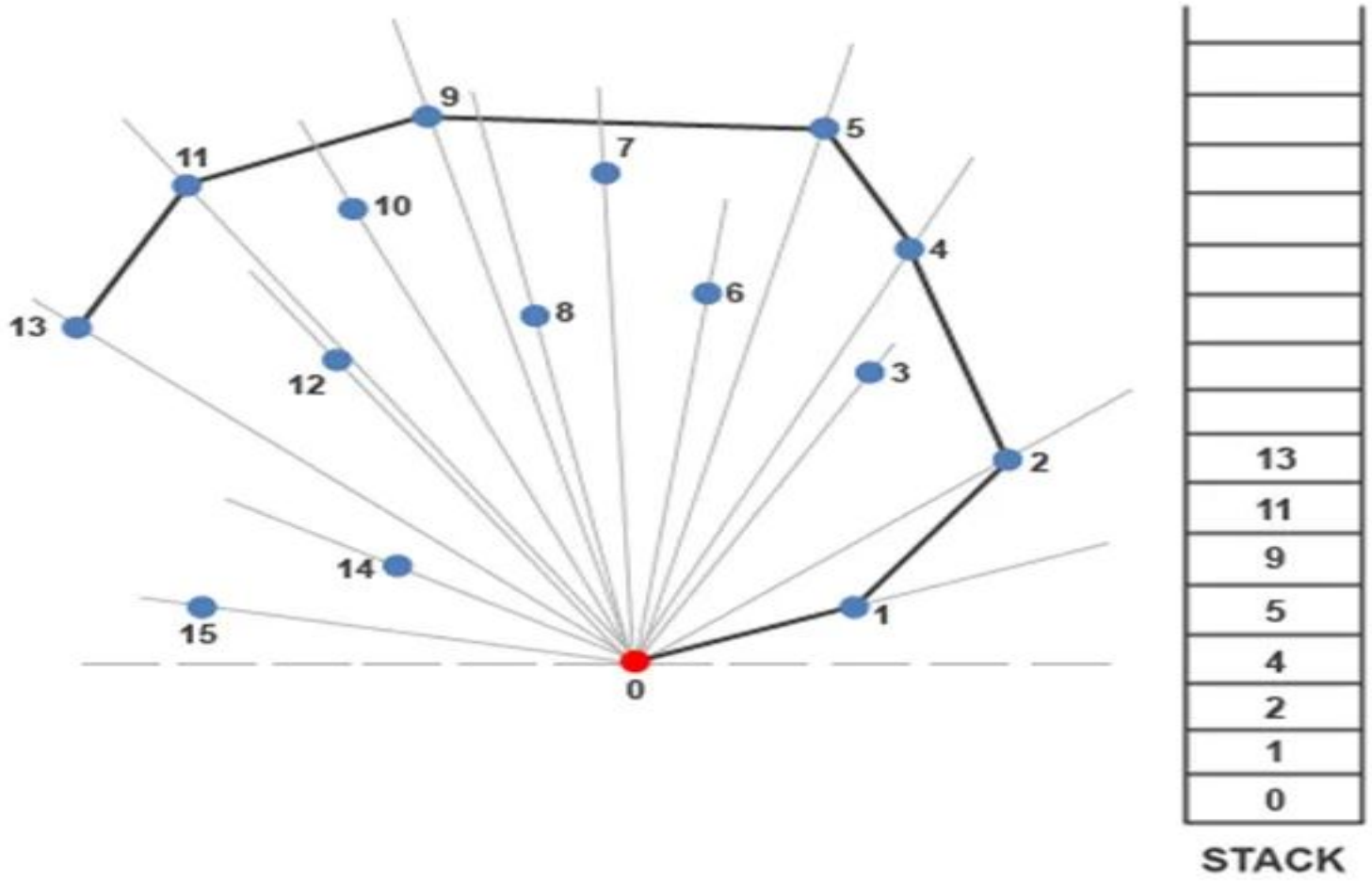
A right turn is required to go to point 13 from point 12.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

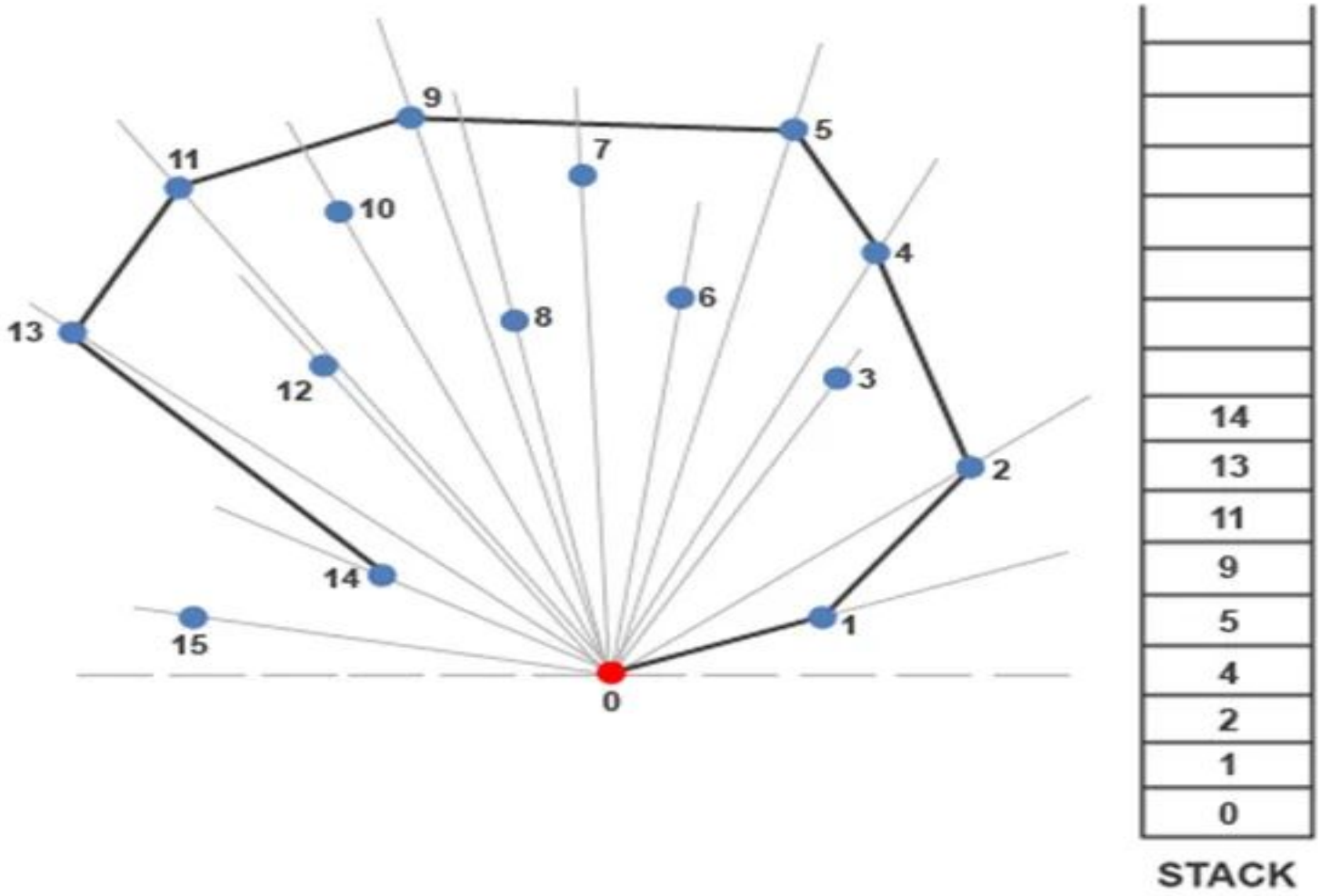
Point 12 is popped from the stack and the path to point 13 from point 11 is examined. Since a left turn is made at point 11, point 13 is pushed onto the stack.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

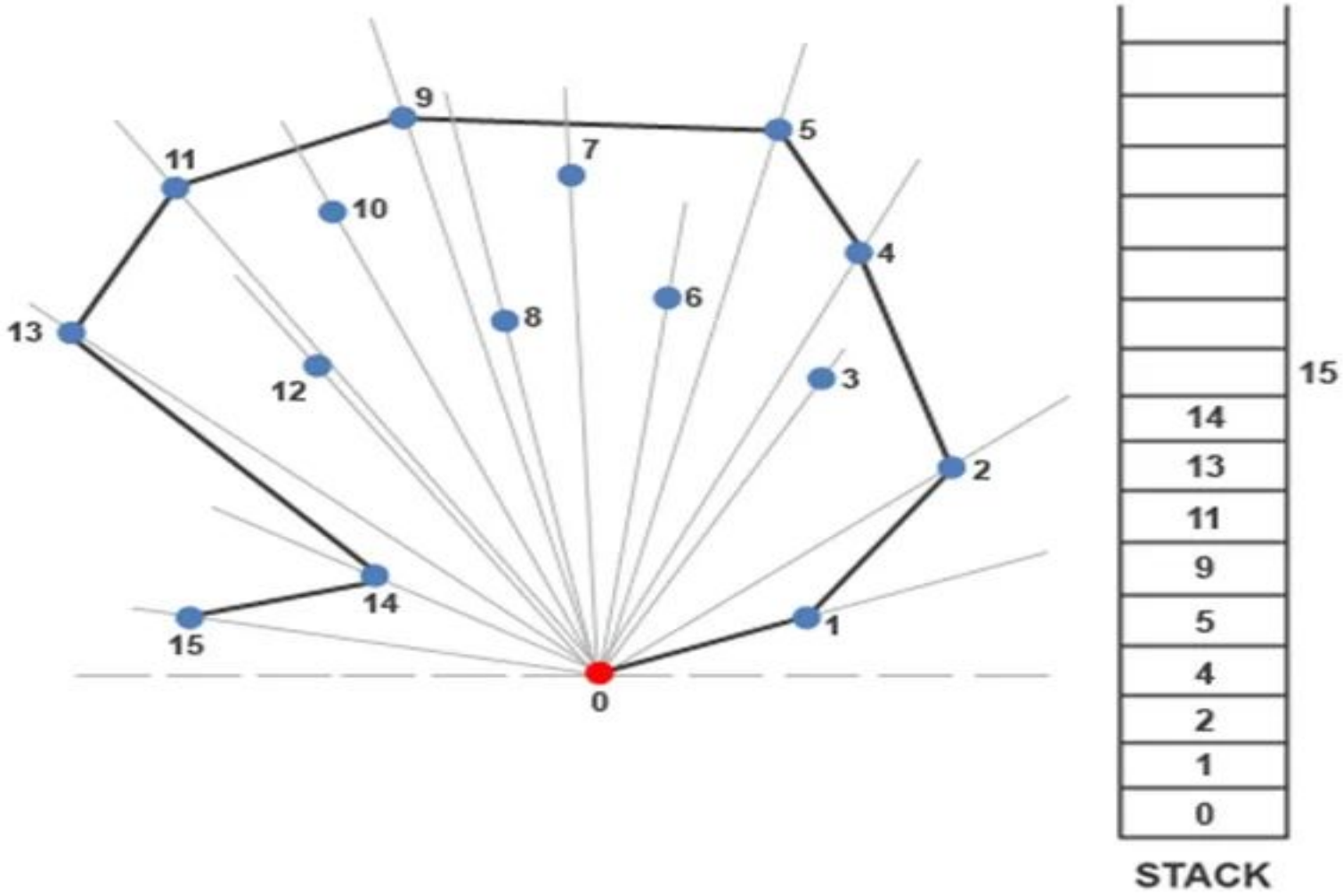
A left turn is made at point 13 to get to point 14, so point 14 is pushed onto the stack.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

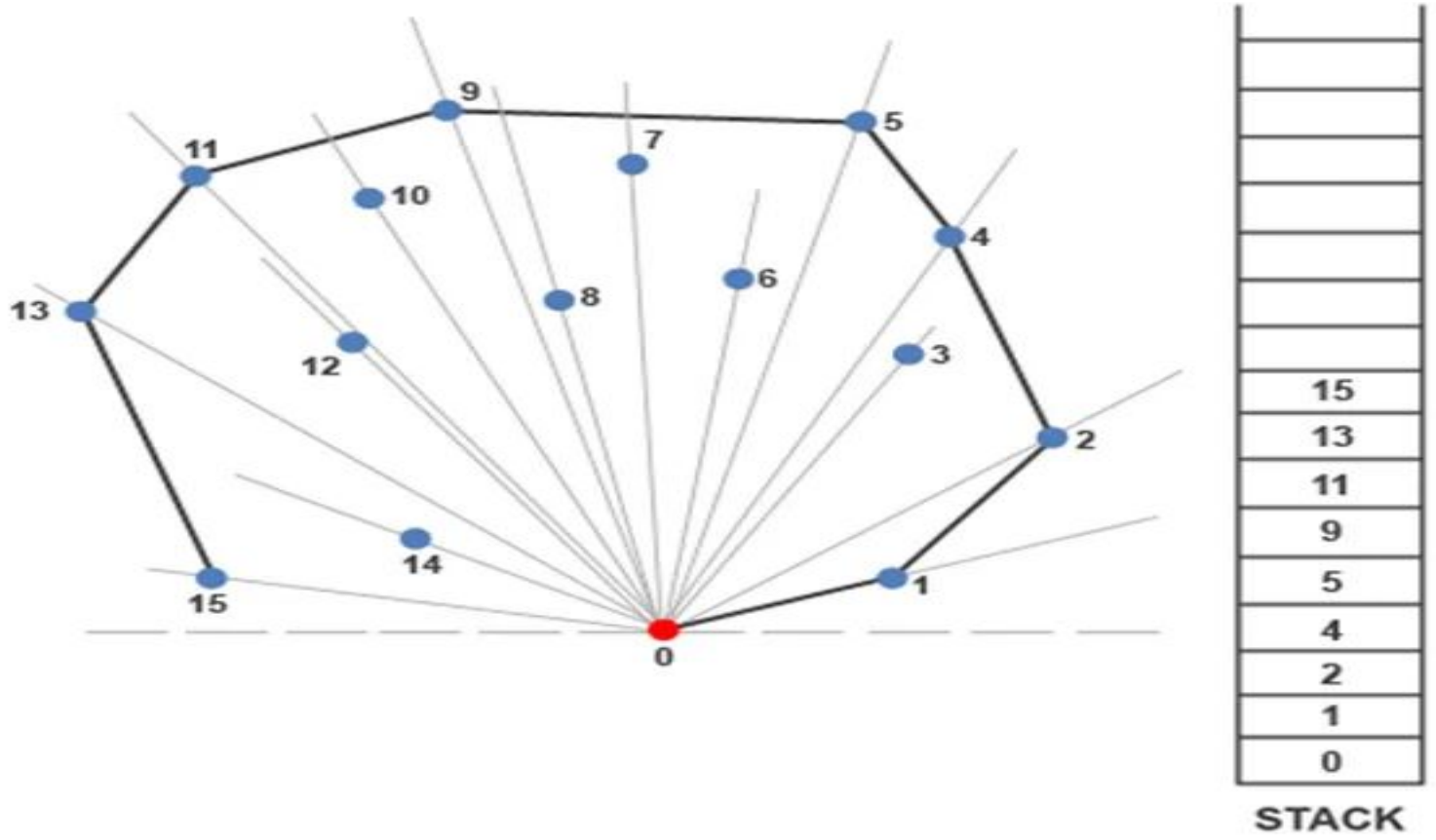
A right turn is required to go from point 14 to point 15.



Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

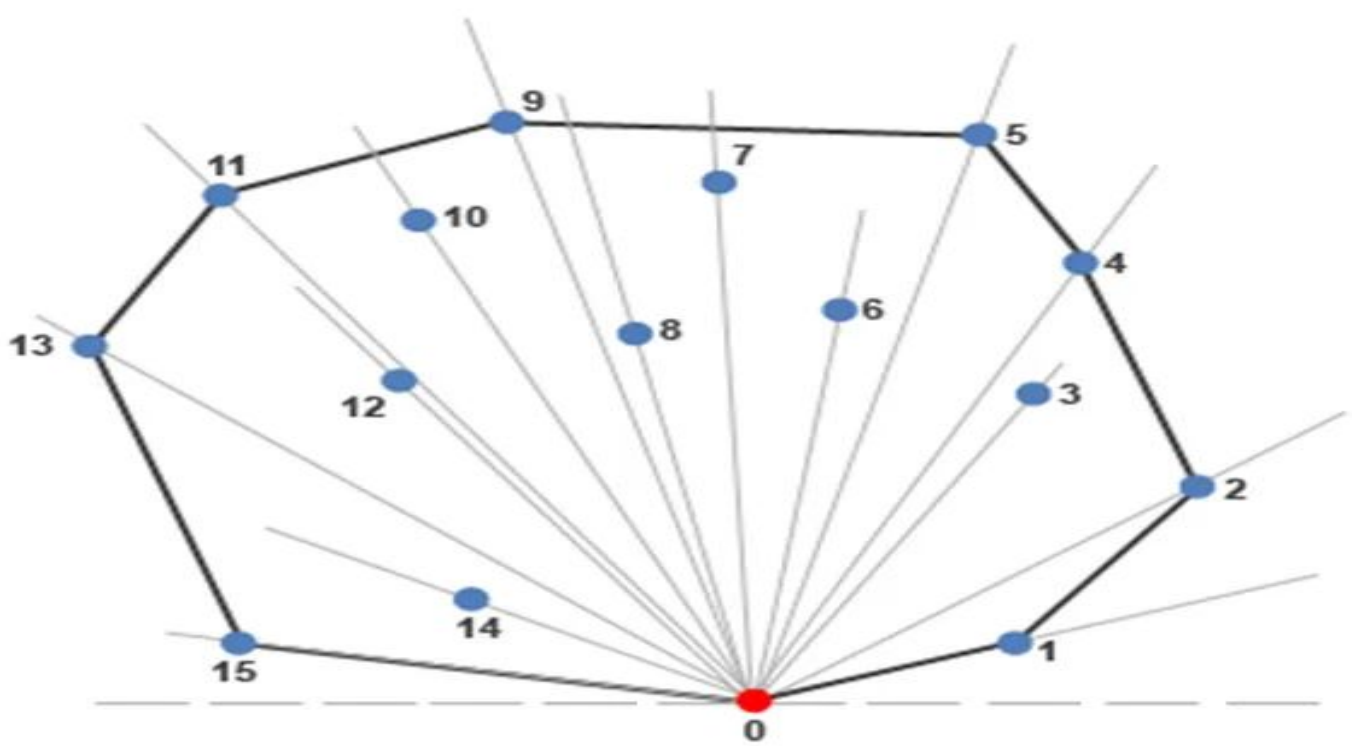
Since a right turn was made at point 14, point 14 is popped from the stack. The path to point 15 from point 13 is examined next. A left turn is made at point 13 to get to point 15, so point 15 is pushed onto the stack.

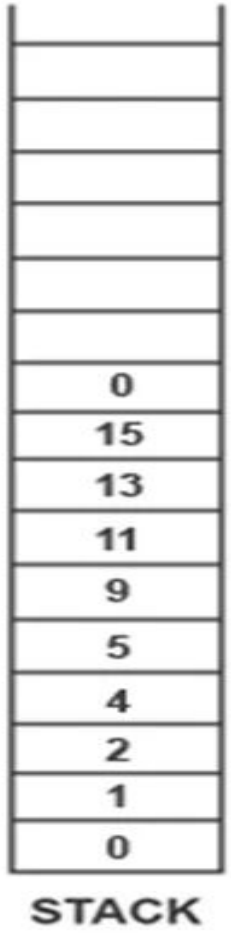


Computational Geometry (Convex Hull)

Graham scan Algorithm (Example)

Going from point 15 to the starting point 0 requires a left turn. Since the initial point was the point that we needed to reach to complete the convex hull, the algorithm ends.





The points that are needed to create the convex hull are 0–1–2–4–5–9–11–13–15.

Computational Geometry (Convex Hull)

Graham scan Algorithm (Complexity)

Complexity:

- Sorting points by polar angle: O(nlogn)
- Constructing the hull: O(n)

Total Complexity: **O(nlogn)**

Jarvis's March (Gift Wrapping Algorithm) – Explore yourself

Jarvis's March is a straightforward algorithm that computes convex hull for a set of points. It relies
on the following two facts:

1. The leftmost point must be one vertex of the convex hull.
2. If point p is a vertex of the convex hull, then the points furthest clockwise and counter-clockwise
are also vertices of the convex hull.

Computational Geometry

Voronoi Diagram

Computational Geometry (Voronoi Diagram)

Voronoi Diagram

- A **Voronoi diagram** is a partitioning of a plane into regions based on the distance to a specific set of points.
- Each region, called a Voronoi cell, contains all the points closer to one particular point (called a site or generator) than to any other site

Key Concepts:

- **Voronoi Cell**: The region containing all points closer to a specific site than to any other site.
- **Duality with Delaunay Triangulation**: The Voronoi diagram is closely related to the Delaunay triangulation, where each edge of the triangulation is perpendicular to the edge of the Voronoi diagram

Applications:

- **Geographic Information Systems (GIS)**: Used for spatial analysis and mapping.
- **Computer Graphics**: Helps in texture mapping and mesh generation.
- **Urban Planning**: Assists in determining the optimal placement of facilities like schools, hospitals, and stores.
- **Biology**: Models the growth patterns of cells and tissues

Computational Geometry (Voronoi Diagram)

- A Voronoi diagram encodes proximity information, that is, what is close to what. Let $P = \{p_1, p_2, \ldots, p_n\}$ be a set of points in the plane (or in any dimensional space), which we call sites. Define V(p_i), the Voronoi cell for p_i, to be the set of points q in the plane that are closer to p_i than to any other site. Tha $\mathcal{V}(p_i) = \{q \mid \|p_i q\| < \|p_j q\|, \forall j \neq i\}$, is define $\|pq\| = \left(\sum_{i=j}^{d} (p_j - q_j)^2\right)^{1/2}$ where denotes the Euclidean distance between points p and q.
- The Voronoi diagram can be defined over any metric and in any dimension.
- Another way to define *V(pi)* is in terms of the intersection of halfplanes. Given two sites p_i and p_j, the set of points that are strictly closer to p_i than to p_j is just the open halfplane whose bounding line is the perpendicular bisector between p_i and p_j. Denote this halfplane h $\mathcal{V}(p_i) = \bigcap_{j \neq i} h(p_i, p_j)$. to see that a point q lies in V(p_i) if and only if q lies within the inter $h(p_i, p_j)$ for all $j \neq i$.
- In otherwords,

Computational Geometry (Voronoi Diagram)

- Since the intersection of halfplanes is a (possibly unbounded) convex polygon, it is easy to see that $V(p_i)$ is a (possibly unbounded) convex polygon.
- Finally, define the Voronoi diagram of P, denoted $Vor(P)$ to be what is left of the plane after we remove all the (open) Voronoi cells.
- The Voronoi diagram consists of a collection of line segments, which may be unbounded, either at one end or both.
- Voronoi Diagram can be use for answering nea neighbor queries, computational morphology and shape analysis, clustering and data minin facility location, multi-dimensional interpolati

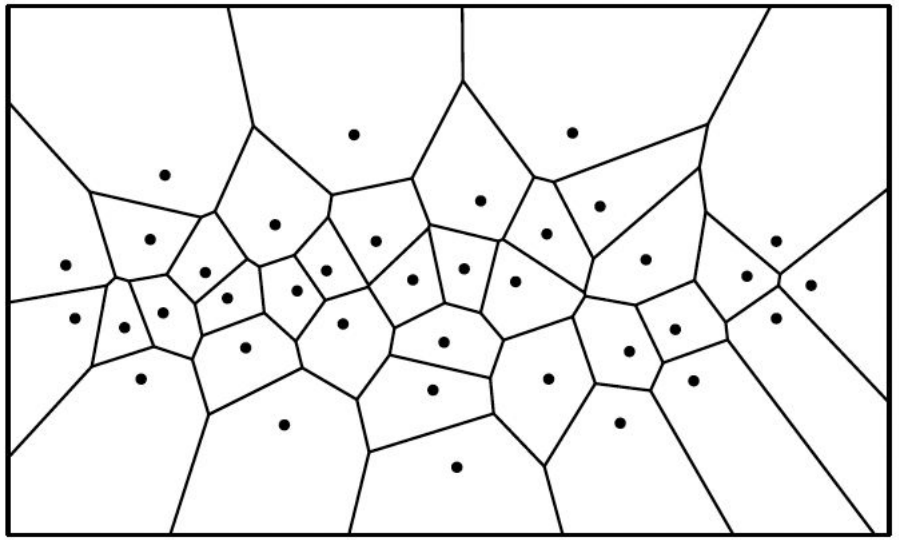
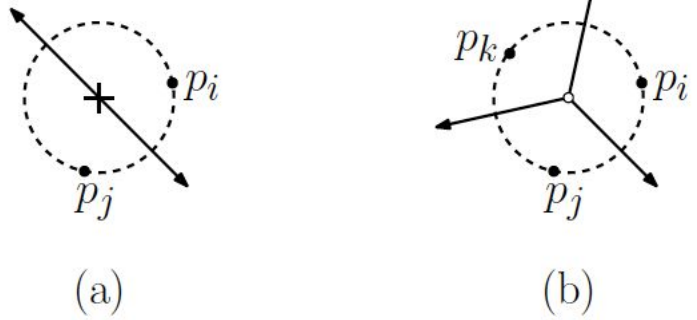


Fig: Voronoi diagram

Computational Geometry (Voronoi Diagram)

Properties of Voronoi Diagram:

- Voronoi complex:
  - Each point on an edge of the Voronoi diagram is equidistant from its two nearest neighbors p_i and p_j. Thus, there is a circle centered at such a point such that p_i and p_j lie on this circle, and no other site is interior to the circle (Fig. a)
- Voronoi vertices:
  - It follows that the vertex at which three Voronoi cells $V(p_i)$, $V(p_j)$, and $V(p_k)$ intersect, called a Voronoi vertex is equidistant from all sites (Fig. b).
  - Thus it is the center of the circle passing through these sit and this circle contains no other sites in its interior.



- Degree:
  - Generally three points in the plane define a unique circle. If we make the general position assumption that no four sites are cocircular, then the vertices of the Voronoi diagram all have degree three.

Computational Geometry (Voronoi Diagram)

- Convex hull:
  - A cell of the Voronoi diagram is unbounded if and only if the corresponding site lies on the convex hull.
  - Thus, given a Voronoi diagram, it is easy to extract the convex hull in linear time.
- Size:
  - If n denotes the number of sites, then the Voronoi diagram is a planar graph (if we imagine all the unbounded edges as going to a common vertex infinity) with exactly n faces. It follows from Euler's formula that the number of Voronoi vertices is roughly *2n* and the number of edges is roughly *3n*.

Computational Geometry (Voronoi Diagram)

Construction of Voronoi Diagram [Most common algorithms are]

1. **Incremental Construction Algorithm**
   - Complexity: $O(n^2)$ (average case O(nlogn) with proper data structures)
2. **Fortune's Sweep Line Algorithm**
   - Complexity: O(nlogn)
3. **Divide and Conquer Algorithm**
   - Complexity: O(nlogn)
4. **Lloyd's Algorithm (Iterative Refinement)**
   - Complexity: Dependent on iterations; often $O(n^2)$
5. **Brute Force Algorithm**
   - Complexity: $O(n^2)$
6. **Dual Delaunay Triangulation Approach**
   - Complexity: O(nlogn)

Construction of Voronoi Diagram(Incremental Approach)

- **Input :** A set of points $P=\{p_1,p_2,\ldots,p_n\}$ in a 2D plane.
- **Output :** A Voronoi diagram representing the partitioning of the plane based on the input points.

Algorithm

Step 1: Initialization

1. **Start with the first point** p_1:
   1. Since there is only one point, the entire plane belongs to p_1.
   2. Initialize the diagram as a single cell for p_1.
2. **Add the second point** p_2:
   1. Compute the perpendicular bisector of the line segment p_1p_2.
   2. This bisector divides the plane into two regions, one for $p_1$and one for p_2.
   3. Update the diagram with these two cells.

Construction of Voronoi Diagram(Incremental Approach)

Step 2: Iterative Addition

- For each subsequent point p_k (k=3,4,…,n):

1. **Identify affected cells**:
   - Determine the Voronoi cells that will be modified by the addition of p_k. These are the cells for points closer to p_k than to their current boundaries.
2. **Compute new edges**:
   - For each affected cell, compute the perpendicular bisector of the line segment between p_k and the cell's generating point.
   - The intersection of these bisectors defines the boundary of the new cell for p_k.
4. **Clip the new cell**:
   - The new Voronoi cell for p_k is bounded by:
     - The edges from the perpendicular bisectors.
     - Any existing Voronoi edges that are closer to other points.
4. **Update affected cells**:
   - For each affected cell, modify its boundaries to include the new edges from p_k.

Construction of Voronoi Diagram(Incremental Approach)

Step 3: Edge Removal

- Remove edges that no longer belong to the Voronoi diagram (i.e., those replaced by new edges from p_k).

Step 4: Final Diagram

- After all points have been added, finalize the diagram by trimming or extending unbounded edges as necessary (e.g., using bounding boxes).

- **How to calculate perpendicular bisector ? (Next Slide)**

1. **Find the midpoint** of p_1 and p_2:

$$\text{Midpoint} = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2}\right)$$

2. **Calculate the slope of the line** connecting p_1 and p_2:

$$\text{slope of line } p_1p_2 = \frac{y_2 - y_1}{x_2 - x_1}$$

(This is the **direct slope** between the two points.)

3. **Find the slope of the perpendicular bisector**: The slope of the perpendicular bisector is the **negative reciprocal** of the slope of the line joining p_1 and p_2:

$$\text{slope of perpendicular bisector} = -\frac{1}{\text{slope of line } p_1p_2}$$

If the line connecting p_1 and p_2 is vertical (i.e., $x_1 = x_2$), the slope of the perpendicular bisector will be **horizontal**, i.e., 0. Similarly, if the line is horizontal, the perpendicular bisector will be vertical.

4. **Equation of the perpendicular bisector**: Using the midpoint and the slope of the perpendicular bisector, the equation of the perpendicular bisector can be written in point-slope form:

$$y - \frac{y_1 + y_2}{2} = m_\perp \left(x - \frac{x_1 + x_2}{2}\right)$$

where $m_\perp$ is the slope of the perpendicular bisector.

Source: Chatgpt

Construction of Voronoi Diagram(Incremental Approach)

Example

Example

Let's say we have two points $p_1(1,1)$ and $p_2(4,5)$. To find the perpendicular bisector:

1. **Midpoint**:

$$\text{Midpoint} = \left(\frac{1+4}{2}, \frac{1+5}{2}\right) = (2.5, 3)$$

2. **Slope of p_1p_2**:

$$\text{slope of } p_1p_2 = \frac{5-1}{4-1} = \frac{4}{3}$$

3. **Slope of the perpendicular bisector**:

$$m_\perp = -\frac{1}{\frac{4}{3}} = -\frac{3}{4}$$

4. **Equation of the perpendicular bisector**: Using the midpoint $(2.5, 3)$ and the slope $-\frac{3}{4}$, the equation is:

$$y - 3 = -\frac{3}{4}(x - 2.5)$$

↓

Now, this bisector can be used to divide the plane into two regions: one for p_1 and one for p_2.

Construction of Voronoi Diagram(Incremental Approach)

Illustration

P={(2,3),(5,8),(1,7),(6,2)
}

Step 1: Start with the first point $(2, 3)$

- Initially, we have only one point, $p1 = (2, 3)$, and the entire plane is assigned to this point.
- **Diagram**: The diagram will consist of a single region that covers the entire plane and belongs to $p1$.

Step 2: Add the second point $(5, 8)$

- **Step 2.1: Compute the perpendicular bisector** between $(2, 3)$ and $(5, 8)$.
  - Midpoint:
$$\left(\frac{2+5}{2}, \frac{3+8}{2}\right) = (3.5, 5.5)$$
  - Slope of line $p1p2$:
$$\frac{8-3}{5-2} = \frac{5}{3}$$
  - Slope of the perpendicular bisector (negative reciprocal):
$$-\frac{3}{5}$$
  - Equation of the perpendicular bisector:
$$y - 5.5 = -\frac{3}{5}(x - 3.5)$$
    Simplifying this equation:
$$y = -\frac{3}{5}(x - 3.5) + 5.5$$
- **Step 2.2: Update the diagram** by adding the perpendicular bisector.
  - The bisector divides the plane into two regions: one for $p1 = (2, 3)$ and one for $p2 = (5, 8)$.

Construction of Voronoi Diagram(Incremental Approach)

Step 3: Add the third point $(1, 7)$

- **Step 3.1: Compute the perpendicular bisector** between $(2, 3)$ and $(1, 7)$.
  - Midpoint:

$$\left(\frac{2+1}{2}, \frac{3+7}{2}\right) = (1.5, 5)$$

  - Slope of line p_1p_3:

$$\frac{7-3}{1-2} = \frac{4}{-1} = -4$$

  - Slope of the perpendicular bisector:

$$\frac{1}{4}$$

  - Equation of the perpendicular bisector:

$$y - 5 = \frac{1}{4}(x - 1.5)$$

    Simplifying this equation:

$$y = \frac{1}{4}(x - 1.5) + 5$$

- **Step 3.2: Compute the perpendicular bisector** between $(5, 8)$ and $(1, 7)$.
  - Midpoint:

$$\left(\frac{5+1}{2}, \frac{8+7}{2}\right) = (3, 7.5)$$

  - Slope of line p_2p_3:

$$\frac{7-8}{1-5} = \frac{-1}{-4} = \frac{1}{4}$$

  - Slope of the perpendicular bisector:

$$-4$$

  - Equation of the perpendicular bisector:

$$y - 7.5 = -4(x - 3)$$

    Simplifying this equation:

$$y = -4(x - 3) + 7.5$$

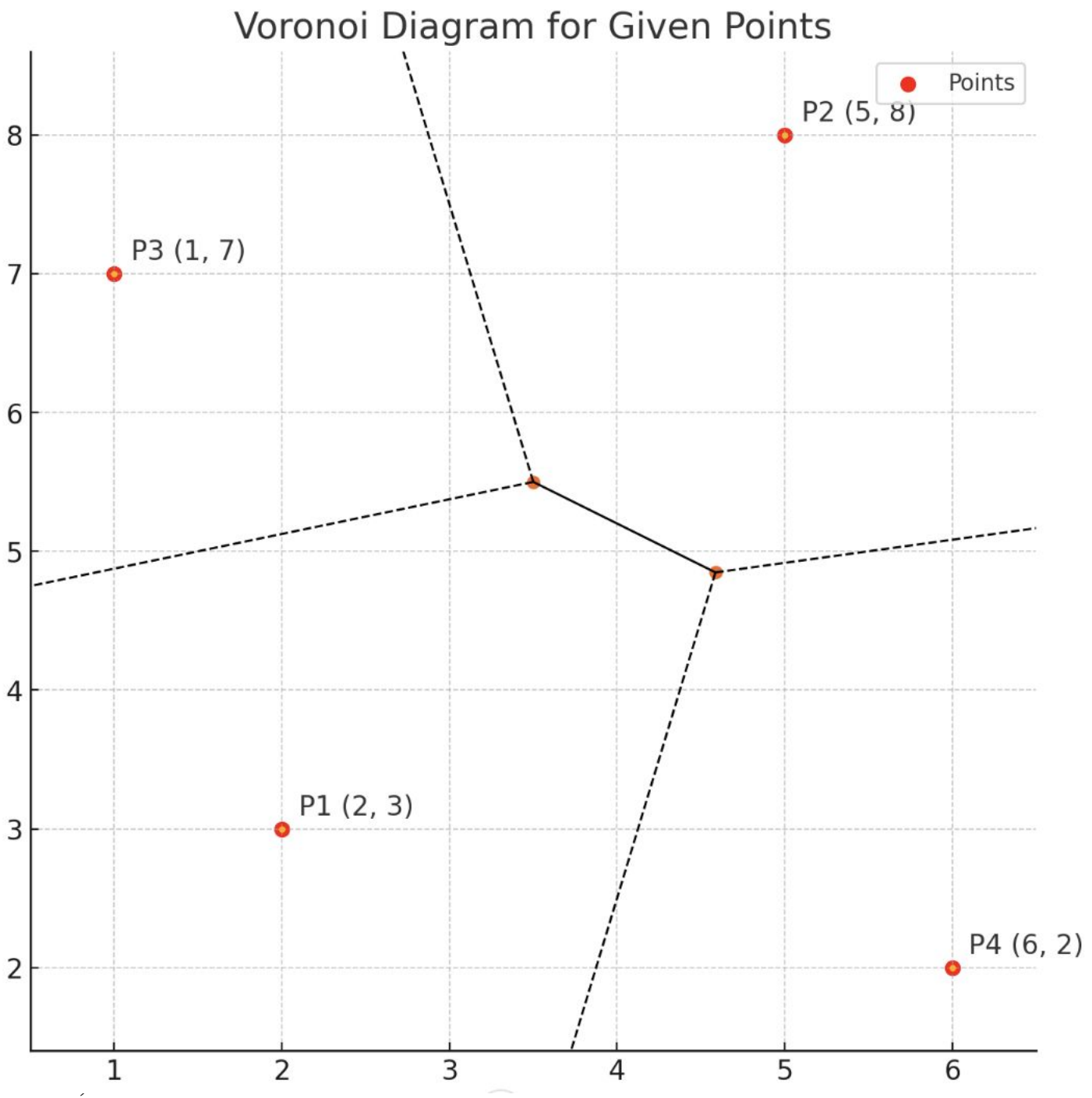
- **Step 3.3: Update the diagram** by adding the two new bisectors.
  - The plane is now divided into three regions: one for each of $p_1 = (2, 3)$, $p_2 = (5, 8)$, and $p_3 = (1, 7)$.

Construction of Voronoi Diagram(Incremental Approach)

Step 4: Add the fourth point $(6, 2)$

- **Step 4.1: Compute the perpendicular bisector** between $(2, 3)$ and $(6, 2)$.
  - Midpoint:
$$\left(\frac{2+6}{2}, \frac{3+2}{2}\right) = (4, 2.5)$$
  - Slope of line $p1p4$:
$$\frac{2-3}{6-2} = \frac{-1}{4}$$
  - Slope of the perpendicular bisector:
$$4$$
  - Equation of the perpendicular bisector:
$$y - 2.5 = 4(x - 4)$$
    Simplifying this equation:
$$y = 4(x - 4) + 2.5$$
- **Step 4.2: Compute the perpendicular bisector** between $(5, 8)$ and $(6, 2)$.
  - Midpoint:
$$\left(\frac{5+6}{2}, \frac{8+2}{2}\right) = (5.5, 5)$$
  - Slope of line $p2p4$:
$$\frac{2-8}{6-5} = \frac{-6}{1} = -6$$
  - Slope of the perpendicular bisector:
$$\frac{1}{6}$$
  - Equation of the perpendicular bisector:
$$y - 5 = \frac{1}{6}(x - 5.5)$$
    Simplifying this equation:
$$y = \frac{1}{6}(x - 5.5) + 5$$
- **Step 4.3: Compute the perpendicular bisector** between $(1, 7)$ and $(6, 2)$.
  - Midpoint:
$$\left(\frac{1+6}{2}, \frac{7+2}{2}\right) = (3.5, 4.5)$$
  - Slope of line $p3p4$:
$$\frac{2-7}{6-1} = \frac{-5}{5} = -1$$
  - Slope of the perpendicular bisector:
$$1$$
  - Equation of the perpendicular bisector:
$$y - 4.5 = 1(x - 3.5)$$
    Simplifying this equation:
$$y = (x - 3.5) + 4.5$$
$$y = x + 1$$
- **Step 4.4: Update the diagram** by adding the three new bisectors.
  - The plane is now divided into four regions, each associated with one of the four points: $p1 = (2, 3)$, $p2 = (5, 8)$, $p3 = (1, 7)$, and $p4 = (6, 2)$.

Fig: Voronoi Diagram of Given Points



Note: Demo in console

Reference

Graham scan Algorithm (Example)

- Convex Hull: Mark de Berg · Otfried Cheong, Marc van Kreveld · Mark Overmars: Computational Geometry: Algorithms and Applications,Third Edition Springer
- Convex Hull Example (Graham Scan): https://www.dinocajic.com/grahams-scan-visually-explained/
- Voronoi Diagram: Computational Geometry Lecture Notes Voronoi Diagrams, Valerie Barr, Hava Siegelmann, Gabor Sarkozy (1990) Michael Horn, Julie Weber (2004)

End of Section 2.6